

NEV-AKE

Algorithm Specifications and Supporting Documentation (version 1.1)

Jiang Zhang¹, Shuqin Fan¹, Kang Yang¹, Jiangxia Ge², Di Yan¹, Debiao He³, Yu Yu^{4,5}, Runqing Xu³, Jing Pan⁶, Yuanmi Chen⁵, Weijia Wang⁷, Cong Peng³, Hui Guo¹, Hui Zhu⁶, and Ruikang Liu⁷

¹ State Key Laboratory of Cryptology, Beijing, China

² China Telecom Quantum Group Ltd., Hefei, China

³ Wuhan University, Wuhan, China

⁴ Shanghai Jiao Tong University, Shanghai, China

⁵ East China Normal University, Shanghai, China

⁶ Xidian University, Xi'an, China

⁷ Shandong University, Qingdao, China

June 26, 2026

Table of Contents

1	Introduction	1
1.1	Feature Statements	1
1.2	Overview of the Generic AKE Construction	2
1.3	Overview of the NEV-AKE Instantiation	4
2	Preliminaries	5
2.1	Notation	5
2.2	Distributions	5
2.3	Defintion of Public-Key Encryption	6
2.4	Definition of Key Encapsulation Mechanism	7
2.5	Two-Round Authenticated Key Exchange	8
2.6	CK^+ Security Model for AKE	9
2.7	Hardness Assumptions	13
3	Generic AKE Protocol from OW-CPA KEM and PKE	15
4	Concrete AKE Instantiation from NTRU and RLWE	18
4.1	Randomized Plaintext Encoding and Decoding	18
4.2	Detailed Descriptions of NEV-PKE-OW and NEV-KEM-OW	20
5	Concrete Parameters	24
6	Implementation Details	25
6.1	Partial NTT Multiplication	25
6.2	Partial NTT Inversion	27
6.3	Symmetric Primitives	27
6.4	Encoding of R_q Elements	28
6.5	Sampling Algorithms	31
6.6	The NEV-PKE-OW and NEV-KEM-OW Schemes	32
7	Performance Evaluation	35
8	Provable Security	36
8.1	Security Proof of GC-AKE in the CK^+ model under the ROM	36
8.2	Security Proof of GC-AKE in the CK^+ model under the QROM	40
8.3	Security Proof of NEV-PKE-OW and NEV-KEM-OW	40
8.4	Security Proof of NEV-AKE in the CK^+ model	42
9	Known Attacks	44
9.1	Attacks against ALWE	44
	Primal Attack.	44
	Dual Attack.	46
	Refined MATZOV Attack ($MATZOV^\dagger$).	48
9.2	Solving the $DNTRU_{n,q,\chi_f,\chi_g,v}$ problem	50
9.3	Solving the $sspRLWE_{n,q,\chi_r,B_\eta,\bar{v}}$ problem	50
9.4	Concrete Security Estimates Against Lattice Attacks	51
9.5	Attacks Exploiting Decryption Failures	51
9.6	Attacks against Symmetric Primitives	55
10	Side-Channel Vulnerabilities and Countermeasures	55
10.1	Security Analysis	56

Key Generation Phase.	56
Decapsulation Phase.	56
10.2 A Discussion on Masking Countermeasures	56
Polynomial Sampling.	57
Masked Pt2Noise Module.	58
Masked Noise2Pt Module.	58
Masked Polynomial Inversion Module.	59
Masked Zero-Testing.	59
A Appendix.	64
A.1 Proof of Lemma 4	64
A.2 Proof of Lemma 5	69
A.3 Proof of Lemma 6	80
A.4 Proof of Lemma 7	90
A.5 Proof of Theorem 3	99

1 Introduction

1.1 Feature Statements

This document first presents a generic construction of Authenticated Key Exchange (AKE), based on Public Key Encryption (PKE) and Key Encapsulation Mechanism (KEM), where both of them satisfy One-Wayness under Chosen-Plaintext Attacks (OW-CPA). Then, we show an AKE instantiation under the NTRU and RLWE assumptions, based on the NEV-KEM scheme, which is an extension and improvement of the known scheme [71].

To instantiate the AKE generic construction, we design the OW-CPA PKE and KEM schemes in the NEV family. The PKE scheme **NEV-PKE-OW** is first designed and proven OW-CPA secure under the NTRU and Subset-Sum Parity Ring Learning with Error (sspRLWE) assumptions, where sspRLWE is an intermediate assumption which is essentially as hard as the standard RLWE assumption under appropriate choices of parameters. Then, **NEV-PKE-OW** is slightly modified to obtain an OW-CPA secure KEM scheme **NEV-KEM-OW** under the same assumptions. Below, we summarize the advantages of the **NEV-AKE** protocol:

- **New generic construction with provable security in the QROM.** We propose a new generic construction of AKE (called **GC-AKE**), which is provably secure in both Random Oracle Model (ROM) and Quantum Random Oracle Model (QROM). Our generic construction allows one to use an OW-CPA secure PKE and an OW-CPA secure KEM as central building blocks. We adopt the well-known CK^+ model [45,29] to prove security of **GC-AKE**. Compared to the state-of-the-art generic AKE construction [38] built from standard PKE/KEM primitives, **GC-AKE** achieves slightly better computational efficiency.
- **Efficient instantiation.** We use the PKE and KEM optimized variants of the NEV scheme in [71], i.e., **NEV-PKE-OW** and **NEV-KEM-OW**, to instantiate the AKE generic construction **GC-AKE**. As a result, we obtain a practical AKE protocol **NEV-AKE** with only two rounds. For the **NEV-PKE-OW** or **NEV-KEM-OW** scheme, both public key and ciphertext only consist of a single ring element, which can be fully stored with 670 bytes for 128-bit classical security without using compressing techniques (risk of patent issues) or LWR-like assumptions.
- **No independence assumptions:** Unlike many existing schemes that rely on a problematic assumption in the analysis of decryption failures, both of **NEV-PKE-OW** and **NEV-KEM-OW** deliberately avoid an error-correcting mechanism whose analysis would depend on the independence of coefficient failures—an assumption rigorously disproven both theoretically and empirically [22]. Instead, **NEV-PKE-OW** and **NEV-KEM-OW** adopt a decryption failure analysis based on the numeric computation of a worst-case union bound, which provides a provably sound upper bound on the failure probability. Consequently, this approach enables more robust parameter selection and substantially enhances the scheme’s reliability in real-world deployments.
- **Conservative security estimates:** Our concrete security analysis adopts a conservative approach, relying primarily on the Core-SVP model introduced in [5]—a standard and widely referenced methodology for estimating the hardness of lattice problems. However, recognizing recent progress in hybrid attacks

on LWE, as discussed in [15], we also account for the MATZOV algorithm [50], which represents a significant improvement in attacks against certain LWE parameter regimes. By including this algorithm in our estimates, we guarantee that our security claims remain valid under the latest cryptanalytic understanding, particularly for long-term security requirements.

- **Easy to implement securely:** NEV-AKE is designed to avoid complex and leakage-prone operations such as higher-order error-correcting codes, (discrete) Gaussian sampling, and floating-/fixed-point arithmetic. Instead, it unifies all parameter sets under the same modulus and fixed Number Theoretic Transform (NTT), ensuring a clean and side-channel-friendly arithmetic foundation.
- **Competitive performance:** The total communication cost of an initiator (resp., a responder) for 128-bit classical security is 1127 bytes (resp., 1127 bytes). Furthermore, the total computation time of an initiator (resp., a responder) for 128-bit classical security is about 0.048 ms (resp., 0.040 ms) for the reference implementation and 0.017 ms (resp., 0.016 ms) for the AVX optimization implementation.
- **Patent relevance:** A key design goal of NEV-AKE is to avoid techniques that carry significant patent risks. These include foundational methods that were prominent in the NIST PQC competition, such as the framework for GAM/LPR-like encryption and rounding-like or reconciliation mechanisms for ciphertext compression. It is worth noting that the underlying PKE and KEM schemes of NEV-AKE differ significantly from NTRU, and most of the relevant patents surrounding NTRU have now expired. In addition, we have filed two Chinese patent applications for NEV-AKE: CN202310188848.0, filed in early 2023, on the application of vector encoding/decoding to NTRU-like schemes; and CN201910340234.3, filed in early 2019, on the application of asymmetric RLWE. To ensure freedom to operate, we have submitted the necessary statements of free use as required.

1.2 Overview of the Generic AKE Construction

There exist several generic AKE constructions built upon KEMs [29,30,38,70,69], and they achieve provable security under different attack models. For the constructions in [29,30], the security is established in the CK^+ model [45], one of the strongest security models for AKE protocols. Specifically, the FSXY protocol [29] requires its underlying KEMs to satisfy IND-CPA and IND-CCA security, with its security proven in the standard model. To improve the efficiency, [30] further proposed a ROM-based FSXY and proved its security in the CK^+ model under the ROM. This proof also relaxes the security requirements of KEMs to OW-CPA and OW-CCA security, where the latter can be obtained by directly applying the Fujisaki-Okamoto (FO) transformation [31]. However, [30] does not provide a QROM security proof in the CK^+ model. Later, based on FSXY, [38] proposed a new AKE construction HKSU and gave a formal security proof in the QROM. Intuitively, [38] decomposed the FO transformation and embedded it directly inside HKSU, which renders HKSU very compact and eliminates several redundant hash evaluations. However, to achieve a tighter QROM security

bound, [38] requires the underlying PKE scheme to be IND-CPA secure, instead of the weaker OW-CPA secure. As for the AKE constructions proposed in [70,69], together with the subsequent works [58,59,48], broadly speaking, they are not designed upon standard OW/IND-CPA secure PKEs/KEMs. Instead, they either introduce particular cryptographic primitives (e.g., 2-Key KEMs, tagged KEMs) or adopt customized security notions (e.g., multi-user, multi-challenge, lossy), to achieve improved communication, stronger security, or tighter security proofs.

Der _{resp} of HKSU	Der _{resp} of GC-AKE
1: $m_{j^*} \xleftarrow{\$} \mathcal{M}$	1: $m_{j^*} \xleftarrow{\$} \mathcal{M}$
2: $\tilde{m} \xleftarrow{\$} \mathcal{M}$	2: $\tilde{m} \xleftarrow{\$} \mathcal{M}$
3: $r := G(\tilde{m})$	3: $r := G(\tilde{m})$
4: $\tilde{c} := \text{PKE-IND.Enc}(\tilde{pk}, \tilde{m}; r)$	4: $(\tilde{c}, K^*) := \text{KEM-OW.Encap}(\tilde{pk})$
5:	5:

Fig. 1: The responder’s key derivation algorithm Der_{resp} for HKSU and GC-AKE. Here $\tilde{pk}$ is the ephemeral public key generated by the initiator. In this figure, PKE-IND denotes an IND-CPA secure PKE scheme, and KEM-OW represents a OW-CPA secure KEM scheme.

In this document, we follow the paradigm of [29,30,38] to design a generic AKE construction based on the OW-CPA secure PKE and KEM, which are more practical. We do not involve any non-standard primitives or customized security notions. In particular, we propose a computation-efficient variant of HKSU, which we call GC-AKE. As shown in Fig. 1, compared with HKSU, GC-AKE uses a KEM scheme to generate the ciphertext $\tilde{c}$, and optimizes the key derivation algorithm Der_{resp} by removing the derandomization step $r := G(\tilde{m})$, making the generation of $\tilde{c}$ more efficient. Essentially, similar to HKSU, GC-AKE decompose and embed the FO transformation into the AKE construction, but further removes one hash evaluation to achieve higher efficiency. It is worth noting that this removal is non-trivial: in the absence of derandomization, the domain-separation technique [43,38] used in the QROM proof of [38] is no longer valid, which requires us to develop a new QROM proof.

For the provable security of GC-AKE, we provide formal ROM and QROM security proofs for GC-AKE in the CK^+ model in Sec. 8. In contrast to the proof for HKSU given in [38], which requires IND-CPA security, our proof for GC-AKE only requires the underlying PKE and KEM schemes to satisfy weaker OW-CPA security. Moreover, unlike HKSU, our proof does not require the underlying PKE scheme to be γ -spread and does not introduce the extra **Punc** operation⁸.

⁸ Roughly speaking, this operation removes one plaintext from the plaintext space, and the security proof for HKSU given in [38] relies on this operation to establish a specific property of *disjoint simulatability*. This in turn enables the application of domain-separation techniques in the QROM to prove the indistinguishability between the challenge key and a uniformly random key.

Remark 1. At a high level, the derandomization step $r := G(\tilde{m})$ employed by HKSU allows it to have a concise QROM security proof with a relatively tight security bound. In our GC-AKE, by contrast, we have to use the One-Way to Hiding (O2H) lemma [67,6] to convert the adversary's distinguishing advantage into the guessing advantage of the plaintext $\tilde{m}$. This yields a somewhat complex proof as well as a looser bound. Nevertheless, GC-AKE only requires OW-CPA security, and as mentioned earlier, GC-AKE also achieves a better generality and computational efficiency. Hence, we believe that the trade-off between practicality and tightness for GC-AKE is acceptable, and that GC-AKE is more friendly to AKE protocol designers. In fact, our instantiation of GC-AKE based on NEV validates this judgment. In particular, in Sec. 5, apart from the construction of parameter sets "C1/C2/C3" and "R1/R2/R3", we further build parameter set "C1*/C2*/C3*" by introducing ciphertext compression into NEV-PKE-OW and NEV-KEM-OW. Since our security proof for GC-AKE only requires OW-CPA security and imposes no constraints on extra structures or additional properties, we are not required to check whether the compressed scheme satisfies certain specialized conditions. As a result, we can directly instantiate GC-AKE with parameter set "C1*/C2*/C3*" to derive an AKE protocol with smaller communication size, achieving a communication reduction of at least 103 bytes.

1.3 Overview of the NEV-AKE Instantiation

Based on the new AKE generic construction, we obtain a concretely efficient AKE instantiation (called NEV-AKE) by instantiating the underlying PKE and KEM schemes with NEV' at Asiacrypt 2023 [71], which is basically a variant of the NTRU encryption using a special polynomial plaintext modulus that allows vector decoding. Specifically, let n be a power of 2, and q be a prime. Let $R = \mathbb{Z}[x]/(x^n+1)$ and $R_q = \mathbb{Z}_q[x]/(x^n+1)$. Let $v = (1 - x^{n/k}) \in R_q^*$ be a ring element, whose inverse is $v^{-1} = \frac{q+1}{2}\bar{v} = \frac{q+1}{2}(1 + x^{n/k} + \dots + x^{(k-1)n/k}) \in R_q^*$. Let $\chi_f, \chi_g, \chi_r, \chi_e$ be four distributions over R . The public key h and the ciphertext c of NEV' have forms of

$$h = vg/(vf' + 1) = g/(f' + v^{-1}), \quad c = hr + e,$$

where $g \xleftarrow{\$} \chi_g, f' \xleftarrow{\$} \chi_f, r \xleftarrow{\$} \chi_r, e \xleftarrow{\$} \chi_e$. Note that by setting $v = p$, the above construction is essentially the same as the original NTRU encryption. For decryption, we first compute $u = (f' + v^{-1})c = gr + f'e + v^{-1}e$. Let $\bar{v} = 1 + x^{n/k} + \dots + x^{(k-1)n/k}$, $e_0 = \bar{v}e \bmod 2$, and $2e_1 = \bar{v}e - e_0$, we have

$$v^{-1} = \frac{q+1}{2}\bar{v}, \quad v^{-1}e = e_1 + \frac{q+1}{2}e_0 \in R_q, \quad \text{and } u = gr + f'e + e_1 + \frac{q+1}{2}e_0 \in R_q.$$

Let $\text{Ext} : R_2 \rightarrow \{0,1\}^{n/k}$ be a function that returns the first n/k coefficients of e_0 . By the nice property of $R_q = \mathbb{Z}_q[x]/(x^n+1)$ and the choice of $\bar{v}$ (and v^{-1}), it is easy to check that e_0 is essentially a polynomial with coefficients consisting of k copies of $\text{Ext}(e_0)$. This fact allows us to use a coefficient vector of u in k dimensions to decode each single coefficient of $\text{Ext}(e_0)$. Moreover, if $\chi_e = B_\eta$ is a center binomial distribution with parameter η , then there exists an efficient

randomized algorithm $\text{Pt2noise}(\cdot, \eta)$ such that (1) sampling an element $e \xleftarrow{\$} B_\eta$ can be equivalently done by first sampling $M \xleftarrow{\$} \{0, 1\}^{n/k}$ and then outputting $e = \text{Pt2noise}(M, \eta)$; (2) for any $e \xleftarrow{\$} \text{Pt2noise}(M, \eta)$, we always have $M = \text{Ext}(\bar{v}e \bmod 2)$. This allows us to show that the above scheme is indeed an OW-CPA secure PKE scheme under the NTRU assumption and the sspRLWE assumption with binomial error distribution. For convenience, we rename the PKE scheme using Pt2noise to encode the message as **NEV-PKE-OW**, which is exactly the PKE scheme **NEV'** in [71]. The **NEV-PKE-OW** scheme can be easily extended to obtain the OW-CPA KEM scheme **NEV-KEM-OW** with slightly better computational performance by replacing $e \xleftarrow{\$} \text{Pt2noise}(M, \eta) \in R_q$ with $e \xleftarrow{\$} B_\eta$.

2 Preliminaries

2.1 Notation

We use κ to denote the security parameter. For an algorithm A , we use $y \leftarrow A(x)$ to denote the procedure in which a polynomial-time algorithm A is given an input x and then outputs y . Let n be a power of 2, and q be a prime. We denote by R the ring $R = \mathbb{Z}[X]/(X^n + 1)$ and by R_q the ring $R_q = \mathbb{Z}_q[X]/(X^n + 1)$. The regular font letters (e.g., a, b) represent elements in R or R_q (including elements in $\mathbb{Z}$ or $\mathbb{Z}_q$), and bold lower-case letters (e.g., $\mathbf{a}, \mathbf{b}$) denote vectors of R or $\mathbb{Z}$ elements. We define an algorithm $\text{Ext}(a, k)$, which takes as input a ring element $a = (a_0, \dots, a_{n-1}) \in R$ and integer $k|n$, and outputs $K = (K_0, \dots, K_{n/k-1}) \in \{0, 1\}^{n/k}$, where $K_i = \sum_{j=0}^{k-1} a_{jn/k+i} \bmod 2$. For a positive integer $\ell \in \mathbb{Z}$, by $[\ell]$ we denote the set $\{0, \dots, \ell - 1\}$. By $r' = r \bmod^\pm q$ we denote the unique element in the range $[-\frac{q-1}{2}, \frac{q-1}{2}]$ such that $r' = r \bmod q$. For an element $w \in \mathbb{Z}_q$, we write $\|w\|_\infty$ to mean $|w \bmod^\pm q|$. The ℓ_∞ and ℓ_2 norms of a ring element $w \in R_q$ is defined as that of its coefficient vector $\mathbf{w} \in \mathbb{Z}_q^n$.

By $x \xleftarrow{\$} \mathcal{D}$ we denote sampling x according to a distribution $\mathcal{D}$ and by $x \xleftarrow{\$} S$ we denote sampling x from a finite set S uniformly. When we write that sampling a polynomial $g \xleftarrow{\$} \mathcal{D}$ from a distribution $\mathcal{D}$ over $\mathbb{Z}$, we mean that sampling each coefficient of g from $\mathcal{D}$ individually. We use $\log_b$ to denote the logarithm function in base b (e.g., 2 or natural constant e) and $\log$ to represent $\log_e$. We say that a function $f : \mathbb{N} \rightarrow [0, 1]$ is *negligible*, if for every positive c and all sufficiently large κ it holds that $f(\kappa) < 1/\kappa^c$. We denote by $\text{negl} : \mathbb{N} \rightarrow [0, 1]$ an (unspecified) negligible function.

2.2 Distributions

Binomial Distribution. The centered binomial distribution B_η with some positive $\eta \in \mathbb{Z}$ is defined as follows:

$$B_\eta = \left\{ \sum_{i=0}^{\eta-1} (a_i - b_i) : (a_0, \dots, a_{\eta-1}, b_0, \dots, b_{\eta-1}) \xleftarrow{\$} \{0, 1\}^{2\eta} \right\}$$

Ternary Distribution. The ternary distribution $\mathcal{T}_\sigma$ with some positive real $\sigma \in (0, 1/2)$ denotes the distribution of sampling a variable $x \in \{-1, 0, 1\}$ with $\Pr[x = 1] = \Pr[x = -1] = \sigma$, and $\Pr[x = 0] = 1 - 2\sigma$. By this notation, we have $\mathcal{T}_{1/3} = \mathcal{U}(\{-1, 0, 1\})$ is the uniform ternary distribution, and $\mathcal{T}_{1/4} = B_1$ is the centered binomial distribution with $\eta = 1$.

Gaussian Distribution. The Gaussian function $\rho_{s,\mathbf{c}}(\mathbf{x})$ over $\mathbb{R}^m$ centered at $\mathbf{c} \in \mathbb{R}^m$ with parameter $s > 0$ is defined as $\rho_{s,\mathbf{c}}(\mathbf{x}) = \exp(-\pi\|\mathbf{x} - \mathbf{c}\|^2/s^2)$. For lattice $\Lambda \subseteq \mathbb{R}^m$, let $\rho_{s,\mathbf{c}}(\Lambda) = \sum_{\mathbf{x} \in \Lambda} \rho_{s,\mathbf{c}}(\mathbf{x})$, and define the discrete Gaussian distribution over Λ as $D_{\Lambda,s,\mathbf{c}}(\mathbf{y}) = \frac{\rho_{s,\mathbf{c}}(\mathbf{y})}{\rho_{s,\mathbf{c}}(\Lambda)}$, where $\mathbf{y} \in \Lambda$. We omit the subscript $\mathbf{c}$ in the above notations if $\mathbf{c} = \mathbf{0}$.

Lemma 1 ([10,46]). *For any real $s, t > 0$, $c \geq 1$, $C = c \cdot \exp(\frac{1-c^2}{2}) < 1$, integer $m > 0$, and any $\mathbf{y} \in \mathbb{R}^m$ we have that $\Pr_{\mathbf{x} \leftarrow D_{\mathbb{Z}^m,s}}[\|\mathbf{x}\|_\infty > t \cdot s] \leq 2e^{-\pi t^2}$.*

Lemma 2 (Special case of [60, Theorem 3.1]). *Let $\mathbf{V} \in \mathbb{Z}^{n \times n}$ be the multiplication matrix corresponding to the ring element $v^{-1} = 1 - x^{n/k} \in R_q$. Let $\alpha, \beta, \gamma > 0$ be reals such that $\alpha \geq \omega(\sqrt{\log n})$, $\gamma = \sqrt{\alpha^2 + 2\beta^2}$ and $\alpha\beta/\gamma > \omega(\sqrt{\log n})$. Consider the following probabilistic experiment:*

Choose $\mathbf{x}_2 \xleftarrow{\$} D_{\mathbb{Z}^n,\beta}$, then choose $\mathbf{x}_1 \xleftarrow{\$} \mathbf{V}\mathbf{x}_2 + D_{\mathbb{Z}^n,\alpha}$.

Then, the marginal distribution of $\mathbf{x}_1$ is statistically close to $D_{\mathbb{Z}^n,\gamma}$.

2.3 Definition of Public-Key Encryption

A Public-Key Encryption (PKE) scheme $\Pi_{\text{PKE}} = (\text{KeyGen}, \text{Enc}, \text{Dec})$ with plaintext space $\mathcal{M}$ consists of the following three polynomial-time algorithms:

- **KeyGen**(1^κ): given a security parameter κ as input, output a pair of public and secret keys (pk, sk) , denoted as $(pk, sk) \leftarrow \text{KeyGen}(1^\kappa)$.
- **Enc**($pk, M; r$): given the public key pk , a plaintext $M \in \mathcal{M}$ and a randomness r (which might be an empty string) as inputs, output a ciphertext C , denoted as $C \leftarrow \text{Enc}(pk, M; r)$ or $C \leftarrow \text{Enc}(pk, M)$ in brief.
- **Dec**(sk, C): given the secret key sk and a ciphertext C as inputs, output a plaintext M' (which might be $\perp$), denoted as $M' \leftarrow \text{Dec}(sk, C)$.

Definition 1 (Encry-collision probability [38]). *For PKE scheme Π_{PKE} , we define the encry-collision probability as*

$$\gamma(\text{Enc}) := \Pr \left[\text{Enc}(pk, M; r) = \text{Enc}(pk, M'; r') : (pk, sk) \leftarrow \text{KeyGen}(1^\kappa), \right. \\ \left. M, M' \xleftarrow{\$} \mathcal{M}, r, r' \xleftarrow{\$} \mathcal{R} \right].$$

On the notion of correctness, we recall the definition in [25], which focuses on worst-case correctness across all possible message choices. This definition is key to defending against attacks that exploit decryption failures.

Definition 2 (δ -correct [25]). We say that a PKE scheme Π_{PKE} with message space $\mathcal{M}$ and randomness space $\mathcal{R}$ is δ -correct or has decryption failure rate δ if

$$\mathbb{E} \left[\max_{m \in \mathcal{M}} \Pr [\text{Dec}(sk, c) \neq m : c = \text{Enc}(pk, m; r), r \leftarrow_{\$} \mathcal{R}] \right] \leq \delta.$$

where the expectation is taken over $(pk, sk) \leftarrow \text{KeyGen}(1^\kappa)$.

Definition 3 (γ -spread [25]). We say that a PKE scheme Π_{PKE} with message space $\mathcal{M}$, ciphertext space $\mathcal{C}$, and randomness space $\mathcal{R}$ is (weakly) γ -spread if

$$\mathbb{E} \left[\max_{m \in \mathcal{M}, c \in \mathcal{C}} \Pr[c = \text{Enc}(pk, m; r) : r \leftarrow_{\$} \mathcal{R}] \right] \leq 2^{-\gamma},$$

where the expectation is taken over $(pk, sk) \leftarrow \text{KeyGen}(1^\kappa)$.

We recall the standard definition of One-Wayness under Chosen-Plaintext Attack (OW-CPA) for PKE schemes, which is modeled by an experiment between a challenger and an adversary $\mathcal{A}$ shown in Fig. 2.

OW-CPA Experiment $\text{Exp}_{\Pi_{\text{PKE}}, \mathcal{A}}^{\text{ow-cpa}}(\kappa)$ for PKE:
1 $(pk, sk) \leftarrow \Pi_{\text{PKE}}.\text{KeyGen}(1^\kappa)$; 2 $M^* \xleftarrow{\\$} \mathcal{M}$; 3 $C^* \leftarrow \Pi_{\text{PKE}}.\text{Enc}(pk, M^*)$; 4 $M' \leftarrow \mathcal{A}(pk, C^*)$; 5 If $M' = M^*$, then return 1, else return 0.

Fig. 2: Experiment of OW-CPA security for PKE

Definition 4 (OW-CPA PKE). We say that a PKE scheme Π_{PKE} is OW-CPA secure if for any PPT adversary $\mathcal{A}$, its advantage

$$\text{Adv}_{\Pi_{\text{PKE}}, \mathcal{A}}^{\text{ow-cpa}}(\kappa) = \Pr [\text{Exp}_{\Pi_{\text{PKE}}, \mathcal{A}}^{\text{ow-cpa}}(\kappa) = 1]$$

is negligible in security parameter κ , where $\text{Exp}_{\Pi_{\text{PKE}}, \mathcal{A}}^{\text{ow-cpa}}(\kappa)$ is depicted in Fig. 2.

2.4 Definition of Key Encapsulation Mechanism

A Key Encapsulation Mechanism (KEM) scheme $\Pi_{\text{KEM}} = (\text{KeyGen}, \text{Encap}, \text{Decap})$ with session key space $\mathcal{K}$ consists of the following three polynomial-time algorithms:

- **KeyGen**(1^κ): given a security parameter κ as input, output a pair of public and secret keys (pk, sk) , denoted as $(pk, sk) \leftarrow \text{KeyGen}(1^\kappa)$.
- **Encap**($pk; r$): given the public key pk , and a randomness r as inputs, output a ciphertext C and a session key $K \in \mathcal{K}$, denoted as $(C, K) \leftarrow \text{Encap}(pk; r)$, or $(C, K) \leftarrow \text{Encap}(pk)$ in brief.

- **Decap**(sk, C): given a secret key sk and a ciphertext C as inputs, output a key K' (which might be a failure symbol $\perp$), denoted as $K' \leftarrow \text{Decap}(sk, C)$.

Definition 5 (Encap/key-collision probability [38,59]). For KEM scheme Π_{KEM} , we define the encap-collision probability as

$$\gamma(\text{Encap}) := \Pr \left[\begin{array}{l} C_1 = C_2 : (pk, sk) \leftarrow \text{KeyGen}(1^\kappa), r_1, r_2 \xleftarrow{\$} \mathcal{R}, \\ (C_1, K_1) \leftarrow \text{Encap}(pk; r_1), (C_2, K_2) \leftarrow \text{Encap}(pk; r_2) \end{array} \right].$$

Here $\mathcal{R}$ denotes the randomness space used by **Encap**. We also define the key-collision probability of Π_{KEM} as

$$\gamma(\text{KeyGen}) := \Pr[pk = pk' : (pk, sk) \leftarrow \text{KeyGen}(1^\kappa), (pk', sk') \leftarrow \text{KeyGen}(1^\kappa)].$$

Definition 6 (δ -correct for KEM). We say that a KEM scheme Π_{KEM} with randomness space $\mathcal{R}$ is δ -correct or has decryption failure rate δ if

$$\mathbb{E} \left[\Pr \left[\text{Decap}(sk, C) \neq K : (C, K) = \text{Encap}(pk, r), r \xleftarrow{\$} \mathcal{R} \right] \right] \leq \delta.$$

where the expectation is taken over $(pk, sk) \leftarrow \text{KeyGen}(1^\kappa)$.

Below, we recall the standard definition of OW-CPA security for KEM schemes, which is modeled by an experiment between a challenger and an adversary $\mathcal{A}$ shown in Fig. 3.

OW-CPA Experiment $\text{Exp}_{\Pi_{\text{KEM}}, \mathcal{A}}^{\text{ow-cpa}}(\kappa)$ for KEM:
1 $(pk, sk) \leftarrow \Pi_{\text{KEM}}.\text{KeyGen}(1^\kappa);$ 2 $(C^*, K^*) \leftarrow \Pi_{\text{KEM}}.\text{Encap}(pk);$ 3 $K' \leftarrow \mathcal{A}(pk, C^*);$ 4 If $K' = K^*$, then return 1, else return 0.

Fig. 3: Experiment of OW-CPA security for KEM

Definition 7 (OW-CPA KEM). We say that a KEM scheme Π_{KEM} is OW-CPA secure if for any PPT adversary $\mathcal{A}$, its advantage

$$\text{Adv}_{\Pi_{\text{KEM}}, \mathcal{A}}^{\text{ow-cpa}}(\kappa) = \Pr [\text{Exp}_{\Pi_{\text{KEM}}, \mathcal{A}}^{\text{ow-cpa}}(\kappa) = 1]$$

is negligible in security parameter κ , where $\text{Exp}_{\Pi_{\text{KEM}}, \mathcal{A}}^{\text{ow-cpa}}(\kappa)$ is defined in Fig. 3.

2.5 Two-Round Authenticated Key Exchange

A two-round Authenticated Key Exchange (AKE) protocol $\Pi_{\text{AKE}} = (\text{AKE}.\text{KeyGen}, \text{AKE}.\text{Init}, \text{AKE}.\text{DER}_{\text{resp}}, \text{AKE}.\text{DER}_{\text{init}})$ is executed in an interactive manner between an initiator P_i and a responder P_j , and consists of the following four polynomial-time algorithms:

- $\text{AKE.KeyGen}(1^\kappa)$: On input a security parameter κ , the key-generation algorithm outputs a public-private key pair (pk, sk) , which is denoted by $(pk, sk) \leftarrow \text{AKE.KeyGen}(1^\kappa)$. In the key-generation phase, P_i is able to run $(pk_i, sk_i) \leftarrow \text{AKE.KeyGen}(1^\kappa)$, and P_j performs $(pk_j, sk_j) \leftarrow \text{AKE.KeyGen}(1^\kappa)$.
- $\text{AKE.Init}(sk_i, pk_j)$: On input a secret key sk_i and a public key pk_j , the initialisation algorithm outputs a message M and a state st , which is denoted by $(M, st) \leftarrow \text{AKE.Init}(sk_i, pk_j)$.
- $\text{AKE.DER}_{\text{resp}}(sk_j, pk_i, M)$: On input a secret key sk_j , a public key pk_i , and a message M , the responder's derivation algorithm outputs a message M' and a session key K' , which is denoted as $(M', K') \leftarrow \text{AKE.DER}_{\text{resp}}(sk_j, pk_i, M)$.
- $\text{AKE.DER}_{\text{init}}(sk_i, st, M')$: On input a secret key sk_i , the secret state st and message M' , the initiator's derivation algorithm outputs a session key K , which is denoted by $K \leftarrow \text{AKE.DER}_{\text{init}}(sk_i, st, M')$.

To run a two-round AKE protocol, the AKE.Init , $\text{AKE.DER}_{\text{resp}}$ and $\text{AKE.DER}_{\text{init}}$ algorithms are executed interactively between two parties P_i and P_j . Specifically, the two parties call the algorithms as follows:

1. The initiator P_i runs $(M, st) \leftarrow \text{AKE.Init}(sk_i, pk_j)$, and then sends M to P_j .
2. The responder P_j runs $(M', K') \leftarrow \text{AKE.DER}_{\text{resp}}(sk_j, pk_i, M)$, and then sends M' to P_i .
3. P_i computes $K \leftarrow \text{AKE.DER}_{\text{init}}(sk_i, st, M')$.

Definition 8 (Correctness). *We say that an AKE protocol Π_{AKE} run between two parties P_i and P_j is δ -correct (or has correctness error δ), if for all $\kappa \in \mathbb{N}$, $(pk_i, sk_i), (pk_j, sk_j) \leftarrow \text{AKE.KeyGen}(1^\kappa)$, the following holds:*

$$\Pr [K \neq K' : (M, st) \leftarrow \text{AKE.Init}(sk_i, pk_j), (M', K') \leftarrow \text{AKE.DER}_{\text{resp}}(sk_j, pk_i, M), K \leftarrow \text{AKE.DER}_{\text{init}}(sk_i, st, M')] \leq \delta(\kappa).$$

2.6 CK^+ Security Model for AKE

We adopt the CK^+ model [45,29] to prove the security of an AKE protocol. This model captures representative advanced attacks such as Key Compromise Impersonation (KCI), the attacks against weak Perfect Forward Secrecy (wPFS), and Maximal EXposure attacks (MEX).

We consider N parties $P_1, \dots, P_N$, where each party P_i holds a (long-term) public-secret key pair (pk_i, sk_i) , and these parties will involve at most S sessions. The sessions run the protocol with access to the party's long-term key material, and also have their own local variables. The local variables of each session, indexed by the integer $\text{sID} \in \{1, \dots, S\}$, are shown below.

- $\text{holder}[\text{sID}]$: the party running the session sID .
- $\text{peer}[\text{sID}]$: the party the session is communicating with.
- $\text{sent}[\text{sID}]$: the message sent by the session sID .
- $\text{rece}[\text{sID}]$: the message received by the session sID .
- $\text{role}[\text{sID}] \in \{\text{initiator}, \text{responder}\}$: a string holds the information whether the session key was derived by $\text{AKE.DER}_{\text{init}}$ or $\text{AKE.DER}_{\text{resp}}$. We say sID is an initiator (resp. responder) session if $\text{role}[\text{sID}]$ is initiator (resp. responder).

- $\text{st}[\text{sID}]$: a string holds (secret) internal state values and intermediary results required by the session. If $\text{role}[\text{sID}] = \text{initiator}$, then $\text{st}[\text{sID}]$ is output by $\text{AKE.DER}_{\text{init}}$, otherwise, $\text{st}[\text{sID}] = \perp$.
- $\text{sKey}[\text{sID}]$: the session key of sID .

Definition 9 (Completed/Matching Sessions). We say that a session sID is completed if and only if $\text{sKey}[\text{sID}] \neq \perp$. We say two sessions sID and sID' are matching if and only if the following requirements hold:

1. $(\text{holder}[\text{sID}], \text{peer}[\text{sID}]) = (\text{peer}[\text{sID}'], \text{holder}[\text{sID}'])$.
2. $(\text{sent}[\text{sID}], \text{rece}[\text{sID}]) = (\text{rece}[\text{sID}'], \text{sent}[\text{sID}'])$.
3. $\text{role}[\text{sID}] \neq \text{role}[\text{sID}']$.

Let $\mathfrak{M}(\text{sID})$ denote the set of session identities which match sID .

Definition 10 (Oracles in the CK^+ Model). In the CK^+ model, the adversary $\mathcal{A}$ is given black-box access to the oracles INIT , DER_{init} , and DER_{resp} , which runs the AKE algorithms AKE.Init , $\text{AKE.DER}_{\text{init}}$, and $\text{AKE.DER}_{\text{resp}}$, respectively. To model the adversary's control of the network, $\mathcal{A}$ is allowed to establish new sessions via EST . Moreover, $\mathcal{A}$ is given black-box access to the oracles REVEAL , REV-STATE , and CORRUPT to reveal certain secret process data. The test (or challenge) oracle queried by $\mathcal{A}$ is denoted as Test , which can be queried only once and returns the session key for $\mathcal{A}$ to distinguish. In the following, for each oracle, we explain its corresponding function.

- $\text{EST}(i, j)$: The query means that $\mathcal{A}$ wants to establish a new session sID for holder i and its peer j . Upon such a query, oracle EST assigns a new session identity $\text{sID} := \text{cnt}$ and sets $\text{holder}[\text{sID}] := i$ and $\text{peer}[\text{sID}] := j$ for $\mathcal{A}$.
- $\text{INIT}(\text{sID})$: The query means that $\mathcal{A}$ asks the oracle to initiate session sID . Then the oracle generates the first round message $(M, \text{st}) \leftarrow \text{AKE.Init}(sk_i, pk_j)$ and replies M to $\mathcal{A}$. Here sk_i is the secret key of $\text{holder}[\text{sID}]$ and pk_j is the public key of $\text{peer}[\text{sID}]$.
- $\text{DER}_{\text{resp}}(\text{sID}, M)$: This query means that $\mathcal{A}$ asks session sID to respond the first-round message M (so $\text{role}[\text{sID}] = \text{responder}$). The oracle invokes $(M', K_j) \leftarrow \text{AKE.DER}_{\text{resp}}(sk_j, pk_i, M)$ and return M' as the second round message to $\mathcal{A}$.
- $\text{DER}_{\text{init}}(\text{sID}, M')$: This query means that $\mathcal{A}$ asks sID to respond the second-round message M' (so $\text{role}[\text{sID}] = \text{initiator}$). The oracle invokes

$$K_i \leftarrow \text{AKE.DER}_{\text{init}}(sk_i, pk_j, M', \text{st}[\text{sID}])$$

to generate the session key $\text{sKey}[\text{sID}] := K_i$. Here sk_i is the secret key of $\text{holder}[\text{sID}]$ and pk_j is the public key of $\text{peer}[\text{sID}]$.

- $\text{REVEAL}(\text{sID})$: It means that $\mathcal{A}$ reveals the session key of session sID . The oracle will return $\text{sKey}[\text{sID}]$ to $\mathcal{A}$.
- $\text{REV-STATE}(\text{sID})$: It means that $\mathcal{A}$ reveals the internal state of session sID . The oracle will return $\text{st}[\text{sID}]$ to $\mathcal{A}$.
- $\text{CORRUPT}(i)$: It means that $\mathcal{A}$ reveals the long-term key of party P_i . The oracle will return sk_i to $\mathcal{A}$.

- **TEST(sID)**: It means that $\mathcal{A}$ chooses sID as the test session and the session key of sID for challenge. The oracle will set $K_0^* := \text{sKey}[\text{sID}]$, sample $K_1^* \leftarrow \mathcal{K}$ and $b \leftarrow \{0, 1\}$, and return K_b^* to $\mathcal{A}$. Note that $\mathcal{A}$ can query **TEST** only once.

By querying **CORRUPT**, $\mathcal{A}$ can corrupt both the test session's holder and peer, which models the possibility of performing KCI attacks and breaking wPFS. In addition, by querying **CORRUPT** and **REV-STATE** with a session, $\mathcal{A}$ can obtain its the internal state, the long-term keys of the holder, the long-term keys of the peer, which models the possibility of performing MEX attacks.

Next, we define a particular property of the sessions in the CK^+ model, which is called freshness. This property is used to rule out certain meaningless attacks launched by the adversary. That is, we only evaluate whether the adversary succeeds when the test session is fresh.

Definition 11 (Freshness). Let sID be a completed session between users P_i and P_j , then we say session sID is fresh if none of the following conditions hold:

1. The $\text{sKey}[\text{sID}]$ is revealed to $\mathcal{A}$, or the $\text{sKey}[\text{sID}']$ for a $\text{sID}' \in \mathfrak{M}(\text{sID})$ is revealed to $\mathcal{A}$.
2. The long-term secret key sk_i of $\text{holder}[\text{sID}]$ and the internal state $\text{st}[\text{sID}]$ are both revealed to $\mathcal{A}$.
3. There exists a $\text{sID}' \in \mathfrak{M}(\text{sID})$ such that both the long-term key sk_j of $\text{peer}[\text{sID}] = \text{holder}[\text{sID}']$ and the internal state $\text{st}[\text{sID}']$ of session sID' are revealed to $\mathcal{A}$.
4. $\mathfrak{M}(\text{sID}) = \emptyset$ and the long-term secret key sk_j of $\text{peer}[\text{sID}]$ is revealed to $\mathcal{A}$.
5. $\mathfrak{M}(\text{sID}) = \emptyset$ and the internal state $\text{st}[\text{sID}]$ is revealed to $\mathcal{A}$. In particular, if the internal state $\text{st}[\text{sID}]$ contains an ephemeral secret key $\tilde{sk}$, this condition is replaced by $\mathfrak{M}(\text{sID}) = \emptyset$ and the state $\text{st}[\text{sID}]/\tilde{sk}$ is revealed to $\mathcal{A}$.

The last condition has a branch depending on whether $\text{st}[\text{sID}]$ contains an ephemeral secret key. This is because the CK^+ model allows a particular partial reveal for the test session. Specifically, if the internal state (of the test session) carries an ephemeral secret key, then revealing this key to $\mathcal{A}$ will not trigger a meaningless attack, which corresponds to the Case 2.(b) of [29, Definition 2].

Remark 2. In fact, this kind of partial reveal seems to have little practical relevance and renders the model description somewhat cumbersome. For this reason, the reformulated CK^+ model given in [38] (i.e., their IND-StAA security) directly removes this partial reveal. In this document, since the CK^+ model is widely adopted for the security proofs of AKE protocols, we choose to retain such partial reveal for the sake of consistency.

Definition 12 (Key Indistinguishability of AKE in the CK^+ Model). We say that an AKE protocol Π_{AKE} with negligible correctness error has the key indistinguishability in the CK^+ model, if for any probabilistic polynomial-time adversary $\mathcal{A}$, its advantage

$$\text{Adv}_{\Pi_{\text{AKE}}}^{\text{KI-CK}^+}(\mathcal{A}) = \left| \Pr \left[1 \leftarrow \text{KeyIND}_{\Pi_{\text{AKE}}}^{\text{A},0} \right] - \Pr \left[1 \leftarrow \text{KeyIND}_{\Pi_{\text{AKE}}}^{\text{A},1} \right] \right|$$

is negligible in the security parameter κ . Here game $\text{KeyIND}_{\Pi_{\text{AKE}}}^{\text{A},b}$ ($b \in \{0, 1\}$) with respect to the adversary $\mathcal{A}$ is defined in Fig. 4.

$\text{KeyIND}_{\Pi_{\text{AKE}}}^{A,b} \quad (b \in \{0, 1\})$	$\text{DER}_{\text{init}}(\text{sID}, M)$
$\text{cnt} := 0 \quad // \text{session counter}$ $\text{sID}^* := 0 \quad // \text{test session's id}$ for $i \in \{1, \dots, N\}$ $(pk_i, sk_i) \leftarrow \text{AKE.KGen}$ $\text{crp}[i] := \text{false} // \text{corruption variables}$ end for $b' \leftarrow \mathcal{A}^{\mathcal{O}_{\text{AKE}}}(pk_1, \dots, pk_N)$ if sID^* is not fresh $\text{return } 0$ return b'	if $\text{holder}[\text{sID}] = \perp \vee \text{st}[\text{sID}] = \perp$, return $\perp$ if $\text{sKey}[\text{sID}] \neq \perp$ $\text{return } \perp \quad // \text{no re-use}$ $\text{role}[\text{sID}] := \text{initiator}$ $(i, j) := (\text{holder}[\text{sID}], \text{peer}[\text{sID}])$ $\text{sKey}[\text{sID}] := \text{AKE.Der}_{\text{init}}(sk_i, pk_j, M, \text{st}[\text{sID}])$ $\text{rece}[\text{sID}] := M$ return $\emptyset$
$\text{EST}((i, j) \in \{1, \dots, N\} \times \{1, \dots, N\})$	$\text{REVEAL}(\text{sID})$
$\text{cnt} := \text{cnt} + 1, \text{sID} := \text{cnt}$ $(i, j) := (\text{holder}[\text{sID}], \text{peer}[\text{sID}])$ $\text{stRev}[\text{sID}] := \text{false} \quad // \text{state reveal variables}$ $\text{keyRev}[\text{sID}] := \text{false} \quad // \text{session key reveal variables}$ return sID	if $\text{sKey}[\text{sID}] = \perp$, return $\perp$ $\text{keyRev}[\text{sID}] := \text{true}$ return $\text{sKey}[\text{sID}]$
$\text{REVEAL-STATE}(\text{sID})$	$\text{CORRUPT}(i \in \{1, \dots, N\})$
if $\text{st}[\text{sID}] = \perp$, return $\perp$ $\text{stRev}[\text{sID}] := \text{true}$ return $\text{st}[\text{sID}]$	$\text{crp}[i] := \text{true}$ return sk_i
$\text{DER}_{\text{resp}}(\text{sID}, M)$	$\text{INIT}(\text{sID})$
if $\text{holder}[\text{sID}] = \perp$, return $\perp$ if $\text{sent}[\text{sID}] \neq \perp$, return $\perp \quad // \text{no re-use}$ if $\text{role}[\text{sID}] = \text{initiator}$, return $\perp$ $\text{role}[\text{sID}] := \text{responder}$ $(j, i) := (\text{holder}[\text{sID}], \text{peer}[\text{sID}])$ $(M', K') := \text{AKE.Der}_{\text{resp}}(sk_j, pk_i, M)$ $\text{sKey}[\text{sID}] := K'$ $(\text{rece}[\text{sID}], \text{sent}[\text{sID}]) := (M, M')$ return M'	if $\text{holder}[\text{sID}] = \perp$ $\text{return } \perp \quad // \text{session not-established}$ if $\text{sent}[\text{sID}] \neq \perp$, return $\perp \quad // \text{no re-use}$ $\text{role}[\text{sID}] := \text{initiator}$ $(i, j) := (\text{holder}[\text{sID}], \text{peer}[\text{sID}])$ $(M, \text{st}) := \text{AKE.Init}(sk_i, pk_j)$ $(\text{sent}[\text{sID}], \text{st}[\text{sID}]) := (M, \text{st})$ return M
	$\text{TEST}(\text{sID}) \quad // \text{only one query}$
	$\text{sID}^* := \text{sID}$ if $\text{sKey}[\text{sID}^*] = \perp$, return $\perp$ $K_0^* := \text{sKey}[\text{sID}^*], K_1^* \leftarrow \mathcal{K}$ return K_b^*

Fig. 4: The security games $\text{KeyIND}_{\Pi_{\text{AKE}}}^{A,b}$ ($b \in \{0, 1\}$). The $\mathcal{O}_{\text{AKE}}$ is a set of oracles defined as $\{\text{EST}, \text{INIT}, \text{DER}_{\text{resp}}, \text{DER}_{\text{init}}, \text{REVEAL}, \text{REV-STATE}, \text{CORRUPT}, \text{TEST}\}$.

2.7 Hardness Assumptions

Let n be a power of 2, and q a prime. Let $R_q = \mathbb{Z}_q[x]/(x^n + 1)$. Let R_q^* denote all invertible ring elements in R_q . Let $\chi_f, \chi_g, \chi_r, \chi_e$ be four probability distributions over R . Let $v \in R_q^*$ be a publicly known small ring element.

The NTRU Assumption. The computational NTRU problem $\text{NTRU}_{n,q,\chi_f,\chi_g,v}$ asks an algorithm, given $h = g/f \in R_q$ as input, to output f' , where $f' \xleftarrow{\$} \chi_f, g \xleftarrow{\$} \chi_g$ and $f = vf' + 1 \in R_q^*$. The decisional NTRU problem $\text{DNTRU}_{n,q,\chi_f,\chi_g,v}$ asks an algorithm to distinguish the following two distributions:

$$\{h = g/f \mid f' \xleftarrow{\$} \chi_f, g \xleftarrow{\$} \chi_g, \text{ and } f = vf' + 1, g \in R_q^*\} \text{ and } \{u \mid u \xleftarrow{\$} R_q^*\}.$$

The computational (resp., decisional) NTRU assumption says that it is hard for any PPT algorithms to solve $\text{NTRU}_{n,q,\chi_f,\chi_g,v}$ (resp., $\text{DNTRU}_{n,q,\chi_f,\chi_g,v}$) with non-negligible advantage over a random guess.

Remark 3. The above definition generalizes the common NTRU assumption with $v = p \in R_q^*$ for some integer p (e.g., $p = 3$ in [36,37,65,17]) with a publicly known ring element $v \in R_q^*$. We note that this generalization is mild up to the choices of the secret key distribution χ_f , because $\text{NTRU}_{n,q,\chi_f,\chi_g,v}$ is essentially equivalent to the standard NTRU problem $\text{NTRU}_{n,q,\chi'_f,\chi_g,p}$ with $\chi'_f = p^{-1}v\chi_f$ (or $\chi_f = pv^{-1}\chi'_f$). In fact, the polynomial $v = x + 2$ was recommended by the authors of the original NTRU cryptosystem as early as 2000 [37], and was investigated in [9,53,39,32,35,65].

Since its introduction [36], the NTRU problem has been studied more than 25 years, and there is no significant algorithmic progress. The decisional NTRU (DNTRU) assumption over the cyclotomic ring $R = \mathbb{Z}_q[x]/(x^n + 1)$, which is also known as the decisional small polynomial ratio (DSPR) assumption, has been extensively used and investigated in [65,47,23,27,13,61]. Notably, Stehlé and Steinfeld [65] showed that the DNTRU assumption indeed holds unconditionally if χ_f, χ_g are discrete Gaussian distributions of standard deviation $\sigma = \omega(n\sqrt{q})$ (We note that their proof mainly focuses on the special case $v = 3$, but it essentially applies to any invertible $v \in R_q^*$). For small secret distributions, a variant of the NTRU problem over $R_q = \mathbb{Z}_q[x]/(x^n + 1)$ is also shown to be at least as hard as the worst-case approximate SVP problem on ideal lattices [61].

The RLWE Assumption. The computational RLWE problem $\text{RLWE}_{n,q,\chi_r,\chi_e}$ asks an algorithm, given a polynomial number of samples from the distribution $\{(a, b = ar + e) \mid a \xleftarrow{\$} R_q, e \xleftarrow{\$} \chi_e\}$ as inputs, to output the secret $r \in R_q$, where $r \xleftarrow{\$} \chi_r$. The decisional RLWE problem $\text{DRLWE}_{n,q,\chi_r,\chi_e}$ asks an algorithm, given a polynomial number of samples to distinguish the following two distributions:

$$\{(a, b = ar + e) \mid a \xleftarrow{\$} R_q, e \xleftarrow{\$} \chi_e\} \text{ and } \{(a, u) \mid a \xleftarrow{\$} R_q, u \xleftarrow{\$} R_q\}.$$

The computational (resp., decisional) RLWE assumption says that it is hard for any PPT algorithms to solve $\text{RLWE}_{n,q,\chi_r,\chi_e}$ (resp., $\text{DRLWE}_{n,q,\chi_r,\chi_e}$) with non-negligible advantage over a random guess.

As an extension of the LWE problem [63], the RLWE problem was first considered in [66,49], and was provably as hard as some hard lattice problems such as the Shortest Vectors Problem (SVP) on ideal lattices.

The Subset-Sum Parity RLWE Assumption. We introduce a variant of the RLWE problem which we call subset-sum parity RLWE (sspRLWE) problem. Formally, the sspRLWE problem $\text{sspRLWE}_{n,q,\chi_r,\chi_e,v}$ asks an algorithm, given an RLWE instance $(a, b = ar + e) \in R_q$ as input, to output $ve \bmod 2 \in R_2$ for some fixed ring element $v \in R_2$. This name comes from the fact that for $R = \mathbb{Z}[X]/(x^n+1)$, the i -th coefficient of $ve \bmod 2 \in R_2$ is essentially equal to the parity of the subset sum $\sum_{v_j=1} e_{(i-j) \bmod n}$ of the coefficient vector $\mathbf{e} = (e_0, \dots, e_{n-1})$ of $e \in R_q$. The sspRLWE assumption says that it is hard for any PPT algorithms to solve $\text{sspRLWE}_{n,q,\chi_r,\chi_e,v}$ with non-negligible advantage over a random guess according to the distribution $\chi' = v\chi_e \bmod 2$.

Remark 4. Our sspRLWE problem can be seen as a generalization of the RLWE problem [25] from a special choice of $v = 1$ to a general chosen $v \in R_2$. On the first hand, the $\text{sspRLWE}_{n,q,\chi_r,\chi_e,v}$ problem is not harder than the corresponding RLWE problem $\text{RLWE}_{n,q,\chi_r,\chi_e}$. On the other hand, if the DRLWE problem $\text{DRLWE}_{n,q,\chi_r,\chi_e}$ is hard, it seems that a RLWE sample $(a, b = ar + e)$ essentially hides all the information about e , and that the best way for a PPT algorithm to solve the sspRLWE problem is to make a random guess on $ve \bmod 2$ according to the distribution $\chi' = v\chi_e \bmod 2$. Moreover, the problem of reducing $\text{DRLWE}_{n,q,\chi_r,\chi_e}$ to $\text{sspRLWE}_{n,q,\chi_r,\chi_e,v}$ can be seen as the problem of solving $\text{DRLWE}_{n,q,\chi_r,\chi_e}$ with modular hints $ve \bmod 2$, and an efficient algorithm to solve $\text{sspRLWE}_{n,q,\chi_r,\chi_e,v}$ may directly lead to a more efficient algorithm to solve $\text{RLWE}_{n,q,\chi_r,\chi_e}$ according to the study in [20,51,14].

For our purpose, we are particularly interested in the problem $\text{sspRLWE}_{n,q,\chi_r,\chi_e,\bar{v}}$ satisfying the following two conditions:

- $\bar{v} = 1 + x^{n/k} + x^{2n/k} + \dots + x^{(k-1)n/k} \in R_2$ for some $k|n$;
- χ_e is the center binomial distribution.

Below, we show that for discrete Gaussian distributions, the $\text{sspRLWE}_{n,q,\chi_r,D_{\mathbb{Z}^n,\gamma},\bar{v}}$ problem is at least as hard as the decisional RLWE problem $\text{DRLWE}_{n,q,\chi_r,D_{\mathbb{Z}^n,\beta}}$ for sufficiently large parameters $\gamma > \beta$, which can be extended to binomial distributions by a standard argument using Rényi divergence [8].

Theorem 1. *Let n, q, k, χ_r and $\bar{v}$ be defined as above. Let α, β, γ be three positive reals satisfying $\alpha \geq \omega(\sqrt{\log n})$, $\gamma = \sqrt{\alpha^2 + 2\beta^2}$, $\alpha\beta/\gamma \geq \omega(\sqrt{\log n})$ and $\gamma\sqrt{n} < q/2$. Let $D_{\mathbb{Z}^n,\beta}, D_{\mathbb{Z}^n,\gamma}$ be two discrete Gaussian distributions with parameter β and γ , respectively. If there is a PPT algorithm $\mathcal{A}$ solving the $\text{sspRLWE}_{n,q,\chi_r,D_{\mathbb{Z}^n,\gamma},\bar{v}}$ problem (with probability negligibly close to 1), then there is another PPT algorithm $\mathcal{B}$ solving the $\text{DRLWE}_{n,q,\chi_r,D_{\mathbb{Z}^n,\beta}}$ problem.*

Proof. It is sufficient to give the description of $\mathcal{B}$. Formally, let $v^{-1} = 1 - x^{n/k}$, and it is easy to check that $v^{-1}\bar{v} = 2 \in R_q = \mathbb{Z}_q[x]/(x^n + 1)$. Now, given a DRLWE tuple $(a, b) \in R_q \times R_q$ as input, $\mathcal{B}$ first randomly chooses a polynomial $e' \in R_q$

from the distribution $D_{\mathbb{Z}^n, \alpha}$, and sets $(a', b') = (v^{-1}a, v^{-1}b + e') \in R_q \times R_q$. Then, it runs algorithm $\mathcal{A}$ with input (a', b') , and obtains $w \in R_2$ from $\mathcal{A}$. Finally, $\mathcal{B}$ returns 1 if $w = \bar{v}e' \bmod 2$, otherwise returns 0.

We now analyze the behavior of algorithm $\mathcal{B}$. First, if $(a, b = ar + e)$ is a real $\text{DRLWE}_{n, q, \chi_r, D_{\mathbb{Z}^n, \beta}}$ tuple, then we have that the coefficients of e are chosen from $D_{\mathbb{Z}^n, \beta}$. By Lemma 2, we have that the distribution of $\hat{e} = v^{-1}e + e'$ is statistically close to $D_{\mathbb{Z}^n, \gamma}$. Since $\gamma\sqrt{n} < q/2$, we have that $\|\hat{e}\|_\infty < q/2$ with probability negligibly close to 1 by Lemma 1, which means that $\hat{e} \bmod q = \hat{e}$ holds with probability negligibly close to 1. Thus, the distribution of $(a' = v^{-1}a, b' = v^{-1}ar + \hat{e}) \in R_q \times R_q$ is statistically close to an $\text{sspRLWE}_{n, q, \chi_r, D_{\mathbb{Z}^n, \gamma}, \bar{v}}$ tuple. Using the fact that $v^{-1}\bar{v} = 2 \in R_q$ and $w = \bar{v}\hat{e} = \bar{v}e' \bmod 2$, we have that $\mathcal{B}$ will return 1 with probability negligibly close to 1. Second, if (a, b) is randomly chosen from $R_q \times R_q$, we have that $(a' = v^{-1}a, b' = v^{-1}b + e')$ is also randomly distributed over $R_q \times R_q$. This means that the probability for any $\mathcal{A}$ to output $w \in R_2$ such that $w = \bar{v}e' \bmod 2$ is negligible in n/k by our choice of $e' \xleftarrow{\$} D_{\mathbb{Z}^n, \alpha}$ with $\alpha \geq \omega(\sqrt{\log n})$. In all, we have shown that $\mathcal{B}$ is a valid distinguisher for $\text{DRLWE}_{n, q, \chi_r, D_{\mathbb{Z}^n, \beta}}$ problem. This completes the proof. $\square$

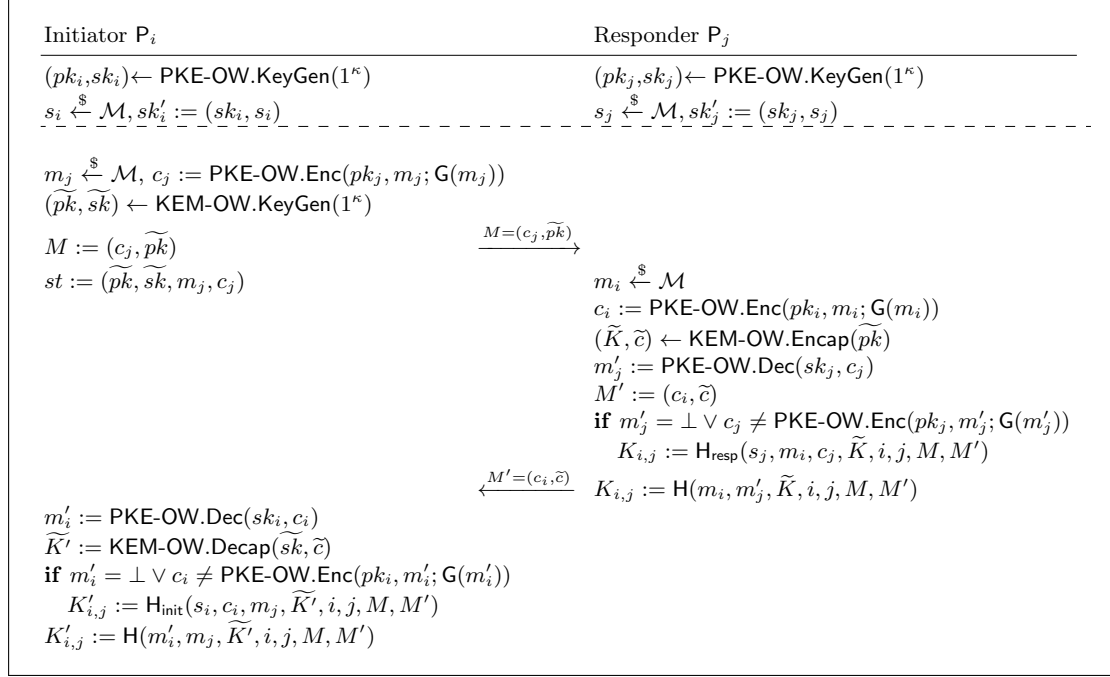
Remark 5. The above proof is essentially obtained by replacing the pair $(a', b') = (2a, 2b + e')$ in the original proof of [71] with $(a', b') = (v^{-1}a, v^{-1}b + e')$. This change allows to obtain a slightly tighter proof. Concretely, by letting $\beta = \alpha$ we have that $\gamma = \sqrt{\alpha^2 + 2\beta^2}$ is about $\sqrt{3}$ times larger than β , which gives a strong theoretical connection between the hardness of sspRLWE and DRLWE . In fact, as discussed in [26] for the concrete hardness of the RLWE2 problem, we also believe that based on the state of the art of how RLWE problems are solved, the constant factor $\sqrt{3}$ is an artificial outcome of the proof, and the $\text{sspRLWE}_{n, q, \chi_r, D_{\mathbb{Z}^n, \gamma}, \bar{v}}$ problem with $\gamma = \beta$ is already as hard as the $\text{DRLWE}_{n, q, \chi_r, D_{\mathbb{Z}^n, \beta}}$ problem. Intuitively, if the DRLWE problem $\text{DRLWE}_{n, q, \chi_r, \chi_e}$ is hard, then for any PPT sspRLWE solver $\mathcal{A}$, the pair $(a, b = ar + e) \in R_q \times R_q$ is already indistinguishable from a uniformly random tuple over $R_q \times R_q$, which indicates that $\mathcal{A}$ should not have noticeable advantage over a trivial solver just making a random guess on $ve \bmod 2$ according to the distribution $\chi' = v\chi_e \bmod 2$.

3 Generic AKE Protocol from OW-CPA KEM and PKE

In this section, we present a generic construction for AKE as shown in Fig. 5, which is named as GC-AKE. This construction employs a OW-CPA secure KEM scheme KEM-OW, a OW-CPA secure PKE scheme PKE-OW, and hash functions

$$H_{\text{init}} : \{0, 1\}^* \rightarrow \mathcal{K}, H_{\text{resp}} : \{0, 1\}^* \rightarrow \mathcal{K}, G : \mathcal{M} \rightarrow \mathcal{R}, H : \{0, 1\}^* \rightarrow \mathcal{K}$$

as building blocks. Here $\mathcal{M}$ (resp. $\mathcal{R}$) is the message (resp randomness) space of PKE-OW, and $\mathcal{K}$ is the key space of GC-AKE. Note that the correctness of GC-AKE directly follows from the correctness of KEM-OW and PKE-OW. In Algorithms 1 to 4, we formally describe the four constituting algorithms of GC-AKE, that is, the GC-AKE.KeyGen, GC-AKE.Init, GC-AKE.Der_{resp}, and GC-AKE.Der_{init}.

Fig. 5: The generic AKE protocol GC-AKE between P_i and P_j .**Algorithm 1:** GC-AKE.KeyGen(1^κ): key generation**Input:** Security parameter κ **Output:** Public key pk_i and secret key sk'_i

- 1 $(pk_i, sk_i) = \text{PKE-OW.KeyGen}(1^\kappa)$;
- 2 $s_i \xleftarrow{\$} \mathcal{M}, sk'_i := (sk_i, s_i)$;
- 3 **return** (pk_i, sk'_i) .

Algorithm 2: GC-AKE.Init(sk'_i, pk_j): initialization**Input:** $sk'_i = (sk_i, s_i), pk_j$ **Output:** (M, st)

- 1 $m_j \xleftarrow{\$} \mathcal{M}$;
- 2 $c_j := \text{PKE-OW.Enc}(pk_j, m_j; G(m_j))$;
- 3 $(\widetilde{pk}, \widetilde{sk}) \leftarrow \text{KEM-OW.KeyGen}(1^\kappa)$;
- 4 $M := (c_j, \widetilde{pk})$;
- 5 $st := (\widetilde{pk}, \widetilde{sk}, m_j, c_j)$;
- 6 **return** (M, st) .

Algorithm 3: GC-AKE.Der_{resp}(sk'_j, pk_i, M): P_j derives a session key

Input: $sk'_j = (sk_j, s_j), pk_i, M = (c_j, \widetilde{pk})$
Output: $(M', K_{i,j})$
 1 $m_i \xleftarrow{\$} \mathcal{M}$;
 2 $c_i := \text{PKE-OW.Enc}(pk_i, m_i; \mathbf{G}(m_i))$;
 3 $(\widetilde{K}, \widetilde{c}) \leftarrow \text{KEM-OW.Encap}(\widetilde{pk})$;
 4 $m'_j := \text{PKE-OW.Dec}(sk_j, c_j)$;
 5 $M' := (c_i, \widetilde{c})$;
 6 **if** $m'_j = \perp \vee c_j \neq \text{PKE-OW.Enc}(pk_j, m'_j; \mathbf{G}(m'_j))$;
 7 $K_{i,j} := \text{H}_{\text{resp}}(s_j, m_i, c_j, \widetilde{K}, i, j, M, M')$;
 8 $K_{i,j} := \text{H}(m_i, m'_j, \widetilde{K}, i, j, M, M')$;
 9 **return** $(M', K_{i,j})$.

Algorithm 4: GC-AKE.Der_{init}(sk'_i, st, M'): P_i derives a session key

Input: $sk'_i = (sk_i, s_i), st = (\widetilde{pk}, \widetilde{sk}, m_j, c_j), M' = (c_i, \widetilde{c})$
Output: $K'_{i,j}$
 1 $m'_i := \text{PKE-OW.Dec}(sk_i, c_i)$;
 2 $\widetilde{K}' := \text{KEM-OW.Decap}(\widetilde{sk}, \widetilde{c})$;
 3 **if** $m'_i = \perp \vee c_i \neq \text{PKE-OW.Enc}(pk_i, m'_i; \mathbf{G}(m'_i))$;
 4 $K'_{i,j} := \text{H}_{\text{init}}(s_i, c_i, m_j, \widetilde{K}', i, j, M, M')$;
 5 $K'_{i,j} := \text{H}(m'_i, m_j, \widetilde{K}', i, j, M, M')$;
 6 **return** $K'_{i,j}$.

In fact, the OW-CPA KEM scheme used by GC-AKE can also be substituted with an OW-CPA PKE scheme. The rationale, for choosing an OW-CPA KEM scheme here, is that our subsequently proposed PKE scheme NEV-PKE-OW naturally induces an OW-CPA KEM scheme NEV-KEM-OW, and the encapsulation algorithm of NEV-KEM-OW is slightly more efficient than the encryption algorithm of NEV-PKE-OW. For this reason, we adopt the OW-CPA KEM scheme here to obtain an AKE protocol with better computational efficiency. Furthermore, since the decapsulation algorithm of NEV-KEM-OW never outputs the failure symbol $\perp$, for simplicity, our GC-AKE (Fig. 5) omits the judgment on whether $\widetilde{K}'$ equals $\perp$, and only checks whether the decryption of c_i and c_j succeeds. We point out that by using an implicit-rejection strategy analogous to that of the HKSU construction [38], namely, introducing extra random oracles to output a pseudorandom session key whenever $\widetilde{K}' = \perp$, the security proof of GC-AKE presented in Sec. 8 can be also generalized to the case where K' may output $\perp$.

In the security proof of GC-AKE presented in Sec. 8, the hash functions H_{init} , H_{resp} , $\mathbf{G}$, and H are modeled as random oracles. Hence, GC-AKE is actually a ROM-based generic AKE construction. As shown in Table 1, compared with several well-known ROM-based generic AKE constructions, our GC-AKE construction has certain advantages in terms of generality and computational efficiency.

- Unlike FSXY [30], which requires an OW-CCA secure KEM together with an OW-CPA secure KEM, GC-AKE only relies on OW-CPA secure PKE and KEM. Meanwhile, we provide the security proof in the QROM for GC-AKE. In terms of computational efficiency, each party in GC-AKE only needs three hash eval-

uations. In contrast, FSXY requires five hash evaluations when adopting the FO transformation to derive an OW-CCA secure KEM.

- Compared with HKSU [38], our GC-AKE eliminates the need for an IND-CPA secure PKE and merely uses an OW-CPA secure PKE and an OW-CPA secure KEM. Moreover, our GC-AKE does not require the underlying PKE to be γ -spread and does not introduce the extra Punc operation. As for the efficiency, the responder of GC-AKE is able to save one hash evaluation relative to HKSU.

Essentially, FSXY, HKSU, and GC-AKE follow the same design paradigm but differ in the use of the FO transformation: FSXY uses it directly, while HKSU and GC-AKE decompose it into modules of derandomization and re-encryption check and then integrate them into the AKE protocols. We point out that, compared with HKSU, our GC-AKE integrates these modules in a more practical manner, and we further provide a security proof merely based on the OW-CPA security. These two features endow GC-AKE with the aforementioned advantages.

Table 1: Comparison of well-known ROM-based AKE constructions. "Requirement" indicates the property needed to be satisfied by the underlying primitives. "Comm." indicates the construction's communication. " x cip + y pk" stands for the communication cost consisting of x ciphertexts and y public keys. "Comp." represents the computational cost of either initiator or responder. " x en/de + y hash" stands for x encryption/decryption or encapsulation/decapsulation operations and y hash evaluations.

Construction	ROM	QROM	Underlying primitives	Requirement	Comm.	Comp. (initiator)	Comp. (responder)
FSXY [30]	✓	\	OW-CCA KEM ^a OW-CPA KEM	\	3 cip + 1 pk	4 en/de + 5 hash	4 en/de + 5 hash
HKSU [38]	✓	✓	IND-CPA PKE	γ -spread Punc	3 cip + 1 pk	4 en/de + 3 hash	4 en/de + 4 hash
GC-AKE	✓	✓	OW-CPA KEM OW-CPA PKE	\	3 cip + 1 pk	4 en/de + 3 hash	4 en/de + 3 hash

^a Without loss of generality, we regard the OW-CCA secure KEM as that obtained by applying the FO transformation, which is exactly the approach adopted in [30].

4 Concrete AKE Instantiation from NTRU and RLWE

In this section, we first describe an OW-CPA secure PKE scheme NEV-PKE-OW from the decisional NTRU and sspRLWE assumptions, and then extend it to get an OW-CPA secure KEM scheme NEV-KEM-OW. By integrating NEV-PKE-OW and NEV-KEM-OW into the generic construction GC-AKE, we directly obtain a concretely efficient AKE instantiation NEV-AKE. We begin by describing our vector decoding technique.

4.1 Randomized Plaintext Encoding and Decoding

Let n be a power of 2, and q be a prime. Let $R = \mathbb{Z}[x]/(x^n + 1)$ and $R_q = \mathbb{Z}_q[x]/(x^n + 1)$. Let $\mathcal{M} = \{0, 1\}^{n/k}$ be the plaintext space. Let $v = (1 - x^{n/k}) \in R_q^*$

be a ring element, whose inverse is $v^{-1} = \frac{q+1}{2}(1 + x^{n/k} + \dots + x^{(k-1)n/k}) \in R_q^*$. Let B_η be the binomial distribution with parameter $\eta \in \mathbb{Z}$. We define a pair of encoding and decoding algorithm (Pt2noise, Noise2Pt) as follows:

- **Pt2noise**(M, η) : given a plaintext $M \in \{0, 1\}^{n/k}$ and an integer η as inputs, first randomly choose $s \xleftarrow{\$} \{0, 1\}^{2n\eta - n/k}$, and parse $s = (s_0, \dots, s_{2k\eta-2})$ as $(2k\eta - 1)$ blocks of n/k bits (i.e., $s_i \in \{0, 1\}^{n/k}$ for all $i \in [2k\eta - 1]$). Then, set $s_{2k\eta-1} = M \oplus (\oplus_{i=0}^{2k\eta-2} s_i) \in \{0, 1\}^{n/k}$, arrange the bit string $(s_0, \dots, s_{2k\eta-1}) \in \{0, 1\}^{2n\eta}$ as a bit array with 2η rows and n columns, and use the 2η bits in the i -th column as the randomness to sample the i -th coefficient of a polynomial $m \in R_q$ from B_η , as depicted in Fig. 6. Finally, return $m = m_0 + m_1x + \dots + m_{n-1}x^{n-1} \in R_q$, where

$$m_{in/k+j} = \sum_{t=0}^{\eta-1} (s_{2i\eta+t,j} - s_{2i\eta+\eta+t,j}) \text{ for } i \in [k], j \in [n/k].$$

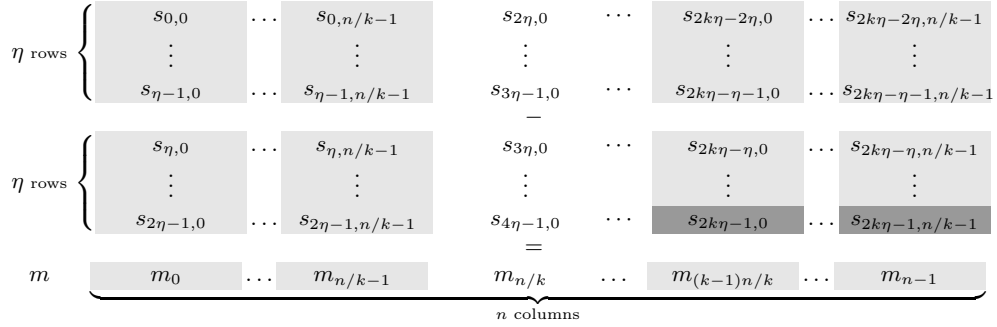


Fig. 6: The bit array for randomized encoding of a plaintext

- **Noise2Pt**(w) : given a polynomial $w = w_0 + w_1x + \dots + w_{n-1}x^{n-1} \in R_q$ as input, first compute $\tilde{w}_i = w_i - \frac{q+1}{2} \bmod^\pm q$ for all $i \in [n]$. Then, compute $t_j = \sum_{i=j \bmod n/k} |\tilde{w}_i|$ for all $j \in [\ell]$. Finally, set

$$M_j = \begin{cases} 1, & \text{if } t_j < \frac{k \cdot (q-1)}{4}; \\ 0, & \text{otherwise,} \end{cases}$$

and return $M = (M_0, \dots, M_{\ell-1}) \in \{0, 1\}^\ell$.

We have the following lemma for the above two algorithms.

Lemma 3. *Let $n, q, k, \eta \in \mathbb{Z}$ and $v \in R_q^*$ be defined as above. If M is uniformly chosen from $\{0, 1\}^{n/k}$, then the coefficient distribution of $m = \text{Pt2noise}(M, \eta)$ is identical to the binomial distribution B_η . Moreover, if $k\eta < \frac{q}{2}$, then for any $m = \text{Pt2noise}(M, \eta)$ and any polynomial $e = e_0 + e_1x + \dots + e_{n-1}x^{n-1} \in R_q$ satisfying the following condition*

$$\left(\sum_{i=j \bmod n/k} |e_i \bmod^\pm q| \right) < \frac{k \cdot (q-1)}{4} - k \frac{k\eta + 1}{2} \text{ for } i \in [n] \text{ and } j \in [n/k] \quad (1)$$

we always have $\text{Noise2Pt}(v^{-1}m + e) = M$.

Proof. The first claim directly follows from the fact that $(s_0, \dots, s_{2k\eta-2})$ are uniformly chosen from $\{0, 1\}^{2n\eta-n/k}$, and given $(s_0, \dots, s_{2k\eta-2})$, $s_{2k\eta-1}$ is also uniformly distributed over $\{0, 1\}^{n/k}$.

Let $\bar{m}$ be a polynomial whose first n/k coefficients are $M = (M_0, \dots, M_{n/k-1}) \in \{0, 1\}^{n/k}$. We now show that $v^{-1}m = v^{-1}\bar{m} + e' \in R_q$ for some $\|e'\|_\infty \leq \frac{k\eta+1}{2}$. Formally, let $\bar{v} = (1 + x^{n/k} + \dots + x^{(k-1)n/k})$, and $u = u_0 + u_1x + \dots + u_{n-1}x^{n-1} = \bar{v}m$, we have that $u_{in/k+j} = \sum_{t \leq i} m_{tn/k+j} - \sum_{t > i} m_{tn/k+j}$ for all $i \in [k], j \in [n/k]$. By the assumption that $k\eta < \frac{q}{2}$, we have that $u_{in/k+j} \in [-\frac{q-1}{2}, \frac{q-1}{2}]$ for all $i \in [k], j \in [n/k]$. Moreover, using a routine calculation one can check that $u_{in/k+j} = \sum_{t=0}^{k-1} m_{tn/k+j} = M_j \bmod 2$ for all $i \in [k], j \in [n/k]$ by the definition of m , and that there exists a polynomial e' such that $u = 2e' + \bar{v}\bar{m}$ and $\|e'\|_\infty \leq \frac{k\eta+1}{2}$ by the definition of $\bar{m}$. We immediately have $w = v^{-1}m + e = v^{-1}\bar{m} + e' + e$ using the fact that $v^{-1} = \frac{q+1}{2}\bar{v}$.

Now, we continue the proof by considering the value of each $M_j \in \{0, 1\}$ for $j \in [n/k]$:

- $M_j = 1$: we have that $w_i = \frac{q+1}{2} + e'_i + e_i$ for all $i = j \bmod n/k$, and that $\tilde{w}_i = w_i - \frac{q+1}{2} = e'_i + e_i \bmod^\pm q$. This means that

$$t_j = \sum_{i=j \bmod n/k} |\tilde{w}_i| = \sum_{i=j \bmod n/k} |e'_i + e_i \bmod^\pm q| < \frac{k \cdot (q-1)}{4},$$

where the inequality holds because $\sum_{i=j \bmod n/k} |e_i \bmod^\pm q| < \frac{k \cdot (q-1)}{4} - k \frac{k\eta+1}{2}$ by assumption and $\|e'\|_\infty \leq \frac{k\eta+1}{2}$ as shown above. This shows that $\text{Noise2Pt}(w)$ will output $M_j = 1$;

- $M_j = 0$: we have that $w_i = e'_i + e_i$ for all $i = j \bmod n/k$, and that $\tilde{w}_i = w_i - \frac{q+1}{2} = e'_i + e_i - \frac{q+1}{2} \bmod^\pm q$. Since we have either $e'_i + e_i = |e'_i + e_i \bmod^\pm q|$ or $e'_i + e_i = q - |e'_i + e_i \bmod^\pm q|$, it is easy to check that $|\tilde{w}_i| \geq \frac{q-1}{2} - |e'_i + e_i \bmod^\pm q|$. This means that

$$t_j = \sum_{i=j \bmod n/k} |\tilde{w}_i| \geq \sum_{i=j \bmod n/k} \left(\frac{q-1}{2} - |e'_i + e_i \bmod^\pm q| \right) > \frac{k \cdot (q-1)}{4},$$

and that $\text{Noise2Pt}(w)$ will output $M_j = 0$. This completes the proof. $\square$

Remark 6. Since $v^{-1}m + e = v^{-1}\bar{m} + e + e'$, the condition (1) in Lemma 3 can actually be relaxed to the following condition:

$$\left(\sum_{i=j \bmod n/k} |e_i + e'_i \bmod^\pm q| \right) < \frac{k \cdot (q-1)}{4} \text{ for } i \in [n] \text{ and } j \in [n/k]. \quad (2)$$

4.2 Detailed Descriptions of **NEV-PKE-OW** and **NEV-KEM-OW**

Let $n, q, k, \eta \in \mathbb{Z}$ and $v \in R_q^*$ be defined as above. Let χ_f, χ_g, χ_r be three distributions over R . We now give our PKE scheme **NEV-PKE-OW**, which consists of the following three algorithms (**KeyGen**, **Enc**, **Dec**):

- **NEV-PKE-OW.KeyGen**(1^κ): given the security parameter κ as inputs, randomly choose $f' \xleftarrow{\$} \chi_f$ and $g \xleftarrow{\$} \chi_g$ such that $f = f' + v^{-1}$ and $g \in R_q^*$ are invertible. Then, return the public key and secret key pair $(pk, sk) = (h = g/f, f) \in R_q \times R_q$.
 - **NEV-PKE-OW.Enc**(pk, M): given the public key $pk = h \in R_q$ and a plaintext $M \in \{0, 1\}^{n/k}$ as inputs, sample $r \xleftarrow{\$} \chi_r$ and $m \xleftarrow{\$} \text{Pt2noise}(M, \eta) \in R_q$. Then, compute and return the ciphertext $c = hr + m$.
- In this document, we will frequently use the notation

$$c = \text{NEV-PKE-OW.Enc}(pk, M; \rho),$$

which means that we run the algorithm **NEV-PKE-OW.Enc**(pk, M) by using $\rho \in \{0, 1\}^\kappa$ as the random seed to a secure pseudorandom generator (PRG) to generate the random coins to sample $r \xleftarrow{\$} \chi_r$ and $m \xleftarrow{\$} \text{Pt2noise}(M, \eta) \in R_q$.

- **NEV-PKE-OW.Dec**(sk, C): given the secret key $sk = f = f' + v^{-1} \in R_q^*$ and a ciphertext $c \in R_q$ as inputs, compute $u = fc$, and $M' = \text{Noise2Pt}(u)$. Finally, return the plaintext $M' \in \{0, 1\}^{n/k}$.

Correctness analysis. Let $\bar{m}$ be a polynomial whose first n/k coefficients are $M = (M_0, \dots, M_{n/k-1}) \in \{0, 1\}^{n/k}$. Then, we have $v^{-1}m = v^{-1}\bar{m} + e' \in R_q$ for some $\|e'\|_\infty \leq \frac{k\eta+1}{2}$. Since we have the following decryption formula

$$w = fc = gr + (f' + v^{-1})m = gr + f'm + v^{-1}m = \underbrace{gr + f'm + e'}_{=e''} + v^{-1}\bar{m},$$

the decryption is correct as long as we set the parameters such that $e'' = (e''_0, \dots, e''_{n-1})$ satisfies

$$\left(\sum_{i=j \bmod n/k} |e''_i \bmod^\pm q| \right) < \frac{k \cdot (q-1)}{4} \text{ for all } i \in [n] \text{ and } j \in [n/k].$$

by the condition (2) in Lemma 3.

For our concrete choices of parameters, we numerically compute a union bound on the probability of decryption failures by using an approach adapted from [5, 11]. Formally, we are more interested in the case of k, n being powers of 2 and χ_f, χ_g, χ_r being centrally symmetric distributions (namely, $-\chi = \chi$). By $S = \mathbb{Z}[Y]/(Y^k + 1)$ we denote the $2k$ -th cyclotomic ring, having rank k over $\mathbb{Z}$. Since $k|n$, we can view S as a subring of $R = \mathbb{Z}[X]/(X^n + 1)$ by identifying $Y = X^{n/k}$. In particular, an element $a \in R$ can be identified to a vector $(\hat{a}_0, \dots, \hat{a}_{n/k-1}) \in S^{n/k}$ such that

$$a(X) = \hat{a}_0(X^{n/k}) + X\hat{a}_1(X^{n/k}) + \dots + X^{n/k-1}\hat{a}_{n/k-1}(X^{n/k}).$$

The coefficients of $\hat{a}_j \in S$ exactly consists of $a_j, a_{j+n/k}, \dots, a_{j+(k-1)n/k}$ of $a \in R$. Using the above notations, the condition for correct decryption can be restated as

$$\|\hat{e}''_j\|_1 < \frac{k \cdot (q-1)}{4} \text{ for all } j \in [n/k].$$

Furthermore, let $y \in S$ be a polynomial ranges over all polynomials with ± 1 coefficients. Then, we have $\|\hat{e}_j''\|_1 = \max_y \langle \hat{e}_j'', y \rangle$ and

$$\Pr \left[\|\hat{e}_j''\|_1 \geq \frac{k \cdot (q-1)}{4} \right] = \Pr \left[\max_y \langle \hat{e}_j'', y \rangle \geq \frac{k \cdot (q-1)}{4} \right].$$

By using a union bound, we have

$$\Pr \left[\|\hat{e}_j''\|_1 \geq \frac{k \cdot (q-1)}{4} \right] \leq \sum_y \Pr \left[\langle \hat{e}_j'', y \rangle \geq \frac{k \cdot (q-1)}{4} \right].$$

Since $e'' = gr + f'e + e'$, we have

$$\hat{e}_j'' = \sum_{i \in [n/k]} Y^{z_j} (\hat{g}_i \hat{r}_{(j-i) \bmod n/k} + \hat{f}_i' \hat{e}_{(j-i) \bmod n/k}) + \hat{e}_j', \quad (3)$$

where the powers z_j are integers. Since all $\hat{g}_i, \hat{f}_i', \hat{r}_i, \hat{e}_i$ have coefficients independently chosen from centrally symmetric distribution (which are invariant under multiplication by Y^{z_i} and by -1), we can numerically compute the probability

$$\begin{aligned} p_j &= \sum_y \Pr \left[\langle \hat{e}_j'', y \rangle \geq \frac{k \cdot (q-1)}{4} \right] \\ &= \sum_y \Pr \left[\left\langle \sum_{i \in [n/k]} (\hat{g}_i \hat{r}_i + \hat{f}_i' \hat{e}_i) + \hat{e}_j', y \right\rangle \geq \frac{k \cdot (q-1)}{4} \right] \end{aligned} \quad (4)$$

by randomly choosing $\hat{g}_i \xleftarrow{\$} \chi_g, \hat{f}_i' \xleftarrow{\$} \chi_f, \hat{r}_i \xleftarrow{\$} \chi_r, \hat{e}_i \xleftarrow{\$} B_\eta$. To address worst-case correctness across all possible messages (Definition 2), our numerical computation actually considers a distribution for $\hat{e}_i = (\hat{e}_{i,0}, \hat{e}_{i,1}, \hat{e}_{i,2}, \hat{e}_{i,3})$, where the first three coefficients are independently drawn from B_η . The last coefficient $\hat{e}_{i,3}$, as defined by our message encoding algorithm $\text{Pt2noise}(M, \eta)$ (see Fig. 6), is drawn from either

$$\{e - (e \bmod 2) : e \xleftarrow{\$} B_{\eta-1} + \mathcal{U}(\{0, 1\})\} \quad \text{or} \quad (5)$$

$$\{e - ((e+1) \bmod 2) : e \xleftarrow{\$} B_{\eta-1} + \mathcal{U}(\{0, 1\})\},$$

depending on the message bit $M_i \in \{0, 1\}$ and the first three coefficients, with $B_0 = 0$. Since the two distributions have the same zero expectation (and, except for $\eta = 1$, the same variance $\eta/2$), the estimated failure probability obtained by our numerical computation is essentially independent of the message bits and matches the case $\hat{e}_i \leftarrow B_\eta$ —which is equivalent to assuming M_i is uniformly random. This implies that our scheme is indeed robust against decryption failure attacks that use maliciously crafted messages.

Finally, by taking a union bound we can obtain an upper bound on the probability of decryption failures

$$1 - \Pr \left[\|\hat{e}_j''\|_1 < \frac{k \cdot (q-1)}{4} \text{ for all } j \in [n/k] \right] \leq n/k \cdot p_j. \quad (6)$$

It is worth to note that the above analysis does not rely on extra assumption on the coefficients distribution of the final error term e'' , and gives a very conservative upper bound on the probability of decryption failures. In particular, it does not need the identically and independently distributed assumption on the coefficients of e'' , which is usually used to simplify the analysis of lattice encryption schemes but was disproved both theoretically and practically in [42]. In fact, our numerical computation shows that, under the independence assumption, even for our simple vector decoding that relies solely on the correlation among four coefficients, the estimated failure probability can be reduced by a factor of up to 2^{16} for our concrete choice of parameters given in Sec. 5.

Remark 7 (Supporting Ciphertext Compression). Since $h \in R_q^*$, the ciphertext in $\text{NEV-PKE-OW.Enc}(pk, M)$ can be equivalently written as $c' = h^{-1}m + r$. In this form, decryption proceeds by first multiplying c' by g , yielding the same decryption formula as before. This conceptual change does not affect the security or correctness analysis. A convenient feature of this formulation is that it naturally supports existing rounding-like compression techniques for ciphertexts. Such techniques essentially introduce an additional error term e' into the ciphertext, which can be directly absorbed into r , i.e., $c' = h^{-1}m + r + e' = h^{-1}m + r'$. This property also extends to other schemes in the NEV family.

However, rounding-like techniques may introduce significant uncertainties into rigorous security analysis (e.g., rounding-based assumptions [55,56]) and correctness analysis (e.g., heuristic independence assumptions [11]). They may also raise patent issues. Therefore, we believe that the non-compression version is a better choice for standardization. Nevertheless, to illustrate the efficiency gains achievable for NEV-AKE under these techniques, we also provide concrete parameters for the compressed variant based on rounding-like assumptions (namely, sspRLWR), denoted as NEV-AKE-C1*, NEV-AKE-C2*, and NEV-AKE-C3* (see Sec. 5). These are intended for those who are comfortable with the aforementioned weaknesses.

Extension to OW-CPA KEM. The above NEV-PKE-OW scheme can be easily extended to obtain the NEV-KEM-OW scheme with slightly better computational performance by replacing $m \xleftarrow{\$} \text{Pt2noise}(M, \eta) \in R_q$ with $m \xleftarrow{\$} B_\eta$ in the encryption algorithm. Specifically, NEV-KEM-OW.KeyGen is the same as NEV-PKE-OW.KeyGen, except that NEV-KEM-OW.Enc and NEV-KEM-OW.Dec are constructed as follows:

- NEV-KEM-OW.Encap(pk): On input $pk = h \in R_q$, sample $r \xleftarrow{\$} \chi_r$ and $m \xleftarrow{\$} B_\eta$, compute a ciphertext $c := hr + m \in R_q$ and a key $K := \text{Ext}(\bar{v}m \bmod 2)$, where $\bar{v} = 1 + x^{n/k} + \dots + x^{(k-1)n/k}$ and Ext is a function that returns the first n/k coefficients of its input. Finally, output (c, K) .
- NEV-KEM-OW.Decap(sk, c): On input the secret key $sk = f = f' + v^{-1} \in R_q^*$ and a ciphertext $c \in R_q$, compute $u = fc$ and $K \leftarrow \text{Noise2Pt}(u)$, and output a key $K \in \{0, 1\}^{n/k}$.

In fact, NEV-KEM-OW.Decap is identical to NEV-PKE-OW.Dec, except for modifying the notation and terminology to comply with the definition of KEM. For correctness, by the definition of Ext and Noise2Pt , it is easy to check that $\text{Ext}(\bar{v}m \bmod$

2) = $\text{Noise2Pt}(v^{-1}m)$. Note that the slight modification of the encryption algorithm has no impact on the provable security of the scheme, as the distribution of $m \xleftarrow{\$} \text{Pt2noise}(M, \eta) \in R_q$ is essentially identical to that of $m \xleftarrow{\$} B_\eta$, when M is uniformly distributed over $\{0, 1\}^{n/k}$. In other words, the provable security of NEV-PKE-OW straightforwardly implies that of NEV-KEM-OW.

5 Concrete Parameters

To select the parameter sets for the NEV-AKE protocol, we only need to choose the concrete parameters for the NEV-KEM-OW and NEV-PKE-OW schemes. In Table 2, we select six parameter sets, divided into three groups of two: "NEV-AKE-C1/C2/C3" (*compact*) and "NEV-AKE-R1/R2/R3" (*recommended*), each targeted $\geq 128/256/512$ -bit classical security (and $\geq 80/128/256$ -bit quantum security). The 4th and 5th columns give the distributions used for key generation and encryption. For the parameter sets NEV-AKE-C1*/C2*/C3*, we incorporate the conceptual modification described in Remark 7: the ciphertext is compressed from R_{769} to R_{256} via

$$c' = \lfloor h^{-1}m \cdot \frac{256}{769} \rfloor_{256}.$$

The resulting distribution $\mathcal{T}_{1/3}^*$, obtained by rounding from $\mathbb{Z}_{769}$ to $\mathbb{Z}_{256}$, closely approximates $\mathcal{T}_{1/3}$. This compression reduces the ciphertext size by roughly 16.7%, but at the cost of introducing a rounding-based hardness assumption and requiring an implicit rejection strategy for the security proof. As an alternative, one may forgo compression entirely and instead set $c' = h^{-1}m + r$ with $r \leftarrow \mathcal{T}_{1/3}$, thus relying on more standard hardness assumptions for both security and correctness. The uncompressed variants, denoted "NEV-AKE-C1/C2/C3", differ from the compressed "NEV-AKE-C1*/C2*/C3*" sets only in using $\mathcal{T}_{1/3}$ instead of the rounded $\mathcal{T}_{1/3}^*$. Their ciphertext size matches the public key size, and their estimated security and decryption failure rates remain essentially the same as those of the compressed versions. Therefore, the parameter sets for "NEV-AKE-C1*/C2*/C3*" are the same as the ones for "NEV-AKE-C1/C2/C3" shown in Table 2.

The 6th column presents the decryption failure probability, which is computed by using a python script following the analysis given in Sec 4.2.⁹ We set our decryption failure probabilities $< 2^{-128}$ with some margin so that it is infeasible to obtain a single decryption failure using at most 2^{80} decryption queries (see the directional failure boosting attacks [21] and Sec. 9.5). The 7-10th columns give the concrete security estimates for the hardness of NTRU and RLWE using the core-SVP model [5], where C/Q denotes the lowest classical/quantum security bits. In the last column, we also present the estimated security bits in a (refined) MATZOV model (see MATZOV[†] in Sec. 9.1) recently shown to have lowered the

⁹ For the NEV-AKE-C1/C2/C3 parameter sets, both the security analysis and the decryption failure computation rely on the same heuristic assumption commonly adopted in rounding-based schemes like Kyber [11]: namely, that each coefficient of the rounding error is randomly and independently distributed. This assumption is distinct from the independence-of-coefficient-failures heuristic, which assumes that decryption failures are identically and independently distributed across coefficients—an assumption that has been disproven both theoretically and empirically [22]. In contrast, the security and correctness analyses for all other parameter sets are completely heuristic-free.

concrete security estimates of Kyber below the NIST requirements [15]. Given the recent progress of dual-hybrid LWE attacks, we believe it is prudent to consider the (refined) MATZOV model in concrete security estimates as also commented in [33]. Table 2 shows that the MATZOV hybrid dual attack largely determines the hardness of the RLWE problem. Although this type of dual attack is not directly applicable to NTRU problems, we believe that selecting NTRU parameters solely based on this assumption would not be a conservative approach. Therefore, we *recommend* using the NEV-AKE-R1, NEV-AKE-R2, and NEV-AKE-R3 parameter sets to ensure that NTRU remains hard even if some variant of the MATZOV hybrid dual attack were to affect the NTRU setting.

Table 2: Concrete Parameter Sets for NEV-AKE

Parameter set	n	q	Key Dist. (χ_f, χ_g)	Enc Dist. (χ_r, χ_e)	Dec. Failure	NTRU Hardness Primal (C,Q)	RLWE Hardness (C,Q)		
							Primal	Dual	MATZOV [†]
NEV-AKE-C1*	512	769	(B_1, B_2)	($\mathcal{T}_{1/3}^*, B_2$)	2^{-160}	(132,116)	(135,118)	(133,117)	(129,103)
NEV-AKE-R1		1409	(B_3, B_3)	(B_3, B_3)	2^{-133}	(135,119)	(135,119)	(133,117)	(134,107)
NEV-AKE-C2*	1024	769	($B_1, \mathcal{T}_{1/8}$)	($\mathcal{T}_{1/3}^*, B_2$)	2^{-162}	(266,234)	(300,264)	(297,261)	(268,217)
NEV-AKE-R2		1409	(B_2, B_2)	(B_2, B_2)	2^{-158}	(284,250)	(284,250)	(282,248)	(261,211)
NEV-AKE-C3*	2048	769	($\mathcal{T}_{1/8}, \mathcal{T}_{1/8}$)	($\mathcal{T}_{1/3}^*, B_1$)	2^{-163}	(546,480)	(610,537)	(607,534)	(521,410)
NEV-AKE-R3		1409	($\mathcal{T}_{1/3}, \mathcal{T}_{1/3}$)	($\mathcal{T}_{1/3}, B_2$)	2^{-147}	(583,513)	(596,525)	(592,521)	(526,424)

6 Implementation Details

We have implemented our NEV-AKE protocol on a x86 platform. We provide a reference C implementation and a vector-optimized version (AVX2 for x86). The following details focus on the implementation choices that most affect performance, specifically for our recommended parameter sets (NEV-R-1/2/3) on x86.

6.1 Partial NTT Multiplication

One costly arithmetic operation in NEV-AKE is to do polynomial multiplication in R_q . Since the use of small modulus, we cannot apply full NTT multiplications in $R_q = \mathbb{Z}[x]/(x^n + 1)$. But because $q - 1 \bmod 128 = 1$ for $q = 1409$, we can still speedup polynomial multiplications by first splitting the polynomials in R_q to a set of sub-polynomials in $R'_q = \mathbb{Z}_q[y]/(y^{64} + 1)$ and then realize a single polynomial multiplication in R_q by using a number of polynomial multiplications in $R'_q = \mathbb{Z}_q[y]/(y^{64} + 1)$, which in turn can be done efficiently using full NTT multiplications.¹⁰ Taking $n = 512$ as an example, by letting $y = x^4$ we can split any two polynomials $a, b \in R_q = \mathbb{Z}[x]/(x^{512} + 1)$ as follows:

$$\begin{aligned} a(x) &= a_0(y) + xa_1(y) + x^2a_2(y) + x^3a_3(y) + x^4a_4(y) + x^5a_5(y) + x^6a_6(y) + x^7a_7(y), \\ b(x) &= b_0(y) + xb_1(y) + x^2b_2(y) + x^3b_3(y) + x^4b_4(y) + x^5b_5(y) + x^6b_6(y) + x^7b_7(y), \end{aligned}$$

¹⁰ For $q \in \{769, 3329\}$, we have $q \bmod 256 = 1$, and the ring $R'_q = \mathbb{Z}_q[y]/(y^{128} + 1)$ can be used to speed up multiplications.

where all the a_i 's and b_i 's are polynomials in $R'_q = \mathbb{Z}_q[y]/(y^{64} + 1)$. Since multiplications between a_i 's and b_j 's can be done using full NTT multiplications in R'_q , we can realize the multiplication between $a(x)$ and $b(x)$ by roughly using 64 NTT multiplications in $R'_q = \mathbb{Z}_q[y]/(y^{64} + 1)$. We can save more NTT multiplications in R'_q by using the Karatsuba method as observed in [73].

In the following, we fix $q = 1409$ to describe the implementation details. To facilitate the above polynomial multiplications, we directly represent each polynomial in $R_q = \mathbb{Z}[x]/(x^n + 1)$ by simply concatenating the coefficient vectors of its split sub-polynomials, which are almost for free when all the coefficients are identically chosen from the same distribution. Moreover, we will keep the split sub-polynomials for the public key, secret key and ciphertext in their NTT forms to save some forward and inverse NTT operations in $R'_q = \mathbb{Z}_q[y]/(y^{64} + 1)$.

Let $\zeta \in \mathbb{Z}_q$ is a primitive 128-th root of unity modulo $q = 1409$. Define $\text{BitRev}_6(k)$ to be the integer represented by bit-reversing the unsigned 6-bit value that corresponds to the input integer $k \in \{0, \dots, 63\}$. Specifically, if $k = k_0 + 2k_1 + 4k_2 + \dots + 32k_5$ with $k_i \in \{0, 1\}$, then $\text{BitRev}_6(k) = k_5 + 2k_4 + 4k_3 + \dots + 32k_0$. The NTT and NTT^{-1} algorithms are defined in Algorithm 5 and 6.

Algorithm 5: NTT(f)

Input: array $f \in \mathbb{Z}_q^n$
Output: array $\hat{f} \in \mathbb{Z}_q^n$

```

1  $\hat{f} := f$ 
2 for  $i := 0; i < n; i := i + 64$  do
3    $k := 0$ 
4   for  $len := 32; len > 1; len := len/2$  do
5     for  $start := 0; start < 64; start := start + len$  do
6        $zeta := \zeta^{\text{BitRev}_6(k)} \bmod q$ 
7        $k := k + 1$ 
8       for  $j := start; j < start + len; j ++$  do
9          $t := zeta \cdot \hat{f}[i + j + len]$ 
10         $\hat{f}[i + j + len] := \hat{f}[i + j] - t$ 
11         $\hat{f}[i + j] := \hat{f}[i + j] + t$ 
12      end
13    end
14  end
15 end
16 return  $\hat{f}$ 

```

Algorithm 6: $\text{NTT}^{-1}(\hat{f})$

Input: array $\hat{f} \in \mathbb{Z}_q^n$
Output: array $f \in \mathbb{Z}_q^n$

```

1  $f := \hat{f}$ 
2 for  $i := 0; i < n; i := i + 64$  do
3    $k := 63$ 
4   for  $len := 1; len \leq 64; len := 2 \cdot len$  do
5     for  $start := 0; start < 64; start := start + len$  do
6        $zeta := \zeta^{\text{BitRev}_6(k)} \bmod q$ 
7        $k := k - 1$ 
8       for  $j := start; j < start + len; j ++$  do
9          $t := f[i + j]$ 
10         $f[i + j] := t + f[i + j + len]$ 
11         $f[i + j + len] := zeta \cdot (t - f[i + j + len])$ 
12      end
13    end
14  end
15 end
16  $f := f \cdot 1387 \bmod q$ ; // multiply every entry by  $1387 \equiv 64^{-1} \bmod q$ 
17 return  $f$ 

```

6.2 Partial NTT Inversion

The other costly arithmetic operation is to do polynomial inversion in R_q to generate the public key. Note that if we can do full NTT multiplications in R_q , this operation can be simply done by using n inversions in $\mathbb{Z}_q$ using the NTT representation. Fortunately, we can still speedup this operation by making full use of partial NTT multiplications given above as shown in [28]. Specifically, given a polynomial $f \in R_q = \mathbb{Z}_q[x]/(x^n + 1)$, by letting $z = x^2$ we can first use Karatsuba with an even/odd split to obtain two sub-polynomials in $\hat{R}_q = \mathbb{Z}_q[z]/(z^{n/2} + 1)$:

$$f(x) = f_0(z) + x f_1(z).$$

Then, the inversion of f in R_q can be done using one polynomial multiplication in R_q and one polynomial inversion in $\hat{R}_q$ because

$$\frac{1}{f(x)} = \frac{f_0(z) - x f_1(z)}{(f_0(z) + x f_1(z))(f_0(z) - x f_1(z))} = \frac{f_0(z) - x f_1(z)}{f_0^2(z) - z f_1^2(z)}.$$

By repeating this process, we can finally reduce the inversion of f to a few polynomial multiplications in R_q and a single polynomial inversion in $R'_q = \mathbb{Z}_q[y]/(y^{64} + 1)$, which in turn can be done using 64 inversions in $\mathbb{Z}_q$.

6.3 Symmetric Primitives

In our default implementations, we use SHA3 and SHAKE256 as the hash function and the Pseudo-Random Generator (PRG), respectively, which are the same as

that of NTRU in the NIST PQC submissions and the NIST ML-KEM standard. Given an input bit string N and an output length d , the function $\text{KECCAK}[c](N, d)$ is defined as [54], we define four hash functions as follows:

$$\begin{aligned}\text{Hash-128}(M) &= \text{KECCAK}[256](M\|01, 128); \\ \text{Hash-256}(M) &= \text{KECCAK}[512](M\|01, 256) = \text{SHA3-256}(M); \\ \text{Hash-512}(M) &= \text{KECCAK}[1024](M\|01, 512) = \text{SHA3-512}(M). \\ \text{Hash-1024}(M) &= \text{KECCAK}[1024](M\|01, 1024).\end{aligned}$$

Then, we define three key-derivation functions as follows:

$$\begin{aligned}\text{XOF-128}(M, d) &= \text{KECCAK}[256](M\|1111, 128) = \text{SHAKE128}(M, 128); \\ \text{XOF-256}(M, d) &= \text{KECCAK}[512](M\|1111, 256) = \text{SHAKE256}(M, 256); \\ \text{XOF-512}(M, d) &= \text{KECCAK}[1024](M\|1111, 512).\end{aligned}$$

The instantiating of PRG, H_1 , H_2 , and H_3 is listed in Table 3.

Table 3: Instantiating of PRG

	$\text{PRG}(M, d)$	$H_1(M)$	$H_2(M)$	$H_3(M)$
NEV-AKE-512	$\text{XOF-128}(M, d)$	$\text{Hash-128}(M)$	$\text{Hash-256}(M)$	$\text{XOF-128}(M, 128)$
NEV-AKE-1024	$\text{XOF-256}(M, d)$	$\text{Hash-256}(M)$	$\text{Hash-512}(M)$	$\text{XOF-256}(M, 256)$
NEV-AKE-2048	$\text{XOF-512}(M, d)$	$\text{Hash-512}(M)$	$\text{Hash-1024}(M)$	$\text{XOF-512}(M, 512)$

6.4 Encoding of R_q Elements

We use the function Encode_{16} (Algorithm 11) to store 16 elements in R_q to the byte form. In particular, we encode coefficients by groups of 16 in 168 bits: each 2 coefficients are encoded into a 21-bit integer which bases $q = 1409$ (For $q = 769$, we can pack 5 coefficients in $\mathbb{Z}_{769}$ into a 48-bit integer [71]). For $n = 512$, this will lead to a reduction of 32 bytes in storing a polynomial in R_q . The decoding function Decode_{16} in Algorithm 12 uses a fact that $\lfloor (1524119 * x) / 2^{31} \rfloor = \lfloor x / 1409 \rfloor$ for any x in 2^{21} to recover the coefficients. For any n coefficients, we pack them to bytes in a usual method as shown in Algorithm 13 and 14. Algorithms 7-10 are intermediate procedures for converting between bit strings, byte strings, and integers.

Algorithm 7: BitsToBytes(y)

Input: a bit string y of length α .

Output: a byte string z of length $\lceil \alpha/8 \rceil$.

```

1  $z \in \mathcal{B}^{\lceil \alpha/8 \rceil} := 0^{\lceil \alpha/8 \rceil};$ 
2 for  $i$  from 0 to  $\alpha - 1$  do
3    $z[\lfloor i/8 \rfloor] := z[\lfloor i/8 \rfloor] + y[i] \cdot 2^{i \bmod 8};$ 
4 end
5 return  $z$ ;
```

Algorithm 8: BytesToBits(z)

Input: a byte string z of length α .**Output:** a bit string y of length 8α .

```

1 for  $i$  from 0 to  $\alpha - 1$  do
2   for  $j$  from 0 to 7 do
3      $y[8i + j] := z[i] \bmod 2$ ;
4      $z[i] := \lfloor z[i]/2 \rfloor$ ;
5   end
6 end
7 return  $y$ ;

```

Algorithm 9: BitsToInteger(y, α)

Input: a positive integer α and a bit string y of length α .**Output:** a nonnegative integer x .

```

1  $x := 0$ ;
2 for  $i$  from 1 to  $\alpha$  do
3    $x := 2x + y[\alpha - i]$ ;
4 end
5 return  $x$ ;

```

Algorithm 10: IntegerToBits(x, α)

Input: a nonnegative integer x and a positive integer α .**Output:** a bit string y of length α .

```

1 for  $i$  from 0 to  $\alpha - 1$  do
2    $y[i] := x \bmod 2$ ;
3    $x := \lfloor x/2 \rfloor$ ;
4 end
5 return  $y$ ;

```

Algorithm 11: Encode₁₆($a_0, a_1, \dots, a_{15}$)

Input: integer $a_0, a_1, \dots, a_{15} \in \mathbb{Z}_q$ **Output:** bit array $b \in \{0, 1\}^{168}$

```

1 for  $i$  from 0 to 7 do
2    $t_i \in \mathbb{Z} := a_{2i+1}q + a_{2i}$ ;
3 end
4  $b := \text{IntegerToBits}(t_0, 21) || \dots || \text{IntegerToBits}(t_7, 21)$ ;
5 return  $b$ ;

```

Algorithm 12: $\text{Decode}_{16}(B)$

Input: byte array $B \in \mathcal{B}^{21}$

Output: integer array $(a_0, a_1, \dots, a_{15}) \in \mathbb{Z}_q^{16}$

```

1  $t_0 \in \mathbb{Z} := (B[2] \bmod 32) \cdot 2^{16} + B[1] \cdot 2^8 + B[0];$ 
2  $t_1 \in \mathbb{Z} := (B[5] \bmod 4) \cdot 2^{19} + B[4] \cdot 2^{11} + B[3] \cdot 2^3 + \lfloor B[2]/32 \rfloor;$ 
3  $t_2 \in \mathbb{Z} := (B[7] \bmod 128) \cdot 2^{14} + B[6] \cdot 2^6 + \lfloor B[5]/4 \rfloor;$ 
4  $t_3 \in \mathbb{Z} := (B[10] \bmod 16) \cdot 2^{17} + B[9] \cdot 2^9 + B[8] \cdot 2 + \lfloor B[7]/128 \rfloor;$ 
5  $t_4 \in \mathbb{Z} := (B[13] \bmod 2) \cdot 2^{20} + B[12] \cdot 2^{12} + B[11] \cdot 2^4 + \lfloor B[10]/16 \rfloor;$ 
6  $t_5 \in \mathbb{Z} := (B[15] \bmod 64) \cdot 2^{15} + B[14] \cdot 2^7 + \lfloor B[13]/2 \rfloor;$ 
7  $t_6 \in \mathbb{Z} := (B[18] \bmod 8) \cdot 2^{18} + B[17] \cdot 2^{10} + B[16] \cdot 2^2 + \lfloor B[15]/64 \rfloor;$ 
8  $t_7 \in \mathbb{Z} := B[20] \cdot 2^{13} + B[19] \cdot 2^5 + \lfloor B[18]/8 \rfloor;$ 
9 for  $i$  from 0 to 7 do
10    $a_{2i+1} := \lfloor t_i \cdot 1524119/2^{31} \rfloor;$ 
11    $a_{2i} := r - a_{2i+1} \cdot 1409;$ 
12 end
13 return  $(a_0, a_1, \dots, a_{15});$ 

```

Algorithm 13: $\text{PolyToBytes}(a)$

Input: polynomial $a \in R_q$

Output: integer array $B \in \mathcal{B}^{21 \cdot \lfloor n/16 \rfloor + \lceil 11 \cdot (n \bmod 16)/8 \rceil}$

```

1  $i := 0;$ 
2 for  $i < \lfloor n/6 \rfloor; i := i + +$  do
3    $b = b || \text{Encode}_{16}(a_{16i}, a_{16i+1}, \dots, a_{16i+15});$ 
4 end
5 for  $j := 0; j < n \bmod 16; j := j + +$  do
6    $b = b || \text{IntegerToBits}(a_{16i+j}, 11);$ 
7 end
8  $B := \text{BitsToBytes}(b);$ 
9 return  $B;$ 

```

Algorithm 14: $\text{PolyFromBytes}(a)$

Input: integer array $B \in \mathcal{B}^{21 \cdot \lfloor n/16 \rfloor + \lceil 11 \cdot (n \bmod 16)/8 \rceil}$

Output: polynomial $a \in R_q$

```

1  $i := 0;$ 
2 for  $i < \lfloor n/16 \rfloor; i := i + +$  do
3    $(a_{16i}, \dots, a_{16i+15}) := \text{Decode}_{16}(B[21i], \dots, B[21i+20]);$ 
4 end
5  $z := \text{BytesToBits}(B[21i], \dots, B[21i + \lceil 11 \cdot (n \bmod 16)/8 \rceil - 1]);$ 
6  $(a_{16i}, \dots, a_{n-1}) := \text{BitsToInteger}(z, 11);$ 
7 return  $B;$ 

```

6.5 Sampling Algorithms

The algorithms of NEV-AKE require two sampling subroutines Binomial Distribution (Algorithm 15) and Ternary Distribution (Algorithm 16-17). Both of these algorithms can be used to convert a stream of uniformly random bytes into a sample from some desired distribution. These algorithms will be invoked with a stream of pseudorandom bytes as the input. Algorithm 18 is used to determine which distribution is adopted according to Table 2.

Algorithm 15: $\text{CBD}_\eta(B)$

Input: byte array $B \in \mathcal{B}^{\lceil n\eta/4 \rceil}$.

Output: array $f \in \mathbb{Z}_q^n$.

```

1  $b := \text{BytesToBits}_q(B)$ ;
2 for  $i := 0; i < n; i++$  do
3    $x := \sum_{j=0}^{\eta-1} b[2i\eta + j]$ ;
4    $y := \sum_{j=0}^{\eta-1} b[2i\eta + \eta + j]$ ;
5    $f[i] := (x - y) \bmod q$ ;
6 end
7 return  $f$ ;

```

Algorithm 16: $\text{Ternary}_{1/8}(\rho, N)$

Input: byte array $B \in \mathbb{B}^{\lceil n/4 \rceil}$.

Output: array $f \in \mathbb{Z}_q^n$.

```

1  $B := \text{PRG}(\rho || N, \lceil n/4 \rceil + \lceil n/8 \rceil)$ ;
2  $f := \text{CBD}_1(B[0], \dots, B[\lceil n/4 \rceil - 1])$ ;
3 for  $i := 0; i < \lceil n/8 \rceil; i++$  do
4    $a := B[\lceil n/4 \rceil + i]$ ;
5   for  $i := 0; i < \lceil n/8 \rceil; i++$  do
6      $b := \text{AND}(a \gg j, 1)$ ;
7      $f[8i + j] := \text{AND}(f[8i + j], -b)$ ;
8   end
9 end
10 return  $f$ ;

```

Algorithm 17: Ternary_{1/3}(ρ, N)

Input: byte array B .
Output: array $f \in \mathbb{Z}_q^n$.

```

1 ctx := PRG.Init();
2 ctx := PRG.Absorb(ctx,  $\rho || N$ );
3 while  $j < n$  do
4    $z := \text{PRG.Squeeze}(\text{ctx}, 1)$ ;
5    $r := \text{BytesToInteger}(z, 1)$ ;
6   if  $r < 243$  then
7     for  $i := 0; i < 5$  and  $j < n; i++$  do
8        $a := \lfloor r/3 \rfloor$ ;
9        $f[j] := 1 - (r - 3a)$ ;
10       $r := a$ ;
11       $j := j + 1$ ;
12    end
13  end
14 end
15 return  $f$ ;
```

Algorithm 18: SamplePoly _{χ} (ρ, N)

Input: byte array B .
Output: array $f \in \mathbb{Z}_q^n$.

```

1 if  $\chi = B_\eta$  then
2    $B := \text{PRG}(\rho || N, \lceil n\eta/4 \rceil)$ ;
3    $f := \text{CBD}_\eta(B)$ ;
4 end
5 else if  $\chi = \mathcal{T}_{1/8}$  then
6    $f := \text{Ternary}_{1/8}(\rho, N)$ ;
7 end
8 else if  $\chi = \mathcal{T}_{1/3}$  then
9    $f := \text{Ternary}_{1/3}(\rho, N)$ ;
10 end
11 return  $f$ ;
```

6.6 The NEV-PKE-OW and NEV-KEM-OW Schemes

This section first describes the NEV-PKE-OW scheme. It is an initialization of the PKE scheme described in Sec. 4.2. NEV-PKE-OW consists of four algorithms: NEV-PKE-OW.KeyGen (Algorithm 19), NEV-PKE-OW.Enc_{internal} (Algorithm 20), NEV-PKE-OW.Enc (Algorithm 21), and NEV-PKE-OW.Dec (Algorithm 22).

Algorithm 19: NEV-PKE-OW.KeyGen()

Output: public key pk , secret key sk .

- 1 $rnd \xleftarrow{\$} \mathcal{B}^{\kappa/8};$
- 2 $\sigma := H_1(rnd);$
- 3 $N := 0;$
- 4 $g := \text{SamplePoly}_{\chi_g}(\sigma, N);$
- 5 $N := N + 1;$
- 6 **if** g is not invertible **then**
- 7 **go back to step 4;**
- 8 **end**
- 9 $f' := \text{SamplePoly}_{\chi_f}(\sigma, N);$
- 10 $N := N + 1;$
- 11 $f := f' + v^{-1};$
- 12 **if** f is not invertible **then**
- 13 **go back to step 9;**
- 14 **end**
- 15 $\hat{g} := \text{NTT}(g);$
- 16 $\hat{f} := \text{NTT}(f);$
- 17 $\hat{f}^{-1} := \text{Inverse}(\hat{f});$
- 18 $\hat{h} := \hat{g} \circ \hat{f}^{-1};$
- 19 $pk := \text{PolyToBytes}(\hat{h});$
- 20 $sk := \text{PolyToBytes}(\hat{f});$
- 21 **return** $(pk, sk);$

Algorithm 20: NEV-PKE-OW.Enc_internal(pk, M, ρ)

Input : the public key pk , the message M , the seed ρ .
Output: the ciphertext c .

- 1 $\hat{h} := \text{PolyFromBytes}(pk);$
- 2 $r := \text{SamplePoly}_{\chi_r}(\rho, 0);$
- 3 $m := \text{Pt2noise}(M, \eta, \rho, 1);$
- 4 $\hat{r} := \text{NTT}(r);$
- 5 $\hat{m} := \text{NTT}(m);$
- 6 $\hat{v} := \hat{h} \circ \hat{r} + \hat{m};$
- 7 $c := \text{PolyToBytes}(\hat{v});$
- 8 **return** $c;$

Algorithm 21: NEV-PKE-OW.Enc(pk, M)

Input : the public key pk , the message M .

Output: the ciphertext c .

```

1  $\rho \xleftarrow{\$} \mathcal{B}^{\kappa/8};$ 
2  $c := \text{NEV-PKE-OW.Enc\_internal}(pk, M, \rho);$ 
3 return  $c$ ;
```

Algorithm 22: NEV-PKE-OW.Dec(sk, c)

Input : the ciphertext c , the private key sk .

Output: the message M' .

```

1  $\hat{f} := \text{PolyFromBytes}(sk);$ 
2  $\hat{v} := \text{PolyFromBytes}(c);$ 
3  $\hat{u} = \hat{f} \circ \hat{v};$ 
4  $u = \text{NTT}^{-1}(\hat{u});$ 
5  $M' = \text{Noise2Pt}(u);$ 
6 return  $M'$ ;
```

The NEV-KEM-OW.KeyGen algorithm of the NEV-KEM-OW scheme is the same as that of NEV-PKE-OW. The NEV-KEM-OW.Encap algorithm is shown in Algorithm 23. We also rewrite the NEV-KEM-OW.Decap algorithm in Algorithm 24, which is essentially identical to the NEV-PKE-OW.Dec algorithm.

Algorithm 23: NEV-KEM-OW.Encap(pk)

Input : a public key pk .

Output: a ciphertext c and a key K .

```

1  $\rho \xleftarrow{\$} \mathcal{B}^{\kappa/8};$ 
2  $\hat{h} := \text{PolyFromBytes}(pk);$ 
3  $r \leftarrow \text{SamplePoly}_{\chi_r}(\rho, 0);$ 
4  $m \leftarrow \text{SamplePoly}_{B_\eta}(\rho, 1);$ 
5  $\hat{r} := \text{NTT}(r);$ 
6  $\hat{m} := \text{NTT}(m);$ 
7  $\hat{v} := \hat{h} \circ \hat{r} + \hat{m};$ 
8  $c := \text{PolyToBytes}(\hat{v});$ 
9  $K \leftarrow \text{Ext}(\bar{v}m \bmod 2);$ 
10 return  $(c, K)$ ;
```

Algorithm 24: NEV-KEM-OW.Decap(sk, c)**Input** : a ciphertext c , the secret key sk .**Output:** a key K .

```

1  $\hat{f} := \text{PolyFromBytes}(sk);$ 
2  $\hat{v} := \text{PolyFromBytes}(c);$ 
3  $\hat{u} = \hat{f} \circ \hat{v};$ 
4  $u = \text{NTT}^{-1}(\hat{u});$ 
5  $K \leftarrow \text{Noise2Pt}(u);$ 
6 return  $K$ ;

```

7 Performance Evaluation

We evaluate the performance of the NEV-AKE protocol on the following platform:

Platform. AMD Ryzen 9 7950X3D processor based on the Zen 4 microarchitecture. The CPU frequency was fixed at 4.20 GHz, with Core Performance Boost and Simultaneous Multithreading (SMT) disabled. All benchmarks were executed in a single-threaded configuration. The software environment consisted of Ubuntu 26.04 LTS and GCC 15.2.0. The implementations were compiled using the flags: `-O3 -march=native -fomit-frame-pointer`.

Table 4 reports the experimental results obtained on the AMD Ryzen 9 7950X3D platform, including both the portable reference implementation (REF, plain C) and the AVX2 implementation. The performance measured in CPU cycles is reported as the average over 30,000 runs of each function. In addition, we report the running time (in milliseconds instead of CPU cycles) of each algorithm of NEV-AKE in Table 5. We note that the KeyGen algorithm needs be run only once by each party. The total computation time of NEV-AKE is less than 0.5 ms for the reference implementation and 0.25 ms for the AVX2 implementation, at the highest 512-bit classical security level (and 256-bit quantum security level).

Table 4: The performance of NEV-AKE

Protocol	REF (CPU cycles)				AVX2 (CPU cycles)				Communication (Bytes)		
	KeyGen	Init	Der _{resp}	Der _{init}	KeyGen	Init	Der _{resp}	Der _{init}	initiator	responder	total
NEV-AKE-C1*	103,958	84,413	168,040	118,770	29,172	22,335	67,147	49,843	1,127	1,127	2,254
NEV-AKE-C1	102,393	76,194	168,445	116,240	29,116	25,793	79,174	58,383	1,230	1,230	2,460
NEV-AKE-R1	84,425	76,177	164,026	110,673	32,973	26,677	76,437	53,219	1,344	1,344	2,688
NEV-AKE-C2*	145,637	136,901	337,690	246,436	42,629	34,940	138,284	111,443	2,253	2,253	4,506
NEV-AKE-C2	145,665	142,173	342,878	246,265	42,573	40,589	159,479	127,136	2,458	2,458	4,916
NEV-AKE-R2	157,824	141,253	324,003	233,321	49,052	39,606	148,047	117,492	2,688	2,688	5,376
NEV-AKE-C3*	328,132	290,504	800,096	621,931	105,636	79,048	396,646	347,869	4,506	4,506	9,012
NEV-AKE-C3	328,196	302,746	824,782	633,520	105,543	90,524	452,143	391,193	4,916	4,916	9,832
NEV-AKE-R3	355,731	318,168	845,509	641,996	107,159	92,334	469,138	396,046	5,376	5,376	10,752

Table 5: The running time of NEV-AKE in milliseconds (ms)

Protocol	REF (ms)				AVX2 (ms)				Communication (Bytes)		
	KeyGen	Init	Der _{resp}	Der _{init}	KeyGen	Init	Der _{resp}	Der _{init}	initiator	responder	total
NEV-AKE-C1*	0.025	0.020	0.040	0.028	0.007	0.005	0.016	0.012	1,127	1,127	2,254
NEV-AKE-C1	0.024	0.018	0.040	0.028	0.007	0.006	0.019	0.014	1,230	1,230	2,460
NEV-AKE-R1	0.020	0.018	0.039	0.026	0.008	0.006	0.018	0.013	1,344	1,344	2,688
NEV-AKE-C2*	0.035	0.033	0.081	0.059	0.010	0.008	0.033	0.027	2,253	2,253	4,506
NEV-AKE-C2	0.035	0.034	0.082	0.059	0.010	0.010	0.038	0.030	2,458	2,458	4,916
NEV-AKE-R2	0.038	0.034	0.077	0.056	0.012	0.009	0.035	0.028	2,688	2,688	5,376
NEV-AKE-C3*	0.078	0.069	0.191	0.148	0.025	0.019	0.095	0.083	4,506	4,506	9,012
NEV-AKE-C3	0.078	0.072	0.197	0.151	0.025	0.022	0.108	0.093	4,916	4,916	9,832
NEV-AKE-R3	0.085	0.076	0.202	0.153	0.026	0.022	0.112	0.094	5,376	5,376	10,752

8 Provable Security

In this section, we first prove the key indistinguishability of the AKE generic construction GC-AKE in the CK^+ model. Here we view the hash functions H_{init} , H_{resp} , G , and H used by GC-AKE as random oracles, and we give proofs of security under both the ROM and QROM. Then, we prove the OW-CPA security of NEV-PKE-OW and NEV-KEM-OW in the standard model. Since NEV-AKE is an instantiation of GC-AKE based on NEV-PKE-OW and NEV-KEM-OW, by combining their proofs in Sec. 8.4, we prove the key indistinguishability of NEV-AKE in the CK^+ model under both the ROM and QROM.

8.1 Security Proof of GC-AKE in the CK^+ model under the ROM

Theorem 2. *Let Π_{PKE} be a δ_P -correct PKE scheme, and Π_{KEM} be a δ_K -correct KEM scheme. Let $\mathcal{A}$ be a key indistinguishability adversary against GC-AKE with Π_{PKE} and Π_{KEM} in the CK^+ model, controlling N users and establishing at most S sessions. Assume that during the running of $\mathcal{A}$, the random oracles H_{init} , H_{resp} , G , and H are queried at most q_I , q_R , q_G , and q_H times, respectively, including queries issued by $\mathcal{A}$ as well as queries issued by other oracles. Then, we can construct an OW-CPA adversary $\mathcal{B}_P$ against Π_{PKE} and an OW-CPA adversary $\mathcal{B}_K$ against Π_{KEM} such that*

$$\begin{aligned}
\text{Adv}_{\text{GC-AKE}}^{\text{KI-CK}^+}(\mathcal{A}) \leq & 2S(SN + 2) \cdot \gamma(\Pi_{PKE}.\text{Enc}) + 4S^3N^2 \cdot \gamma(\Pi_{KEM}.\text{KeyGen}) \\
& + 2S^3N^2 \cdot \gamma(\Pi_{KEM}.\text{Encap}) + 4S^2N^2 \cdot \delta_K + 12S^2N^2(q_G + 1) \cdot \delta_P \\
& + \frac{6S^2N^2(q_R + q_I)}{|\mathcal{M}|} + 2S^2N^2q_H \cdot \text{Adv}_{\Pi_{KEM}}^{\text{OW-CPA}}(\mathcal{B}_K) \\
& + 18S^2N^2(q_G + 2q_H + q_I + q_R) \cdot \text{Adv}_{\Pi_{PKE}}^{\text{OW-CPA}}(\mathcal{B}_P).
\end{aligned}$$

Here $\gamma(\Pi_{PKE}.\text{Enc})$, $\gamma(\Pi_{KEM}.\text{KeyGen})$, and $\gamma(\Pi_{KEM}.\text{Encap})$ are defined in Definition 1, 5. The running times of $\mathcal{B}_P$ and $\mathcal{B}_K$ are roughly the same as those of $\mathcal{A}$.

Proof. In the security game $\text{KeyIND}_{\Pi_{\text{AKE}}}^{A,b}$ (Fig. 4) of the CK^+ model, the adversary $\mathcal{A}$ can control the inputs of oracles INIT and DER_{resp} , but cannot manipulate their internal computations. Hence, $\mathcal{A}$ is unable to directly control the outputs of INIT and DER_{resp} . This fact indicates that if the test session sID^* has a matching session (i.e., $\mathfrak{M}(\text{sID}^*) \neq \emptyset$), sID^* must be executed by honestly running the AKE protocol Π_{AKE} . On the contrary, if no matching session exists for sID^* , $\mathcal{A}$ must have tampered with this test session. That is, at least one of the messages M and M' handled by sID^* is constructed by $\mathcal{A}$ itself. Based on this observation, we decompose $\mathcal{A}$'s advantage according to whether the test session sID^* is tampered with, as detailed below.

$$\begin{aligned} \text{Adv}_{\text{GC-AKE}}^{\text{KI-CK}^+}(\mathcal{A}) &= \left| \Pr \left[1 \leftarrow \text{KeyIND}_{\Pi_{\text{AKE}}}^{A,0} \right] - \Pr \left[1 \leftarrow \text{KeyIND}_{\Pi_{\text{AKE}}}^{A,1} \right] \right| \\ &= \left| \Pr \left[1 \leftarrow \text{KeyIND}_{\Pi_{\text{AKE}}}^{A,0} \wedge \mathfrak{M}(\text{sID}^*) \neq \emptyset \right] \right. \\ &\quad \left. - \Pr \left[1 \leftarrow \text{KeyIND}_{\Pi_{\text{AKE}}}^{A,1} \wedge \mathfrak{M}(\text{sID}^*) \neq \emptyset \right] \right| \end{aligned} \quad (7)$$

$$+ \left| \Pr \left[1 \leftarrow \text{KeyIND}_{\Pi_{\text{AKE}}}^{A,0} \wedge \mathfrak{M}(\text{sID}^*) = \emptyset \right] \right. \\ \left. - \Pr \left[1 \leftarrow \text{KeyIND}_{\Pi_{\text{AKE}}}^{A,1} \wedge \mathfrak{M}(\text{sID}^*) = \emptyset \right] \right|. \quad (8)$$

Since $\text{KeyIND}_{\Pi_{\text{AKE}}}^{A,b}$ only outputs 1 when sID^* is fresh (Definition 11), and a fresh sID^* with $\mathfrak{M}(\text{sID}^*) \neq \emptyset$ means that the adversary $\mathcal{A}$ does not obtain both the long-term secret key of the initiator for sID^* and the internal state of sID^* . Thus, by defining events

sk: $\mathcal{A}$ has obtained the long-term secret key sk_i of the initiator of sID^* ,
st: $\mathcal{A}$ has obtained the internal state $\text{st}[\text{sID}^*]$ of sID^* ,

we can further decompose Eq. (7) as follows:

$$\begin{aligned} \text{Eq. (7)} &= \left| \Pr \left[1 \leftarrow \text{KeyIND}_{\Pi_{\text{AKE}}}^{A,0} \wedge \mathfrak{M}(\text{sID}^*) \neq \emptyset \wedge \neg(sk \wedge st) \right] \right. \\ &\quad \left. - \Pr \left[1 \leftarrow \text{KeyIND}_{\Pi_{\text{AKE}}}^{A,1} \wedge \mathfrak{M}(\text{sID}^*) \neq \emptyset \wedge \neg(sk \wedge st) \right] \right| \\ &\leq \left| \Pr \left[1 \leftarrow \text{KeyIND}_{\Pi_{\text{AKE}}}^{A,0} \wedge \mathfrak{M}(\text{sID}^*) \neq \emptyset \wedge \neg st \right] \right. \\ &\quad \left. - \Pr \left[1 \leftarrow \text{KeyIND}_{\Pi_{\text{AKE}}}^{A,1} \wedge \mathfrak{M}(\text{sID}^*) \neq \emptyset \wedge \neg st \right] \right| \quad (\text{Case 1}) \\ &\quad + \left| \Pr \left[1 \leftarrow \text{KeyIND}_{\Pi_{\text{AKE}}}^{A,0} \wedge \mathfrak{M}(\text{sID}^*) \neq \emptyset \wedge \neg sk \wedge st \right] \right. \\ &\quad \left. - \Pr \left[1 \leftarrow \text{KeyIND}_{\Pi_{\text{AKE}}}^{A,1} \wedge \mathfrak{M}(\text{sID}^*) \neq \emptyset \wedge \neg sk \wedge st \right] \right|. \quad (\text{Case 2}) \end{aligned}$$

As for the Eq. (8), we can decompose it with respect to the role of sID^* :

$$\begin{aligned} \text{Eq. (8)} &= \left| \Pr \left[1 \leftarrow \text{KeyIND}_{\Pi_{\text{AKE}}}^{A,0} \wedge \mathfrak{M}(\text{sID}^*) = \emptyset \wedge \text{role}[\text{sID}^*] = \text{initiator} \right] \right. \\ &\quad \left. - \Pr \left[1 \leftarrow \text{KeyIND}_{\Pi_{\text{AKE}}}^{A,1} \wedge \mathfrak{M}(\text{sID}^*) = \emptyset \wedge \text{role}[\text{sID}^*] = \text{initiator} \right] \right| \quad (\text{Case 3}) \\ &\quad + \left| \Pr \left[1 \leftarrow \text{KeyIND}_{\Pi_{\text{AKE}}}^{A,0} \wedge \mathfrak{M}(\text{sID}^*) = \emptyset \wedge \text{role}[\text{sID}^*] = \text{responder} \right] \right. \\ &\quad \left. - \Pr \left[1 \leftarrow \text{KeyIND}_{\Pi_{\text{AKE}}}^{A,1} \wedge \mathfrak{M}(\text{sID}^*) = \emptyset \wedge \text{role}[\text{sID}^*] = \text{responder} \right] \right|. \quad (\text{Case 4}) \end{aligned} \quad (9)$$

The above equations indicate that if we derive upper bounds for **Case 1** to **Case 4** respectively, we can directly combine these bounds to obtain the upper bound for $\text{Adv}_{\Pi_{\text{AKE}}}^{\text{KI-CK}^+}(\mathcal{A})$. Hence, our subsequent strategy is to derive upper bounds for **Case 1** to **Case 4** based on the OW-CPA security of Π_{PKE} and Π_{KEM} , which correspond following Lemma 4 to Lemma 7.

Lemma 4. *For **Case 1**, where the adversary $\mathcal{A}$ cannot obtain the internal state $\text{st}[\text{sID}^*]$, we have $\text{Case 1} \leq 2S \cdot \gamma(\Pi_{\text{PKE}}.\text{Enc}) + 4S^2N^2 \cdot \delta_{\text{P}} + 2S^2N^2 \cdot \delta_{\text{K}} + 2S^2N^2q_{\text{H}} \cdot \text{Adv}_{\Pi_{\text{KEM}}}^{\text{OW-CPA}}(\mathcal{B}_{\text{K}})$. Here $\mathcal{B}_{\text{K}}$ has roughly the same running time as $\mathcal{A}$.*

Proof (sketch). Since the internal state $\text{st}[\text{sID}^*]$ contains the ephemeral secret key $\widetilde{sk}$ generated by the initiator, in **Case 1** this secret key remains hidden from the adversary $\mathcal{A}$ even if $\mathcal{A}$ is allowed to query oracles REVEAL, REV-STATE, CORRUPT. This implies we can equivalently replace the ciphertext $\widetilde{c}$ within M' processed by the test session with the challenge ciphertext c^* provided by the OW-CPA challenger of Π_{KEM} , and replace the $\widetilde{K'}$ used to generate $K'_{i,j}$ by the key K^* encapsulated by c^{*11} . At this point, when $b = 0$, the oracle TEST in game $\text{KeyIND}_{\Pi_{\text{AKE}}}^{\mathcal{A},b}$ returns the key $K'_{i,j} = \text{H}(m'_i, m_j, K^*, i, j, M, M')$ produced by the test session; when $b = 1$, the oracle TEST outputs a uniformly random value of $\mathcal{K}$. Hence, by simulating H on-the-fly, we can construct an OW-CPA adversary $\mathcal{B}_{\text{K}}$ and convert $\mathcal{A}$'s distinguishing advantage into $\mathcal{B}_{\text{K}}$'s guessing advantage, thus completing the proof. Specifically, $\mathcal{B}_{\text{K}}$ invokes $\mathcal{A}$, simulates the environment required by it, uses list L_{H} to simulate H on-the-fly, and finally uniformly samples an element from L_{H} to form its guess for K^* . The full proof is given in Appendix A.1. $\square$

Lemma 5. *For **Case 2**, where the adversary $\mathcal{A}$ cannot obtain the long-term secret key of the initiator for sID^* , we have*

$$\begin{aligned} \text{Case 2} \leq & 2S \cdot \gamma(\Pi_{\text{PKE}}.\text{Enc}) + 2S^3N^2 \cdot (\gamma(\Pi_{\text{KEM}}.\text{KeyGen}) + \gamma(\Pi_{\text{KEM}}.\text{Encap})) \\ & + 4S^2N^2(q_{\text{G}} + 1) \cdot \delta_{\text{P}} + 2S^2N^2 \cdot \delta_{\text{K}} + \frac{2S^2N^2(q_{\text{R}} + q_{\text{I}})}{|\mathcal{M}|} \\ & + 2S^2N^2(q_{\text{G}} + q_{\text{H}}) \cdot \text{Adv}_{\Pi_{\text{PKE}}}^{\text{OW-CPA}}(\mathcal{B}). \end{aligned}$$

Here $\mathcal{B}$ has roughly the same running time as $\mathcal{A}$.

Proof (sketch). The proof of this lemma is similar to that of Lemma 4. The key difference is that in this proof the long-term secret key sk_{i^*} belonging to the initiator of sID^* is hidden from the adversary $\mathcal{A}$. Hence, we need to replace the c_{i^*} inside M' with the challenge ciphertext c^* provided by the OW-CPA challenger of Π_{PKE} . We point out that, unlike an ephemeral secret key $\widetilde{sk}$ which is only used for a single session, a long-term secret key sk_{i^*} can be used to create sessions outside the test session. To simulate these additional sessions, the reduction algorithm $\mathcal{B}$ in this proof has to simulate the sk_{i^*} -based decryption operations in those sessions while only possessing the corresponding public key pk_{i^*} . Nevertheless, our scheme derandomizes the encryption of m_{i^*} , performs a re-encryption check when

¹¹ Here we implicitly require the decapsulation of c^* to be correct, i.e., $\Pi_{\text{KEM}}.\text{Decap}(c^*) = K^*$, which holds up to an error term δ_{K} .

decrypting c_{i^*} , and uses an implicit-rejection method to generate the session key whenever decryption fails. We can thus apply the domain-separation technique [43,38] similarly to simulate all decryption operations using only pk_{i^*} . The full proof of this lemma is given in Appendix A.2. $\square$

Lemma 6. *For Case 3, where $\mathfrak{M}(\text{sID}^*) = \emptyset$ and $\text{role}[\text{sID}^*] = \text{initiator}$, we have*

$$\begin{aligned} \text{Case 3} \leq & 2S^2N \cdot \gamma(\Pi_{\text{KEM}}.\text{KeyGen}) + \frac{2SN(q_R + q_I)}{|\mathcal{M}|} \\ & + 4SN(q_G + 1) \cdot \delta_P + 8SN(q_G + 2q_H + q_I) \cdot \text{Adv}_{\Pi_{\text{PKE}}}^{\text{OW-CPA}}(\mathcal{B}). \end{aligned}$$

Here $\mathcal{B}$ has roughly the same running time as $\mathcal{A}$.

Proof (sketch). Denote j^* as the index of the test session sID^* 's peer. Then, by Definition 11, the event that $\mathfrak{M}(\text{sID}^*) = \emptyset$, $\text{role}[\text{sID}^*] = \text{initiator}$, and sID^* is fresh implies that the adversary $\mathcal{A}$ never queries CORRUPT with j^* to get the long-term secret key sk_{j^*} and never obtains the value m_{j^*} stored in the internal state $\text{st}[\text{sID}^*]$. Hence, in Case 3, the value m_{j^*} used to generate $K'_{i,j} = H(m'_i, m_{j^*}, \widetilde{K'}, i, j, M, M')$ remains hidden from the adversary $\mathcal{A}$. Now, similar to the proof of Lemma 5, we can prove this lemma by using the domain-separation technique [43,38]. The full proof is given in Appendix A.3. $\square$

Lemma 7. *For Case 4, where $\mathfrak{M}(\text{sID}^*) = \emptyset$ and $\text{role}[\text{sID}^*] = \text{responder}$, we have*

$$\begin{aligned} \text{Case 4} \leq & 2S^2N \cdot \gamma(\Pi_{\text{PKE}}.\text{Enc}) + \frac{2SN(q_R + q_I)}{|\mathcal{M}|} \\ & + 4SN(q_G + 1) \cdot \delta_P + 8SN(q_G + q_H + q_R) \cdot \text{Adv}_{\Pi_{\text{PKE}}}^{\text{OW-CPA}}(\mathcal{B}). \end{aligned}$$

Here $\mathcal{B}$ has roughly the same running time as $\mathcal{A}$.

Proof (sketch). The proof of this lemma is similar to that of Lemma 6. The key difference is that in this proof the long-term secret key sk_{i^*} belonging to the peer of sID^* is hidden from the adversary $\mathcal{A}$. Hence, we need to replace the c_{i^*} inside M' with the challenge ciphertext c^* provided by the OW-CPA challenger of Π_{PKE} . The full proof of this lemma is given in Appendix A.4. $\square$

At this point, by combining Lemma 4, Lemma 5, Lemma 6, Lemma 7, and folding the OW-CPA adversaries against Π_{PKE} into one adversary $\mathcal{B}_P$, we obtain

$$\begin{aligned} \text{Adv}_{\text{GC-AKE}}^{\text{KI-CK}^+}(\mathcal{A}) \leq & 2S(SN + 2) \cdot \gamma(\Pi_{\text{PKE}}.\text{Enc}) + 4S^3N^2 \cdot \gamma(\Pi_{\text{KEM}}.\text{KeyGen}) \\ & + 2S^3N^2 \cdot \gamma(\Pi_{\text{KEM}}.\text{Encap}) + 4S^2N^2 \cdot \delta_K + 12S^2N^2(q_G + 1) \cdot \delta_P \\ & + \frac{6S^2N^2(q_R + q_I)}{|\mathcal{M}|} + 2S^2N^2q_H \cdot \text{Adv}_{\Pi_{\text{KEM}}}^{\text{OW-CPA}}(\mathcal{B}_K) \\ & + 18S^2N^2(q_G + 2q_H + q_I + q_R) \cdot \text{Adv}_{\Pi_{\text{PKE}}}^{\text{OW-CPA}}(\mathcal{B}_P). \end{aligned}$$

This completes the proof of Theorem 2. $\square$

8.2 Security Proof of GC-AKE in the CK⁺ model under the QROM

In this section, we prove the QROM version of Theorem 2, in which the adversary $\mathcal{A}$ has quantum access to the random oracles. Our proof strategy is identical to that of Theorem 2: we partition $\mathcal{A}$ into four cases based on the set $\mathfrak{M}(\text{slD}^*)$, and then derive an upper bound on $\mathcal{A}$'s advantage for each case. In fact, the key point here is how to respectively establish the QROM versions of Lemma 4, Lemma 5, Lemma 6, and Lemma 7.

Theorem 3. *Let Π_{PKE} be a δ_{P} -correct PKE scheme, and Π_{KEM} be a δ_{K} -correct KEM scheme. Let $\mathcal{A}$ be a key indistinguishability adversary against GC-AKE with Π_{PKE} and Π_{KEM} in the CK⁺ model, controlling N users and establishing at most S sessions. Assume that during the running of $\mathcal{A}$, the random oracles H_{init} , H_{resp} , G , and H are queried at most q_{I} , q_{R} , q_{G} , and q_{H} times, respectively, including quantum queries issued by $\mathcal{A}$ as well as classical queries issued by other oracles. Then, we can construct an OW-CPA adversary $\mathcal{B}_{\text{P}}$ against Π_{PKE} and an OW-CPA adversary $\mathcal{B}_{\text{K}}$ against Π_{KEM} such that*

$$\begin{aligned} \text{Adv}_{\text{GC-AKE}}^{\text{KI-CK}^+}(\mathcal{A}) \leq & 2S(SN + 2) \cdot \gamma(\Pi_{\text{PKE}}.\text{Enc}) + 4S^3N^2 \cdot \gamma(\Pi_{\text{KEM}}.\text{KeyGen}) \\ & + 2S^3N^2 \cdot \gamma(\Pi_{\text{KEM}}.\text{Encap}) + 4S^2N^2 \cdot \delta_{\text{K}} + 96S^2N^2(q_{\text{G}} + 3) \cdot \delta_{\text{P}} \\ & + \frac{8S^2N^2(q_{\text{R}} + q_{\text{I}})}{\sqrt{|\mathcal{M}|}} + 8S^2N^2(q_{\text{H}} + 1) \cdot \sqrt{\text{Adv}_{\Pi_{\text{KEM}}}^{\text{OW-CPA}}(\mathcal{B}_{\text{K}})} \\ & + 52S^2N^2(q_{\text{G}} + 2q_{\text{H}} + q_{\text{I}} + q_{\text{R}} + 1)^{1.5} \cdot \sqrt{\text{Adv}_{\Pi_{\text{PKE}}}^{\text{OW-CPA}}(\mathcal{B}_{\text{P}})}. \end{aligned}$$

Here $\gamma(\Pi_{\text{PKE}}.\text{Enc})$, $\gamma(\Pi_{\text{KEM}}.\text{KeyGen})$, and $\gamma(\Pi_{\text{KEM}}.\text{Encap})$ are defined in Definition 1, 5. The running times of $\mathcal{B}_{\text{P}}$ and $\mathcal{B}_{\text{K}}$ are roughly the same as those of $\mathcal{A}$.

Proof. The full proof of this theorem is given in Appendix A.5. $\square$

8.3 Security Proof of NEV-PKE-OW and NEV-KEM-OW

Below, we prove the OW-CPA security of the NEV-PKE-OW scheme and the NEV-KEM-OW scheme in the standard model.

Theorem 4. *Let $n, q, k, \eta \in \mathbb{Z}$ and $v = 1 - x^{n/k}$, $\bar{v} = (1 + x^{n/k} + \dots + x^{(k-1)n/k}) \in R_q$ be defined as above. Let χ_f, χ_g, χ_r be three probability distributions over R_q . Let $\mathcal{A}$ be an OW-CPA adversary against the NEV-PKE-OW scheme. Then, we can construct two new adversaries $\mathcal{B}_1$ and $\mathcal{B}_2$ such that*

$$\text{Adv}_{\text{NEV-PKE-OW}}^{\text{OW-CPA}}(\mathcal{A}) = \text{Adv}_{n,q,\chi_f,\chi_g,v}^{\text{DNTRU}}(\mathcal{B}_1) + \text{Adv}_{n,q,\chi_r,B_\eta,\bar{v}}^{\text{sspRLWE}}(\mathcal{B}_2).$$

The running times of $\mathcal{B}_1$ and $\mathcal{B}_2$ are roughly the same as those of $\mathcal{A}$.

Proof. We introduce a sequence of games as shown in Fig. 7 to prove this theorem.

Game $\mathbf{G}_0$: This game is the OW-CPA security game of NEV-PKE-OW with the adversary $\mathcal{A}$. Obviously, we have

$$\text{Adv}_{\text{NEV-PKE-OW}}^{\text{OW-CPA}}(\mathcal{A}) = \Pr[1 \leftarrow \mathbf{G}_0]. \quad (10)$$

Game $\mathbf{G}_0$	Game $\mathbf{G}_1$	Game $\mathbf{G}_2$	Game $\mathbf{G}_3$
1. $f' \leftarrow \chi_f, g \leftarrow \chi_g$ $f = f' + v^{-1} \in R_q^*$ $g \in R_q^*$ $(pk, sk) = (h = g/f, f)$	1. $f' \leftarrow \chi_f, g \leftarrow \chi_g$ $f = vf' + 1 \in R_q^*$ $g \in R_q^*$ $(pk, sk) = (h = vg/f, f)$	1. $f' \leftarrow \chi_f, g \leftarrow \chi_g, u \leftarrow R_q^*$ $f = vf' + 1 \in R_q^*$ $g \in R_q^*$ $(pk, sk) = (h = u, f)$	1. $f' \leftarrow \chi_f, g \leftarrow \chi_g, u \leftarrow R_q^*$ $f = vf' + 1 \in R_q^*$ $g \in R_q^*$ $(pk, sk) = (h = u, f)$
2. $M^* \leftarrow \{0, 1\}^{n/k}$ $r_1^* \leftarrow \{0, 1\}^n, r_2^* \leftarrow \chi_r$ $m^* \leftarrow \text{Pt2noise}(M^*, \eta, r_1)$ $c^* = hr_2^* + m^*$	2. $M^* \leftarrow \{0, 1\}^{n/k}$ $r_1^* \leftarrow \{0, 1\}^n, r_2^* \leftarrow \chi_r$ $m^* \leftarrow \text{Pt2noise}(M^*, \eta, r_1)$ $c^* = hr_2^* + m^*$	2. $M^* \leftarrow \{0, 1\}^{n/k}$ $r_1^* \leftarrow \{0, 1\}^n, r_2^* \leftarrow \chi_r$ $m^* \leftarrow \text{Pt2noise}(M^*, \eta, r_1)$ $c^* = hr_2^* + m^*$	2. $m^* \leftarrow B_\eta$ $r_2^* \leftarrow \chi_r$ $M^* \leftarrow \text{Ext}(\bar{v}m^* \bmod 2)$ $c^* = hr_2^* + m^*$
3. $M' \leftarrow \mathcal{A}(pk, c^*)$	3. $M' \leftarrow \mathcal{A}(pk, c^*)$	3. $M' \leftarrow \mathcal{A}(pk, c^*)$	3. $M' \leftarrow \mathcal{A}(pk, c^*)$
4. return $\llbracket M' = M^* \rrbracket$	4. return $\llbracket M' = M^* \rrbracket$	4. return $\llbracket M' = M^* \rrbracket$	4. return $\llbracket M' = M^* \rrbracket$

Fig. 7: Games $\mathbf{G}_0$ to $\mathbf{G}_3$ in the proof of Theorem 4.

Game $\mathbf{G}_1$: This game is the same as game $\mathbf{G}_0$, except that f is computed as $vf' + 1$ and h is computed as vg/f . Since $v \in R_q^*$ and the adversary $\mathcal{A}$ only gets $pk(=h)$, this is only a conceptual change, and thus

$$\Pr[1 \leftarrow \mathbf{G}_0] = \Pr[1 \leftarrow \mathbf{G}_1]. \quad (11)$$

Game $\mathbf{G}_2$: This game is the same as game $\mathbf{G}_1$, except that the public key pk is replaced with a uniformly random value over R_q^* .

Let $\mathcal{B}_1$ be a new adversary against the $\text{DNTRU}_{n,q,\chi_f,\chi_g,v}$ assumption, which is defined as follows: after getting the instance h' (g/f or a uniformly random value over R_q^*), $\mathcal{B}_1$ runs game $\mathbf{G}_1$ from the second step, replaces the public key pk with vh' , and finally outputs $\llbracket M' = M^* \rrbracket$. Obviously, $\mathcal{B}_1$ runs game $\mathbf{G}_1$ when $h' = g/f$, and runs game $\mathbf{G}_2$ when h' is a uniformly random value over R_q^* . The running time of $\mathcal{B}_1$ is about that of $\mathcal{A}$ and

$$|\Pr[1 \leftarrow \mathbf{G}_1] - \Pr[1 \leftarrow \mathbf{G}_2]| = \text{Adv}_{n,q,\chi_f,\chi_g,v}^{\text{DNTRU}}(\mathcal{B}_1). \quad (12)$$

Game $\mathbf{G}_3$: This game is the same as game $\mathbf{G}_2$, except that the m^* is first sampled according to B_η and then M^* is computed as $\text{Ext}(\bar{v}m^* \bmod 2)$. Here $\text{Ext} : R_2 \rightarrow \{0, 1\}^{n/k}$ is a function that returns the first n/k coefficients of its input.

According to Lemma 3, the (m^*, M^*) of game $\mathbf{G}_2$ has the same distribution as that of game $\mathbf{G}_3$, and hence we have

$$\Pr[1 \leftarrow \mathbf{G}_2] = \Pr[1 \leftarrow \mathbf{G}_3]. \quad (13)$$

Note that the element $\bar{v}$ we employ satisfies that $\bar{v}m^* \bmod 2$ is essentially a polynomial with coefficients consisting of k copies of $\text{Ext}(\bar{v}m^* \bmod 2)$. Hence we can directly derive $\bar{v}m^* \bmod 2$ after getting $\text{Ext}(\bar{v}m^* \bmod 2)$. At this point, we define a new adversary $\mathcal{B}_2$ against the $\text{sspRLWE}_{n,q,\chi_r,B_\eta,\bar{v}}$ assumption as follows: after getting the instance $(h, hr_2^* + m^*)$, $\mathcal{B}_2$ runs game $\mathbf{G}_3$ from the third step, and generates its guess for $\bar{v}m^* \bmod 2$ using the output M' from $\mathcal{A}$. Now, we have

$$\Pr[1 \leftarrow \mathbf{G}_3] = \text{Adv}_{n,q,\chi_r,B_\eta,\bar{v}}^{\text{sspRLWE}}(\mathcal{B}_2). \quad (14)$$

By combining Eq. (10) to Eq. (14), we obtain

$$\text{Adv}_{\text{NEV-PKE-OW}}^{\text{OW-CPA}}(\mathcal{A}) = \text{Adv}_{n,q,\chi_f,\chi_g,v}^{\text{DNTRU}}(\mathcal{B}_1) + \text{Adv}_{n,q,\chi_r,B_\eta,\bar{v}}^{\text{sspRLWE}}(\mathcal{B}_2).$$

This completes the proof of Theorem 4. $\square$

Remark 8. Note that game $\mathbf{G}_3$ requires u to be invertible; thus, the sspRLWE assumption used above has an additional condition that h is invertible. For our parameter sets, the probability that h is non-invertible for a uniformly chosen $h \xleftarrow{\$} R_q$ is significant. Consequently, the condition introduced here does not make much difference to the security of the sspRLWE assumption in practice.

Theorem 5. *Let $n, q, k, \eta \in \mathbb{Z}$ and $v = 1 - x^{n/k}$, $\bar{v} = (1 + x^{n/k} + \dots + x^{(k-1)n/k}) \in R_q$ be defined as above. Let χ_f, χ_g, χ_r be three probability distributions over R_q . Let $\mathcal{A}$ be an OW-CPA adversary against the NEV-KEM-OW scheme. Then, we can construct two new adversaries $\mathcal{B}_1$ and $\mathcal{B}_2$ such that*

$$\text{Adv}_{\text{NEV-KEM-OW}}^{\text{OW-CPA}}(\mathcal{A}) = \text{Adv}_{n,q,\chi_f,\chi_g,v}^{\text{DNTRU}}(\mathcal{B}_1) + \text{Adv}_{n,q,\chi_r,B_\eta,\bar{v}}^{\text{sspRLWE}}(\mathcal{B}_2).$$

The running times of $\mathcal{B}_1$ and $\mathcal{B}_2$ are roughly the same as those of $\mathcal{A}$.

Proof. The proof is totally similar to that of Theorem 4. We omit the details here. $\square$

8.4 Security Proof of NEV-AKE in the CK^+ model

Recall that NEV-AKE is obtained by integrating NEV-KEM-OW and NEV-PKE-OW into the generic construction GC-AKE. Hence, by combining Theorem 2, Theorem 4, Theorem 5, and folding the two DNTRU (resp. sspRLWE) adversaries into one DNTRU (resp. sspRLWE) adversary, we obtain the following theorem, which proves the key indistinguishability of NEV-AKE in the CK^+ model under the ROM.

Theorem 6. *Let $n, q, k, \eta \in \mathbb{Z}$ and $v = 1 - x^{n/k}$, $\bar{v} = (1 + x^{n/k} + \dots + x^{(k-1)n/k}) \in R_q$ be defined as above. Let χ_f, χ_g, χ_r be three probability distributions over R_q . Let $\mathcal{A}$ be a key indistinguishability adversary against the NEV-AKE in the CK^+ model, controlling N users and establishing at most S sessions. Assume that during the running of $\mathcal{A}$, the random oracles H_{init} , H_{resp} , G , and H are queried at most q_I , q_R , q_G , and q_H times, respectively, including queries issued by $\mathcal{A}$ as well as queries issued by other oracles. Then, we can construct a DNTRU adversary $\mathcal{B}_1$ and a sspRLWE adversary $\mathcal{B}_2$ such that*

$$\begin{aligned} \text{Adv}_{\text{NEV-AKE}}^{\text{KI-CK}^+}(\mathcal{A}) \leq & 2S(SN + 2) \cdot \gamma(\text{NEV-PKE-OW.Enc}) \\ & + 4S^3N^2 \cdot \gamma(\text{NEV-KEM-OW.KeyGen}) \\ & + 2S^3N^2 \cdot \gamma(\text{NEV-KEM-OW.Encap}) + \frac{6S^2N^2(q_R + q_I)}{|\mathcal{M}|} \\ & + 4S^2N^2 \cdot \delta_K + 12S^2N^2(q_G + 1) \cdot \delta_P \\ & + 40S^2N^2(q_G + 2q_H + q_I + q_R) \cdot \text{Adv}_{n,q,\chi_f,\chi_g,v}^{\text{DNTRU}}(\mathcal{B}_1) \\ & + 40S^2N^2(q_G + 2q_H + q_I + q_R) \cdot \text{Adv}_{n,q,\chi_r,B_\eta,\bar{v}}^{\text{sspRLWE}}(\mathcal{B}_2). \end{aligned}$$

Here δ_P (resp. δ_K) denotes the decryption failure rate of NEV-PKE-OW (resp. NEV-KEM-OW). The values $\gamma(\text{NEV-PKE-OW.Enc})$, $\gamma(\text{NEV-KEM-OW.KeyGen})$, and $\gamma(\text{NEV-KEM-OW.Encap})$ are defined in Definition 1, 5. The running times of $\mathcal{B}_1$ and $\mathcal{B}_2$ are roughly the same as those of $\mathcal{A}$.

Similarly, by combining Theorem 3, Theorem 4, and Theorem 5, we obtain the following theorem, which proves the key indistinguishability of NEV-AKE in the CK^+ model under the QROM.

Theorem 7. *Let $n, q, k, \eta \in \mathbb{Z}$ and $v = 1 - x^{n/k}$, $\bar{v} = (1 + x^{n/k} + \dots + x^{(k-1)n/k}) \in R_q$ be defined as above. Let χ_f, χ_g, χ_r be three probability distributions over R_q . Let $\mathcal{A}$ be a key indistinguishability adversary against the NEV-AKE in the CK^+ model, controlling N users and establishing at most S sessions. Assume that during the running of $\mathcal{A}$, the random oracles H_{init} , H_{resp} , G , and H are queried at most q_1 , q_R , q_G , and q_H times, respectively, including quantum queries issued by $\mathcal{A}$ as well as classical queries issued by other oracles. Then, we can construct a DNTRU adversary $\mathcal{B}_1$ and a sspRLWE adversary $\mathcal{B}_2$ such that*

$$\begin{aligned} \text{Adv}_{\text{NEV-AKE}}^{\text{KI-CK}^+}(\mathcal{A}) \leq & 2S(SN + 2) \cdot \gamma(\text{NEV-PKE-OW.Enc}) \\ & + 4S^3N^2 \cdot \gamma(\text{NEV-KEM-OW.KeyGen}) \\ & + 2S^3N^2 \cdot \gamma(\text{NEV-KEM-OW.Encap}) + \frac{8S^2N^2(q_R + q_1)}{\sqrt{|\mathcal{M}|}} \\ & + 4S^2N^2 \cdot \delta_K + 96S^2N^2(q_G + 3) \cdot \delta_P \\ & + 120S^2N^2(q_G + 2q_H + q_1 + q_R + 1)^{1.5} \cdot \sqrt{\text{Adv}_{n,q,\chi_f,\chi_g,v}^{\text{DNTRU}}(\mathcal{B}_1)} \\ & + 120S^2N^2(q_G + 2q_H + q_1 + q_R + 1)^{1.5} \cdot \sqrt{\text{Adv}_{n,q,\chi_r,B_{\eta},\bar{v}}^{\text{sspRLWE}}(\mathcal{B}_2)}. \end{aligned}$$

Here δ_P (resp. δ_K) denotes the decryption failure rate of NEV-PKE-OW (resp. NEV-KEM-OW). The values $\gamma(\text{NEV-PKE-OW.Enc})$, $\gamma(\text{NEV-KEM-OW.KeyGen})$, and $\gamma(\text{NEV-KEM-OW.Encap})$ are defined in Definition 1, 5. The running times of $\mathcal{B}_1$ and $\mathcal{B}_2$ are roughly the same as those of $\mathcal{A}$.

In the above two theorems, $\gamma(\text{NEV-PKE-OW.Enc})$, $\gamma(\text{NEV-KEM-OW.KeyGen})$, and $\gamma(\text{NEV-KEM-OW.Encap})$ all appear in the security bounds. We now clarify that all these three values are negligible.

- $\gamma(\text{NEV-PKE-OW.Enc})$: By Definition 1 and the construction of NEV-PKE-OW, we need to estimate the probability of

$$hr + \text{Pt2noise}(M, \eta) = hr' + \text{Pt2noise}(M', \eta),$$

where $r, r' \xleftarrow{\$} \chi_r$, $M, M' \xleftarrow{\$} \{0, 1\}^{n/k}$, and h is the public key sampled by NEV-PKE-OW.KeyGen. In particular, h is an invertible element in R_q . Hence, the above equation implies

$$r - r' = h^{-1}(\text{Pt2noise}(M', \eta) - \text{Pt2noise}(M, \eta)),$$

For the distribution χ_r used in this document (Table 2), the $r - r'$ still has high min-entropy. Hence, the probability that the above equation holds is negligible, which further implies that value $\gamma(\text{NEV-PKE-OW.Enc})$ is also negligible.

- $\gamma(\text{NEV-KEM-OW.KeyGen})$: The negligibility of this value is straightforward, since f' sampled from χ_f and g sampled from χ_g both have high min-entropy.
- $\gamma(\text{NEV-KEM-OW.Encap})$: By a very similar derivation to $\gamma(\text{NEV-PKE-OW.Enc})$, the $\gamma(\text{NEV-KEM-OW.Encap})$ here is also negligible.

For the sake of simplicity, we omit detailed estimation of $\gamma(\text{NEV-PKE-OW.Enc})$, $\gamma(\text{NEV-KEM-OW.KeyGen})$, $\gamma(\text{NEV-KEM-OW.Encap})$ for each parameter set of NEV.

9 Known Attacks

As shown in Sec. 8, the NEV-AKE protocol is provably secure under the hardness of the NTRU and sspRLWE problems, in both the classical and quantum random oracle models. Leveraging the (tight) reduction from RLWE to sspRLWE presented in Sec. 2.7, together with the fact that NTRU can be viewed as a special case of RLWE, we first examine the most effective known attacks against RLWE problems, and then discuss how these attacks can be adapted to NTRU and sspRLWE. Since the state-of-the-art attacks on RLWE do not exploit the lattice structure, we begin by considering the best known attacks against Asymmetric Learning With Errors (ALWE) [72].

9.1 Attacks against ALWE

Solvers for ALWE mainly include primal attacks, dual attacks (against the underlying lattice problems) and direct solving algorithms such as BKW and Arora-Ge [4]. BKW and Arora-Ge attacks need sub-exponentially (or even exponentially) many samples, and thus they are not relevant to the public-key cryptography scenario where only a restricted amount of samples are available. Therefore, for analyzing and evaluating practical lattice-based crypto-systems, we typically consider primal attacks and dual attacks.

Since any bounded centrally symmetric distribution can be regarded as subgaussian for a certain parameter, for simplicity and without loss of generality, we consider the case that secret vector and error vector in $\text{ALWE}_{n,q,m,\alpha_1,\alpha_2}$ are sampled from subgaussians with parameters α_1 and α_2 respectively. Formally, the problem is to recover $\mathbf{s}$ from samples

$$(\mathbf{A}, \mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e}) \in \mathbb{Z}_q^{m \times n} \times \mathbb{Z}_q^m,$$

where $\mathbf{A} \xleftarrow{\$} \mathbb{Z}_q^{m \times n}$, $\mathbf{s} \leftarrow \chi_{\alpha_1}^n$ and $\mathbf{e} \leftarrow \chi_{\alpha_2}^m$. In the following, we first consider the primal/dual attacks adapted to the asymmetric setting in [72], and then consider a variant of the MATZOV attack in [15] also adapted to the asymmetric setting.

Primal Attack. The basic intuition behind primal attack is to convert ALWE problem to the corresponding bounded distance decoding (BDD) problem or shortest vector problem by embedding. First, define a m -dimensional lattice

$$\Lambda = \{\mathbf{y} \in \mathbb{Z}^m \mid \mathbf{y} = \mathbf{A}\mathbf{x} \bmod q, \mathbf{x} \in \mathbb{Z}_q^n\}.$$

For $m > n$ the $m \times n$ matrix $\mathbf{A}$ has full rank with high probability. Assume that the first n rows of $\mathbf{A}$ are independent (otherwise swap the rows to make it), and denote by $\mathbf{A}_1 \in \mathbb{Z}_q^{n \times n}$ the submatrix by keeping the first n rows of $\mathbf{A}$, i.e., $\mathbf{A} = \begin{pmatrix} \mathbf{A}_1 \\ \mathbf{A}_2 \end{pmatrix}$. Denote $\mathbf{A}_1$'s inverse by $\mathbf{A}_1^{-1} \in \mathbb{Z}_q^{n \times n}$, and let $\mathbf{A}' = \begin{pmatrix} \mathbf{I}_n \\ \mathbf{A}_2 \mathbf{A}_1^{-1} \end{pmatrix}$. Then, we have

$$\begin{aligned} \Lambda &= \{\mathbf{y} | \mathbf{y} = \mathbf{A}\mathbf{x} \bmod q, \mathbf{x} \in \mathbb{Z}_q^n\} \\ &= \{\mathbf{y} | \mathbf{y} = \mathbf{A}'\mathbf{x} \bmod q, \mathbf{x} \in \mathbb{Z}_q^n\} \subset \mathbb{Z}^m, \end{aligned}$$

and the columns of the following matrix

$$\mathbf{B} = \begin{pmatrix} \mathbf{I}_n & 0 \\ \mathbf{A}_2 \mathbf{A}_1^{-1} & q\mathbf{I}_{m-n} \end{pmatrix} \in \mathbb{Z}^{m \times m}$$

constitutes a basis of Λ . It is easy to see $\det(\Lambda) = q^{m-n}$. We have by $\mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e} \bmod q$ that the distance between vector $\mathbf{b}$ and lattice Λ is $\|\mathbf{e}\|$. If we could find the closest lattice point $\mathbf{u} \in \Lambda$ to $\mathbf{b}$, then we have $\mathbf{e} = \mathbf{b} - \mathbf{u}$ with high probability. In other words, the problem can be reduced to bounded distance decoding (BDD) problem on Λ , and then one applies the nearest planes algorithm [46] to solve. Furthermore, using Kannan's embedding [44,2] one can reduce BDD to unique shortest vector problem (uSVP). In detail, consider lattice Λ' spanned by the column vectors of the following matrix:

$$\mathbf{B}' = \begin{pmatrix} \mathbf{I}_n & 0 & \mathbf{b} \\ \mathbf{A}_2 \mathbf{A}_1^{-1} & q\mathbf{I}_{m-n} & t \\ 0 & 0 & t \end{pmatrix} \in \mathbb{Z}^{(m+1) \times (m+1)},$$

where $t \in \mathbb{Z}$ is an adjustable parameter. Theoretically, the algorithm works best when $t = \|\mathbf{e}\|$. However, empirical experiments suggest that best performance is achieved when $t = 1$, which is the typical value we choose. In this case, we have that $\mathbf{v} = (\mathbf{e}^T, 1)^T \in \mathbb{Z}^{m+1}$ is the unique shortest vector in Λ' with high probability. To summarize, we work on uSVP problem in Λ' to find out the error vector $\mathbf{e} \in \mathbb{Z}^m$, which is in turn used to solve the system of linear congruences to recover $\mathbf{s} \in \mathbb{Z}_q^n$, i.e., $\mathbf{A}\mathbf{s} = \mathbf{b} - \mathbf{e}$. Besides, since every element in vector $\mathbf{e}$ is sampled from subgaussian χ_{α_2} , we have $\|\mathbf{v}\| \approx \|\mathbf{e}\| \approx \alpha_2 \sqrt{m}$, namely, $\mathbf{v}$ is of small norm.

One can also make use of the fact that $\mathbf{s} \stackrel{\$}{\leftarrow} \chi_{\alpha_1}^n$ has small norm in practical lattice-based cryptosystems by considering a variant of the LWE embedding [7,5]. Formally, let $c = \alpha_2/\alpha_1$ and consider lattice

$$\Lambda = \left\{ \mathbf{v} = \begin{pmatrix} c\mathbf{x} \\ \mathbf{y} \\ \alpha_2 z \end{pmatrix} \in \mathbb{R}^{n+m+1} \left| (\mathbf{A}\|\mathbf{I}_m\| - \mathbf{b})\mathbf{u} = 0 \bmod q, \mathbf{u} = \begin{pmatrix} \mathbf{x} \\ \mathbf{y} \\ z \end{pmatrix} \in \mathbb{Z}^{n+m+1} \right. \right\},$$

Then the column vectors of matrix

$$\mathbf{B} = \begin{pmatrix} c\mathbf{I}_n & 0 & 0 \\ -\mathbf{A} & q\mathbf{I}_m & \mathbf{b} \\ 0 & 0 & \alpha_2 \end{pmatrix} \in \mathbb{R}^{(n+m+1) \times (n+m+1)},$$

constitutes a basis of Λ . We observe that Λ is of dimension $m + n + 1$, $\det(\Lambda) = \alpha_2 c^n q^m$, and that $\mathbf{v} = (c\mathbf{s}^T, \mathbf{e}^T, \alpha_2)^T \in \mathbb{R}^{m+n+1}$ is a short vector in Λ (and $\|\mathbf{v}\| \approx \alpha_2 \sqrt{n + m + 1}$). Therefore, we recover $\mathbf{s}$ by solving uSVP on lattice Λ .

The Cost of Primal Attack. As shown above, the computational costs of primal attack is mainly dominated by the cost of solving uSVP problem. The best known SVP solver is the BKZ- b lattice basis reduction and its variants. Given a d -dimensional basis as input, the algorithm translates the lattice basis reduction problem to the SVP problem on b -dimensional sublattice ($b < d$). BKZ- b repeats the SVP solving procedure $O(d)$ times (the actual number may depend on the specific algorithm used) on different b -dimensional sublattices to reduce the norm of the basis (of the original d -dimensional basis). Therefore, the cost of BKZ- b basis reduction is mainly decided by SVP solving algorithm for b -dimensional lattice.

Assume that the reduced basis obtained from running BKZ- b on d -dimensional lattice basis $\mathbf{B} \in \mathbb{Z}^{d \times d}$ satisfies the Geometry Series Assumption (GSA), which is the best scenario for adversary. Let $\hat{\mathbf{B}} = (\hat{\mathbf{b}}_1, \dots, \hat{\mathbf{b}}_d)$ be a reduced basis, then we have

$$\|\hat{\mathbf{b}}_1\| = \delta^d \det(\Lambda)^{1/d}, \quad \|\hat{\mathbf{b}}_i^*\| = \delta^{-2d(i-1)/(d-1)} \|\hat{\mathbf{b}}_1\|,$$

where $\delta = ((\pi b)^{1/b} \cdot b / 2\pi e)^{\frac{1}{2(b-1)}} [18]$, $\hat{\mathbf{B}}^* = (\hat{\mathbf{b}}_1^*, \dots, \hat{\mathbf{b}}_d^*)$ is the orthogonalized matrix of $\hat{\mathbf{B}}$ (i.e., $\hat{\mathbf{b}}_1 = \hat{\mathbf{b}}_1^*$). Specifically, the literatures [5, 2] show that when the norm of the projection of the unique shortest vector $\mathbf{v}$ into the space spanned by the last b orthogonal vectors $(\hat{\mathbf{b}}_{d-b+1}^*, \dots, \hat{\mathbf{b}}_d^*)$ is less than $\|\hat{\mathbf{b}}_{d-b+1}^*\|$, then the BKZ- b lattice basis reduction recovers $\mathbf{v}$.

The norm of $\mathbf{v}$'s projection into the space spanned by b orthogonal vectors $(\hat{\mathbf{b}}_{d-b+1}^*, \dots, \hat{\mathbf{b}}_d^*)$ approximately equals $\ell \sqrt{b/d}$, where $\ell = \|\mathbf{v}\|$. That is, we assume every component of vector $\mathbf{v}$'s projection is of roughly the same size. In this case, the computational cost of primal attack is mainly decided by the minimal value of b that satisfies the constraint

$$\ell \sqrt{b/d} \leq \delta^{(-d^2 + 2db - d)/(d-1)} \det(\Lambda)^{1/d}. \quad (15)$$

Following [5], our estimation of the complexity of solving uSVP on d -dimensional lattice is based on that of solving SVP on b -dimensional lattice, where b satisfy inequality (15) (namely, the Core-SVP model). Furthermore, we use $\text{cost}_b = 2^{0.292b}$ [5, 11] and $\text{cost}_b = 2^{0.257b}$ [16] to estimate the complexities of classical and quantum algorithms (for solving SVP on b -dimensional lattice) respectively.

Dual Attack. Dual attack translates the LWE problem to a ℓ_2 -norm SIS problem, whose solution is used to solve the decisional LWE by distinguishing between subgaussians (for certain parameters) and uniform distribution over $\mathbb{Z}_q^n$. The first dual attack was given in [52], which only cares about the error distribution of the LWE-like problem, and thus does not distinguish between LWE and ALWE. In other words, the original dual attack solves ALWE the same way as it does to LWE, and it appears more efficient when $\alpha_1 \gg \alpha_2$. Formally, let

$$\Lambda = \{\mathbf{x} \in \mathbb{Z}^m \mid \mathbf{A}^T \mathbf{x} = \mathbf{0} \bmod q\},$$

be a lattice of dimension $d = m \gg n$. Note that $\mathbf{A}$ has full rank with high probability and we assume the submatrix $\mathbf{A}_1 \in \mathbb{Z}_q^{n \times n}$ made up by the first n columns of $\mathbf{A}^T$ has rank n and $\mathbf{A}^T = (\mathbf{A}_1 \| \mathbf{A}_2) \in \mathbb{Z}_q^{n \times m}$. Let $\mathbf{A}' = (\mathbf{I}_n \| \mathbf{A}_1^{-1} \mathbf{A}_2)$. Then, we have that

$$\Lambda = \{\mathbf{x} \in \mathbb{Z}^m | \mathbf{A}^T \mathbf{x} = \mathbf{0} \bmod q\} = \{\mathbf{x} \in \mathbb{Z}^m | \mathbf{A}' \mathbf{x} = \mathbf{0} \bmod q\},$$

and the column vectors of

$$\mathbf{B} = \begin{pmatrix} q\mathbf{I}_n & -\mathbf{A}_1^{-1} \mathbf{A}_2 \\ 0 & \mathbf{I}_{m-n} \end{pmatrix} \in \mathbb{Z}^{m \times m}$$

constitute a basis of lattice Λ and $\det(\Lambda) = q^n$. The dual attack first solves the SIS problem by finding out a short vector $\mathbf{v} \in \mathbb{Z}^m$ with $\mathbf{A}^T \mathbf{v} = \mathbf{0} \bmod q$ in lattice Λ with basis $\mathbf{B}$. It then computes $u = \langle \mathbf{v}, \mathbf{b} \rangle = \langle \mathbf{v}, \mathbf{e} \rangle \bmod q$ in order to distinguish between subgaussian and uniform distributions. Specifically, if the norm $\ell = \|\mathbf{v}\|$ is relatively small, then the value of $\langle \mathbf{v}, \mathbf{e} \rangle$ is relatively small too and u can be seen as random variable drawn from subgaussian distribution with standard variance $\ell\alpha_2$, which can be distinguished from uniform distribution on $\mathbb{Z}_q$ with probability $4 \exp(-2\pi^2\tau^2)$, where $\tau = \ell\alpha_2/q$.

Similar as for primal attacks, we can also make use of the fact that $\mathbf{s} \stackrel{\$}{\leftarrow} \chi_{\alpha_1}^n$ has small norm [5] to optimize the dual attack. Formally, define lattice

$$\Lambda = \{(c\mathbf{x}^T, \mathbf{y}^T)^T \in \mathbb{R}^{m+n} \mid \mathbf{A}^T \mathbf{x} = \mathbf{y} \bmod q, \mathbf{x} \in \mathbb{Z}^m, \mathbf{y} \in \mathbb{Z}^n\},$$

where Λ has dimension $d = m + n$ and $c = \frac{\alpha_2}{\alpha_1}$. Then, the column vectors of matrix

$$\mathbf{B} = \begin{pmatrix} c\mathbf{I}_m & 0 \\ \mathbf{A}^T & q\mathbf{I}_n \end{pmatrix} \in \mathbb{R}^{(m+m) \times (m+n)}$$

constitutes a basis of lattice Λ with $\det(\Lambda) = c^m q^n$. Likewise, we find out a short vector $\mathbf{v} = (c\mathbf{x}^T, \mathbf{y}^T)^T \in \mathbb{R}^{m+n}$ with $\mathbf{A}^T \mathbf{x} = \mathbf{y} \bmod q$ from $(m + n)$ -dimensional lattice Λ with basis $\mathbf{B}$, and then compute

$$u = \langle \mathbf{x}, \mathbf{b} \rangle = \langle \mathbf{y}, \mathbf{s} \rangle + \langle c\mathbf{x}, \mathbf{e}/c \rangle \bmod q,$$

and therefore distinguish the above u from uniform. Specifically, the values of $\langle \mathbf{y}, \mathbf{s} \rangle$ and $\langle \mathbf{x}, \mathbf{e} \rangle$ are relatively small for small $\ell = \|\mathbf{v}\|$. Thus, $u = \langle \mathbf{y}, \mathbf{s} \rangle + \langle c\mathbf{x}, \mathbf{e}/c \rangle$ can be almost regarded as following subgaussian distribution with parameter $\ell\alpha_1$. This enable efficient algorithms that distinguish the subgaussian-distributed u from uniform with probability $4 \exp(-2\pi^2\tau^2)$, where $\tau = \ell\alpha_1/q$.

The Cost of Dual Attack. As shown above, the computational costs of dual attack is mainly dominated by the cost of solving SIS problem and transforming distinguishing attacks into secret recovery attacks (with success probability more than half) for the ALWE problem. We use the model of primal attack to estimate the cost of solving SIS problem. That is, applying BKZ- b to reduce the basis of d -dimensional lattice yields a short vector $\mathbf{v}$ of norm $\ell = \|\mathbf{v}\| = \delta^d \det(\Lambda)^{1/d}$.

Substituting this into the previous analysis, we distinguish ALWE problem with advantage $\epsilon = 4\exp(-2\pi^2\tau^2)$, where τ is fully decided by ℓ and the actual dual attack algorithm. To obtain the final secret recovery attack with probability greater than $1/2$, we need $1/\epsilon^2$ short vectors. Taking into account that each invocation of the sieving yields $2^{0.2075b}$ short vectors, we need to repeat it $R = 1/\max(1, 1/(2^{0.2075b}\epsilon^2))$ times.

Following the cost estimation framework for primal attacks, we adopt the Core-SVP model, which uses the complexity of solving SVP on a b -dimensional lattice as a (conservative) estimate for solving SVP on a d -dimensional lattice. Specifically, we set $\text{cost}_b = 2^{0.292b}$ for classical algorithms and $\text{cost}_b = 2^{0.257b}$ for quantum algorithms when solving SVP on b -dimensional lattices. Under this model, the complexity of dual attack is estimated as

$$1/\max(1, 1/(2^{0.2075b} \cdot 16\exp(-4\pi^2\tau^2))) \cdot \text{cost}_b, \quad (16)$$

where τ is decided by b and other parameters of the actual dual attack.

Refined MATZOV Attack (MATZOV[†]). The MATZOV attack given in [50] is basically a variant of the hybrid-dual attack which splits the attack into three parts: a BKZ lattice reduction stage, a guessing stage and a FFT-based distinguishing stage. Formally, given a LWE problem

$$(\mathbf{A}, \mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e}) \in \mathbb{Z}_q^{m \times n} \times \mathbb{Z}_q^m,$$

where $\mathbf{A} \xleftarrow{\$} \mathbb{Z}_q^{m \times n}$, $\mathbf{s} \leftarrow \chi_{\alpha_1}^n$ and $\mathbf{e} \leftarrow \chi_{\alpha_2}^m$. Let $n_{\text{lat}}, n_{\text{enu}}, n_{\text{fft}}$ be integers satisfying $n = n_{\text{lat}} + n_{\text{enu}} + n_{\text{fft}}$. The matrix $\mathbf{A}$ and secret $\mathbf{s}$ can be divided accordingly:

$$\mathbf{A} = [\mathbf{A}_{\text{enu}}, \mathbf{A}_{\text{fft}}, \mathbf{A}_{\text{lat}}] \in \mathbb{Z}_q^{m \times n_{\text{enu}}} \times \mathbb{Z}_q^{m \times n_{\text{fft}}} \times \mathbb{Z}_q^{m \times n_{\text{lat}}},$$

$$\mathbf{s} = \begin{pmatrix} \mathbf{s}_{\text{enu}} \\ \mathbf{s}_{\text{fft}} \\ \mathbf{s}_{\text{lat}} \end{pmatrix} \in \mathbb{Z}_q^{n_{\text{enu}}} \times \mathbb{Z}_q^{n_{\text{fft}}} \times \mathbb{Z}_q^{n_{\text{lat}}}.$$

The attack works by first performing BKZ lattice reduction on the lattice generated by

$$\mathbf{B}_{\text{lat}} = \begin{pmatrix} \mathbf{I}_m & \mathbf{0} \\ \mathbf{A}_{\text{lat}}^T & q\mathbf{I}_{n_{\text{lat}}} \end{pmatrix} \in \mathbb{Z}_q^{(m+n_{\text{lat}}) \times (m+n_{\text{lat}})}$$

to obtain a set $\mathcal{S}$ of N vectors $(\mathbf{x}, \mathbf{y}) \in \mathbb{Z}_q^{m+n_{\text{lat}}}$ with small Euclidean norm such that $\mathbf{y} = \mathbf{A}_{\text{lat}}^T \mathbf{x}$. Then, for each vector $(\mathbf{x}, \mathbf{y}) \in \mathcal{S}$, by the equation $\mathbf{b} = \mathbf{A}_{\text{enu}}\mathbf{s}_{\text{enu}} + \mathbf{A}_{\text{fft}}\mathbf{s}_{\text{fft}} + \mathbf{A}_{\text{lat}}\mathbf{s}_{\text{lat}} + \mathbf{e}$ we have that

$$\underbrace{\langle \mathbf{x}, \mathbf{b} - \mathbf{A}_{\text{enu}}\mathbf{s}_{\text{enu}} \rangle}_{= \hat{b}} = \underbrace{\langle \mathbf{x}, \mathbf{A}_{\text{fft}}\mathbf{s}_{\text{fft}} \rangle}_{= \langle \hat{\mathbf{a}}, \mathbf{s}_{\text{fft}} \rangle} + \underbrace{\langle \mathbf{x}, \mathbf{e} \rangle + \langle \mathbf{y}, \mathbf{s}_{\text{lat}} \rangle}_{= \hat{e}}, \quad (17)$$

where $\hat{\mathbf{a}} = \mathbf{A}_{\text{fft}}^T \mathbf{x}$, $\hat{b} = \mathbf{x}^T(\mathbf{b} - \mathbf{A}_{\text{enu}}\mathbf{s}_{\text{enu}})$ and $\hat{e} = \mathbf{x}^T \mathbf{e} + \mathbf{y}^T \mathbf{s}_{\text{lat}}$. Clearly, if $(\mathbf{e}, \mathbf{s}_{\text{lat}})$ has small Euclidean norm, then $\hat{e}$ is small. Thus, by correctly guessing $\mathbf{s}_{\text{enu}}$, we can

obtain a new LWE sample $\hat{b} = \hat{\mathbf{a}}^T \mathbf{s}_{n_{\text{fft}}} + \hat{e}$ in much smaller dimension $n_{\text{fft}} \ll n$ for each $(\mathbf{x}, \mathbf{y}) \in \mathcal{S}$. Finally, the attack ends by running a FFT-based distinguishing algorithm over $\mathbb{Z}_q^{n_{\text{fft}}}$ which can determine whether a guessed value for $\mathbf{s}_{\text{enu}}$ is correct or not. The core improvement of the MATZOV attack over previous hybrid-dual attacks such as [34] is the introduction of modulus-switching to speedup the FFT-based distinguishing stage, which greatly reduces the FFT cost from previous $O(n_{\text{fft}} q^{n_{\text{fft}}})$ to $O(n_{\text{fft}} p^{n_{\text{fft}}})$ for some much smaller $p \ll q$ by switching the obtained LWE samples in Equation (17) from $\mathbb{Z}_q^{n_{\text{fft}}}$ to $\mathbb{Z}_p^{n_{\text{fft}}}$. The main problem of the MAZOV attack is the requirement of some controversial assumptions in the complexity analysis [24, 62].

Recently, Carrier et al. [15] replaced the modulus switching part of the MATZOV attack with a lossy source/quantizing approach, and obtained a variant that is supported by experimentally validated assumptions. Specifically, given integers $k_{\text{fft}} \ll n_{\text{fft}}$ and a linear code generated by some $\mathbf{G} \in \mathbb{Z}_q^{n_{\text{fft}} \times k_{\text{fft}}}$ such that for each $\mathbf{A}_{\text{fft}}^T \mathbf{x}$ there exists a vector $\mathbf{u} \in \mathbb{Z}_q^{k_{\text{fft}}}$ and a small vector $\mathbf{e}_{\mathbf{x}} \in \mathbb{Z}_q^{n_{\text{fft}}}$ satisfying $\mathbf{A}_{\text{fft}}^T \mathbf{x} = \mathbf{G} \mathbf{u} + \mathbf{e}_{\mathbf{x}}$, we can rewrite Equation (17) as follows:

$$\underbrace{\langle \mathbf{x}, \mathbf{b} - \mathbf{A}_{\text{enu}} \mathbf{s}_{\text{enu}} \rangle}_{= \hat{b}} = \underbrace{\langle \mathbf{u}, \mathbf{G}^T \mathbf{s}_{\text{fft}} \rangle}_{= \langle \mathbf{u}, \mathbf{s}'_{\text{fft}} \rangle} + \underbrace{\langle \mathbf{x}, \mathbf{e} \rangle + \langle \mathbf{y}, \mathbf{s}_{\text{lat}} \rangle + \langle \mathbf{e}_{\mathbf{x}}, \mathbf{s}_{\text{fft}} \rangle}_{= \hat{e}'}. \quad (18)$$

Now, the attack can be finished by running the FFT-based distinguishing algorithm over $\mathbb{Z}_q^{k_{\text{fft}}}$, and the FFT cost is reduced from previous $O(n_{\text{fft}} q^{n_{\text{fft}}})$ to $O(k_{\text{fft}} q^{k_{\text{fft}}})$. In [15], Carrier et al. demonstrated that their variant of the MATZOV attack is not only supported by experimental evidence but also outperforms the original MATZOV attack [50]. Thus, for more conservative concrete security estimates, one should consider the MATZOV attack in [15], as also remarked in [33].

Since Carrier et al.'s attack [15] only considers symmetric distributions (namely, $\alpha_1 = \alpha_2$), we slightly adapt their attack to handle asymmetric distributions. Formally, let $c = \alpha_2 / \alpha_1$, we scale the lattice used in the BKZ lattice reduction stage as follows:

$$\mathbf{B}'_{\text{lat}} = \begin{pmatrix} c\mathbf{I}_m & \mathbf{0} \\ \mathbf{A}_{\text{lat}}^T & q\mathbf{I}_{n_{\text{lat}}} \end{pmatrix} \in \mathbb{R}^{(m+n_{\text{lat}}) \times (m+n_{\text{lat}})}.$$

By doing this, we can model the distribution of $\hat{e}'$ in Equation (18) as a Gaussian distribution with parameter $\ell \alpha_1$, where $\ell = \|(c\mathbf{x}, \mathbf{y}, \mathbf{e}_{\mathbf{x}})\|_2$ for each $\mathbf{y} = \mathbf{A}_{n_{\text{lat}}}^T \mathbf{x}$. For ease of presentation, we refer to our above refined MATZOV attack as MATZOV[†].

The Cost of MATZOV[†] Attack. As shown in [15], the total cost of the MATZOV[†] attack depends on the cost T_{sample} for obtaining the set $\mathcal{S}$ of N short vectors, R for the expected number of times to guess $\mathbf{s}_{\text{enu}}$ correctly, T_{dec} for decoding $\mathbf{e}_{\mathbf{x}}$ from $\mathbf{A}_{\text{fft}}^T \mathbf{x}$, and T_{fft} for running the FFT-based distinguishing algorithm, namely,

$$T_{\text{cost}} = T_{\text{sample}} + R \cdot (N \cdot T_{\text{dec}} + T_{\text{fft}}),$$

where $T_{\text{sample}} = 2^{0.292b}$ and $N = 2^{0.2075b}$ for performing SVP sieving algorithm in b -dimensional sublattice. For quantum security estimates, in addition to considering quantum SVP sieving cost $T'_{\text{sample}} = 2^{0.257b}$, we additionally assume that the

guessing-then-FFT part can achieve a quantum quadratic speedup [3], namely,

$$T'_{\text{cost}} = T'_{\text{sample}} + \sqrt{R \cdot (N \cdot T_{\text{dec}} + T_{\text{fft}})}.$$

We note that this will only make our concrete estimates more conservative. Because T_{dec} and T_{fft} depend strongly on the modulus size, and R on the sparsity of the secret vector, the above analysis immediately imply that the MATZOV[†] attack may be particularly advantageous for sparse secrets and small modulus.

9.2 Solving the $\text{DNTRU}_{n,q,\chi_f,\chi_g,v}$ problem

As discussed in Sec. 2.7, for the setting that $v = 1 - x^{n/k} \in R_q$ is invertible, the NTRU problem $\text{DNTRU}_{n,q,\chi_f,\chi_g,v}$ is essentially equivalent to the standard NTRU problem (with $v = 3$) up to the choices of the secret key distribution. Moreover, the ℓ_2 -norm of $vf' + 1$ is only roughly about $\sqrt{2}$ times larger than that of f' , which is small as long as f' is chosen from a small distribution. Thus, one can either solve the NTRU problem by taking $f = vf' + 1$ as whole just as in the standard lattice attacks against the NTRU problem with secret distributions $(\chi'_f = v\chi_f, \chi_g)$ in lattice $\mathcal{L}_0^\perp(h)$ for $h = g/f$, or solve the BDD problem on the shifted lattice $\mathcal{L}_h^\perp(-vh)$ by treating it as an RLWE instance $(vh, h = -vhf' + g)$ with secret distribution χ_f and noise distribution χ_g , where

$$\mathcal{L}_c^\perp(a) = \{(u, w) \in R_q = \mathbb{Z}[x]/(x^n + 1) : au + w = c \in R_q\}.$$

We adopt the latter approach for concrete estimates of our NTRU problem, since the norm of the short vector $(g, f', 1)$ in the latter case—which is independent of v —is smaller than that of $(f = vf' + 1, -g)$ in the former case. Finally, dual-like attacks—including MATZOV[†]—are not directly applicable to NTRU problems [1].

9.3 Solving the $\text{sspRLWE}_{n,q,\chi_r,B_\eta,\bar{v}}$ problem

In Sec. 2.7, we have shown that the sspRLWE problem is polynomially equivalent to the standard DRLWE problem under a reduction that is significantly tighter than reductions exist from RLWE to a shortest vector problem. Moreover, as discussed in [26] regarding the concrete hardness of the RLWE2 problem (which can be seen as a special case of the sspRLWE problem), we believe that, given the current state of the art in solving DRLWE problems, the best known attack against the $\text{sspRLWE}_{n,q,\chi_r,B_\eta,\bar{v}}$ problem must also solve the $\text{DRLWE}_{n,q,\chi_r,B_\eta}$ problem. First, if the $\text{DRLWE}_{n,q,\chi_r,B_\eta}$ problem is hard, then a given DRLWE sample $(a, b = ar + e)$ is indistinguishable from uniform. This implies that no PPT algorithm can extract useful information about e from such a sample. Second, according to the study in [20,51,14], if there exists an algorithm that solves the $\text{sspRLWE}_{n,q,\chi_r,B_\eta,\bar{v}}$ problem with lower complexity than directly solving the $\text{DRLWE}_{n,q,\chi_r,B_\eta}$ problem, then this would immediately yield a new, more efficient algorithm for the $\text{DRLWE}_{n,q,\chi_r,B_\eta}$ problem—which is not known to exist. Therefore, we directly estimate the concrete hardness of the $\text{sspRLWE}_{n,q,\chi_r,B_\eta,\bar{v}}$ problem by treating it as a standard DRLWE problem $\text{DRLWE}_{n,q,\chi_r,B_\eta}$.

9.4 Concrete Security Estimates Against Lattice Attacks

The complexity of the attacks depends on the attack type, the number of samples m , and the block size $b \in \mathbb{Z}$ used in the BKZ- b algorithm. Therefore, to obtain a conservative overall security estimation for NEV under the nine concrete parameter sets, we enumerate all possible values of m and b . In detail, Table 6 estimates the complexities of the underlying NTRU and RLWE problems against the aforementioned attacks, where the concrete security estimates against MATZOV[†] are derived from a code that modifies the publicly available implementation by Carrier et al. [15], which can be found at <https://github.com/kevin-carrier/CodedDualAttack>. For comparison, we also provide the figures against the original MATZOV attack [50] given by the LWE estimator [4]. As shown in Table 6, the MATZOV[†] attack appears especially efficient for sparse keys and small moduli—particularly in the high-dimension setting—and it nearly dominates the concrete estimates for all parameters.

Table 6: The Concrete Security Estimates of NEV-AKE (where C and Q denotes classical and quantum security, respectively)

Parameter Set	RLWE Hardness (m, b, sec)				NTRU Hardness (m, b, sec)
	Primal	Dual	MATZOV	MATZOV [†]	Primal
NEV-AKE-C1	C: (461,463,135) Q: (461,463,118)	C: (476,457,133) Q: (476,457,117)	C: (512,442,129) Q: (512,445,115)	C: (448,432,129) Q: (426,399,103)	C: (446,453,132) Q: (446,453,116)
NEV-AKE-R1	C: (486,464,135) Q: (486,464,119)	C: (491,458,133) Q: (491,458,117)	C: (512,450,131) Q: (512,453,116)	C: (504,455,134) Q: (467,414,107)	C: (486,464,135) Q: (486,464,119)
NEV-AKE-C2	C: (806,1030,300) Q: (806,1030,264)	C: (841,1019,297) Q: (841,1019,261)	C: (1024,953,279) Q: (1024,966,248)	C: (729,914,268) Q: (688,841,217)	C: (751,912,266) Q: (751,912,234)
NEV-AKE-R2	C: (826,975,284) Q: (826,975,250)	C: (831,966,282) Q: (831,966,248)	C: (1024,916,268) Q: (1024,934,240)	C: (860,892,261) Q: (740,817,211)	C: (826,975,284) Q: (826,975,250)
NEV-AKE-C3	C: (1242,2090,610) Q: (1242,2090,537)	C: (1282,2080,607) Q: (1282,2080,534)	C: (2048,1936,565) Q: (2048,1966,505)	C: (1366,1782,521) Q: (1162,1590,410)	C: (1296,1871,546) Q: (1296,1871,480)
NEV-AKE-R3	C: (1421,2043,596) Q: (1421,2043,525)	C: (1491,2029,592) Q: (1491,2029,521)	C: (2048,1897,554) Q: (2048,1925,494)	C: (1386,1797,526) Q: (1262,1638,422)	C: (1431,1998,583) Q: (1431,1998,513)

9.5 Attacks Exploiting Decryption Failures

In line with standard practice for practical lattice-based schemes such as Kyber [11], we select concrete parameters that achieve a negligible decryption failure probability to ensure compactness. Although our estimation model is quite conservative, as discussed in Sec. 4.2, it only considers the scenario where all random coins are honestly sampled from the intended distributions. In practice, however, an adversary may depart from this setting and search across many different values of M until finding one for which the induced random coins ρ result in a decryption failure, thus leaking information about the secret key. More alarmingly, after triggering the first failure, the adversary can exploit the directional failure boosting attack [21] to obtain additional decryption failures at a significantly reduced cost.

Consequently, to remain on the safe side, one must ensure that even a single failure is difficult to provoke.

We follow the approach of [11] to demonstrate that, for our concrete parameters in Table 2, the corresponding probability of decryption failures is sufficiently low to resist attacks that exploit such failures. Our analysis crucially relies on the simple decryption formula of the underlying PKE and KEM schemes underlying NEV-AKE. Formally, from inequalities (4) and (6) in Sec. 4.2, the decryption failure is determined by the term

$$\begin{aligned} p_j &= \sum_y \Pr \left[\langle \hat{e}_j'', y \rangle \geq \frac{k \cdot (q-1)}{4} \right] \\ &= \sum_y \Pr \left[\left\langle \sum_{i \in [n/k]} (\hat{g}_i \hat{r}_i + \hat{f}_i' \hat{e}_i) + \hat{e}_j', y \right\rangle \geq \frac{k \cdot (q-1)}{4} \right], \end{aligned}$$

where the probability is taken over the random choice of $\hat{g}_i \xleftarrow{\$} \chi_g, \hat{f}_i' \xleftarrow{\$} \chi_f, \hat{r}_i \xleftarrow{\$} \chi_r, \hat{e}_i \xleftarrow{\$} B_\eta$. For a fixed y , let

$$p_{j,y} = \Pr \left[\left\langle \sum_{i \in [n/k]} (\hat{g}_i \hat{r}_i + \hat{f}_i' \hat{e}_i) + \hat{e}_j', y \right\rangle \geq \frac{k \cdot (q-1)}{4} \right],$$

and $t_y = \left\langle \sum_{i \in [n/k]} (\hat{g}_i \hat{r}_i + \hat{f}_i' \hat{e}_i), y \right\rangle$. Clearly, the probability of $p_{j,y}$ is primarily determined by the value of t_y . Moreover, let $\hat{\mathbf{R}}_i, \hat{\mathbf{E}}_i$ be the coefficient matrices of $\hat{r}_i, \hat{e}_i$ defined via multiplication over the subring $S = \mathbb{Z}[Y]/(Y^k + 1)$, and let $\hat{\mathbf{g}}_i, \hat{\mathbf{f}}_i', \mathbf{y}$ be the coefficient vectors of $\hat{g}_i, \hat{f}_i', y$ in column representation. Then,

$$\begin{aligned} t_y &= \left\langle \sum_{i \in [n/k]} (\hat{g}_i \hat{r}_i + \hat{f}_i' \hat{e}_i), y \right\rangle = \left\langle \sum_{i \in [n/k]} (\hat{\mathbf{R}}_i \hat{\mathbf{g}}_i + \hat{\mathbf{E}}_i \hat{\mathbf{f}}_i'), \mathbf{y} \right\rangle \\ &= \mathbf{y}^T \begin{pmatrix} \hat{\mathbf{R}}_1 \dots \hat{\mathbf{R}}_{n/k} & \hat{\mathbf{E}}_1 \dots \hat{\mathbf{E}}_{n/k} \end{pmatrix} \begin{pmatrix} \hat{\mathbf{g}}_1 \\ \vdots \\ \mathbf{g}_{n/k} \\ \hat{\mathbf{f}}_1' \\ \vdots \\ \hat{\mathbf{f}}_{n/k}' \end{pmatrix}. \end{aligned} \quad (19)$$

For clarity, we first consider the simplified case where $\chi_g = \chi_f$ is a subgaussian distribution with parameter σ —this exactly matches our parameter sets NEV-AKE-C3 and NEV-AKE-R1/R2/R3. We then explain how to generalize the analysis to the broader setting. Define

$$\mathbf{v}_y = \mathbf{y}^T \begin{pmatrix} \hat{\mathbf{R}}_1 \dots \hat{\mathbf{R}}_{n/k} & \hat{\mathbf{E}}_1 \dots \hat{\mathbf{E}}_{n/k} \end{pmatrix} \in \mathbb{Z}^{2n}.$$

By concentration inequalities for subgaussian variables [68, 5], we have

$$\Pr[t_y > \kappa \sigma \|\mathbf{v}_y\|] \leq e^{-\kappa^2/2}. \quad (20)$$

For each $y \in \{0, 1\}^k$, we compute $p_{j,y}$ numerically and denote the largest such value by $p_{j,y}^* = \max_y p_{j,y}$. Setting the right-hand side of inequality (20) equal to $p_{j,y}^*$ gives a specific value $\kappa = \kappa^*$ (reported in Table 7).

Note that the adversary can make at most a bounded number N of decryption queries—for example, $N = 2^{80}$ as specified in the NGCC Call for Proposals [40]. To obtain a decryption failure, the adversary must increase $\|\mathbf{v}_y\|$ by a factor c (e.g., by appropriately choosing (r, e) and M). This factor c reduces κ^* by the same factor in inequality (20), so that the resulting failure probability becomes at least $1/N$. Let c^* denote the threshold value satisfying

$$e^{-(\kappa^*/c^*)^2/2} = 2^{-80-\log(n/k)-k}, \quad (21)$$

where the terms $\log(n/k) + k$ account for the union bound over $y \in \{0, 1\}^k$ in equation (4) and over $j \in [n/k]$ in equation (6). This union bound is introduced artificially for the analysis of decryption failures and only relaxes the conditions under which the adversary can trigger a decryption failure within $N = 2^{80}$ queries. The resulting values of c^* for our concrete parameters are listed in Table 7.

Now it suffices to estimate the probability of obtaining a vector $\mathbf{v}_y$ whose norm is c times larger than the expected value. Let

$$\mathbf{v}_{i,y} = \mathbf{y}^T (\hat{\mathbf{R}}_i \hat{\mathbf{E}}_i) \in \mathbb{Z}^{2k}.$$

All $\mathbf{v}_{i,y}$ are identically distributed, and

$$\|\mathbf{v}_y\|^2 = \sum_{i \in \{1, \dots, n/k\}} \|\mathbf{v}_{i,y}\|^2.$$

Let D_y be the distribution of $\|\mathbf{v}_{i,y}\|^2$ with mean μ . Let $M_{D_y}(\theta) = \mathbb{E}_{X \sim D_y} e^{\theta X}$ the moment-generating function of D_y , assumed finite for some $\theta > 0$. Then, by the Chernoff-Cramer inequality [12], we have that

$$\Pr[\|\mathbf{v}_y\|^2 \geq c^2 \cdot n\mu/k] \leq e^{-n(c^2\mu\theta - \log(M_{D_y}(\theta)))/k} \quad (22)$$

We perform a numerical computation of $\|\mathbf{v}_{i,y}\|^2$ to determine the distribution (D_y, μ) . Since an adversary may maliciously choose $M \in \{0, 1\}^{n/k}$ to influence the distribution of $\hat{\mathbf{E}}_i$ and thereby increase decryption failures, our numerical computation accounts for the choice of M as well. Specifically, the i -th bit M_i of M affects only the single matrix $\hat{\mathbf{E}}_i$. Hence, we can enumerate all possibilities $M_i \in \{0, 1\}$ and $y \in \{0, 1\}^k$ in the numerical computation. For the subsequent analysis, we fix the pair (M_i, y) that maximizes the probability in equation (22) — this choice represents the most favorable conditions for the adversary's attack.

Finally, the total cost for the adversary to trigger a single decryption failure is given by

$$\min_{c \geq c^*} \frac{1}{e^{-(\kappa^*/c)^2/2} \cdot e^{-n(c^2\mu\theta - \log(M_{D_y}(\theta)))/k}}$$

in the classical setting, and

$$\min_{c \geq c^*} \frac{1}{e^{-(\kappa^*/c)^2/2} \cdot \sqrt{e^{-n(c^2\mu\theta - \log(M_{D_y}(\theta)))/k}}}$$

in the quantum setting. Here, we directly assume that Grover’s algorithm yields a square-root speed-up for quantum search, which only makes our analysis more conservative.

Now, we show how to extend the above analysis for the case $\chi_g \neq \chi_f$. Formally, let χ_g and χ_f be subgaussian distributions with parameter σ_g and σ_f , respectively. By the properties of subgaussian distributions, we have $\frac{\sigma_f}{\sigma_g}\chi_g$ is subgaussian with parameter σ_f . Using the identity

$$\begin{aligned} t_y &= \left\langle \sum_{i \in [n/k]} (\hat{\mathbf{R}}_i \hat{\mathbf{g}}_i + \hat{\mathbf{E}}_i \hat{\mathbf{f}}'_i), \mathbf{y} \right\rangle \\ &= \mathbf{y}^T \left(\frac{\sigma_g}{\sigma_f} \hat{\mathbf{R}}_1, \dots, \frac{\sigma_g}{\sigma_f} \hat{\mathbf{R}}_{n/k}, \hat{\mathbf{E}}_1, \dots, \hat{\mathbf{E}}_{n/k} \right) \begin{pmatrix} \frac{\sigma_f}{\sigma_g} \hat{\mathbf{g}}_1 \\ \vdots \\ \frac{\sigma_f}{\sigma_g} \hat{\mathbf{g}}_{n/k} \\ \hat{\mathbf{f}}'_1 \\ \vdots \\ \hat{\mathbf{f}}'_{n/k} \end{pmatrix}, \end{aligned}$$

we can handle this general case naturally by instead performing a numerical computation of $\|\mathbf{v}'_{i,y}\|^2$, where

$$\mathbf{v}'_{i,y} = \mathbf{y}^t \left(\frac{\sigma_g}{\sigma_f} \hat{\mathbf{R}}_i \hat{\mathbf{E}}_i \right) \in \mathbb{Z}^{2k}.$$

We develop a Python script to compute the adversary’s optimal cost—attained at $c = c^*$ for all parameter sets—and report the resulting values in Table 7. In this table, θ is the constant appearing in the Chernoff–Cramér inequality, μ^* denotes the optimal μ for the adversary’s choice of $M_i \in \{0, 1\}$, and C and Q represent the minimum classical and quantum security bits, respectively. These results confirm that our concrete parameters maintain the claimed security levels against attacks that exploit decryption failures.

Table 7: Cost to trigger a single decryption failure within 2^{80} decryption queries

Parameter set	Dec. Failure	$p_{j,y}^*$	κ^*	c^*	μ^*	θ	Bit Security (C,Q)
NEV-AKE-C1	2^{-161}	2^{-170}	15.35	1.37	37.47	0.061	(288, 184)
NEV-AKE-R1	2^{-134}	2^{-145}	14.18	1.26	48	0.024	(151, 115)
NEV-AKE-C2	2^{-162}	2^{-173}	15.49	1.37	21.37	0.059	(324, 202)
NEV-AKE-R2	2^{-158}	2^{-170}	15.35	1.36	32	0.05	(369, 224)
NEV-AKE-C3	2^{-163}	2^{-175}	15.58	1.37	18.74	0.152	(1098, 589)
NEV-AKE-R3	2^{-147}	2^{-160}	14.89	1.31	26.67	0.056	(542, 311)

9.6 Attacks against Symmetric Primitives

The overall security of **NEV-AKE** relies not only on the hardness of the underlying mathematical problems, but also closely on the security of several symmetric primitives it invokes, such as collision-resistant hash functions, pseudorandom number generators, and key derivation functions. In Sec. 6.3, we instantiate these primitives with SHA3 and SHAKE256. This instantiation rests on a clear security assumption: they provide the required collision resistance, preimage resistance, and pseudorandomness at the level matching our concrete security parameter in both the classical and quantum setting.

Crucially, **NEV-AKE** is designed and proven secure by treating these symmetric primitives as idealised black boxes throughout the security reduction. This means that the proof does not depend on the internal details or specific algebraic structure of SHA3 or SHAKE256, but only on its external input-output behaviour and the expected security metrics. Such a highly modular abstraction offers strong cryptographic agility: in the future, should any major cryptanalytic breakthrough against SHA3 or SHAKE256 emerge—for instance, collision or preimage attacks significantly more efficient than anticipated—that would undermine the prescribed security strength in either the classical or quantum setting, we would not need to redesign the core public-key framework of **NEV-AKE** nor invalidate its existing security reduction. In practice, one would simply replace SHA3 or SHAKE256 with another well-analysed, standardised algorithms that provide an equivalent interface, and the overall scheme architecture and core logic would remain unchanged, thereby swiftly restoring and maintaining the actual security margin of the scheme.

10 Side-Channel Vulnerabilities and Countermeasures

To the best of our knowledge, existing design and implementation of **NEV-KEM-OW** and **NEV-PKE-OW** follow a constant-time design paradigm, where the execution time is independent of secret-dependent values. Consequently, timing-based side-channel leakage is not expected to be exploitable in practice.

Regarding physical side channels such as power consumption and electromagnetic leakage, although there exist constructions that aim to achieve leakage resilience, they often incur prohibitive overhead and are therefore impractical for real-world deployments. Instead, we argue that having a clear and systematic understanding of the inherent side-channel vulnerabilities of the algorithm, together with targeted and efficient countermeasures, is of primary importance for achieving practical security.

NEV-KEM-OW and **NEV-PKE-OW** share similar structures in their KeyGen and Decap (resp., Dec) procedures. As a result, they exhibit essentially the same side-channel leakage behaviors, and admit equivalent countermeasure designs. To avoid redundant exposition, we present our analysis only for **NEV-KEM-OW**, and the same reasoning applies to **NEV-PKE-OW** in a straightforward manner.

10.1 Security Analysis

Key Generation Phase. In NEV-KEM-OW, sensitive information mainly includes the secret key sk and its derived intermediate variables during computation. Among them, the secret key vector sk is the most critical confidential data, whose security directly determines the overall security of the scheme. Fig. 8 illustrate NEV-KEM-OW key generation flow, where red denotes sensitive information and operations, and blue denotes non-sensitive information.

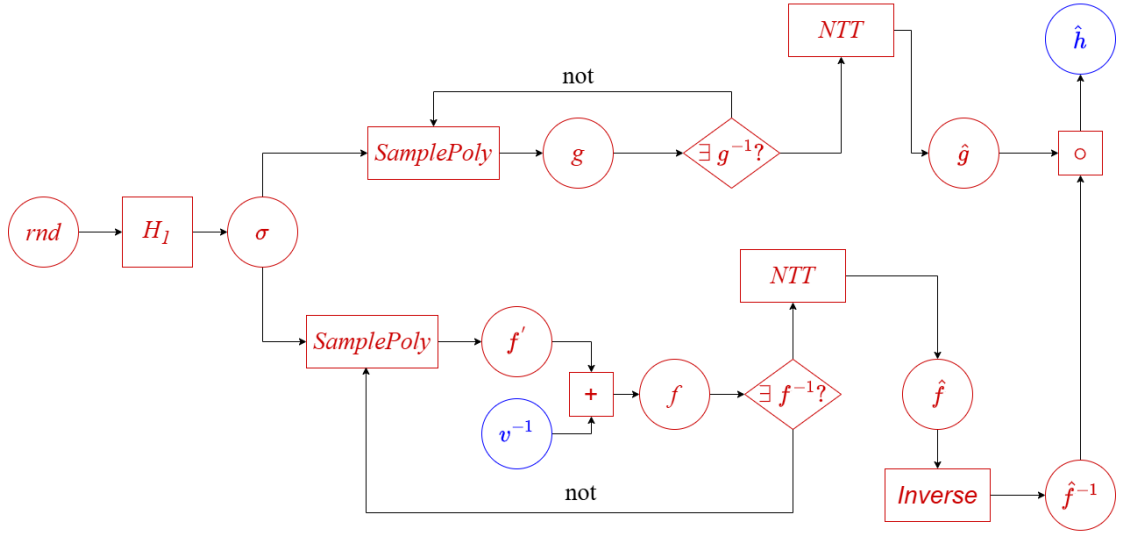


Fig. 8: NEV-KEM-OW key-generation procedure and side-channel vulnerabilities

During key generation, two polynomials f' and g are first randomly sampled and checked for invertibility. Since both f' and g are sensitive, the sampling procedure and invertibility testing must be protected. Subsequently, these polynomials are transformed into the NTT domain, where polynomial inversion is performed. Therefore, both the NTT transformation and inversion operations in the NTT domain require protection.

Decapsulation Phase. During the decapsulation procedure in NEV-KEM-OW, the secret key sk and its derived variables remain sensitive throughout the computation. In fact, the secret key sk is first multiplied with the ciphertext c in the NTT domain, then the result is transformed back to the arithmetic domain, and the plaintext is recovered via the Noise2Pt algorithm. Throughout the decapsulation process, the NTT operations and Noise2Pt require protection:

10.2 A Discussion on Masking Countermeasures

In the propagation paths of sensitive information in NEV-KEM-OW, linear operations such as polynomial addition and multiplication in the NTT domain can be computed in a share-wise manner.

In contrast, nonlinear operations, including polynomial sampling, polynomial inversion, invertibility testing, Noise2Pt, Pt2Noise, and ciphertext comparison, constitute the main challenges for masking and require dedicated secure designs. In the remainder of this subsection, we denote by d the masking order.

Polynomial Sampling. The polynomial sampling procedure in NEV-KEM-OW first generates random bytes using a PRG, then produces polynomials following a binomial distribution via the CBD algorithm, and finally generates polynomials with specific distributions using the Ternary algorithm.

Masked PRG. The PRG is implemented using the Keccak algorithm. Masking for Keccak has been extensively studied in the literature. Existing masking schemes for Keccak are well developed, and their main computational overhead is typically dominated by the nonlinear layer, resulting in a cost of $O(d^2)$ for a masking order of d .

Masked CBD Function. The CBD_η function maps random bits to polynomial coefficients following a discrete Gaussian-like distribution. The standard method computes $f_i = a - b$, where a and b are Hamming weights of bit partitions (say, x and y). The main challenge of Masked CBD Function is the conversion from Boolean-masked inputs to arithmetic outputs.

A straightforward approach is proposed in [57]. It first applies multiple Boolean-to-Arithmetic (B2A) conversions to map the input into the arithmetic domain, where the Hamming weight is subsequently computed. More precisely, for a k -bit Boolean-masked input with two shares (x_1, x_2) , the bits are processed individually. For each bit position j , a one-bit B2A conversion is performed to transform the j -th bit from Boolean masking to arithmetic masking, yielding arithmetic shares. The masked Hamming weight is then obtained by summing over all k bit positions. This approach requires $2k$ B2A calls and incurs a complexity of $O(kd^2)$. A more efficient method is introduced in [64]. Leveraging a bit-slicing technique, it computes the Hamming weight directly in the Boolean domain, and performs only a single B2A conversion at the end, with input bit-width $\lambda = \log_2(k + 1) + 1$, to convert the result into the arithmetic domain. This approach achieves a reduced complexity of $O(d^2 \log_2 k)$.

Masked Ternary Function. The Ternary function first invokes the PRG and CBD functions. Since the masking strategies for these components have already been given, we only need to replace the AND operation with masked multiplication. For masked multiplication [41] proposed an NI-secure masked multiplication scheme with complexity $O(d^2)$, requiring $\frac{d(d-1)}{2}$ random values and no additional storage. The Ternary function invokes AND a total of $\frac{N}{8} \cdot \frac{N}{8}$ times, and thus the total complexity of masked multiplication is $O(N^2 d^2)$. The CBD module inside the Ternary function has been discussed in previous parts. In general, the cost overhead is approximately $O(d^2)$.

Masked Pt2Noise Module. The Pt2Noise module converts a plaintext bit string M of length $\ell = n/k$ to be encoded into a polynomial m over the ring $\mathbb{Z}_q[x]/(x^n+1)$ whose coefficients follow a centered binomial distribution. More specifically, the algorithm consists of the following three stages:

1. Use XOR random masking to mix the plaintext. This stage consists entirely of linear operations.
2. Randomize and encode the plaintext bit matrix. This part partitions and permutes the bit stream block by block and is also linear.
3. Generate the plaintext polynomial m following a centered binomial distribution. This part corresponds to the CBD module. This function inside the Ternary function has been discussed in previous parts. In general, the cost overhead is approximately $O(d^2)$.

In general, the cost overhead is approximately $O(d^2 \log_2 k)$.

Masked Noise2Pt Module. The Noise2Pt module converts a polynomial $w = w_0 + w_1x + \dots + w_{n-1}x^{n-1}$ whose coefficients follow a centered binomial distribution into a plaintext bit string M of length $\ell = n/k$.

For each coefficient, the procedure consists of three stages:

1. **Centering:** $\tilde{w}_i = w_i - \frac{q+1}{2} \bmod \pm q$,
2. **Aggregation:** $t_j = \sum_{i \equiv j \bmod (n/k)} |\tilde{w}_i|$;
3. **Decision:** $M_j = \begin{cases} 1, & t_j < \frac{k(q-1)}{4}, \\ 0, & \text{otherwise.} \end{cases}$

First of all, we consider the masked centering. The centering operation is essentially a conditional reduction: $\tilde{w}_i = \begin{cases} w_i, & w_i < \frac{q}{2}, \\ w_i - q, & \text{otherwise,} \end{cases}$ which is nonlinear and cannot be implemented directly in the arithmetic masked domain. A practical approach is to convert this decision into the Boolean domain using the most significant bit (MSB): $\text{MSB}(w_i) = \begin{cases} 1, & w_i < \frac{q}{2}, \\ 0, & \text{otherwise} \end{cases}$. For w_i in arithmetic masking form $w_i = w_i^{(1)} + \dots + w_i^{(d)} \bmod q$, the procedure is as follows:

1. Compute masked shares of $w_i - \frac{q+1}{2}$.
2. Convert to Boolean masking using an Arithmetic-to-Boolean (A2B) conversion (e.g., [64]), with complexity $O(d^2 \log k)$, where k is the bit-length.
3. Extract the MSB in the Boolean domain share-wise.
4. Compute the centered value using masked multiplication: $\tilde{w}_i = w_i \cdot \text{MSB}(w_i) + (w_i - q) \cdot (1 - \text{MSB}(w_i))$.

Therefore, the cost of centering a single coefficient is $O(d^2 \log p)$.

Then, we consider the masked absolute value and aggregation. The computation of $t_j = \sum |\tilde{w}_i|$ consists of following two steps.

- Step 1: Absolute value $|\tilde{w}_i| = \begin{cases} \tilde{w}_i, & \tilde{w}_i < \frac{q}{2}, \\ q - \tilde{w}_i, & \text{otherwise} \end{cases}$ is nonlinear and implemented in the Boolean domain using MSB: $|\tilde{w}_i| = \tilde{w}_i \cdot (1 - \text{MSB}(w_i))$. Note that the A2B conversion computed during centering can be reused here. A masked refreshing step is applied to obtain fresh Boolean shares with cost $O(d^2)$, followed by one masked multiplication with cost $O(d^2)$.
- Step 2: Summation. Since summation is not modulo q , it must be performed in a larger domain $\mathbb{Z}_r$ such that $r > k^2$.
 - First, convert $|\tilde{w}_i|$ from Boolean to arithmetic masking in $\mathbb{Z}_r$ using B2A with overhead complexity of $O(d^2 \log(\log r))$.
 - Then directly perform masked summation $t_j = \sum |\tilde{w}_i|$.
 Thus, the total complexity of aggregation is $O(d^2 \log(\log r))$.

Finally, we consider the masked decision. It is also nonlinear and must be implemented in the Boolean domain. This requires an A2B conversion with complexity $O(d^2 \log(\log r))$ and a Boolean comparison. Therefore, the total complexity of the decision step is dominated by A2B conversion $O(d^2 \log(\log r))$.

Masked Polynomial Inversion Module. Polynomial inversion is performed through a sequence of polynomial multiplications. Each polynomial multiplication can be protected using a standard masked multiplication scheme such as ISW [41]. Concretely, the multiplication of two shared values is carried out securely in the masked domain, with a cost of $O(d^2)$ per multiplication.

In addition, polynomial inversion requires checking whether the coefficients in the NTT domain are zero, in order to determine invertibility. Since these coefficients are secret-shared values over the prime field, this step reduces to a masked zero-testing problem.

Masked Zero-Testing. Let $x \in \mathbb{F}_p$ be a secret value represented in arithmetic masking with d shares:

$$x = x_1 + x_2 + \dots + x_d \bmod p.$$

The goal is to determine whether $x = 0$, i.e., to compute a Boolean output

$$b = \begin{cases} 1, & x = 0, \\ 0, & x \neq 0, \end{cases}$$

where the output b is not required to be masked.

Coron et al. [19] gives an approach for this problem. The approach iteratively multiplies by random invertible elements u_i and performs linear mask refreshing, thereby transforming the shares into a form satisfying

$$u_1 \cdot \dots \cdot u_d \cdot x = B_1 + \dots + B_d \bmod q.$$

Afterwards, all shares are added together to reconstruct b . If the original secret satisfies $x = 0$, then $B = 0$; otherwise, B is a uniformly random nonzero value in $\mathbb{Z}_q^*$. Then, one can directly check if B is zero or not, without leaking the value of x . This approach incurs an overhead of complexity $O(d^2)$.

References

1. Martin R. Albrecht, Benjamin R. Curtis, Amit Deo, Alex Davidson, Rachel Player, Eamonn W. Postlethwaite, Fernando Virdia, and Thomas Wunderer. Estimate all the LWE, NTRU schemes! Cryptology ePrint Archive, Paper 2018/331, 2018.
2. Martin R. Albrecht, Florian Göpfert, Fernando Virdia, and Thomas Wunderer. Revisiting the expected cost of solving usvp and applications to lwe. In Tsuyoshi Takagi and Thomas Peyrin, editors, *Advances in Cryptology – ASIACRYPT 2017*, pages 297–322, Cham, 2017. Springer International Publishing.
3. Martin R. Albrecht and Yixin Shen. Quantum augmented dual attack. Cryptology ePrint Archive, Paper 2022/656, 2022.
4. Martin R. Albrecht, Rachel Player, and Sam Scott. On the concrete hardness of learning with errors. In *Journal of Mathematical Cryptology*, 9:169–203, oct 2015.
5. Erdem Alkim, Léo Ducas, Thomas Pöppelmann, and Peter Schwabe. Post-quantum key exchange—a new hope. In *USENIX Security Symposium 2016*, 2016.
6. Andris Ambainis, Mike Hamburg, and Dominique Unruh. Quantum security proofs using semi-classical oracles. In *Advances in Cryptology – CRYPTO 2019*, pages 269–295. Springer, 2019.
7. Shi Bai and Steven D. Galbraith. Lattice decoding attacks on binary lwe. In Willy Susilo and Yi Mu, editors, *Information Security and Privacy*, pages 322–337, Cham, 2014. Springer International Publishing.
8. Shi Bai, Adeline Langlois, Tancrede Lepoint, Damien Stehlé, and Ron Steinfeld. Improved security proofs in lattice-based cryptography: Using the rényi divergence rather than the statistical distance. In Tetsu Iwata and Jung Hee Cheon, editors, *Advances in Cryptology – ASIACRYPT 2015*, pages 3–24. Springer, 2015.
9. Daniel V. Bailey, Daniel Coffin, Adam Elbirt, Joseph H. Silverman, and Adam D. Woodbury. NTRU in constrained devices. In *CHES 2001*, pages 262–272. Springer, 2001.
10. W. Banaszczyk. New bounds in some transference theorems in the geometry of numbers. *Mathematische Annalen*, 296:625–635, 1993.
11. J. Bos, L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, J. M. Schanck, P. Schwabe, G. Seiler, and D. Stehle. Crystals-Kyber: A CCA-secure module-lattice-based KEM. In *2018 IEEE European Symposium on Security and Privacy (EuroS&P)*, pages 353–367, 2018.
12. Stéphane Boucheron, Gábor Lugosi, and Pascal Massart. *Concentration inequalities: A nonasymptotic theory of independence*. Oxford University Press, 2013.
13. Zvika Brakerski and Nico Döttling. Lossiness and entropic hardness for ring-LWE. In *Theory of Cryptography Conference*, pages 1–27. Springer, 2020.
14. Jinzheng Cao, Haodong Jiang, and Qingfeng Cheng. Refined attack on LWE with hints: Constructing lattice via gaussian elimination. In Yael Tauman Kalai and Seny F. Kamara, editors, *Advances in Cryptology – CRYPTO 2025*, LNCS, pages 385–416. Springer, 2025.
15. Kevin Carrier, Charles Meyer-Hilfiger, Yixin Shen, and Jean-Pierre Tillich. Assessing the impact of a variant of matzov’s dual attack on kyber. In *Advances in Cryptology – CRYPTO 2025*, pages 444–476. Springer, 2025.
16. André Chailloux and Johanna Loyer. Lattice sieving via quantum random walks. In Mehdi Tibouchi and Huaxiong Wang, editors, *Advances in Cryptology – ASIACRYPT 2021*, pages 63–91, Cham, 2021. Springer International Publishing.
17. Cong Chen, Oussama Danba, Jeffrey Hoffstein, Andreas Hülsing, Joost Rijneveld, John M. Schanck, Tsunekazu Saito, Peter Schwabe, William Whyte, Keita Xagawa, Takashi Yamakawa, and Zhenfei Zhang. NTRU-submission to the NIST post-quantum project, 2019.
18. Yuanmi Chen. Lattice reduction and concrete security of fully homomorphic encryption. *Dept. Informatique, ENS, Paris, France, PhD thesis*, 2013.
19. Jean-Sébastien Coron, François Gérard, Simon Montoya, and Rina Zeitoun. High-order polynomial comparison and masking lattice-based encryption. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2023(1):153–192, 2023.
20. Dana Dachman-Soled, Léo Ducas, Huijing Gong, and Mélissa Rossi. LWE with side information: Attacks and concrete security estimation. In Daniele Micciancio and Thomas Ristenpart, editors, *Advances in Cryptology – CRYPTO 2020*, pages 329–358. Springer, 2020.
21. Jan-Pieter D’Anvers, Mélissa Rossi, and Fernando Virdia. (one) failure is not an option: Bootstrapping the search for failures in lattice-based encryption schemes. In Anne Canteaut and Yuval Ishai, editors, *Advances in Cryptology – EUROCRYPT 2020*, pages 3–33. Springer, 2020.
22. Jan-Pieter D’Anvers, Frederik Vercauteren, and Ingrid Verbauwhede. The impact of error dependencies on ring/mod-lwe/lwr based schemes. In *International Conference on Post-Quantum Cryptography*, pages 103–115. Springer, 2019.

23. Léo Ducas, Vadim Lyubashevsky, and Thomas Prest. Efficient identity-based encryption over NTRU lattices. In Palash Sarkar and Tetsu Iwata, editors, *Advances in Cryptology – ASIACRYPT 2014*, pages 22–41. Springer, 2014.
24. Léo Ducas and Ludo N. Pulles. Does the dual-sieve attack on learning with errors even work? In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology – CRYPTO 2023*, pages 37–69, Cham, 2023. Springer Nature Switzerland.
25. Julien Duman, Kathrin Hövelmanns, Eike Kiltz, Vadim Lyubashevsky, Gregor Seiler, and Dominique Unruh. A thorough treatment of highly-efficient NTRU instantiations. Cryptology ePrint Archive, Paper 2021/1352, 2021.
26. Julien Duman, Kathrin Hövelmanns, Eike Kiltz, Vadim Lyubashevsky, Gregor Seiler, and Dominique Unruh. A thorough treatment of highly-efficient ntru instantiations. In *IACR International Conference on Public-Key Cryptography*, pages 65–94. Springer, 2023.
27. Pierre-Alain Fouque, Jeffrey Hoffstein, Paul Kirchner, Vadim Lyubashevsky, Thomas Pornin, Thomas Prest, Thomas Ricosset, Gregor Seiler, William Whyte, and Zhenfei Zhang. Falcon: Fast-fourier lattice-based compact signatures over NTRU, 2016.
28. Pierre-Alain Fouque, Paul Kirchner, Thomas Pornin, and Yang Yu. BAT: Small and fast kem over NTRU lattices. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2022(2):240–265, 2022.
29. Atsushi Fujioka, Koutarou Suzuki, Keita Xagawa, and Kazuki Yoneyama. Strongly secure authenticated key exchange from factoring, codes, and lattices. In Marc Fischlin, Johannes Buchmann, and Mark Manulis, editors, *PKC 2012*, LNCS, pages 467–484. Springer, 2012.
30. Atsushi Fujioka, Koutarou Suzuki, Keita Xagawa, and Kazuki Yoneyama. Practical and post-quantum authenticated key exchange from one-way secure key encapsulation mechanism. In Kefei Chen, Qi Xie, Weidong Qiu, Ninghui Li, and Wen-Guey Tzeng, editors, *ASIA CCS 2013*, pages 83–94. ACM, 2013.
31. Eiichiro Fujisaki and Tatsuaki Okamoto. Secure integration of asymmetric and symmetric encryption schemes. In Michael Wiener, editor, *Advances in Cryptology – CRYPTO 1999*, pages 537–554. Springer, 1999.
32. Nicolas Gama and Phong Q. Nguyen. New chosen-ciphertext attacks on NTRU. In Tatsuaki Okamoto and Xiaoyun Wang, editors, *PKC 2007*, pages 89–106. Springer, 2007.
33. Lewis Glabush, Patrick Longa, Michael Naehrig, Chris Peikert, Douglas Stebila, and Fernando Virdia. FrodoKEM: A CCA-secure learning with errors key encapsulation mechanism. *IACR Communications in Cryptology*, 2(3), 2025.
34. Qian Guo and Thomas Johansson. Faster dual lattice attacks for solving lwe with applications to crystals. In Mehdi Tibouchi and Huaxiong Wang, editors, *Advances in Cryptology – ASIACRYPT 2021*, pages 33–62, Cham, 2021. Springer International Publishing.
35. Jens Hermans, Frederik Vercauteren, and Bart Preneel. Speed records for NTRU. In Josef Pieprzyk, editor, *Topics in Cryptology - CT-RSA 2010*, pages 73–88. Springer, 2010.
36. Jeffrey Hoffstein, Jill Pipher, and Joseph H. Silverman. NTRU: A ring-based public key cryptosystem. In Joe P. Buhler, editor, *Algorithmic Number Theory*, pages 267–288. Springer, 1998.
37. Jeffrey Hoffstein and Joseph H. Silverman. Optimizations for NTRU. In Joe P. Buhler, editor, *Proc. the Conf. on Public Key Cryptography and Computational Number Theory*, pages 77–88. Springer, 2000.
38. Kathrin Hövelmanns, Eike Kiltz, Sven Schäge, and Dominique Unruh. Generic authenticated key exchange in the quantum random oracle model. In Aggelos Kiayias, Markulf Kohlweiss, Petros Wallden, and Vassilis Zikas, editors, *PKC 2020*, Lecture Notes in Computer Science, pages 389–422. Springer, 2020.
39. Nick Howgrave-Graham, Phong Q. Nguyen, David Pointcheval, John Proos, Joseph H. Silverman, Ari Singer, and William Whyte. The impact of decryption failures on the security of NTRU encryption. In Dan Boneh, editor, *Advances in Cryptology – CRYPTO 2003*, pages 226–246. Springer, 2003.
40. China Institute of Commercial Cryptography Standards. Call for proposals for the next-generation public-key cryptographic algorithms, 2025.
41. Yuval Ishai, Amit Sahai, and David A. Wagner. Private circuits: Securing hardware against probing attacks. In Dan Boneh, editor, *Advances in Cryptology - CRYPTO 2003, 23rd Annual International Cryptology Conference, Santa Barbara, California, USA, August 17-21, 2003, Proceedings*, Lecture Notes in Computer Science, pages 463–481. Springer, 2003.
42. Frederik Vercauteren Jan-Pieter D’Anvers and Ingrid Verbauwhede. The impact of error dependencies on ring/mod-lwe/lwr based schemes. In *International Conference on Post-Quantum Cryptography*, pages 103–115. Springer, 2019.

43. Haodong Jiang, Zhenfeng Zhang, Long Chen, Hong Wang, and Zhi Ma. IND-CCA-secure key encapsulation mechanism in the quantum random oracle model, revisited. In Hovav Shacham and Alexandra Boldyreva, editors, *CRYPTO 2018*, volume 10993 of *LNCS*, pages 96–125. Springer, 2018.
44. Ravi Kannan. Minkowski’s convex body theorem and integer programming. *Mathematics of Operations Research*, 12(3):415–440, 1987.
45. Hugo Krawczyk. HMQV: A high-performance secure diffie-hellman protocol. In Victor Shoup, editor, *CRYPTO 2005*, LNCS, pages 546–566. Springer, 2005.
46. Richard Lindner and Chris Peikert. Better key sizes (and attacks) for LWE-based encryption. In Aggelos Kiayias, editor, *Topics in Cryptology – CT-RSA 2011*, volume 6558 of *LNCS*, pages 319–339. Springer, 2011.
47. Adriana López-Alt, Eran Tromer, and Vinod Vaikuntanathan. On-the-fly multiparty computation on the cloud via multikey fully homomorphic encryption. In *STOC 2012*, pages 1219–1234, 2012.
48. You Lyu and Shengli Liu. Two-message authenticated key exchange from public-key encryption. In Gene Tsudik, Mauro Conti, Kaitai Liang, and Georgios Smaragdakis, editors, *Computer Security - ESORICS 2023 - 28th European Symposium on Research in Computer Security, The Hague, The Netherlands, September 25-29, 2023, Proceedings, Part I*, volume 14344 of *Lecture Notes in Computer Science*, pages 414–434. Springer, 2023.
49. Vadim Lyubashevsky, Chris Peikert, and Oded Regev. On ideal lattices and learning with errors over rings. In Henri Gilbert, editor, *Advances in Cryptology – EUROCRYPT 2010*, volume 6110 of *LNCS*, pages 1–23. Springer, 2010.
50. IDF MATZOV. Report on the security of lwe: improved dual lattice attack. <https://zenodo.org/record/6412487>, 2022.
51. Alexander May and Julian Nowakowski. Too many hints—when LLL breaks LWE. In *Advances in Cryptology – ASIACRYPT 2023*, pages 106–137. Springer, 2023.
52. Daniele Micciancio and Oded Regev. Lattice-based cryptography. In DanielJ. Bernstein, Johannes Buchmann, and Erik Dahmen, editors, *Post-Quantum Cryptography*, pages 147–191. Springer Berlin Heidelberg, 2009.
53. Phong Q. Nguyen and David Pointcheval. Analysis and improvements of NTRU encryption paddings. In Moti Yung, editor, *Advances in Cryptology – CRYPTO 2002*, pages 210–225. Springer, 2002.
54. NIST. FIPS PUB 202: SHA-3 Standard: Permutation-based hash and extendable-output functions. Federal Information Processing Standards Publication 202, U.S. Department of Commerce, 2015.
55. NIST. Status report on the second round of the nist post-quantum cryptography standardization process, 2020.
56. NIST. Status report on the third round of the nist post-quantum cryptography standardization process, 2022.
57. Tobias Oder, Tobias Schneider, Thomas Pöppelmann, and Tim Güneysu. Practical cca2-secure and masked ring-lwe implementation. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2018(1):142–174, 2018.
58. Jiaxin Pan, Benedikt Wagner, and Runzhi Zeng. Lattice-based authenticated key exchange with tight security. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology - CRYPTO 2023 - 43rd Annual International Cryptology Conference, CRYPTO 2023, Santa Barbara, CA, USA, August 20-24, 2023, Proceedings, Part V*, volume 14085 of *Lecture Notes in Computer Science*, pages 616–647. Springer, 2023.
59. Jiaxin Pan, Benedikt Wagner, and Runzhi Zeng. Tighter security for generic authenticated key exchange in the QROM. In Jian Guo and Ron Steinfeld, editors, *Advances in Cryptology - ASIACRYPT 2023 - 29th International Conference on the Theory and Application of Cryptology and Information Security, Guangzhou, China, December 4-8, 2023, Proceedings, Part IV*, volume 14441 of *Lecture Notes in Computer Science*, pages 401–433. Springer, 2023.
60. Chris Peikert. An efficient and parallel gaussian sampler for lattices. In Tal Rabin, editor, *Advances in Cryptology – CRYPTO 2010*, volume 6223 of *LNCS*, pages 80–97. Springer, 2010.
61. Alice Pellet-Mary and Damien Stehlé. On the hardness of the NTRU problem. In Mehdi Tibouchi and Huaxiong Wang, editors, *Advances in Cryptology – ASIACRYPT 2021*, pages 3–35. Springer, 2021.
62. Amaury Pouly and Yixin Shen. Provable dual attacks on?learning with?errors. In Marc Joye and Gregor Leander, editors, *Advances in Cryptology – EUROCRYPT 2024*, pages 256–285, Cham, 2024. Springer Nature Switzerland.
63. Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. In *STOC 2005*, pages 84–93. ACM, 2005.

64. Tobias Schneider, Clara Paglialonga, Tobias Oder, and Tim Güneysu. Efficiently masking binomial sampling at arbitrary orders for lattice-based crypto. In Dongdai Lin and Kazue Sako, editors, *Public-Key Cryptography - PKC 2019 - 22nd IACR International Conference on Practice and Theory of Public-Key Cryptography, Beijing, China, April 14-17, 2019, Proceedings, Part II*, Lecture Notes in Computer Science, pages 534–564. Springer, 2019.
65. Damien Stehlé and Ron Steinfeld. Making NTRU as secure as worst-case problems over ideal lattices. In Kenneth Paterson, editor, *Advances in Cryptology – EUROCRYPT 2011*, volume 6632 of *LNCS*, pages 27–47. Springer, 2011.
66. Damien Stehlé, Ron Steinfeld, Keisuke Tanaka, and Keita Xagawa. Efficient public key encryption based on ideal lattices. In Mitsuru Matsui, editor, *Advances in Cryptology – ASIACRYPT 2009*, volume 5912 of *LNCS*, pages 617–635. Springer, 2009.
67. Dominique Unruh. Revocable quantum timed-release encryption. *J. ACM*, 62(6):49:1–49:76, 2015.
68. Roman Vershynin. Introduction to the non-asymptotic analysis of random matrices. In Yonina C. Eldar and Gitta Kutyniok, editors, *Compressed Sensing: Theory and Applications*, pages 210–268. Cambridge University Press, 2012.
69. Haiyang Xue, Man Ho Au, Rupeng Yang, Bei Liang, and Haodong Jiang. Compact authenticated key exchange in the quantum random oracle model. *Cryptology ePrint Archive*, Paper 2020/1282, 2020.
70. Haiyang Xue, Xianhui Lu, Bao Li, Bei Liang, and Jingnan He. Understanding and constructing AKE via double-key key encapsulation mechanism. In Thomas Peyrin and Steven D. Galbraith, editors, *Advances in Cryptology - ASIACRYPT 2018 - 24th International Conference on the Theory and Application of Cryptology and Information Security, Brisbane, QLD, Australia, December 2-6, 2018, Proceedings, Part II*, volume 11273 of *Lecture Notes in Computer Science*, pages 158–189. Springer, 2018.
71. Jiang Zhang, Dengguo Feng, and Di Yan. NEV: Faster and smaller NTRU encryption using vector decoding. In Jian Guo and Ron Steinfeld, editors, *Advances in Cryptology – ASIACRYPT 2023*, pages 157–189. Springer Nature Singapore, 2023.
72. Jiang Zhang, Yu Yu, Shuqin Fan, Zhenfeng Zhang, and Kang Yang. Tweaking the asymmetry of asymmetric-key cryptography on lattices: Kems and signatures of smaller sizes. In Aggelos Kiayias, Markulf Kohlweiss, Petros Wallden, and Vassilis Zikas, editors, *PKC 2020*, pages 37–65. Springer, 2020.
73. Yiming Zhu, Zhen Liu, and Yanbin Pan. When NTT meets karatsuba: Preprocess-then-NTT technique revisited. In Debin Gao, Qi Li, Xiaohong Guan, and Xiaofeng Liao, editors, *Information and Communications Security*, pages 249–264. Springer, 2021.

Appendix A

A.1 Proof of Lemma 4

Proof. To prove this lemma, we introduce a sequence of games $\mathbf{G}_0^b$ to $\mathbf{G}_8^b$ ($b \in \{0, 1\}$) as shown in Fig. 9. In these games, H_{init} , H_{resp} , G , and H are random oracles, and the adversary $\mathcal{A}$ can also query a set of oracles $\mathcal{O}_{\text{AKE}} = \{\text{EST}, \text{INIT}, \text{DER}_{\text{resp}}, \text{DER}_{\text{init}}, \text{REVEAL}, \text{REV-STATE}, \text{CORRUPT}, \text{TEST}\}$.

Game $\mathbf{G}_0^b$: This game is the same as $\text{KeyIND}_{\Pi_{\text{AKE}}}^{\mathcal{A}, b}$ (Fig. 4), except that it additionally checks whether $\mathfrak{M}(\text{sID}^*) \neq \emptyset$ and $\neg \text{st}$ occurs, and directly returns 0 if this check fails. Thus,

$$\Pr[1 \leftarrow \mathbf{G}_0^b] = \Pr[1 \leftarrow \text{KeyIND}_{\Pi_{\text{AKE}}}^{\mathcal{A}, b} \wedge \mathfrak{M}(\text{sID}^*) \neq \emptyset \wedge \neg \text{st}]. \quad (23)$$

Game $\mathbf{G}_1^b$: Based on game $\mathbf{G}_0^b$, this game also checks whether $|\mathfrak{M}(\text{sID}^*)| = 1$, and directly returns 0 if this check fails.

Let $\text{Bad}_{\mathfrak{M}}$ denote the event that $|\mathfrak{M}(\text{sID}^*)| > 1$. If $\text{Bad}_{\mathfrak{M}}$ does not occur, the number of elements of the set $\mathfrak{M}(\text{sID}^*)$ is at most 1 throughout the execution, which implies games $\mathbf{G}_0^b$ and $\mathbf{G}_1^b$ are equivalent. We next analyze the probability of $\text{Bad}_{\mathfrak{M}}$ according to the role of sID^* .

- If $\text{role}[\text{sID}^*] = \text{initiator}$, by the definition of the matching session (Definition 9), $\text{Bad}_{\mathfrak{M}}$ occurring implies that the adversary $\mathcal{A}$ has performed the following operations:

- $\mathcal{A}$ establishes a new session sID' , which is different from another session sID contained in the set $\mathfrak{M}(\text{sID}^*)$. Here $\text{role}[\text{sID}] = \text{responder}$, $\text{rece}[\text{sID}] = \text{sent}[\text{sID}^*]$, $\text{sent}[\text{sID}] = \text{rece}[\text{sID}^*]$, and $\text{sent}[\text{sID}] = (c_i, \tilde{c})$ is generated by querying DER_{resp} . Then, $\mathcal{A}$ queries DER_{resp} with $(\text{sID}', \text{sent}[\text{sID}^*])$ to label sID' as the responder session, and finally obtains the return value $(c'_i, \tilde{c}')$ equals to $\text{sent}[\text{sID}](= (c_i, \tilde{c}))$.

During this process, the oracle DER_{resp} always generates and outputs two ciphertexts of Π_{PKE} and Π_{KEM} honestly. Hence, the probability that DER_{resp} produces the same output is bounded by $\gamma(\Pi_{\text{PKE}}.\text{Enc}) \cdot \gamma(\Pi_{\text{KEM}}.\text{Encap})$, where $\gamma(\Pi_{\text{PKE}}.\text{Enc})$ is the encry-collision probability of Π_{PKE} , and $\gamma(\Pi_{\text{KEM}}.\text{Encap})$ is the encap-collision probability of Π_{KEM} (Definition 1, 5). Since $\mathcal{A}$ can establish at most $S-1$ sessions apart from the test session, the probability that $\text{Bad}_{\mathfrak{M}}$ occurs under $\text{role}[\text{sID}^*] = \text{initiator}$ is at most $(S-1) \cdot \gamma(\Pi_{\text{PKE}}.\text{Enc}) \cdot \gamma(\Pi_{\text{KEM}}.\text{Encap})$.

- If $\text{role}[\text{sID}^*] = \text{responder}$, by the definition of the matching session (Definition 9), $\text{Bad}_{\mathfrak{M}}$ occurring implies that $\mathcal{A}$ has performed the following operations: $\mathcal{A}$ establishes a new session sID' , which is different from another session sID contained in the set $\mathfrak{M}(\text{sID}^*)$; then $\mathcal{A}$ queries INIT with sID' to label sID' as the initiator session, and finally obtains the return value $(c'_j, \tilde{p}k')$ equals to $\text{sent}[\text{sID}](= (c_j, \tilde{p}k))$, where $\text{sent}[\text{sID}]$ is also generated by querying INIT . During this process, the oracle INIT always generates a ciphertext of Π_{PKE} and a public key of Π_{KEM} honestly. Hence, the probability that INIT produces the same output is bounded

GAMES $\mathbf{G}_0^b\text{-}\mathbf{G}_8^b$ ($b \in \{0, 1\}$)	Session(i^*, j^*) $// \mathbf{G}_3^b\text{-}\mathbf{G}_4^b$
<pre> 1, $\text{cnt} := 0, \text{slD}^* := 0$ 2, $\mathbf{H}_{\text{init}}, \mathbf{H}_{\text{resp}}, \mathbf{H} \xleftarrow{\\$} \text{Fcs}[\{0, 1\}^*, \mathcal{K}], \mathbf{G} \xleftarrow{\\$} \text{Fcs}[\mathcal{M}, \mathcal{R}]$ $s_{\text{init}}, s_{\text{resp}} \xleftarrow{\\$} \{1, \dots, S\} // \mathbf{G}_2^b\text{-}\mathbf{G}_8^b$ $i^*, j^* \xleftarrow{\\$} \{1, \dots, N\} // \mathbf{G}_2^b\text{-}\mathbf{G}_8^b$ $r^* \xleftarrow{\\$} \mathcal{K} // \mathbf{G}_6^b$ 3, for $i \in \{1, \dots, N\}$ $(pk_i, sk_i) \leftarrow \Pi_{\text{PKE}}.\text{KeyGen}, s_i \xleftarrow{\\$} \mathcal{M}$ $sk_i' := (sk_i, s_i), \text{crp}[i] := \text{false}$ end for run Session(i^*, j^*) $// \mathbf{G}_3^b$ 4, $b' \leftarrow \mathcal{A}^{\text{AKE}, \mathbf{H}_{\text{init}}, \mathbf{H}_{\text{resp}}, \mathbf{G}, \mathbf{H}}(pk_1, \dots, pk_N)$ 5, if slD^* is not fresh, return 0 if $\mathfrak{M}(\text{slD}^*) = \emptyset \vee \text{stRev}[\text{slD}^*] = \text{ture}$, return 0 $// \mathbf{G}_0^b\text{-}\mathbf{G}_8^b$ if $\mathfrak{M}(\text{slD}^*) \neq 1$, return 0 $// \mathbf{G}_1^b\text{-}\mathbf{G}_8^b$ if $\text{role}[s_{\text{init}}] \neq \text{initiator} \vee \text{role}[s_{\text{resp}}] \neq \text{responder} // \mathbf{G}_2^b\text{-}\mathbf{G}_8^b$ $\vee \{s_{\text{init}}, s_{\text{resp}}\} \neq \{\text{slD}^*\} \cup \mathfrak{M}(\text{slD}^*)$ $\vee (i^*, j^*) \neq (\text{holder}[s_{\text{init}}], \text{peer}[s_{\text{init}}])$ return 0 6, return b' $// \mathbf{G}_2^b\text{-}\mathbf{G}_8^b$ </pre>	<pre> 1, $m_{j^*} \xleftarrow{\\$} \mathcal{M}, c_{j^*} := \Pi_{\text{PKE}}.\text{Enc}(pk_{j^*}, m_{j^*}; \mathbf{G}(m_{j^*}))$ $(\widetilde{pk}^*, \widetilde{sk}^*) \leftarrow \Pi_{\text{KEM}}.\text{KeyGen}, \widetilde{M} := (\widetilde{pk}^*, c_{j^*})$ 2, $m_{i^*} \xleftarrow{\\$} \mathcal{M}, c_{i^*} := \Pi_{\text{PKE}}.\text{Enc}(pk_{i^*}, m_{i^*}; \mathbf{G}(m_{i^*}))$ $(\widetilde{c}^*, K^*) \leftarrow \Pi_{\text{KEM}}.\text{Encap}(\widetilde{pk}^*), \widetilde{M}' := (c_{i^*}, \widetilde{c}^*)$ $m_{j^*}' := \Pi_{\text{PKE}}.\text{Dec}(sk_{j^*}, c_{j^*})$ if $c_{j^*} \neq \Pi_{\text{PKE}}.\text{Enc}(pk_{j^*}, m_{j^*}'; \mathbf{G}(m_{j^*}'))$ $K_{\text{resp}} := \mathbf{H}_{\text{resp}}(s_{j^*}, m_{i^*}, c_{j^*}, K^*, i^*, j^*, \widetilde{M}, \widetilde{M}')$ $K_{\text{resp}} := \mathbf{H}(m_{i^*}, m_{j^*}', K^*, i^*, j^*, \widetilde{M}, \widetilde{M}')$ 3, $m_{i^*}' := \Pi_{\text{PKE}}.\text{Dec}(sk_{i^*}, c_{i^*})$ $K' := \Pi_{\text{KEM}}.\text{Decap}(\widetilde{sk}^*, \widetilde{c}^*)$ if $c_{i^*} \neq \Pi_{\text{PKE}}.\text{Enc}(pk_{i^*}, m_{i^*}'; \mathbf{G}(m_{i^*}')) \vee K' = \perp$ $K_{\text{init}} := \mathbf{H}_{\text{init}}(s_{i^*}, c_{i^*}, m_{j^*}, \widetilde{c}^*, i^*, j^*, \widetilde{M}, \widetilde{M}')$ $K_{\text{init}} := \mathbf{H}(m_{i^*}', m_{j^*}, K', i^*, j^*, \widetilde{M}, \widetilde{M}')$ </pre>
<pre> Session(i^*, j^*) $// \mathbf{G}_5^b\text{-}\mathbf{G}_8^b$ 1, $m_{j^*} \xleftarrow{\\$} \mathcal{M}, c_{j^*} := \Pi_{\text{PKE}}.\text{Enc}(pk_{j^*}, m_{j^*}; \mathbf{G}(m_{j^*}))$ $(\widetilde{pk}^*, \widetilde{sk}^*) \leftarrow \Pi_{\text{KEM}}.\text{KeyGen}, \widetilde{M} := (\widetilde{pk}^*, c_{j^*})$ 2, $m_{i^*} \xleftarrow{\\$} \mathcal{M}, c_{i^*} := \Pi_{\text{PKE}}.\text{Enc}(pk_{i^*}, m_{i^*}; \mathbf{G}(m_{i^*}))$ $(\widetilde{c}^*, K^*) \leftarrow \Pi_{\text{KEM}}.\text{Encap}(\widetilde{pk}^*), \widetilde{M}' := (c_{i^*}, \widetilde{c}^*)$ 3, $K_{\text{resp}} := \mathbf{H}(m_{i^*}, m_{j^*}, K^*, i^*, j^*, \widetilde{M}, \widetilde{M}')$ $// \mathbf{G}_5^b$ $K_{\text{init}} := \mathbf{H}(m_{i^*}, m_{j^*}, K^*, i^*, j^*, \widetilde{M}, \widetilde{M}')$ $// \mathbf{G}_5^b$ $K_{\text{resp}} = K_{\text{init}} := r^*$ $// \mathbf{G}_6^b\text{-}\mathbf{G}_7^b$ $K_{\text{resp}} = K_{\text{init}} := 0$ $// \mathbf{G}_8^b$ </pre>	<pre> INIT(slD) 1, if holder[slD] = $\perp \vee$ sent[slD] $\neq \perp$, return $\perp$ 2, role[slD] := initiator 3, if slD = s_{init} $// \mathbf{G}_3^b\text{-}\mathbf{G}_8^b$ (sent[slD], st[slD]) := $(\widetilde{M}, (\widetilde{M}, \widetilde{sk}^*, m_{j^*}))$ $// \mathbf{G}_3^b$ (sent[slD], st[slD]) := $(\widetilde{M}, (\widetilde{M}, 0, m_{j^*}))$ $// \mathbf{G}_4^b\text{-}\mathbf{G}_8^b$ return $\widetilde{M}$ else $(i, j) := (\text{holder}[\text{slD}], \text{peer}[\text{slD}])$ $(M, st) \leftarrow \text{GC-AKE}.\text{Init}(sk_i, pk_j)$ (sent[slD], st[slD]) := (M, st) return M </pre>
<pre> DER_{resp}(slD, M) 1, if holder[slD] = $\perp \vee$ sKey[slD] $\neq \perp$ $\vee \text{role}[\text{slD}] = \text{initiator}$, return $\perp$ 2, role[slD] := responder 3, if slD = $s_{\text{resp}} \wedge M = \widetilde{M}$ $// \mathbf{G}_3^b\text{-}\mathbf{G}_8^b$ sKey[slD] := K_{resp} $// \mathbf{G}_3^b\text{-}\mathbf{G}_8^b$ (sent[slD], rece[slD]) := $(\widetilde{M}', \widetilde{M})$ $// \mathbf{G}_3^b\text{-}\mathbf{G}_8^b$ return $\widetilde{M}'$ $// \mathbf{G}_3^b\text{-}\mathbf{G}_8^b$ else $(j, i) := (\text{holder}[\text{slD}], \text{peer}[\text{slD}])$ $(M', K) \leftarrow \text{GC-AKE}.\text{Der}_{\text{resp}}(sk_j, pk_i, M)$ sKey[slD] := K (sent[slD], rece[slD]) := (M', M), return M' </pre>	<pre> DER_{init}(slD, M') 1, if holder[slD] = $\perp \vee$ st[slD] = $\perp$ $\vee \text{sKey}[\text{slD}] \neq \perp$; return $\perp$ 2, if slD = $s_{\text{init}} \wedge M' = \widetilde{M}'$ $// \mathbf{G}_3^b\text{-}\mathbf{G}_8^b$ sKey[slD] := K_{init}, rece[slD] := $\widetilde{M}'$ $// \mathbf{G}_3^b\text{-}\mathbf{G}_8^b$ else $(i, j) := (\text{holder}[\text{slD}], \text{peer}[\text{slD}])$ sKey[slD] $\leftarrow \text{GC-AKE}.\text{Der}_{\text{init}}(sk_i, pk_j, M', \text{st}[\text{slD}])$ rece[slD] := M' </pre>
	<pre> TEST(slD) $//$ only one query 1, slD* := slD $//$ test session's id 2, if sKey[slD*] = $\perp$, return $\perp$ 3, $K_0^* := \text{sKey}[\text{slD}^*], K_1^* \xleftarrow{\\$} \mathcal{K}$ $K_0^* := r^*, K_1^* \xleftarrow{\\$} \mathcal{K} // \mathbf{G}_7^b\text{-}\mathbf{G}_8^b$ 4, return K_b^* </pre>

Fig. 9: Games $\mathbf{G}_0^b$ to $\mathbf{G}_8^b$ used in the proof of Lemma 4. Here for brevity, we only list the oracles TEST, INIT, DER_{resp} and DER_{init} that need to be modified in our game sequence. The definitions of all other unlisted oracles are identical to those given in Fig. 4.

by $\gamma(\Pi_{\text{PKE}}.\text{Enc}) \cdot \gamma(\Pi_{\text{KEM}}.\text{KeyGen})$, where $\gamma(\Pi_{\text{KEM}}.\text{KeyGen})$ denotes the key-collision probability of Π_{KEM} (Definition 5). Since $\mathcal{A}$ can establish at most $S-1$ sessions apart from the test session, the probability that $\text{Bad}_{\mathfrak{M}}$ occurs under $\text{role}[\text{sID}^*] = \text{responder}$ is at most $(S-1) \cdot \gamma(\Pi_{\text{PKE}}.\text{Enc}) \cdot \gamma(\Pi_{\text{KEM}}.\text{KeyGen})$.

Combining the above analysis, we have

$$\begin{aligned}
& |\Pr[1 \leftarrow \mathbf{G}_0^b] - \Pr[1 \leftarrow \mathbf{G}_1^b]| \\
& \leq \Pr[\text{Bad}_{\mathfrak{M}}] \\
& = \Pr[\text{Bad}_{\mathfrak{M}} \wedge \text{role}[\text{sID}^*] = \text{initiator}] + \Pr[\text{Bad}_{\mathfrak{M}} \wedge \text{role}[\text{sID}^*] = \text{responder}] \\
& \leq (S-1) \cdot \gamma(\Pi_{\text{PKE}}.\text{Enc}) \cdot (\gamma(\Pi_{\text{KEM}}.\text{Encap}) + \gamma(\Pi_{\text{KEM}}.\text{KeyGen})).
\end{aligned} \tag{24}$$

Game $\mathbf{G}_2^b$: This game is the same as $\mathbf{G}_1^b$, except that at the very beginning, it uniformly samples two sessions $s_{\text{init}}, s_{\text{resp}}$ from $\{1, \dots, S\}$ and two indexes i^*, j^* from $\{1, \dots, N\}$, and checks whether $\text{role}[s_{\text{init}}] = \text{initiator}$, $\text{role}[s_{\text{resp}}] = \text{responder}$, $\{s_{\text{init}}, s_{\text{resp}}\} = \{\text{sID}^*\} \cup \mathfrak{M}(\text{sID}^*)$, and $(i^*, j^*) = (\text{holder}[s_{\text{init}}], \text{peer}[s_{\text{init}}])$ at the end. If such check fails, game $\mathbf{G}_2^b$ directly returns 0.

In game $\mathbf{G}_1^b$, the additional check $\mathfrak{M}(\text{sID}^*) \neq \emptyset \wedge |\mathfrak{M}(\text{sID}^*)| = 1$ ensures that the set $\mathfrak{M}(\text{sID}^*)$ contains exactly one session, which we denote by msID^* . Since msID^* is the matching session of sID^* , the roles of sID^* and msID^* must be one initiator and the other responder. With this observation, in game $\mathbf{G}_2^b$, we make a random guess at the beginning of the game, hoping to directly obtain sID^* and msID^* (i.e., the s_{init} and s_{resp}). In addition, we also guess that the holder and peer of session s_{init} are i^* and j^* , respectively. These guessing steps are necessary; with it, we can embed the challenge ciphertext of Π_{KEM} into the key negotiation process between s_{init} and s_{resp} . As for the difference between games $\mathbf{G}_1^b$ and $\mathbf{G}_2^b$, we have

$$\Pr[1 \leftarrow \mathbf{G}_1^b] = S^2 N^2 \cdot \Pr[1 \leftarrow \mathbf{G}_2^b]. \tag{25}$$

Game $\mathbf{G}_3^b$: This game is the same as $\mathbf{G}_2^b$, except that it introduces a new procedure $\text{Session}(i^*, j^*)$ and modifies the computations of INIT , DER_{resp} and DER_{init} according to the internal random values used by $\text{Session}(i^*, j^*)$, as detailed below:

- $\text{Session}(i^*, j^*)$ honestly simulates the key negotiation process between users P_{i^*} and P_{j^*} . Here, P_{i^*} acts as the initiator; it generates the ephemeral key pair $(\widetilde{pk}^*, \widetilde{sk}^*)$ of Π_{KEM} and returns $\widetilde{M} := (\widetilde{pk}^*, c_{j^*})$ to P_{j^*} . As the responder, P_{j^*} generates the key K_{resp} after receiving $\widetilde{M}$ and sends back $\widetilde{M}' := (c_{i^*}, \widetilde{c}^*)$ to P_{i^*} . Finally, P_{i^*} derives its session key K_{init} .
- For INIT , if the input is s_{init} , it sets $(\text{sent}[s_{\text{init}}], \text{st}[s_{\text{init}}]) := (\widetilde{M}, (\widetilde{M}, \widetilde{sk}^*, m_{j^*}))$ and returns $\widetilde{M}$. For DER_{init} , if the input is $(s_{\text{init}}, \widetilde{M}')$, it sets $\text{sKey}[s_{\text{init}}] := K_{\text{init}}$ and $\text{rece}[s_{\text{init}}] := \widetilde{M}'$.
- For DER_{resp} , if the input is $(s_{\text{resp}}, \widetilde{M})$, it sets $\text{sKey}[s_{\text{resp}}] := K_{\text{resp}}$, $(\text{sent}[s_{\text{resp}}], \text{rece}[s_{\text{resp}}]) := (\widetilde{M}', \widetilde{M})$, and then returns $\widetilde{M}'$.

For all remaining inputs, INIT , DER_{resp} and DER_{init} perform their computations in accordance with the specifications given in Fig. 4.

In game $\mathbf{G}_3^b$, we actually simulate the key negotiation process between s_{init} and s_{resp} in advance by executing $\text{Session}(i^*, j^*)$. We then perform checks on the inputs of INIT , DER_{resp} and DER_{init} , and insert certain random values generated during this simulated negotiation into the lists sKey , sent , rece , and st . Since outputting 1 in both $\mathbf{G}_2^b$ and $\mathbf{G}_3^b$ guarantees $\{s_{\text{init}}, s_{\text{resp}}\} = \{\text{sID}^*\} \cup \mathfrak{M}(\text{sID}^*)$ and $(i^*, j^*) = (\text{holder}[s_{\text{init}}], \text{peer}[s_{\text{init}}])$, the key negotiation process of the test session sID^* in $\mathbf{G}_2^b$ is equivalent to that simulated by $\text{Session}(i^*, j^*)$ in $\mathbf{G}_3^b$. Meanwhile, $\mathbf{G}_3^b$ appropriately inserts corresponding random values into the lists sKey , sent , rece , and st . Hence, the modifications introduced in $\mathbf{G}_3^b$ relative to $\mathbf{G}_2^b$ do not change the adversary's view under the condition that the game outputs 1, and thus,

$$\Pr [1 \leftarrow \mathbf{G}_2^b] = \Pr [1 \leftarrow \mathbf{G}_3^b]. \quad (26)$$

Game $\mathbf{G}_4^b$: In this game, when the oracle INIT is queried with s_{init} , the $\text{st}[s_{\text{init}}]$ is set as $(\widetilde{M}, 0, m_{j^*})$, rather than $(\widetilde{M}, \widetilde{sk}^*, m_{j^*})$. Since in **Case 1** the event st does not occur, the adversary $\mathcal{A}$ never queries REVEAL-STATE with s_{init} to get the internal state $\text{st}[s_{\text{init}}]$ if the game finally outputs 1, and thus $\mathcal{A}$ cannot detect our modification to $\text{st}[s_{\text{init}}]$. Therefore, we have

$$\Pr [1 \leftarrow \mathbf{G}_3^b] = \Pr [1 \leftarrow \mathbf{G}_4^b]. \quad (27)$$

Game $\mathbf{G}_5^b$: This game is the same as $\mathbf{G}_4^b$, except that in $\text{Session}(i^*, j^*)$, it no longer performing decryption on c_{j^*} and c_{i^*} as well as decapsulation on $\widetilde{c}^*$, and directly set $K_{\text{resp}} = K_{\text{init}} := \text{H}(m_{i^*}, m_{j^*}, K^*, i^*, j^*, \widetilde{M}, \widetilde{M}')$. Notice that if the decryptions of c_{j^*} and c_{i^*} and the decapsulation of $\widetilde{c}^*$ all succeed without errors, that is, decrypting c_{j^*} (resp. c_{i^*}) yields m_{j^*} (resp. m_{i^*}) and decapsulating $\widetilde{c}^*$ yields K^* , then games $\mathbf{G}_4^b$ and $\mathbf{G}_5^b$ are equivalent. Hence, we have

$$\begin{aligned} & |\Pr [1 \leftarrow \mathbf{G}_4^b] - \Pr [1 \leftarrow \mathbf{G}_5^b]| \\ & \leq 2 \cdot \mathbb{E}_{(pk, sk) \leftarrow \Pi_{\text{PKE}}.\text{KeyGen}} \left[\Pr_{m \leftarrow \mathcal{M}} \left[\begin{array}{l} \Pi_{\text{PKE}}.\text{Dec}(sk, c) \neq m, \\ c = \Pi_{\text{PKE}}.\text{Enc}(pk, m; \mathbf{G}(m)) \end{array} \right] \right] \\ & \quad \mathbf{G} \xleftarrow{\$} \text{Fcs}[\mathcal{M}, \mathcal{R}] \\ & + \mathbb{E}_{(pk, sk) \leftarrow \Pi_{\text{KEM}}.\text{KeyGen}} \Pr \left[\begin{array}{l} \Pi_{\text{KEM}}.\text{Decap}(sk, c) \neq K, \\ (c, K) \leftarrow \Pi_{\text{KEM}}.\text{Encap}(pk) \end{array} \right] \\ & \leq 2 \cdot \delta_{\text{P}} + \delta_{\text{K}}. \end{aligned} \quad (28)$$

Game $\mathbf{G}_6^b$: This game is the same as $\mathbf{G}_5^b$, except that in $\text{Session}(i^*, j^*)$, it directly set $K_{\text{resp}} = K_{\text{init}} := r^*$, where r^* is a fresh uniformly random value in $\mathcal{K}$ sampled at the game beginning.

Based on the basic rules of the random oracle, games $\mathbf{G}_5^b$ and $\mathbf{G}_6^b$ are equivalent as long as H is never queried with $(*, *, K^*, \dots)$ during the running of the adversary $\mathcal{A}$. Hence, by defining the event

$\text{Bad}_{K^*}^b$: During the running of $\mathcal{A}$, H is ever queried with $(*, *, K^*, \dots)$,

where $b \in \{0, 1\}$ indicates whether the oracle **TEST** returns K_0^* or K_1^* , we have

$$|\Pr[1 \leftarrow \mathbf{G}_5^b] - \Pr[1 \leftarrow \mathbf{G}_6^b]| \leq \Pr[\text{Bad}_{K^*}^b]. \quad (29)$$

As for the $\Pr[\text{Bad}_{K^*}^b]$, we first construt a OW-CPA adversary $\mathcal{B}_b$ ($b \in \{0, 1\}$) against Π_{KEM} as follows, and then bound $\Pr[\text{Bad}_{K^*}^b]$ by using $\mathcal{B}_b$'s advantage.

- The input of $\mathcal{B}_b$ is $(\widetilde{pk}^*, \widetilde{c}^*)$, where $\widetilde{pk}^*$ is the public key and $\widetilde{c}^*$ is the challenge ciphertext.
- $\mathcal{B}_b$ runs game $\mathbf{G}_6^b$, simulates the random oracles $\mathbf{H}_{\text{init}}$, $\mathbf{H}_{\text{resp}}$, $\mathbf{G}$, and $\mathbf{H}$ on-th-fly by using the record list $\mathbf{L}_I$, $\mathbf{L}_R$, $\mathbf{L}_G$, and $\mathbf{L}_H$, respectively.
- When running **Session** (i^*, j^*) , $\mathcal{B}_b$ replaces the $(\widetilde{pk}^*, \widetilde{c}^*)$ generated there by its own input.
- After $\mathcal{A}$ returns its output b' , $\mathcal{B}_b$ uniformly samples an element from $\mathbf{L}_H$, which we denote as $(*, *, K, \dots)$, and then returns K as its final output.

We stress that although $\mathcal{B}_b$ dose not have the secret key $\widetilde{sk}^*$ corresponding to $\widetilde{pk}^*$, it can still run game $\mathbf{G}_6^b$ normally, as the computations of **Session** (i^*, j^*) and **DER** $_{\text{init}}$ in game $\mathbf{G}_6^b$ no longer require $\widetilde{sk}^*$. Obviously, for any $b \in \{0, 1\}$, the running time of $\mathcal{B}_b$ is roughly the same as those of $\mathcal{A}$, and $\text{Bad}_{K^*}^b$ occurring implies that $\mathcal{B}_b$ has an advantage at least $1/q_H$ to successfully guess the key K^* encapsulated by $\widetilde{c}^*$. Thus,

$$\Pr[\text{Bad}_{K^*}^b] \leq q_H \cdot \text{Adv}_{\Pi_{\text{KEM}}}^{\text{OW-CPA}}(\mathcal{B}_b). \quad (30)$$

Game $\mathbf{G}_7^b$: This game is the same as $\mathbf{G}_6^b$, except that the oracle **TEST** directly sets K_0^* to r^* , rather than $\text{sKey}[\text{sID}^*]$. Since outputting 1 in game $\mathbf{G}_6^b$ guarantees $\{s_{\text{init}}, s_{\text{resp}}\} = \{\text{sID}^*\} \cup \mathfrak{M}(\text{sID}^*)$ and $(i^*, j^*) = (\text{holder}[s_{\text{init}}], \text{peer}[s_{\text{init}}])$, the K_0^* in game $\mathbf{G}_6^b$ is also r^* if it finally outputs 1. Thus, game $\mathbf{G}_7^b$ is only a conceptual change and

$$\Pr[1 \leftarrow \mathbf{G}_6^b] = \Pr[1 \leftarrow \mathbf{G}_7^b]. \quad (31)$$

Game $\mathbf{G}_8^b$: This game is the same as $\mathbf{G}_7^b$, except that in **Session** (i^*, j^*) , it sets $K_{\text{resp}} = K_{\text{init}} := 0$. Similar to the above game hop, since outputting 1 in game $\mathbf{G}_7^b$ guarantees $\{s_{\text{init}}, s_{\text{resp}}\} = \{\text{sID}^*\} \cup \mathfrak{M}(\text{sID}^*)$ and sID^* is fresh, the adversary $\mathcal{A}$ never queries **REVEAL** with s_{init} or s_{resp} to get the K_{init} or K_{resp} if the game finally outputs 1, and thus $\mathcal{A}$ cannot detect our change in game $\mathbf{G}_8^b$. Hence, we have

$$\Pr[1 \leftarrow \mathbf{G}_7^b] = \Pr[1 \leftarrow \mathbf{G}_8^b]. \quad (32)$$

In game $\mathbf{G}_8^b$, K_0^* and K_1^* are both uniformly random values in $\mathcal{K}$, and thus

$$\Pr[1 \leftarrow \mathbf{G}_8^0] = \Pr[1 \leftarrow \mathbf{G}_8^1]. \quad (33)$$

Finally, by combining Eq. (23) to Eq. (33), and folding adversaries $\mathcal{B}_0$ and $\mathcal{B}_1$ into one adversary $\mathcal{B}$, we obtain

$$\text{Case 1} \leq 2S \cdot \gamma(\Pi_{\text{PKE}}. \text{Enc}) + 4S^2 N^2 \cdot \delta_P + 2S^2 N^2 \cdot \delta_K + 2S^2 N^2 q_H \cdot \text{Adv}_{\Pi_{\text{KEM}}}^{\text{OW-CPA}}(\mathcal{B}).$$

This completes the proof of Lemma 4. $\square$

A.2 Proof of Lemma 5

Proof. To prove this lemma, we introduce a sequence of games $\mathbf{G}_0^b$ to $\mathbf{G}_8^b$ ($b \in \{0, 1\}$) as shown in Fig. 10. In these games, H_{init} , H_{resp} , G , and H are random oracles, and the adversary $\mathcal{A}$ can also query a set of oracles $\mathcal{O}_{\text{AKE}} = \{\text{EST}, \text{INIT}, \text{DER}_{\text{resp}}, \text{DER}_{\text{init}}, \text{REVEAL}, \text{REV-STATE}, \text{CORRUPT}, \text{TEST}\}$.

Game $\mathbf{G}_0^b$: This game is the same as $\text{KeyIND}_{\Pi_{\text{AKE}}}^{\mathcal{A}, b}$ (Fig. 4), except that it additionally checks whether $\mathfrak{M}(\text{sID}^*) \neq \emptyset$ and $\neg sk \wedge \text{st}$ occurs, and directly returns 0 if the check fails. Thus,

$$\Pr[1 \leftarrow \mathbf{G}_0^b] = \Pr\left[1 \leftarrow \text{KeyIND}_{\Pi_{\text{AKE}}}^{\mathcal{A}, b} \wedge \mathfrak{M}(\text{sID}^*) \neq \emptyset \wedge \neg sk \wedge \text{st}\right]. \quad (34)$$

Game $\mathbf{G}_1^b$: Based on game $\mathbf{G}_0^b$, this game also checks whether $|\mathfrak{M}(\text{sID}^*)| = 1$, and directly returns 0 if this check fails.

Let $\text{Bad}_{\mathfrak{M}}$ denote the event that $|\mathfrak{M}(\text{sID}^*)| > 1$. If $\text{Bad}_{\mathfrak{M}}$ does not occur, the number of elements of the set $\mathfrak{M}(\text{sID}^*)$ is at most 1 throughout the execution, which implies games $\mathbf{G}_0^b$ and $\mathbf{G}_1^b$ are equivalent. We next analyze the probability of $\text{Bad}_{\mathfrak{M}}$ according to the role of sID^* .

- If $\text{role}[\text{sID}^*] = \text{initiator}$, by the definition of the matching session (Definition 9), $\text{Bad}_{\mathfrak{M}}$ occurring implies that the adversary $\mathcal{A}$ has performed the following operations:
 - $\mathcal{A}$ establishes a new session sID' , which is different from another session sID contained in the set $\mathfrak{M}(\text{sID}^*)$. Here $\text{role}[\text{sID}] = \text{responder}$, $\text{rece}[\text{sID}] = \text{sent}[\text{sID}^*]$, $\text{sent}[\text{sID}] = \text{rece}[\text{sID}^*]$, and $\text{sent}[\text{sID}] = (c_i, \tilde{c})$ is generated by querying DER_{resp} . Then, $\mathcal{A}$ queries DER_{resp} with $(\text{sID}', \text{sent}[\text{sID}^*])$ to label sID' as the responder session, and finally obtains the return value $(c'_i, \tilde{c}')$ equals to $\text{sent}[\text{sID}] (= (c_i, \tilde{c}))$.

During this process, the oracle DER_{resp} always generates and outputs two ciphertexts of Π_{PKE} and Π_{KEM} honestly. Hence, the probability that DER_{resp} produces the same output is bounded by $\gamma(\Pi_{\text{PKE}}.\text{Enc}) \cdot \gamma(\Pi_{\text{KEM}}.\text{Encap})$, where $\gamma(\Pi_{\text{PKE}}.\text{Enc})$ is the encry-collision probability of Π_{PKE} , and $\gamma(\Pi_{\text{KEM}}.\text{Encap})$ is the encap-collision probability of Π_{KEM} (Definition 1, 5). Since $\mathcal{A}$ can establish at most $S-1$ sessions apart from the test session, the probability that $\text{Bad}_{\mathfrak{M}}$ occurs under $\text{role}[\text{sID}^*] = \text{initiator}$ is at most $(S-1) \cdot \gamma(\Pi_{\text{PKE}}.\text{Enc}) \cdot \gamma(\Pi_{\text{KEM}}.\text{Encap})$.

- If $\text{role}[\text{sID}^*] = \text{responder}$, by the definition of the matching session (Definition 9), $\text{Bad}_{\mathfrak{M}}$ occurring implies that $\mathcal{A}$ has performed the following operations: $\mathcal{A}$ establishes a new session sID' , which is different from another session sID contained in the set $\mathfrak{M}(\text{sID}^*)$; then $\mathcal{A}$ queries INIT with sID' to label sID' as the initiator session, and finally obtains the return value $(c'_j, \widetilde{pk'})$ equals to $\text{sent}[\text{sID}] (= (c_j, \widetilde{pk}))$, where $\text{sent}[\text{sID}]$ is also generated by querying INIT . During this process, the oracle INIT always generates a ciphertext of Π_{PKE} and a public key of Π_{KEM} honestly. Hence, the probability that INIT produces the same output is bounded by $\gamma(\Pi_{\text{PKE}}.\text{Enc}) \cdot \gamma(\Pi_{\text{KEM}}.\text{KeyGen})$, where $\gamma(\Pi_{\text{KEM}}.\text{KeyGen})$ denotes the key-collision probability of Π_{KEM} (Definition 5). Since $\mathcal{A}$ can establish at most $S-1$

GAMES $\mathbf{G}_0^b$ - $\mathbf{G}_{15}^b$ ($b \in \{0, 1\}$)	INIT(sID)
<pre> 1, cnt := 0 sID* := 0 2, H_{init}, H_{resp}, H $\xleftarrow{\\$}$ Fcs[{0, 1}[*], K] G $\xleftarrow{\\$}$ Fcs[M, R] s_{init}, s_{resp} $\xleftarrow{\\$}$ {1, ..., S} //G₂^b-G₁₅^b i[*], j[*] $\xleftarrow{\\$}$ {1, ..., N} //G₂^b-G₁₅^b Flag := 0 //G₇^b G := G_{good} //G₈^b-G₁₁^b r* $\xleftarrow{\\$}$ R //G₁₃^b-G₁₅^b k* $\xleftarrow{\\$}$ K //G₁₃^b-G₁₅^b 3, for i ∈ {1, ..., N} (pk_i, sk_i) ← Π_{PKE}.KeyGen, s_i $\xleftarrow{\\$}$ M sk_i' := (sk_i, s_i), crp[i] := false end for run Session(i[*], j[*]) //G₃^b-G₁₅^b 4, b' ← A^{OAKE}.H_{init}, H_{resp}, G, H(pk₁, ..., pk_N) 5, if sID* is not fresh, return 0 if M(sID*) = ∅ ∨ sk ∨ stRev[sID*] = false //G₀^b-G₁₅^b return 0 //G₀^b-G₁₅^b if M(sID*) ≠ 1, return 0 //G₁^b-G₁₅^b if role[s_{init}] ≠ initiator ∨ role[s_{resp}] ≠ responder //G₂^b-G₁₅^b ∨ {s_{init}, s_{resp}} ≠ {sID*} ∪ M(sID*) //G₂^b-G₁₅^b ∨ (i[*], j[*]) ≠ (holder[s_{init}], peer[s_{init}]) //G₂^b-G₁₅^b return 0 //G₂^b-G₁₅^b 6, return b'</pre>	<pre> 1, if holder[sID] = ⊥ ∨ sent[sID] ≠ ⊥, return ⊥ 2, role[sID] := initiator 3, if sID = s_{init} //G₃^b-G₁₅^b (sent[s_{init}], st[s_{init}]) := (M̃, (M̃, sk̃*, m_j*)) //G₃^b-G₁₅^b return M̃ else (i, j) := (holder[sID], peer[sID]) (M, st) ← GC-AKE.Init(sk_i, pk_j) if M = M̃, Flag := 1 //G₇^b-G₁₅^b (sent[sID], st[sID]) := (M, st) return M</pre>
DER _{init} (sID, M')	DER _{resp} (sID, M)
<pre> 1, if holder[sID] = ⊥ ∨ st[sID] = ⊥ ∨ sKey[sID] ≠ ⊥ return ⊥ 2, if sID = s_{init} ∧ M' = M̃' //G₃^b-G₁₅^b sKey[s_{init}] := K_{init} //G₃^b-G₁₅^b rece[s_{init}] := M̃' //G₃^b-G₁₅^b else (i, j) := (holder[sID], peer[sID]) Parse st[sID] as (M, sk̃, m_j) Parse M' as (c_i, c̃) if i = i*, j ≠ i*, K_i := H₁(c_i, m_j, K', i, j, M, M') //G₁₁^b-G₁₅^b if i = i*, j = i*, K_i := H₃(c_i, c_j, K', i, j, M, M') //G₁₁^b-G₁₅^b m_i' := Π_{PKE}.Dec(sk_i, c_i) K' := Π_{KEM}.Decap(sk̃, c̃) if m_i' = ⊥ ∨ Π_{PKE}.Enc(pk_i, m_i'; G(m_i')) ≠ c_i K_i := H_{init}(s_i, c_i, m_j, c̃, i, j, M, M') if i = i*, K_i := F₁(c_i, m_j, c̃, i, j, M, M') //G₆^b-G₁₅^b else K_i := H(m_i', m_j, K', i, j, M, M') else K_i := H̃(m_i', m_j, K', i, j, M, M') //G₁₀^b-G₁₅^b sKey[sID] := K_i rece[sID] := M'</pre>	<pre> 1, if holder[sID] = ⊥ ∨ sKey[sID] ≠ ⊥ ∨ role[sID] = initiator, return ⊥ 2, role[sID] := responder 3, if sID = s_{resp} ∧ M = M̃ //G₃^b-G₁₅^b sKey[s_{resp}] := K_{resp} //G₃^b-G₁₅^b (sent[s_{resp}], rece[s_{resp}]) := (M̃', M̃) //G₃^b-G₁₅^b return M̃' //G₃^b-G₁₅^b else (j, i) := (holder[sID], peer[sID]) Parse M as (pk̃, c_j) m_i $\xleftarrow{\\$}$ M, c_i := Π_{PKE}.Enc(pk_i, m_i; G(m_i)) (c̃, K*) := Π_{KEM}.Encap(pk̃), M' := (c_i, c*) if (M, M') = (M̃, M̃'), Flag := 1 //G₇^b-G₁₅^b if j = i*, i ≠ i*, K_r := H₂(m_i, c_j, K*, i, j, M, M') //G₁₁^b-G₁₅^b if j = i*, i = i*, K_r := H₃(c_i, c_j, K*, i, j, M, M') //G₁₁^b-G₁₅^b m_j' := Π_{PKE}.Dec(sk_j, c_j) if m_j' = ⊥ ∨ Π_{PKE}.Enc(pk_j, m_j; G(m_j')) ≠ c_j K_r := H_{resp}(s_j, m_i, c_j, K*, i, j, M, M') if j = i*, K_r := F_R(m_i, c_j, K*, i, j, M, M') //G₆^b-G₁₅^b else K_r := H(m_i, m_j', K*, i, j, M, M') else K_r := H̃(m_i, m_j', K*, i, j, M, M') //G₁₀^b-G₁₅^b sKey[sID] := K_r, (sent[sID], rece[sID]) := (M', M) return M'</pre>
TEST(sID)	TEST(sID) //only one query
<pre> 1, sID* := sID //test session's id 2, if sKey[sID*] = ⊥, return ⊥ 3, K₀* := sKey[sID*], K₁* ← K K₀* := k*, K₁* ← K //G₁₄^b-G₁₅^b 4, return K_b*</pre>	<pre> CORRUPT(i ∈ {1, ..., N}) crp[i] := true if i = i*, return 0 //G₅^b-G₁₅^b return (sk_i, s_i)</pre>

Fig. 10: Games $\mathbf{G}_0^b$ to $\mathbf{G}_{15}^b$ used in the proof of Lemma 5. Here for brevity, we only list the oracles TEST, CORRUPT, INIT, DER_{resp} and DER_{init} that need to be modified in our game sequence. The definitions of all other unlisted oracles are identical to those given in Fig. 4.

sessions apart from the test session, the probability that $\text{Bad}_{\mathfrak{M}}$ occurs under $\text{role}[\text{sID}^*] = \text{responder}$ is at most $(S - 1) \cdot \gamma(\Pi_{\text{PKE}}.\text{Enc}) \cdot \gamma(\Pi_{\text{KEM}}.\text{KeyGen})$.

Combining the above analysis, we have

$$\begin{aligned}
& |\Pr[1 \leftarrow \mathbf{G}_0^b] - \Pr[1 \leftarrow \mathbf{G}_1^b]| \\
& \leq \Pr[\text{Bad}_{\mathfrak{M}}] \\
& = \Pr[\text{Bad}_{\mathfrak{M}} \wedge \text{role}[\text{sID}^*] = \text{initiator}] + \Pr[\text{Bad}_{\mathfrak{M}} \wedge \text{role}[\text{sID}^*] = \text{responder}] \\
& \leq (S - 1) \cdot \gamma(\Pi_{\text{PKE}}.\text{Enc}) \cdot (\gamma(\Pi_{\text{KEM}}.\text{Encap}) + \gamma(\Pi_{\text{KEM}}.\text{KeyGen})).
\end{aligned} \tag{35}$$

Game $\mathbf{G}_2^b$: This game is the same as $\mathbf{G}_1^b$, except that at the very beginning, it uniformly samples two sessions $s_{\text{init}}, s_{\text{resp}}$ from $\{1, \dots, S\}$ and two indexes i^*, j^* from $\{1, \dots, N\}$, and checks whether $\text{role}[s_{\text{init}}] = \text{initiator}$, $\text{role}[s_{\text{resp}}] = \text{responder}$, $\{s_{\text{init}}, s_{\text{resp}}\} = \{\text{sID}^*\} \cup \mathfrak{M}(\text{sID}^*)$, and $(i^*, j^*) = (\text{holder}[s_{\text{init}}], \text{peer}[s_{\text{init}}])$ at the end. If such check fails, game $\mathbf{G}_2^b$ directly returns 0.

In game $\mathbf{G}_1^b$, the additional check $\mathfrak{M}(\text{sID}^*) \neq \emptyset \wedge |\mathfrak{M}(\text{sID}^*)| = 1$ ensures that the set $\mathfrak{M}(\text{sID}^*)$ contains exactly one session, which we denote by msID^* . Since msID^* is the matching session of sID^* , the roles of sID^* and msID^* must be one initiator and the other responder. With this observation, in game $\mathbf{G}_2^b$, we make a random guess at the beginning of the game, hoping to directly obtain sID^* and msID^* (i.e., the s_{init} and s_{resp}). In addition, we also guess that the holder and peer of session s_{init} are i^* and j^* , respectively. These guessing steps are necessary; with it, we can embed the challenge ciphertext of Π_{PKE} into the key negotiation process between s_{init} and s_{resp} . As for the difference between games $\mathbf{G}_1^b$ and $\mathbf{G}_2^b$, we have

$$\Pr[1 \leftarrow \mathbf{G}_1^b] = S^2 N^2 \cdot \Pr[1 \leftarrow \mathbf{G}_2^b]. \tag{36}$$

Game $\mathbf{G}_3^b$: This game is the same as $\mathbf{G}_2^b$, except that it introduces a new procedure $\text{Session}(i^*, j^*)$ as shown in Fig. 11 and modifies the computations of INIT , DER_{resp} and DER_{init} according to the internal random values generated by $\text{Session}(i^*, j^*)$, as detailed below:

- $\text{Session}(i^*, j^*)$ honestly simulates the key negotiation process between users P_{i^*} and P_{j^*} . Here, P_{i^*} acts as the initiator; it generates the ephemeral key pair $(\widetilde{pk}^*, \widetilde{sk}^*)$ of Π_{KEM} and returns $\widetilde{M} := (\widetilde{pk}^*, c_{j^*})$ to P_{j^*} . As the responder, P_{j^*} generates the key K_{resp} after receiving $\widetilde{M}$ and sends back $\widetilde{M}' := (c_{i^*}, \widetilde{c}^*)$ to P_{i^*} . Finally, P_{i^*} derives its session key K_{init} .
- For INIT , if the input is s_{init} , it sets $(\text{sent}[s_{\text{init}}], \text{st}[s_{\text{init}}]) = (\widetilde{M}, (\widetilde{M}, \widetilde{sk}^*, m_{j^*}))$ and returns $\widetilde{M}$. For DER_{init} , if the input is $(s_{\text{init}}, \widetilde{M}')$, it sets $\text{sKey}[s_{\text{init}}] := K_{\text{init}}$ and $\text{rece}[s_{\text{init}}] := \widetilde{M}'$.
- For DER_{resp} , if the input is $(s_{\text{resp}}, \widetilde{M})$, it sets $\text{sKey}[s_{\text{resp}}] := K_{\text{resp}}$, $(\text{sent}[s_{\text{resp}}], \text{rece}[s_{\text{resp}}]) := (\widetilde{M}', \widetilde{M})$, and then returns $\widetilde{M}'$.

For all remaining inputs, INIT , DER_{resp} and DER_{init} perform their computations in accordance with the specifications given in Fig. 4.

Session(i^*, j^*) // $\mathbf{G}_3^b$	Session(i^*, j^*) // $\mathbf{G}_4^b - \mathbf{G}_{15}^b$
1, $m_{j^*} \xleftarrow{\$} \mathcal{M}$, $c_{j^*} := \Pi_{\text{PKE}}.\text{Enc}(pk_{j^*}, m_{j^*}; \mathbf{G}(m_{j^*}))$ $(\widetilde{pk}^*, \widetilde{sk}^*) \leftarrow \Pi_{\text{KEM}}.\text{KeyGen}$, $\widetilde{M} := (\widetilde{pk}^*, c_{j^*})$	1, $m_{j^*} \xleftarrow{\$} \mathcal{M}$ $c_{j^*} := \Pi_{\text{PKE}}.\text{Enc}(pk_{j^*}, m_{j^*}; \mathbf{G}(m_{j^*}))$ $(\widetilde{pk}^*, \widetilde{sk}^*) \leftarrow \Pi_{\text{KEM}}.\text{KeyGen}$ $\widetilde{M} := (\widetilde{pk}^*, c_{j^*})$
2, $m_{i^*} \xleftarrow{\$} \mathcal{M}$, $c_{i^*} := \Pi_{\text{PKE}}.\text{Enc}(pk_{i^*}, m_{i^*}; \mathbf{G}(m_{i^*}))$ $(\widetilde{c}^*, K^*) \leftarrow \Pi_{\text{KEM}}.\text{Encap}(\widetilde{pk}^*)$, $\widetilde{M}' := (c_{i^*}, \widetilde{c}^*)$ $m'_{j^*} := \Pi_{\text{PKE}}.\text{Dec}(sk_{j^*}, c_{j^*})$ if $c_{j^*} \neq \Pi_{\text{PKE}}.\text{Enc}(pk_{j^*}, m'_{j^*}; \mathbf{G}(m'_{j^*}))$ $K_{\text{resp}} := \mathbf{H}_{\text{resp}}(s_{j^*}, m_{i^*}, c_{j^*}, K^*, i^*, j^*, \widetilde{M}, \widetilde{M}')$ $K_{\text{resp}} := \mathbf{H}(m_{i^*}, m'_{j^*}, K^*, i^*, j^*, \widetilde{M}, \widetilde{M}')$	2, $m_{i^*} \xleftarrow{\$} \mathcal{M}$ $c_{i^*} := \Pi_{\text{PKE}}.\text{Enc}(pk_{i^*}, m_{i^*}; \mathbf{G}(m_{i^*}))$ $c_{i^*} := \Pi_{\text{PKE}}.\text{Enc}(pk_{i^*}, m_{i^*}; r^*)$ // $\mathbf{G}_{13}^b - \mathbf{G}_{15}^b$ $(\widetilde{c}^*, K^*) \leftarrow \Pi_{\text{KEM}}.\text{Encap}(\widetilde{pk}^*)$ $\widetilde{M}' := (c_{i^*}, \widetilde{c}^*)$
3, $m'_{i^*} := \Pi_{\text{PKE}}.\text{Dec}(sk_{i^*}, c_{i^*})$ $K' := \Pi_{\text{KEM}}.\text{Decap}(\widetilde{sk}^*, \widetilde{c}^*)$ if $c_{i^*} \neq \Pi_{\text{PKE}}.\text{Enc}(pk_{i^*}, m'_{i^*}; \mathbf{G}(m'_{i^*}))$ $K_{\text{init}} := \mathbf{H}_{\text{init}}(s_{i^*}, c_{i^*}, m_{j^*}, \widetilde{c}^*, i^*, j^*, \widetilde{M}, \widetilde{M}')$ $K_{\text{init}} := \mathbf{H}(m'_{i^*}, m_{j^*}, K', i^*, j^*, \widetilde{M}, \widetilde{M}')$	3, $K_{\text{resp}} := \mathbf{H}(m_{i^*}, m_{j^*}, K^*, i^*, j^*, \widetilde{M}, \widetilde{M}')$ $K_{\text{init}} := \mathbf{H}(m_{i^*}, m_{j^*}, K^*, i^*, j^*, \widetilde{M}, \widetilde{M}')$ $K_{\text{resp}} = K_{\text{init}} := k^*$ // $\mathbf{G}_{13}^b - \mathbf{G}_{15}^b$ $K_{\text{resp}} = K_{\text{init}} := 0$ // $\mathbf{G}_{15}^b$

Fig. 11: The procedure $\text{Session}(i^*, j^*)$ used in the proof of Lemma 5.

In game $\mathbf{G}_3^b$, we simulate the key negotiation process between s_{init} and s_{resp} in advance by executing $\text{Session}(i^*, j^*)$. We then perform several checks on the inputs of INIT , DER_{resp} and DER_{init} , and insert certain random values generated during this simulated negotiation into the lists sKey , sent , rece , and st . Since outputting 1 in both $\mathbf{G}_2^b$ and $\mathbf{G}_3^b$ guarantees $\{s_{\text{init}}, s_{\text{resp}}\} = \{\text{sID}^*\} \cup \mathcal{M}(\text{sID}^*)$ and $(i^*, j^*) = (\text{holder}[s_{\text{init}}], \text{peer}[s_{\text{init}}])$, the key negotiation process of the test session sID^* in $\mathbf{G}_2^b$ is equivalent to that simulated by $\text{Session}(i^*, j^*)$ in $\mathbf{G}_3^b$. Meanwhile, $\mathbf{G}_3^b$ appropriately inserts corresponding random values into the lists sKey , sent , rece , and st . Hence, the modifications introduced in $\mathbf{G}_3^b$ relative to $\mathbf{G}_2^b$ do not change the adversary's view under the condition that the game outputs 1, and thus,

$$\Pr[1 \leftarrow \mathbf{G}_2^b] = \Pr[1 \leftarrow \mathbf{G}_3^b]. \quad (37)$$

Game $\mathbf{G}_4^b$: This game is the same as $\mathbf{G}_3^b$, except that in $\text{Session}(i^*, j^*)$, it no longer performing decryption on c_{j^*} and c_{i^*} as well as decapsulation on $\widetilde{c}^*$, and directly set $K_{\text{resp}} = K_{\text{init}} := \mathbf{H}(m_{i^*}, m_{j^*}, K^*, i^*, j^*, \widetilde{M}, \widetilde{M}')$. Notice that if the decryptions of c_{j^*} and c_{i^*} and the decapsulation of $\widetilde{c}^*$ all succeed without errors, that is, decrypting c_{j^*} (resp. c_{i^*}) yields m_{j^*} (resp. m_{i^*}) and decapsulating $\widetilde{c}^*$ yields K^* , then games $\mathbf{G}_3^b$ and $\mathbf{G}_4^b$ are equivalent. Hence, we have

$$\begin{aligned}
& |\Pr[1 \leftarrow \mathbf{G}_4^b] - \Pr[1 \leftarrow \mathbf{G}_5^b]| \\
& \leq 2 \cdot \mathbb{E}_{(pk, sk) \leftarrow \Pi_{\text{PKE}}.\text{KeyGen}} \left[\Pr_{m \xleftarrow{\$} \mathcal{M}} \left[\Pi_{\text{PKE}}.\text{Dec}(sk, c) \neq m, \right. \right. \\
& \quad \left. \left. c = \Pi_{\text{PKE}}.\text{Enc}(pk, m; \mathbf{G}(m)) \right] \right] \\
& \quad \mathbb{G} \xleftarrow{\$} \text{Fcs}[\mathcal{M}, \mathcal{R}] \\
& \quad + \mathbb{E}_{(pk, sk) \leftarrow \Pi_{\text{KEM}}.\text{KeyGen}} \Pr \left[\Pi_{\text{KEM}}.\text{Decap}(sk, c) \neq K, \right. \\
& \quad \left. (c, K) \leftarrow \Pi_{\text{KEM}}.\text{Encap}(pk) \right] \\
& \leq 2 \cdot \delta_P + \delta_K.
\end{aligned} \quad (38)$$

The $\text{Session}(i^*, j^*)$ of game $\mathbf{G}_4^b$ is clearly described in Fig. 11.

Game $\mathbf{G}_5^b$: This game is the same as $\mathbf{G}_4^b$, except that the oracle CORRUPT directly returns 0 for the input i^* . Notice that outputting 1 in game $\mathbf{G}_4^b$ guarantees $\{s_{\text{init}}, s_{\text{resp}}\} = \{\text{sID}^*\} \cup \mathfrak{M}(\text{sID}^*)$ and $(i^*, j^*) = (\text{holder}[s_{\text{init}}], \text{peer}[s_{\text{init}}])$. Therefore, in the **Case 2** we considered, the game outputting 1 implies that the adversary $\mathcal{A}$ never queries CORRUPT with i^* to get the long-term secret key (sk_{i^*}, s_{i^*}) of $\mathbf{P}_{i^*}$. That is to say, under the condition that the game outputting 1, $\mathcal{A}$ cannot detect our modification to CORRUPT . Thus

$$\Pr [1 \leftarrow \mathbf{G}_4^b] = \Pr [1 \leftarrow \mathbf{G}_5^b]. \quad (39)$$

Game $\mathbf{G}_6^b$: This game introduces two new random oracles F_R and F_I , and modifies the keys produced by DER_{resp} and DER_{init} . For $\text{DER}_{\text{resp}}(\text{sID}, M)$, if $(\text{sID}, M) \neq (s_{\text{resp}}, \widetilde{M})$, $\text{holder}[\text{sID}] = i^*$, and decryption of c_j fails, K_r is set to $F_R(m_i, c_j, K^*, i, j, M, M')$; for $\text{DER}_{\text{init}}(\text{sID}, M')$, if $(\text{sID}, M') \neq (s_{\text{init}}, \widetilde{M}')$, $\text{holder}[\text{sID}] = i^*$, and decryption of c_i fails, K_i is set to $F_I(c_i, m_j, \widetilde{c}, i, j, M, M')$.

It is easy to see that these two games are equivalent, if oracles H_{resp} and H_{init} are never queried with input $(s_{i^*}, \dots)$ during the running of the adversary $\mathcal{A}$. Since in both $\mathbf{G}_5^b$ and $\mathbf{G}_6^b$, s_{i^*} is uniformly sampled from $\mathcal{M}$ and the running of $\mathcal{A}$ no longer depends on s_{i^*} , we have

$$|\Pr [1 \leftarrow \mathbf{G}_5^b] - \Pr [1 \leftarrow \mathbf{G}_6^b]| \leq \frac{q_R + q_I}{|\mathcal{M}|}. \quad (40)$$

Game $\mathbf{G}_7^b$: This game is the same as $\mathbf{G}_6^b$, except that it adds a flag in oracles INIT and DER_{resp} . Specifically, $\mathbf{G}_7^b$ defines $\text{Flag} := 0$ at the game begins, and set $\text{Flag} = 1$ if $\text{INIT}(\text{sID})$ with $\text{sID} \neq s_{\text{init}}$ generates a M equals to $\widetilde{M}$, or $\text{DER}_{\text{resp}}(\text{sID}, M)$ with $(\text{sID}, M) \neq (s_{\text{resp}}, \widetilde{M})$ generates a M' such that $(M, M') = (\widetilde{M}, \widetilde{M}')$. Game $\mathbf{G}_7^b$ finally outputs 0 if $\text{Flag} = 1$.

Although the adversary $\mathcal{A}$ can control the inputs of oracles INIT and DER_{resp} , they generate M and M' by honestly executing the protocol GC-AKE. This implies that if the Flag of game $\mathbf{G}_7^b$ finally equals 1, the following event must occur:

Two independent runs of $\Pi_{\text{KEM}}.\text{KeyGen}$ produce the same public key, or two independent runs of $\Pi_{\text{KEM}}.\text{Encap}(pk)$ produce the same ciphertexts.

At this point, by using the $\gamma(\Pi_{\text{KEM}}.\text{KeyGen})$ and $\gamma(\Pi_{\text{KEM}}.\text{Encap})$ defined in Definition 5, together with the fact that $\mathcal{A}$ can establish at most S sessions, we have

$$|\Pr [1 \leftarrow \mathbf{G}_6^b] - \Pr [1 \leftarrow \mathbf{G}_7^b]| \leq S \cdot \gamma(\Pi_{\text{KEM}}.\text{KeyGen}) + S \cdot \gamma(\Pi_{\text{KEM}}.\text{Encap}). \quad (41)$$

Before defining the next game, we first introduce some parameters that will be used later. For a fixed (pk, sk) pair sampled by $\Pi_{\text{PKE}}.\text{KeyGen}$ and a $m \in \mathcal{M}$, we define sets

$$\mathcal{R}_{\text{bad}}(pk, sk, m) := \{r \in \mathcal{R} : \Pi_{\text{PKE}}.\text{Dec}(sk, \Pi_{\text{PKE}}.\text{Enc}(pk, m; r)) \neq m\}$$

and

$$\mathcal{R}_{\text{good}}(pk, sk, m) := \mathcal{R} \setminus \mathcal{R}_{\text{bad}}(pk, sk, m). \quad (42)$$

Here $\mathcal{M}/\mathcal{R}$ is the message/randomness space of Π_{PKE} . Then, we define two parameters

$$\delta(pk, sk, m) := \frac{|\mathcal{R}_{\text{bad}}(pk, sk, m)|}{|\mathcal{R}|} \text{ and } \delta(pk, sk) := \max_{m \in \mathcal{M}} \delta(pk, sk, m). \quad (43)$$

By the definition of δ -correct given in Definition 2, we have

$$\mathbb{E}_{(pk, sk) \leftarrow \Pi_{\text{PKE}}.\text{KeyGen}}[\delta(pk, sk)] \leq \delta_P$$

since Π_{PKE} is δ_P -correct.

Game $\mathbf{G}_8^b$: This game is the same as $\mathbf{G}_7^b$, except that $\mathbf{G}$ is set as $\mathbf{G}_{\text{good}}$, a function with domain $\mathcal{M}$ and codomain $\mathcal{R}$, and $\mathbf{G}_{\text{good}}(m)$ is uniformly sampled from $\mathcal{R}_{\text{good}}(pk_{i^*}, sk_{i^*}, m)$ for any $m \in \mathcal{M}$.

For $\mathbf{G} \xleftarrow{\$} \text{Fcs}[\mathcal{M}, \mathcal{R}]$ and a fixed (pk_{i^*}, sk_{i^*}) sampled by $\Pi_{\text{KEM}}.\text{KeyGen}$, we construct the following algorithm $\mathcal{B}_b^{\mathbf{G}}(pk_{i^*}, sk_{i^*})$ ($b \in \{0, 1\}$) with classical oracle access to $\mathbf{G}$:

- $\mathcal{B}_b^{\mathbf{G}}(pk_{i^*}, sk_{i^*})$ runs game $\mathbf{G}_7^b$ and finally returns the game's output. In particular, $\mathcal{B}_b$ simulates random oracles $\mathbf{H}_{\text{init}}$, $\mathbf{H}_{\text{resp}}$, $\mathbf{F}_I$, $\mathbf{F}_R$, and $\mathbf{H}$ on-the-fly, replaces the (pk_{i^*}, sk_{i^*}) generated in game $\mathbf{G}_7^b$ by its own input, and replaces calls to $\mathbf{G}$ in game $\mathbf{G}_7^b$ by calls to its own oracle.

Here $\mathcal{B}_b$ makes at most $q_{\mathbf{G}}$ queries to its oracle $\mathbf{G}$ and

$$\Pr[1 \leftarrow \mathbf{G}_7^b : (pk_{i^*}, sk_{i^*})] = \Pr\left[1 \leftarrow \mathcal{B}_b^{\mathbf{G}}(pk_{i^*}, sk_{i^*}) : \mathbf{G} \xleftarrow{\$} \text{Fcs}[\mathcal{M}, \mathcal{R}]\right]. \quad (44)$$

The $\Pr[1 \leftarrow \mathbf{G} : (pk, sk)]$ used here and follows denotes the probability that game $\mathbf{G}$ outputs 1 under a fixed (pk, sk) pair. Since game $\mathbf{G}_8^b$ is the same as game $\mathbf{G}_7^b$ except that $\mathbf{G}$ is changed into $\mathbf{G}_{\text{good}}$, we have

$$\Pr[1 \leftarrow \mathbf{G}_8^b : (pk_{i^*}, sk_{i^*})] = \Pr\left[1 \leftarrow \mathcal{B}_b^{\mathbf{G}_{\text{good}}}(pk_{i^*}, sk_{i^*})\right]. \quad (45)$$

Let $\mathbf{N}_1 : \mathcal{M} \rightarrow \{0, 1\}$ be a random function such that $\Pr[\mathbf{N}_1(m) = 1] = \delta(pk, sk, m)$ and $\Pr[\mathbf{N}_1(m) = 0] = 1 - \delta(pk, sk, m)$ for any $m \in \mathcal{M}$. Here the value $\delta(pk, sk, m)$ is defined in Eq. (43). Let $\mathbf{N}_2 : \mathcal{M} \rightarrow \{0, 1\}$ be a constant function such that $\mathbf{N}_2(m) = 0$ for any $m \in \mathcal{M}$. Then, by running $\mathcal{B}_b^{\mathbf{G}}(pk_{i^*}, sk_{i^*})$, we construct an algorithm $\mathcal{C}_b^{\mathbf{N}}(pk_{i^*}, sk_{i^*})$ ($\mathbf{N} \in \{\mathbf{N}_1, \mathbf{N}_2\}$) as shown in Fig. 12. Here $\mathcal{C}_b^{\mathbf{N}}(pk_{i^*}, sk_{i^*})$ introduces a new oracle $\mathbf{K}$ relates to its quantum oracle $\mathbf{N}$ and replaces $\mathbf{G}$ by $\mathbf{K}$. Now by the definitions of algorithms $\mathcal{B}_b$ and $\mathcal{C}_b$, we have

$$\begin{aligned} & \left| \Pr\left[1 \leftarrow \mathcal{B}_b^{\mathbf{G}}(pk_{i^*}, sk_{i^*}) : \mathbf{G} \xleftarrow{\$} \text{Fcs}[\mathcal{M}, \mathcal{R}]\right] - \Pr\left[1 \leftarrow \mathcal{B}_b^{\mathbf{G}_{\text{good}}}(pk_{i^*}, sk_{i^*})\right] \right| \\ &= \left| \Pr\left[1 \leftarrow \mathcal{C}_b^{\mathbf{N}_1}(pk_{i^*}, sk_{i^*})\right] - \Pr\left[1 \leftarrow \mathcal{C}_b^{\mathbf{N}_2}(pk_{i^*}, sk_{i^*})\right] \right| \\ &\stackrel{(a)}{\leq} q_{\mathbf{G}} \cdot \max_{m \in \mathcal{M}} \delta(pk_{i^*}, sk_{i^*}, m) \\ &\stackrel{(b)}{=} q_{\mathbf{G}} \cdot \delta(pk_{i^*}, sk_{i^*}). \end{aligned} \quad (46)$$

Here (a) follows from the fact that if $\mathcal{C}_b$ never queries its oracle with any m satisfying $N(m) = 1$, then $\mathcal{C}_b^{N_1}$ and $\mathcal{C}_b^{N_2}$ are equivalent. (b) uses the definition of $\delta(pk, sk)$ shown in Eq. (43). By combining the above equation with Eq. (44) and Eq. (45), we obtain

$$|\Pr[1 \leftarrow \mathbf{G}_7^b : (pk_{i^*}, sk_{i^*})] - \Pr[1 \leftarrow \mathbf{G}_8^b : (pk_{i^*}, sk_{i^*})]| \leq q_G \cdot \delta(pk_{i^*}, sk_{i^*}).$$

By averaging over $(pk_{i^*}, sk_{i^*}) \leftarrow \Pi_{\text{PKE}}.\text{KeyGen}$, we finally obtain

$$|\Pr[1 \leftarrow \mathbf{G}_7^b] - \Pr[1 \leftarrow \mathbf{G}_8^b]| \leq q_G \cdot \delta_P. \quad (47)$$

$\mathcal{C}_b^N(pk_{i^*}, sk_{i^*}) \ b \in \{0, 1\}$	$K(m)$
1 : $G \xleftarrow{\$} \text{Fcs}[\mathcal{M}, \mathcal{R}]$	1 : if $N(m) = 0$
2 : $G := K$	2 : return $r \xleftarrow{\$} \mathcal{R}_{\text{good}}(pk_{i^*}, sk_{i^*}, m)$
3 : $b \leftarrow \mathcal{B}_b^G(pk_{i^*}, sk_{i^*})$	3 : if $N(m) = 1$
4 : return b	4 : return $r \xleftarrow{\$} \mathcal{R}_{\text{bad}}(pk_{i^*}, sk_{i^*}, m)$

Fig. 12: The algorithm $\mathcal{C}_b^N(pk_{i^*}, sk_{i^*})$ used in the proof of Lemma 5.

Game $\mathbf{G}_9^b$: Unlike game $\mathbf{G}_8^b$, this game modifies the definition of random oracle H as follows.

$$H(m_1, m_2, K, i, j, M, M') := \begin{cases} \widetilde{H}(m_1, m_2, K, i, j, M, M') & (M, M') = (\widetilde{M}, \widetilde{M}'), \\ \widehat{H}(m_1, m_2, K, i, j, M, M') & (M, M') \neq (\widetilde{M}, \widetilde{M}'). \end{cases} \quad (48)$$

Here $\widehat{H}$ is further decomposed as:

$$\begin{aligned} & \widehat{H}(m_1, m_2, K, i, j, M, M') \\ := & \begin{cases} H_1(\Pi_{\text{PKE}}.\text{Enc}(pk_{i^*}, m_1; G(m_1)), m_2, K, i, j, M, M') & i = i^*, j \neq i^*, \\ H_2(m_1, \Pi_{\text{PKE}}.\text{Enc}(pk_{i^*}, m_2; G(m_2)), K, i, j, M, M') & i \neq i^*, j = i^*, \\ H_3 \left(\begin{array}{l} \Pi_{\text{PKE}}.\text{Enc}(pk_{i^*}, m_1; G(m_1)), \\ \Pi_{\text{PKE}}.\text{Enc}(pk_{i^*}, m_2; G(m_2)), K, i, j, M, M' \end{array} \right) & i = i^*, j = i^*, \\ H_4(m_1, m_2, K, i, j, M, M') & \text{otherwise.} \end{cases} \end{aligned} \quad (49)$$

In addition, game $\mathbf{G}_9^b$ also modifies the definition of random oracles F_I and F_R as follows.

$$\begin{aligned} F_I(c, m, K, i, j, M, M') &= \begin{cases} F_I^1(c, \Pi_{\text{PKE}}.\text{Enc}(pk_{i^*}, m; G(m)), K, i, j, M, M') & i = j = i^*, \\ F_I^2(c, m, K, i, j, M, M') & \text{otherwise,} \end{cases} \\ F_R(m, c, K, i, j, M, M') &= \begin{cases} F_R^1(\Pi_{\text{PKE}}.\text{Enc}(pk_{i^*}, m; G(m)), c, K, i, j, M, M') & i = j = i^*, \\ F_R^2(m, c, K, i, j, M, M') & \text{otherwise.} \end{cases} \end{aligned} \quad (50)$$

In the above definitions, $\widetilde{H}$, H_1 , H_2 , H_3 , H_4 , F_I^1 , F_I^2 , F_R^1 , and F_R^2 are fresh random oracles cannot be directly accessed by the adversary $\mathcal{A}$.

Notice that $\mathbf{G}$ has been set as $\mathbf{G}_{\text{good}}$ in games $\mathbf{G}_8^b$ and $\mathbf{G}_9^b$, where $\mathbf{G}_{\text{good}}(m)$ is uniformly sampled from $\mathcal{R}_{\text{good}}(pk_{i^*}, sk_{i^*}, m)$ for any $m \in \mathcal{M}$. Hence, by the definition of $\mathcal{R}_{\text{good}}(pk_{i^*}, sk_{i^*}, m)$ given in Eq. (42), we can conclude that the function $g(\cdot) := \Pi_{\text{PKE}}.\text{Enc}(pk_{i^*}, \cdot; \mathbf{G}(\cdot))$ is an injective function. Thus, by the basic rules of random oracles, game $\mathbf{G}_9^b$ is only a conceptual change and

$$\Pr [1 \leftarrow \mathbf{G}_8^b] = \Pr [1 \leftarrow \mathbf{G}_9^b]. \quad (51)$$

Game $\mathbf{G}_{10}^b$: This game is the same as $\mathbf{G}_9^b$, except that it modifies the keys produced by DER_{resp} and DER_{init} . For $\text{DER}_{\text{resp}}(\text{sID}, M)$, if $(\text{sID}, M) \neq (s_{\text{resp}}, \widetilde{M})$ and decryption of c_j succeeds, the key K_r is set to $\widehat{\mathbf{H}}(m_i, m'_j, K^*, i, j, M, M')$; for $\text{DER}_{\text{init}}(\text{sID}, M')$, if $(\text{sID}, M') \neq (s_{\text{init}}, \widetilde{M}')$ and decryption of c_i succeeds, the key K_i is set to $\widehat{\mathbf{H}}(m'_i, m_j, K', i, j, M, M')$.

Recall that outputting 1 in game $\mathbf{G}_9^b$ guarantees the **Flag** finally must be 0. In the following, we explain that the oracles DER_{resp} and DER_{init} cannot produce a key as $\mathbf{H}(\dots, \widetilde{M}, \widetilde{M}')$ under the condition that **Flag** = 0.

- For DER_{resp} , **Flag** = 0 implies that its input (sID, M) and internally generated M' do not satisfy $(M, M') = (\widetilde{M}, \widetilde{M}')$, so this oracle cannot produce a key as $\mathbf{H}(\dots, \widetilde{M}, \widetilde{M}')$.
- For DER_{init} , if its input (sID, M') satisfies $M' \neq \widetilde{M}'$, it is obvious that this oracle cannot produce a key as $\mathbf{H}(\dots, \widetilde{M}, \widetilde{M}')$. Otherwise, the condition $\text{sID} \neq s_{\text{init}}$ necessarily holds; nevertheless, **Flag** = 0 implies that the M generated by $\text{INIT}(\text{sID})$ is guaranteed to be different from $\widetilde{M}$, so this oracle still cannot produce a key as $\mathbf{H}(\dots, \widetilde{M}, \widetilde{M}')$.

In game $\mathbf{G}_9^b$, we have modified the definition of $\mathbf{H}$, which is replaced by $\widehat{\mathbf{H}}$ if the (M, M') of the input does not equal $(\widetilde{M}, \widetilde{M}')$. Hence, under the condition that the game outputting 1, $\mathcal{A}$ cannot detect our modification in game $\mathbf{G}_{10}^b$. Thus

$$\Pr [1 \leftarrow \mathbf{G}_9^b] = \Pr [1 \leftarrow \mathbf{G}_{10}^b]. \quad (52)$$

Game $\mathbf{G}_{11}^b$: This game is the same as $\mathbf{G}_{10}^b$, except that it modifies the keys produced by DER_{resp} and DER_{init} as follows.

- For $\text{DER}_{\text{resp}}(\text{sID}, M)$, if $(\text{sID}, M) \neq (s_{\text{resp}}, \widetilde{M})$, $j = i^*$, and $i \neq i^*$ the key K_r is directly set to $\mathbf{H}_2(m_i, c_j, K^*, i, j, M, M')$. Here $(j, i) = (\text{holder}[\text{sID}], \text{peer}[\text{sID}])$.
- For $\text{DER}_{\text{resp}}(\text{sID}, M)$, if $(\text{sID}, M) \neq (s_{\text{resp}}, \widetilde{M})$, $j = i^*$, and $i = i^*$ the key K_r is directly set to $\mathbf{H}_3(c_i, c_j, K^*, i, j, M, M')$. Here $(j, i) = (\text{holder}[\text{sID}], \text{peer}[\text{sID}])$.
- For $\text{DER}_{\text{init}}(\text{sID}, M')$, if $(\text{sID}, M') \neq (s_{\text{init}}, \widetilde{M}')$, $i = i^*$, and $j \neq i^*$, the key K_i is directly set to $\mathbf{H}_1(c_i, m_j, K', i, j, M, M')$. Here $(i, j) = (\text{holder}[\text{sID}], \text{peer}[\text{sID}])$.
- For $\text{DER}_{\text{init}}(\text{sID}, M')$, if $(\text{sID}, M') \neq (s_{\text{init}}, \widetilde{M}')$, $i = i^*$, and $j = i^*$, the key K_i is directly set to $\mathbf{H}_3(c_i, c_j, K', i, j, M, M')$. Here $(i, j) = (\text{holder}[\text{sID}], \text{peer}[\text{sID}])$.

In this game, the secret key sk_{i^*} is no longer used to generate the final key K_r or K_i .

In game $\mathbf{G}_{10}^b$, the final key produced by DER_{resp} for the input (sID, M) where $M = (\widetilde{pk}, c_j)$, $(\text{sID}, M) \neq (s_{\text{resp}}, \widetilde{M})$ and $j = i^*$ is

$$K_r = \begin{cases} F_R^2(m_i, c_j, K^*, i, j, M, M') & j = i^*, i \neq i^*, c_j \text{ decryption fails} & \text{(Branch 1)}, \\ H_2(m_i, c_j, K^*, i, j, M, M') & j = i^*, i \neq i^*, c_j \text{ decryption succeeds} & \text{(Branch 2)}, \\ F_R^1(c_i, c_j, K^*, i, j, M, M') & j = i^*, i = i^*, c_j \text{ decryption fails} & \text{(Branch 3)}, \\ H_3(c_i, c_j, K^*, i, j, M, M') & j = i^*, i = i^*, c_j \text{ decryption succeeds} & \text{(Branch 4)}. \end{cases}$$

Similarly, the final key produced by DER_{init} in game $\mathbf{G}_{10}^b$ for the input (sID, M') where $M' = (c_i, c^*)$, $(\text{sID}, M') \neq (s_{\text{init}}, \widetilde{M}')$ and $i = i^*$ is

$$K_i = \begin{cases} F_1^2(c_i, m_j, K', i, j, M, M') & i = i^*, j \neq i^*, c_i \text{ decryption fails} & \text{(Branch 5)}, \\ H_1(c_i, m_j, K', i, j, M, M') & i = i^*, j \neq i^*, c_i \text{ decryption succeeds} & \text{(Branch 6)}, \\ F_1^1(c_i, c_j, K', i, j, M, M') & i = i^*, j = i^*, c_i \text{ decryption fails} & \text{(Branch 7)}, \\ H_3(c_i, c_j, K', i, j, M, M') & i = i^*, j = i^*, c_i \text{ decryption succeeds} & \text{(Branch 8)}. \end{cases}$$

In the above equations, " c decryption fails" means $\Pi_{\text{PKE}}.\text{Dec}(sk_{i^*}, c) = \perp$, or $m := \Pi_{\text{PKE}}.\text{Dec}(sk_{i^*}, c) \neq \perp$ together with $\Pi_{\text{PKE}}.\text{Enc}(pk_{i^*}, m; \mathbf{G}(m)) \neq c$. In contrast, " c decryption succeeds" means $m := \Pi_{\text{PKE}}.\text{Dec}(sk_{i^*}, c) \neq \perp$ together with $\Pi_{\text{PKE}}.\text{Enc}(pk_{i^*}, m; \mathbf{G}(m)) = c$.

We first focus on **Branch 1** and **2**, for which the inputs to F_R^2 and H_2 are disjoint. Concretely, the input c_j to F_R^2 always incurs decryption failure, while the input c_j to H_2 always decrypts correctly. Note that the adversary $\mathcal{A}$ can only obtain outputs of H_2 by querying H or CORRUPT , but by the definition of H given in Eq. (48), Eq. (49), and the fact that $\mathbf{G}$ is replaced by $\mathbf{G}_{\text{good}}$, $\mathcal{A}$ can never obtain $H_2(m, c, \dots)$ where c incurs decryption failure. Furthermore, by the definition of DER_{resp} , F_R^2 is only used to generate a key when c_j fails decryption. Therefore, even if $\mathcal{A}$ is allowed to query CORRUPT , it can never obtain $F_R^2(m, c, \dots)$ where c decrypts correctly. It follows that the input domain of F_R^2 and H_2 are disjoint in game $\mathbf{G}_{10}^b$. We can hence equivalently merge **Branch 1** and **2**: whenever $j = i^*$ and $i \neq i^*$, we directly set $K_r := H_2(m_i, c_j, K^*, i, j, M, M')$, removing the step that uses the secret key sk_{i^*} to verify whether c_j decrypts correctly. The analysis for **Branch 5** and **6** is analogous to that for **Branch 1** and **2**, and these two branches can also be merged with responses returned from H_1 , again without relying on sk_{i^*} .

We now turn to **Branch 3**, **4**, **7**, and **8**. First, the ciphertext c_i in **Branch 4** always decrypts correctly, since inside DER_{resp} , $c_i = \Pi_{\text{PKE}}.\text{Enc}(pk_{i^*}, m_i, \mathbf{G}(m_i))$ and the $\mathbf{G}$ is replaced by $\mathbf{G}_{\text{good}}$. Similarly, c_j in **Branch 8** is guaranteed to decrypt correctly, as the c_j used by DER_{init} is generated by INIT , where $c_j = \Pi_{\text{PKE}}.\text{Enc}(pk_{i^*}, m_j, \mathbf{G}(m_j))$ and the $\mathbf{G}$ is replaced by $\mathbf{G}_{\text{good}}$. Observe that for the two ciphertexts c_i and c_j involved in **Branch 3** and **7**, at least one of them results in decryption failure. Consequently, the random oracle inputs for these two branches are disjoint from those of **Branch 4** and **8**. As for H_3 , although $\mathcal{A}$ can obtain outputs of H_3 by querying H or CORRUPT , by the definition of H given in Eq. (48), Eq. (49), and the fact that $\mathbf{G}$ is replaced by $\mathbf{G}_{\text{good}}$, $\mathcal{A}$ can never obtain $H_3(c, c', \dots)$ where either c or c' fails decryption. Regarding F_R^1 and F_1^1 , $\mathcal{A}$ can only learn their outputs indirectly by querying CORRUPT . From Eq. (50) and the definitions of DER_{resp} and DER_{init} , any $F_R^1(c, c')$ obtained by $\mathcal{A}$ must satisfy that

c decrypts correctly while c' incurs decryption failure; in contrast, any $F_1^1(c, c')$ obtained by $\mathcal{A}$ must satisfy that c incurs decryption failure while c' decrypts correctly. This implies that the input domains of H_3 , F_R^1 and F_1^1 are pairwise disjoint in game $\mathbf{G}_{10}^b$. We can hence equivalently merge **Branch 3, 4, 7, and 8**: we directly use H_3 to generate K_r or K_i when $i = j = i^*$, removing the step that uses sk_{i^*} to verify whether c_i and c_j decrypts correctly. Combining the above analysis for **Branch 1 to 8**, K_r and K_i can be equivalently generated as follows:

$$\begin{aligned} K_r &= \begin{cases} H_2(m_i, c_j, K^*, i, j, M, M') & j = i^*, i \neq i^*, \\ H_3(c_i, c_j, K^*, i, j, M, M') & j = i^*, i = i^*, \end{cases} \\ K_i &= \begin{cases} H_1(c_i, m_j, K', i, j, M, M') & i = i^*, j \neq i^*, \\ H_3(c_i, c_j, K', i, j, M, M') & i = i^*, j = i^*. \end{cases} \end{aligned}$$

This exactly matches the construction of game $\mathbf{G}_{11}^b$, leading to

$$\Pr[1 \leftarrow \mathbf{G}_{10}^b] = \Pr[1 \leftarrow \mathbf{G}_{11}^b]. \quad (53)$$

Game $\mathbf{G}_{12}^b$: In this game, we switch back to using $G \xleftarrow{\$} \text{Fcs}[\mathcal{M}, \mathcal{R}]$ instead of G_{good} . With the same analysis of the game hop $\mathbf{G}_7^b$ to $\mathbf{G}_8^b$, we have

$$|\Pr[1 \leftarrow \mathbf{G}_{11}^b] - \Pr[1 \leftarrow \mathbf{G}_{12}^b]| \leq q_G \cdot \delta_P. \quad (54)$$

Game $\mathbf{G}_{13}^b$: This game is the same as $\mathbf{G}_{12}^b$, except that in $\text{Session}(i^*, j^*)$ it replaces $G(m_{i^*}^*)$ by r^* and directly sets $K_{\text{resp}} = K_{\text{init}} := k^*$, where r^* (resp. k^*) is a fresh uniformly random value in $\mathcal{R}$ (resp. $\mathcal{K}$) sampled at the game beginning. The $\text{Session}(i^*, j^*)$ of game $\mathbf{G}_{13}^b$ is clearly described in Fig. 11.

The definition of H in these two games is shown in Eq. (48) and Eq. (49). Thus, the K_{resp} and K_{init} computed by $\text{Session}(i^*, j^*)$ in game $\mathbf{G}_{12}^b$ is actually $\tilde{H}(m_{i^*}^*, m_{j^*}^*, K^*, i^*, j^*, \tilde{M}, \tilde{M}')$. Based on the basic properties of the random oracle, games $\mathbf{G}_{12}^b$ and $\mathbf{G}_{13}^b$ are equivalent as long as the following event does not occur.

$\text{Bad}_{m_{i^*}}^b$: During the running of $\mathbf{G}_{13}^b$, G is ever queried with m_{i^*} or $\tilde{H}$ is ever queried with $(m_{i^*}, \dots)$.

As for the probability $\Pr[\text{Bad}_{m_{i^*}}^b]$, we first construt a OW-CPA adversary $\mathcal{B}_b$ ($b \in \{0, 1\}$) against Π_{PKE} as follows, and then bound $\Pr[\text{Bad}_{m_{i^*}}^b]$ by using $\mathcal{B}_b$'s advantage.

- The input of $\mathcal{B}_b$ is (pk_{i^*}, c_{i^*}) , where pk_{i^*} is the public key and c_{i^*} is the challenge ciphertext.
- $\mathcal{B}_b$ runs game $\mathbf{G}_{13}^b$, simulates the random oracles on-the-fly.
- When running $\text{Session}(i^*, j^*)$, $\mathcal{B}_b$ replaces the (pk_{i^*}, c_{i^*}) generated there by its own input.
- After $\mathcal{A}$ returns its output b' , $\mathcal{B}_b$ uniformly samples an element from the record lists of the random oracles H and G , which we denote as $(m, \dots)$, and then returns m as its final output.

We stress that although $\mathcal{B}_b$ does not have the secret key sk_{i^*} corresponding to pk_{i^*} , it can still run game $\mathbf{G}_{13}^b$ normally, as the computations of this game no longer require sk_{i^*} . Obviously, for any $b \in \{0, 1\}$, the running time of $\mathcal{B}_b$ is roughly the same as those of $\mathcal{A}$, and $\text{Bad}_{m_{i^*}}^b$ occurring implies that $\mathcal{B}_b$ has an advantage at least $1/(q_H + q_G)$ to successfully guess the plaintext m_{i^*} of c_{i^*} . Thus, we have

$$\begin{aligned} |\Pr[1 \leftarrow \mathbf{G}_{12}^b] - \Pr[1 \leftarrow \mathbf{G}_{13}^b]| &\leq \Pr[\text{Bad}_{m_{i^*}}^b] \\ &\leq (q_H + q_G) \cdot \text{Adv}_{\Pi_{\text{PKE}}}^{\text{OW-CPA}}(\mathcal{B}_b). \end{aligned} \quad (55)$$

Game $\mathbf{G}_{14}^b$: This game is the same as $\mathbf{G}_{13}^b$, except that the oracle TEST directly sets K_0^* as k^* , rather than $\text{sKey}[\text{sID}^*]$. Since outputting 1 in game $\mathbf{G}_{13}^b$ guarantees $\{s_{\text{init}}, s_{\text{resp}}\} = \{\text{sID}^*\} \cup \mathfrak{M}(\text{sID}^*)$ and $(i^*, j^*) = (\text{holder}[s_{\text{init}}], \text{peer}[s_{\text{init}}])$, the K_0^* in game $\mathbf{G}_{13}^b$ is also k^* if it finally outputs 1. Thus, game $\mathbf{G}_{14}^b$ is only a conceptual change and

$$\Pr[1 \leftarrow \mathbf{G}_{13}^b] = \Pr[1 \leftarrow \mathbf{G}_{14}^b]. \quad (56)$$

Game $\mathbf{G}_{15}^b$: This game is the same as $\mathbf{G}_{14}^b$, except that in $\text{Session}(i^*, j^*)$ it sets $K_{\text{resp}} = K_{\text{init}} := 0$. The $\text{Session}(i^*, j^*)$ of game $\mathbf{G}_{15}^b$ is clearly described in Fig. 11. Similar to the above game hop, since outputting 1 in game $\mathbf{G}_{14}^b$ guarantees $\{s_{\text{init}}, s_{\text{resp}}\} = \{\text{sID}^*\} \cup \mathfrak{M}(\text{sID}^*)$ and sID^* is fresh, the adversary $\mathcal{A}$ never queries REVEAL with s_{init} or s_{resp} to get the K_{init} or K_{resp} if the game finally outputs 1, and thus $\mathcal{A}$ cannot detect our change in game $\mathbf{G}_{15}^b$. Therefore, we have

$$\Pr[1 \leftarrow \mathbf{G}_{14}^b] = \Pr[1 \leftarrow \mathbf{G}_{15}^b]. \quad (57)$$

In game $\mathbf{G}_{15}^b$, K_0^* and K_1^* are both uniformly random values in $\mathcal{K}$, and thus

$$\Pr[1 \leftarrow \mathbf{G}_{15}^0] = \Pr[1 \leftarrow \mathbf{G}_{15}^1]. \quad (58)$$

At this point, by combining Eq. (34) to Eq. (41), Eq. (47), Eq. (51) to Eq. (58), and folding adversaries $\mathcal{B}_0$ and $\mathcal{B}_1$ into one adversary $\mathcal{B}$, we obtain

$$\begin{aligned} \text{Case 2} &\leq 2S \cdot \gamma(\Pi_{\text{PKE}}.\text{Enc}) + 2S^3 N^2 \cdot (\gamma(\Pi_{\text{KEM}}.\text{KeyGen}) + \gamma(\Pi_{\text{KEM}}.\text{Encap})) \\ &\quad + 4S^2 N^2 (q_G + 1) \cdot \delta_P + 2S^2 N^2 \cdot \delta_K + \frac{2S^2 N^2 (q_R + q_I)}{|\mathcal{M}|} \\ &\quad + 2S^2 N^2 (q_G + q_H) \cdot \text{Adv}_{\Pi_{\text{PKE}}}^{\text{OW-CPA}}(\mathcal{B}). \end{aligned}$$

This completes the proof of Lemma 5. $\square$

A.3 Proof of Lemma 6

Proof. To prove this lemma, we introduce a sequence of games $\mathbf{G}_0^b$ to $\mathbf{G}_{13}^b$ ($b \in \{0, 1\}$) as shown in Fig. 13. In these games, $\mathbf{H}_{\text{init}}$, $\mathbf{H}_{\text{resp}}$, $\mathbf{G}$, and $\mathbf{H}$ are random oracles, and the adversary $\mathcal{A}$ can also query a set of oracles $\mathcal{O}_{\text{AKE}} = \{\text{EST}, \text{INIT}, \text{DER}_{\text{resp}}, \text{DER}_{\text{init}}, \text{REVEAL}, \text{REV-STATE}, \text{CORRUPT}, \text{TEST}\}$.

Game $\mathbf{G}_0^b$: This game is the same as $\text{KeyIND}_{\Pi_{\text{AKE}}}^{\mathcal{A}, b}$ (Fig. 4), except that it additionally checks whether $\mathfrak{M}(\text{sID}^*) = \emptyset \wedge \text{role}[\text{sID}^*] = \text{initiator}$ occurs, and directly returns 0 if this check fails. Thus,

$$\Pr [1 \leftarrow \mathbf{G}_0^b] = \Pr \left[1 \leftarrow \text{KeyIND}_{\Pi_{\text{AKE}}}^{\mathcal{A}, b} \wedge \mathfrak{M}(\text{sID}^*) = \emptyset \wedge \text{role}[\text{sID}^*] = \text{initiator} \right]. \quad (59)$$

Game $\mathbf{G}_1^b$: This game is the same as $\mathbf{G}_0^b$, except that at the very beginning, it uniformly samples a session s_{init} from $\{1, \dots, S\}$ and an index j^* from $\{1, \dots, N\}$, and checks whether $\text{sID}^* = s_{\text{init}}$ and $\text{peer}(\text{sID}^*) = j^*$ at the end. If such check fails, game $\mathbf{G}_1^b$ directly returns 0.

In game $\mathbf{G}_1^b$, we make a random guess at the beginning, hoping to directly obtain sID^* and $\text{peer}(\text{sID}^*)$ (i.e., the s_{init} and j^*). This guessing step is necessary; with it, we can embed the challenge ciphertext of Π_{PKE} into the computation of $\text{INIT}(s_{\text{init}})$. As for the difference between games $\mathbf{G}_0^b$ and $\mathbf{G}_1^b$, we have

$$\Pr [1 \leftarrow \mathbf{G}_0^b] = SN \cdot \Pr [1 \leftarrow \mathbf{G}_1^b]. \quad (60)$$

Game $\mathbf{G}_2^b$: This game is the same as $\mathbf{G}_1^b$, except that it introduces a new procedure $\text{Challenge}(s_{\text{init}}, j^*)$ and modifies the computations of INIT according to the internal random values generated by $\text{Challenge}(s_{\text{init}}, j^*)$, as detailed below:

- $\text{Challenge}(s_{\text{init}}, j^*)$ honestly simulates the first phase of the session sID^* , namely the execution of $\text{INIT}(\text{sID}^*)$ with responder $\mathbf{P}_{j^*}$. Specifically, it samples $m_{j^*} \xleftarrow{\$} \mathcal{M}$, $(\widetilde{pk}^*, \widetilde{sk}^*) \leftarrow \Pi_{\text{KEM}}.\text{KeyGen}$, computes $c_{j^*} := \Pi_{\text{PKE}}.\text{Enc}(pk_{j^*}, m_{j^*}; \mathbf{G}(m_{j^*}))$, and finally sets $\widetilde{M} := (\widetilde{pk}^*, c_{j^*})$.
- For INIT , if the input is s_{init} , it sets $(\text{sent}[s_{\text{init}}], \text{st}[s_{\text{init}}]) := (\widetilde{M}, (\widetilde{M}, \widetilde{sk}^*, m_{j^*}))$ and returns $\widetilde{M}$.

For all remaining input, INIT is in accordance with the specification given in Fig. 4.

In game $\mathbf{G}_2^b$, we simulate $\text{INIT}(\text{sID}^*)$ in advance by executing $\text{Challenge}(s_{\text{init}}, j^*)$. We then perform checks on the inputs of INIT , and insert certain random values generated during this simulation into the lists sent and st . Since outputting 1 in both $\mathbf{G}_1^b$ and $\mathbf{G}_2^b$ guarantees $\text{sID}^* = s_{\text{init}}$ and $\text{peer}(\text{sID}^*) = j^*$, the computation of $\text{INIT}(\text{sID}^*)$ in $\mathbf{G}_1^b$ is equivalent to that simulated by $\text{Challenge}(s_{\text{init}}, j^*)$ in $\mathbf{G}_2^b$. Meanwhile, $\mathbf{G}_2^b$ appropriately inserts corresponding random values into the lists sent and st . As a result, the modifications introduced in $\mathbf{G}_2^b$ relative to $\mathbf{G}_1^b$ do not change the adversary's view under the condition that the game outputs 1, thus,

$$\Pr [1 \leftarrow \mathbf{G}_1^b] = \Pr [1 \leftarrow \mathbf{G}_2^b]. \quad (61)$$

GAMES $\mathbf{G}_0^b\text{-}\mathbf{G}_{13}^b$ ($b \in \{0, 1\}$)	INIT(sID)
1, $\text{cnt} := 0, \text{sID}^* := 0$ 2, $\text{H}_{\text{init}}, \text{H}_{\text{resp}}, \text{H} \xleftarrow{\$} \text{Fcs}[\{0, 1\}^*, \mathcal{K}], \text{G} \xleftarrow{\$} \text{Fcs}[\mathcal{M}, \mathcal{R}]$ $\text{s}_{\text{init}} \xleftarrow{\$} \{1, \dots, S\}$ // $\mathbf{G}_1^b\text{-}\mathbf{G}_{13}^b$ $j^* \xleftarrow{\$} \{1, \dots, N\}$ // $\mathbf{G}_1^b\text{-}\mathbf{G}_{13}^b$ $\text{G} := \text{G}_{\text{good}}$ // $\mathbf{G}_5^b\text{-}\mathbf{G}_7^b$ $k^* \xleftarrow{\$} \mathcal{K}$ // $\mathbf{G}_{10}^b\text{-}\mathbf{G}_{13}^b$ $r^* \xleftarrow{\$} \mathcal{R}$ // $\mathbf{G}_{11}^b\text{-}\mathbf{G}_{13}^b$ 3, for $i \in \{1, \dots, N\}$ $(pk_i, sk_i) \leftarrow \Pi_{\text{PKE}}.\text{KeyGen}, s_i \xleftarrow{\$} \mathcal{M}$ $sk'_i := (sk_i, s_i), \text{crp}[i] := \text{false}$ end for run Challenge(s_{init}, j^*) // $\mathbf{G}_2^b\text{-}\mathbf{G}_{13}^b$ 4, $b' \leftarrow \mathcal{A}^{\mathcal{O}_{\text{AKE}}, \text{H}_{\text{init}}, \text{H}_{\text{resp}}, \text{G}, \text{H}}(pk_1, \dots, pk_N)$ 5, if sID* is not fresh, return 0 if $\mathfrak{M}(\text{sID}^*) \neq \emptyset \vee \text{role}[\text{sID}^*] = \text{responder}$, return 0 // $\mathbf{G}_0^b\text{-}\mathbf{G}_{13}^b$ if $\text{sID}^* \neq s_{\text{init}} \vee \text{peer}[\text{sID}^*] \neq j^*$, return 0 // $\mathbf{G}_1^b\text{-}\mathbf{G}_{13}^b$ 6, return b' DER _{resp} (sID, M) 1, if holder[sID] = $\perp \vee \text{sKey}[\text{sID}] \neq \perp$ $\vee \text{role}[\text{sID}] = \text{initiator}$, return $\perp$ 2, $\text{role}[\text{sID}] := \text{responder}$ 3, $(j, i) := (\text{holder}[\text{sID}], \text{peer}[\text{sID}])$ Parse M as $(\widetilde{pk}, c_j)$ $m_i \xleftarrow{\$} \mathcal{M}, c_i := \Pi_{\text{PKE}}.\text{Enc}(pk_i, m_i; \text{G}(m_i))$ $(\widetilde{c}, K^*) = \Pi_{\text{KEM}}.\text{Encap}(\widetilde{pk}), M' := (c_i, \widetilde{c})$ $(\text{sent}[\text{sID}], \text{rece}[\text{sID}]) := (M', M)$ if sID and s_{init} are matching // $\mathbf{G}_9^b\text{-}\mathbf{G}_{13}^b$ $K_r \xleftarrow{\$} \mathcal{K}$ // $\mathbf{G}_9^b\text{-}\mathbf{G}_{13}^b$ if $j = j^*, i \neq j^*, K_r := \text{H}_2(m_i, c_j, K^*, i, j, M, M')$ // $\mathbf{G}_7^b\text{-}\mathbf{G}_{13}^b$ if $j = j^*, i = j^*, K_r := \text{H}_3(c_i, c_j, K^*, i, j, M, M')$ // $\mathbf{G}_7^b\text{-}\mathbf{G}_{13}^b$ $m'_j := \Pi_{\text{PKE}}.\text{Dec}(sk_j)$ if $m'_j = \perp \vee \Pi_{\text{PKE}}.\text{Enc}(pk_j, m_j; \text{G}(m'_j)) \neq c_j$ $K_r := \text{H}_{\text{resp}}(s_j, m_i, c_j, K^*, i, j, M, M')$ if $j = j^*, K_r := \text{F}_r(m_i, c_j, K^*, i, j, M, M')$ // $\mathbf{G}_4^b\text{-}\mathbf{G}_{13}^b$ else $K_r := \text{H}(m_i, m'_j, K^*, i, j, M, M')$ $\text{sKey}[\text{sID}] := K_i$ return M' Challenge(s_{init}, j^*) 1, $m_{j^*} \xleftarrow{\$} \mathcal{M}$ 2, $c_{j^*} := \Pi_{\text{PKE}}.\text{Enc}(pk_{j^*}, m_{j^*}; \text{G}(m_{j^*}))$ $c_{j^*} := \Pi_{\text{PKE}}.\text{Enc}(pk_{j^*}, m_{j^*}; r^*)$ // $\mathbf{G}_{11}^b\text{-}\mathbf{G}_{13}^b$ 3, $(\widetilde{pk}^*, \widetilde{sk}^*) \leftarrow \Pi_{\text{KEM}}.\text{KeyGen}, \widetilde{M} := (\widetilde{pk}^*, c_{j^*})$	1, if holder[sID] = $\perp \vee \text{sent}[\text{sID}] \neq \perp$, return $\perp$ 2, $\text{role}[\text{sID}] := \text{initiator}$ 3, if sID = s_{init} // $\mathbf{G}_2^b\text{-}\mathbf{G}_{13}^b$ $(\text{sent}[s_{\text{init}}], \text{st}[s_{\text{init}}]) := (\widetilde{M}, (\widetilde{M}, \widetilde{sk}^*, m_{j^*}))$ // $\mathbf{G}_2^b$ $(\text{sent}[s_{\text{init}}], \text{st}[s_{\text{init}}]) := (\widetilde{M}, (\widetilde{M}, \widetilde{sk}^*, 0))$ // $\mathbf{G}_3^b\text{-}\mathbf{G}_{13}^b$ return $\widetilde{M}$ else $(i, j) := (\text{holder}[\text{sID}], \text{peer}[\text{sID}])$ $(M, st) \leftarrow \text{GC-AKE}.\text{Init}(sk_i, pk_j)$ $(\text{sent}[\text{sID}], \text{st}[\text{sID}]) := (M, st)$ return M DER _{init} (sID, M') 1, if holder[sID] = $\perp \vee \text{st}[\text{sID}] = \perp \vee \text{sKey}[\text{sID}] \neq \perp$ return $\perp$ 2, $(i, j) := (\text{holder}[\text{sID}], \text{peer}[\text{sID}])$ Parse st[sID] as $(M = (\widetilde{pk}, c_j), \widetilde{sk}, m_j)$ if sID $\neq s_{\text{init}}, \widetilde{pk} = \widetilde{pk}^*, K_i \xleftarrow{\$} \mathcal{K}$ // $\mathbf{G}_9^b\text{-}\mathbf{G}_{13}^b$ if sID = $s_{\text{init}}, m_j := m_{j^*}$ // $\mathbf{G}_3^b\text{-}\mathbf{G}_9^b$ if sID = $s_{\text{init}}, K_i := k^*$ // $\mathbf{G}_{10}^b\text{-}\mathbf{G}_{13}^b$ if sID = $s_{\text{init}}, K_i := 0$ // $\mathbf{G}_{13}^b$ $\text{rece}[\text{sID}] := M'$ if sID = $s_{\text{init}}, s_{\text{init}}$ has a match; $K_i \xleftarrow{\$} \mathcal{K}$ // $\mathbf{G}_9^b\text{-}\mathbf{G}_{13}^b$ Parse M' as $(c_i, \widetilde{c})$ if $i = j^*, j \neq j^*, K_i := \text{H}_1(c_i, m_j, K', i, j, M, M')$ // $\mathbf{G}_7^b\text{-}\mathbf{G}_{13}^b$ if $i = j^*, j = j^*, K_i := \text{H}_3(c_i, c_j, K', i, j, M, M')$ // $\mathbf{G}_7^b\text{-}\mathbf{G}_{13}^b$ $m'_i := \Pi_{\text{PKE}}.\text{Dec}(sk_i, c_i)$ $K' := \Pi_{\text{KEM}}.\text{Decap}(\widetilde{sk}, \widetilde{c})$ if $m'_i = \perp \vee \Pi_{\text{PKE}}.\text{Enc}(pk_i, m'_i; \text{G}(m'_i)) \neq c_i$ $K_i := \text{H}_{\text{init}}(s_i, c_i, m_j, \widetilde{c}, i, j, M, M')$ if $i = j^*, K_i := \text{F}_1(c_i, m_j, \widetilde{c}, i, j, M, M')$ // $\mathbf{G}_4^b\text{-}\mathbf{G}_{13}^b$ else $K_i := \text{H}(m'_i, m_j, K', i, j, M, M')$ $\text{sKey}[\text{sID}] := K_i$ TEST(sID) // only one query 1, $\text{sID}^* := \text{sID}$ // test session's id 2, if sKey[sID*] = $\perp$, return $\perp$ 3, $K_0^* := \text{sKey}[\text{sID}^*], K_1^* \xleftarrow{\$} \mathcal{K}$ $K_0^* := k^*, K_1^* \xleftarrow{\$} \mathcal{K}$ // $\mathbf{G}_{12}^b\text{-}\mathbf{G}_{13}^b$ 4, return K_0^* CORRUPT($i \in \{1, \dots, N\}$) $\text{crp}[i] := \text{true}$ if $i = j^*$, return 0 // $\mathbf{G}_3^b\text{-}\mathbf{G}_{13}^b$ return (sk_i, s_i)

Fig. 13: Games $\mathbf{G}_0^b$ to $\mathbf{G}_{13}^b$ used in the proof of Lemma 6. Here for brevity, we only list the oracles TEST, CORRUPT, INIT, DER_{resp} and DER_{init} that need to be modified in our game sequence. The definitions of all other unlisted oracles are identical to those given in Fig. 4.

Game $\mathbf{G}_3^b$: Compared with $\mathbf{G}_2^b$, this game has three changes: the oracle CORRUPT directly returns 0 for the input j^* ; the $\text{st}[s_{\text{init}}]$ is replaced by $(\widetilde{M}, \widetilde{sk}^*, 0)$; for $\text{DER}_{\text{init}}(\text{sID}, M')$, if $\text{sID} = s_{\text{init}}$, the m_j parsed from $\text{st}[\text{sID}]$ is replaced by m_j^* .

Notice that outputting 1 in game $\mathbf{G}_2^b$ ensures $\mathfrak{M}(\text{sID}^*) = \emptyset$, $\text{role}[\text{sID}^*] = \text{initiator}$, $\text{sID}^* = s_{\text{init}}$, $\text{peer}(\text{sID}^*) = j^*$, and sID^* is fresh. Therefore, by Definition 11, the game outputting 1 implies that the adversary $\mathcal{A}$ never queries CORRUPT with j^* to get the long-term secret key sk_{j^*} , and never obtains the value m_{j^*} stored in the internal state $\text{st}[s_{\text{init}}]$. That is to say, under the condition that the game outputting 1, $\mathcal{A}$ cannot detect our modification to CORRUPT and $\text{st}[s_{\text{init}}]$. Thus

$$\Pr[1 \leftarrow \mathbf{G}_2^b] = \Pr[1 \leftarrow \mathbf{G}_3^b]. \quad (62)$$

We point out that the third changes made in $\mathbf{G}_3^b$ serves to maintain consistency of DER_{init} . Although $\mathcal{A}$ is unable to obtain the m_{j^*} stored in $\text{st}[s_{\text{init}}]$, DER_{init} still needs to compute key K_i with m_{j^*} when its input satisfies $\text{sID} = s_{\text{init}}$. Accordingly, we directly set $m_j := m_j^*$.

Game $\mathbf{G}_4^b$: This game introduces two new random oracles $\mathbf{F}_R$ and $\mathbf{F}_I$, and modifies the keys produced by DER_{resp} and DER_{init} . For $\text{DER}_{\text{resp}}(\text{sID}, M)$, if $\text{holder}[\text{sID}] = j^*$ and decryption of c_j fails, $K_r := \mathbf{F}_R(m_i, c_j, K^*, i, j, M, M')$; for $\text{DER}_{\text{init}}(\text{sID}, M')$, if $\text{holder}[\text{sID}] = j^*$ and decryption of c_i fails, $K_i := \mathbf{F}_I(c_i, m_j, \widetilde{c}, i, j, M, M')$.

It is easy to see that these two games are equivalent, if oracles $\mathbf{H}_{\text{resp}}$ and $\mathbf{H}_{\text{init}}$ are never queried with input $(s_{j^*}, \dots)$ during the running of the adversary $\mathcal{A}$. Since in both $\mathbf{G}_3^b$ and $\mathbf{G}_4^b$, s_{j^*} is uniformly sampled from $\mathcal{M}$ and the running of $\mathcal{A}$ no longer depends on s_{j^*} , we have

$$|\Pr[1 \leftarrow \mathbf{G}_3^b] - \Pr[1 \leftarrow \mathbf{G}_4^b]| \leq \frac{q_R + q_I}{|\mathcal{M}|}. \quad (63)$$

Game $\mathbf{G}_5^b$: This game is the same as $\mathbf{G}_4^b$, except that $\mathbf{G}$ is set as $\mathbf{G}_{\text{good}}$, a function with domain $\mathcal{M}$ and codomain $\mathcal{R}$, and $\mathbf{G}_{\text{good}}(m)$ is uniformly sampled from $\mathcal{R}_{\text{good}}(pk_{j^*}, sk_{j^*}, m)$ (Eq. (42)) for any $m \in \mathcal{M}$. With the same analysis of the game hop $\mathbf{G}_7^b$ to $\mathbf{G}_8^b$ in the proof of Lemma 5, we have

$$|\Pr[1 \leftarrow \mathbf{G}_4^b] - \Pr[1 \leftarrow \mathbf{G}_5^b]| \leq q_G \cdot \delta_P. \quad (64)$$

Game $\mathbf{G}_6^b$: Unlike game $\mathbf{G}_5^b$, this game modifies the definition of random oracle $\mathbf{H}$ as follows.

$$\begin{aligned} & \mathbf{H}(m_1, m_2, K, i, j, M, M') \\ = & \begin{cases} \mathbf{H}_1(\Pi_{\text{PKE}}.\text{Enc}(pk_{j^*}, m_1; \mathbf{G}(m_1)), m_2, K, i, j, M, M') & i = j^*, j \neq j^*, \\ \mathbf{H}_2(m_1, \Pi_{\text{PKE}}.\text{Enc}(pk_{j^*}, m_2; \mathbf{G}(m_2)), K, i, j, M, M') & i \neq j^*, j = j^*, \\ \mathbf{H}_3 \left(\begin{array}{l} \Pi_{\text{PKE}}.\text{Enc}(pk_{j^*}, m_1; \mathbf{G}(m_1)), \\ \Pi_{\text{PKE}}.\text{Enc}(pk_{j^*}, m_2; \mathbf{G}(m_2)), K, i, j, M, M' \end{array} \right) & i = j^*, j = j^*, \\ \mathbf{H}_4(m_1, m_2, K, i, j, M, M') & \text{otherwise.} \end{cases} \end{aligned} \quad (65)$$

In addition, $\mathbf{G}_6^b$ also modifies the definition of random oracles F_I and F_R as follows.

$$\begin{aligned} F_I(c, m, K, i, j, M, M') &= \begin{cases} F_I^1(c, \Pi_{\text{PKE}}.\text{Enc}(pk_{j^*}, m; G(m)), K, i, j, M, M') & i = j = j^*, \\ F_I^2(c, m, K, i, j, M, M') & \text{otherwise,} \end{cases} \\ F_R(m, c, K, i, j, M, M') &= \begin{cases} F_R^1(\Pi_{\text{PKE}}.\text{Enc}(pk_{j^*}, m; G(m)), c, K, i, j, M, M') & i = j = j^*, \\ F_R^2(m, c, K, i, j, M, M') & \text{otherwise.} \end{cases} \end{aligned} \quad (66)$$

In the above definitions, $H_1, H_2, H_3, H_4, F_I^1, F_I^2, F_R^1$, and F_R^2 are fresh random oracles cannot be directly accessed by the adversary $\mathcal{A}$.

Notice that G has been set as G_{good} in games $\mathbf{G}_5^b$ and $\mathbf{G}_6^b$, where $G_{\text{good}}(m)$ is uniformly sampled from $\mathcal{R}_{\text{good}}(pk_{j^*}, sk_{j^*}, m)$ for any $m \in \mathcal{M}$. Hence, by the definition of $\mathcal{R}_{\text{good}}(pk_{j^*}, sk_{j^*}, m)$ given in Eq. (42), we can conclude that the function $g(\cdot) := \Pi_{\text{PKE}}.\text{Enc}(pk_{j^*}, \cdot; G(\cdot))$ is an injective function. Thus, by the basic rules of random oracles, game $\mathbf{G}_6^b$ is only a conceptual change and

$$\Pr [1 \leftarrow \mathbf{G}_5^b] = \Pr [1 \leftarrow \mathbf{G}_6^b]. \quad (67)$$

Game $\mathbf{G}_7^b$: This game is the same as $\mathbf{G}_6^b$, except that it modifies the keys produced by DER_{resp} and DER_{init} as follows.

- For $\text{DER}_{\text{resp}}(\text{sID}, M)$, if $j = j^*$, and $i \neq j^*$ the key K_r is directly set to $H_2(m_i, c_j, K^*, i, j, M, M')$. Here $(j, i) = (\text{holder}[\text{sID}], \text{peer}[\text{sID}])$.
- For $\text{DER}_{\text{resp}}(\text{sID}, M)$, if $j = j^*$, and $i = j^*$ the key K_r is directly set to $H_3(c_i, c_j, K^*, i, j, M, M')$. Here $(j, i) = (\text{holder}[\text{sID}], \text{peer}[\text{sID}])$.
- For $\text{DER}_{\text{init}}(\text{sID}, M')$, if $i = j^*$, and $j \neq j^*$, the key K_i is directly set to $H_1(c_i, m_j, K', i, j, M, M')$. Here $(i, j) = (\text{holder}[\text{sID}], \text{peer}[\text{sID}])$.
- For $\text{DER}_{\text{init}}(\text{sID}, M')$, if $i = j^*$, and $j = j^*$, the key K_i is directly set to $H_3(c_i, c_j, K', i, j, M, M')$. Here $(i, j) = (\text{holder}[\text{sID}], \text{peer}[\text{sID}])$.

In this game, the secret key sk_{j^*} is no longer used to generated the final key K_r/K_i .

In game $\mathbf{G}_6^b$, the final key produced by DER_{resp} for the input (sID, M) where $M = (pk, c_j)$, $(j, i) = (\text{holder}[\text{sID}], \text{peer}[\text{sID}])$ and $j = j^*$ is

$$K_r = \begin{cases} F_R^2(m_i, c_j, K^*, i, j, M, M') & j = j^*, i \neq j^*, c_j \text{ decryption fails} & \text{(Branch 1)}, \\ H_2(m_i, c_j, K^*, i, j, M, M') & j = j^*, i \neq j^*, c_j \text{ decryption succeeds} & \text{(Branch 2)}, \\ F_R^1(c_i, c_j, K^*, i, j, M, M') & j = j^*, i = j^*, c_j \text{ decryption fails} & \text{(Branch 3)}, \\ H_3(c_i, c_j, K^*, i, j, M, M') & j = j^*, i = j^*, c_j \text{ decryption succeeds} & \text{(Branch 4)}. \end{cases}$$

Similarly, the final key produced by DER_{init} in game $\mathbf{G}_6^b$ for the input (sID, M') where $M' = (c_i, c^*)$, $(i, j) = (\text{holder}[\text{sID}], \text{peer}[\text{sID}])$ and $i = j^*$ is

$$K_i = \begin{cases} F_I^2(c_i, m_j, K', i, j, M, M') & i = j^*, j \neq j^*, c_i \text{ decryption fails} & \text{(Branch 5)}, \\ H_1(c_i, m_j, K', i, j, M, M') & i = j^*, j \neq j^*, c_i \text{ decryption succeeds} & \text{(Branch 6)}, \\ F_I^1(c_i, c_j, K', i, j, M, M') & i = j^*, j = j^*, c_i \text{ decryption fails} & \text{(Branch 7)}, \\ H_3(c_i, c_j, K', i, j, M, M') & i = j^*, j = j^*, c_i \text{ decryption succeeds} & \text{(Branch 8)}. \end{cases}$$

In the above equations, " c decryption fails" means $\Pi_{\text{PKE}}.\text{Dec}(sk_{j^*}, c) = \perp$, or $m := \Pi_{\text{PKE}}.\text{Dec}(sk_{j^*}, c) \neq \perp$ together with $\Pi_{\text{PKE}}.\text{Enc}(pk_{j^*}, m; \mathbf{G}(m)) \neq c$. In contrast, " c decryption succeeds" means $m := \Pi_{\text{PKE}}.\text{Dec}(sk_{j^*}, c) \neq \perp$ together with $\Pi_{\text{PKE}}.\text{Enc}(pk_{j^*}, m; \mathbf{G}(m)) = c$.

We first focus on **Branch 1** and **2**, for which the inputs to $\mathbf{F}_R^2$ and $\mathbf{H}_2$ are disjoint. Concretely, the input c_j to $\mathbf{F}_R^2$ always incurs decryption failure, while the input c_j to $\mathbf{H}_2$ always decrypts correctly. Note that the adversary $\mathcal{A}$ can only obtain outputs of $\mathbf{H}_2$ by querying $\mathbf{H}$ or **CORRUPT**, but by the definition of $\mathbf{H}$ given in Eq. (65), and the fact that $\mathbf{G}$ is replaced by $\mathbf{G}_{\text{good}}$, $\mathcal{A}$ can never obtain $\mathbf{H}_2(m, c, \dots)$ where c incurs decryption failure. Furthermore, by the definition of DER_{resp} , $\mathbf{F}_R^2$ is only used to generate a key when c_j fails decryption. Therefore, even if $\mathcal{A}$ is allowed to query **CORRUPT**, it can never obtain $\mathbf{F}_R^2(m, c, \dots)$ where c decrypts correctly. It follows that the input domain of $\mathbf{F}_R^2$ and $\mathbf{H}_2$ are disjoint in game $\mathbf{G}_6^b$. We can hence equivalently merge **Branch 1** and **2**: whenever $j = j^*$ and $i \neq j^*$, we directly set $K_r = \mathbf{H}_2(m_i, c_j, K^*, i, j, M, M')$, removing the step that uses the secret key sk_{j^*} to verify whether c_j decrypts correctly. The analysis for **Branch 5** and **6** is analogous to that for **Branch 1** and **2**, and these two branches can also be merged with responses returned from $\mathbf{H}_1$, again without relying on sk_{j^*} .

We now turn to **Branch 3, 4, 7, and 8**. First, the ciphertext c_i in **Branch 4** always decrypts correctly, since inside DER_{resp} , $c_i = \Pi_{\text{PKE}}.\text{Enc}(pk_{j^*}, m_i, \mathbf{G}(m_i))$ and the $\mathbf{G}$ is replaced by $\mathbf{G}_{\text{good}}$. Similarly, c_j in **Branch 8** is guaranteed to decrypt correctly, as the c_j used by DER_{init} is generated by **INIT**, where $c_j = \Pi_{\text{PKE}}.\text{Enc}(pk_{j^*}, m_j, \mathbf{G}(m_j))$ and the $\mathbf{G}$ is replaced by $\mathbf{G}_{\text{good}}$. Observe that for the two ciphertexts c_i and c_j involved in **Branch 3** and **7**, at least one of them results in decryption failure. Consequently, the random oracle inputs for these two branches are disjoint from those of **Branch 4** and **8**. As for $\mathbf{H}_3$, although $\mathcal{A}$ can obtain outputs of $\mathbf{H}_3$ by querying $\mathbf{H}$ or **CORRUPT**, by the definition of $\mathbf{H}$ given in Eq. (65), and the fact that $\mathbf{G}$ is replaced by $\mathbf{G}_{\text{good}}$, $\mathcal{A}$ can never obtain $\mathbf{H}_3(c, c', \dots)$ where either c or c' fails decryption. Regarding $\mathbf{F}_R^1$ and $\mathbf{F}_I^1$, $\mathcal{A}$ can only learn their outputs indirectly by querying **CORRUPT**. From Eq. (66) and the definitions of DER_{resp} and DER_{init} , any $\mathbf{F}_R^1(c, c', \dots)$ obtained by $\mathcal{A}$ must satisfy that c decrypts correctly while c' incurs decryption failure; in contrast, any $\mathbf{F}_I^1(c, c', \dots)$ obtained by $\mathcal{A}$ must satisfy that c incurs decryption failure while c' decrypts correctly. This implies that the input domains of $\mathbf{H}_3$, $\mathbf{F}_R^1$ and $\mathbf{F}_I^1$ are pairwise disjoint in game $\mathbf{G}_6^b$. We can hence equivalently merge **Branch 3, 4, 7, and 8**: we directly use $\mathbf{H}_3$ to generate K_r or K_i when $i = j = j^*$, removing the step that uses sk_{j^*} to verify whether c_i and c_j decrypts correctly. Combining the above analysis for **Branch 1** to **8**, K_r and K_i can be equivalently generated as follows:

$$\begin{aligned} K_r &= \begin{cases} \mathbf{H}_2(m_i, c_j, K^*, i, j, M, M') & j = j^*, i \neq j^*, \\ \mathbf{H}_3(c_i, c_j, K^*, i, j, M, M') & j = j^*, i = j^*, \end{cases} \\ K_i &= \begin{cases} \mathbf{H}_1(c_i, m_j, K', i, j, M, M') & i = j^*, j \neq j^*, \\ \mathbf{H}_3(c_i, c_j, K', i, j, M, M') & i = j^*, j = j^*. \end{cases} \end{aligned}$$

This exactly matches the construction of game $\mathbf{G}_7^b$, leading to

$$\Pr [1 \leftarrow \mathbf{G}_6^b] = \Pr [1 \leftarrow \mathbf{G}_7^b]. \quad (68)$$

Game $\mathbf{G}_8^b$: In this game, we switch back to using $\mathbf{G} \stackrel{\$}{\leftarrow} \text{Fcs}[\mathcal{M}, \mathcal{R}]$ instead of $\mathbf{G}_{\text{good}}$. With the same analysis of the game hop $\mathbf{G}_7^b$ to $\mathbf{G}_8^b$ in the proof of Lemma 5,

$$|\Pr[1 \leftarrow \mathbf{G}_7^b] - \Pr[1 \leftarrow \mathbf{G}_8^b]| \leq q_{\mathbf{G}} \cdot \delta_{\mathbf{P}}. \quad (69)$$

Game $\mathbf{G}_9^b$: This game is the same as $\mathbf{G}_8^b$, except that it modifies the keys produced by DER_{init} and DER_{resp} :

- For DER_{init} with input $(\text{sID} \neq s_{\text{init}}, M')$, if the ephemeral public key $\widetilde{pk}$ parsed from $\text{st}[\text{sID}]$ equals $\widetilde{pk}^*$, it directly sets K_i to a uniformly random value of $\mathcal{K}$.
- For DER_{init} with input (s_{init}, M') , if there exists a $\text{sID} \neq s_{\text{init}}$ such that sID and s_{init} become matching after performing the assignment $\text{rece}(s_{\text{init}}) := M'$, it directly sets K_i to a uniformly random value of $\mathcal{K}$.
- For DER_{resp} with input (sID, M) , if sID and s_{init} become matching after performing the assignment $(\text{sent}(\text{sID}), \text{rece}(\text{sID})) := (M', M)$, it sets K_r to a uniformly random value of $\mathcal{K}$.

These modifications are introduced to guarantee those sessions distinct from s_{init} never query the random oracle with the same (M, M') used by s_{init} for generating the final key; this property will be used in our subsequent game transitions.

Notice that outputting 1 in game $\mathbf{G}_8^b$ ensures $\mathfrak{M}(\text{sID}^*) = \emptyset$, $\text{role}[\text{sID}^*] = \text{initiator}$, and $\text{sID}^* = s_{\text{init}}$. Hence, on the event that the game outputs 1, the adversary $\mathcal{A}$ cannot detect the latter two modifications made in game $\mathbf{G}_9^b$. Regarding the first modification, since $\text{st}[\text{sID}]$ is honestly generated by $\text{INIT}(\text{sID})$, the probability that it also produce $\widetilde{pk}^*$ is at most $\gamma(\Pi_{\text{KEM}}.\text{KeyGen})$, and hence we have

$$|\Pr[1 \leftarrow \mathbf{G}_8^b] - \Pr[1 \leftarrow \mathbf{G}_9^b]| \leq S \cdot \gamma(\Pi_{\text{KEM}}.\text{KeyGen}). \quad (70)$$

Game $\mathbf{G}_{10}^b$: This game is the same as $\mathbf{G}_9^b$, except that in $\text{DER}_{\text{init}}(s_{\text{init}}, M')$ it no longer performs $m_j := m_{j^*}$ and directly sets $K_i := k^*$. Here k^* is a fresh uniformly random value in $\mathcal{K}$ sampled at the game beginning. Note that in this game, the plaintext m_{j^*} is no longer used by $\text{DER}_{\text{init}}(s_{\text{init}}, M')$ to generated the final key K_i .

Let $(i, j) := (\text{holder}[\text{sID}], \text{peer}[\text{sID}])$. Define **Error** as the event such that, just after the query $\text{DER}_{\text{init}}(s_{\text{init}}, M')$ finishes the assignment $(\text{sent}(s_{\text{init}}), \text{rece}(s_{\text{init}})) := (\widetilde{M}, M')$, there exists a session $\text{sID} \neq s_{\text{init}}$ for which sID and s_{init} constitute a matching session pair. Now, according to the definition of game $\mathbf{G}_9^b$, the final key K_i generated by $\text{DER}_{\text{init}}(s_{\text{init}}, M' = (c_i, \widetilde{c}))$ in this game is:

$$K_i = \begin{cases} k \stackrel{\$}{\leftarrow} \mathcal{K} & \text{Error occurs,} \\ k' & \text{Error does not occur,} \end{cases}$$

where

$$k' := \begin{cases} H_{\text{init}}(s_i, c_i, m_{j^*}, K', i, j, M, M') & i \neq j^*, j = j^*, c_i \text{ decryption fails,} \\ H_2(m'_i, c_j^*, K', i, j, M, M') & i \neq j^*, j = j^*, c_i \text{ decryption succeeds,} \\ H_3(c_i, c_j^*, K', i, j, M, M') & i = j^*, j = j^*, \\ H_{\text{init}}(s_i, c_i, m_{j^*}, K', i, j, M, M') & i \neq j^*, j \neq j^*, c_i \text{ decryption fails,} \\ H_4(m'_i, m_{j^*}, K', i, j, M, M') & i \neq j^*, j \neq j^*, c_i \text{ decryption succeeds,} \\ H_1(c_i, m_{j^*}, K', i, j, M, M') & i = j^*, j \neq j^*. \end{cases} \quad (71)$$

In game $\mathbf{G}_9^b$, $\neg\text{Error}$ implies that just after executing $\text{DER}_{\text{init}}(s_{\text{init}}, M')$, s_{init} does not form a matching session with any session established prior to this execution. Consequently, for all random oracle queries submitted by DER_{resp} to derivate key before this execution, the value (M, M') contained in these queries must differ from the $(\widetilde{M}, M')$ used by s_{init} . We recall that first and third modifications introduced in $\mathbf{G}_9^b$ further guarantee that the (M, M') appearing in key-derivation random oracle queries issued by DER_{init} and subsequent DER_{resp} is distinct from the $(\widetilde{M}, M')$ used by s_{init} . Therefore, conditioned on $\neg\text{Error}$, the adversary $\mathcal{A}$ cannot obtain k' via the strategy of first querying DER_{init} or DER_{resp} and then querying REVEAL . Instead, $\mathcal{A}$ can only obtain k' either by querying H_{init} with input $(*, *, m_{j^*}, \dots)$, or by querying H with input $(*, m, \dots)$ where $m = m_{j^*}$ or $m \neq m_{j^*} \wedge \Pi_{\text{PKE}}.\text{Enc}(pk_{j^*}, m; G(m)) = c_{j^*}$. Define events

$\text{Bad}_{\mathbf{G}, m_j^*}^b$ ($b \in \{0, 1\}$): During the running of $\mathcal{A}$, either H_{init} is ever queried with input $(*, *, m_{j^*}, \dots)$ or H is ever queried with input $(*, m, \dots)$ where $m \in S_{\mathbf{G}, m_j^*}$ and $S_{\mathbf{G}, m_j^*} := \{m_{j^*}^*\} \cup \{m : m \neq m_{j^*} \wedge \Pi_{\text{PKE}}.\text{Enc}(pk_{j^*}, m, G(m)) = c_{j^*}\}$.

Here b denotes whether the oracle TEST returns K_0^* or K_1^* . In fact, the above analysis implies that, conditioned on $\neg\text{Error} \wedge \neg\text{Bad}_{\mathbf{G}, m_j^*}^b$, $\mathcal{A}$'s view in game $\mathbf{G}_9^b$ is independent of the random oracle values in all branches corresponding to k' (Eq. (71)). Consequently, these values are uniformly random in $\mathcal{A}$'s view, and thus k' can be equivalently repalced by

$$k' = \begin{cases} k_1 \xleftarrow{\$} \mathcal{K} & i \neq j^*, j = j^*, c_i \text{ decryption fails,} \\ k_2 \xleftarrow{\$} \mathcal{K} & i \neq j^*, j = j^*, c_i \text{ decryption succeeds,} \\ k_3 \xleftarrow{\$} \mathcal{K} & i = j^*, j = j^*, \\ k_4 \xleftarrow{\$} \mathcal{K} & i \neq j^*, j \neq j^*, c_i \text{ decryption fails,} \\ k_5 \xleftarrow{\$} \mathcal{K} & i \neq j^*, j \neq j^*, c_i \text{ decryption succeeds,} \\ k_6 \xleftarrow{\$} \mathcal{K} & i = j^*, j \neq j^*. \end{cases}$$

At this point, the operation " $m_j := m_{j^*}$ " can be directly omitted since the generation of k' no longer uses it. Note that k' given in the above equation is obviously still a uniformly random vale of $\mathcal{K}$. Thus, we have equivalently transitioned to the setting of game $\mathbf{G}_{10}^b$, which indicates that

$$\Pr \left[1 \leftarrow \mathbf{G}_9^b \wedge \neg\text{Error} \wedge \neg\text{Bad}_{\mathbf{G}, m_j^*}^b \right] = \Pr \left[1 \leftarrow \mathbf{G}_{10}^b \wedge \neg\text{Error} \wedge \neg\text{Bad}_{\mathbf{G}, m_j^*}^b \right], \quad (72)$$

$$\Pr \left[\neg\text{Error} \wedge \neg\text{Bad}_{\mathbf{G}, m_j^*}^b : \mathbf{G}_9^b \right] = \Pr \left[\neg\text{Error} \wedge \neg\text{Bad}_{\mathbf{G}, m_j^*}^b : \mathbf{G}_{10}^b \right]. \quad (73)$$

Since the event **Error** depends only on the game execution prior to the query $\text{DER}_{\text{init}}(s_{\text{init}}, M')$, and games $\mathbf{G}_9^b$ and $\mathbf{G}_{10}^b$ are identical up to that point, we have $\Pr[\neg \text{Error} : \mathbf{G}_9^b] = \Pr[\neg \text{Error} : \mathbf{G}_{10}^b]$. Combining this equation with Eq. (73),

$$\Pr[\neg \text{Error} \wedge \text{Bad}_{\mathbf{G}, m_j^*}^b : \mathbf{G}_9^b] = \Pr[\neg \text{Error} \wedge \text{Bad}_{\mathbf{G}, m_j^*}^b : \mathbf{G}_{10}^b]. \quad (74)$$

Now we can compute

$$\begin{aligned} & |\Pr[1 \leftarrow \mathbf{G}_9^b] - \Pr[1 \leftarrow \mathbf{G}_{10}^b]| \\ & \stackrel{(a)}{=} |\Pr[1 \leftarrow \mathbf{G}_9^b \wedge \neg \text{Error}] - \Pr[1 \leftarrow \mathbf{G}_{10}^b \wedge \neg \text{Error}]| \\ & \stackrel{(b)}{=} |\Pr[1 \leftarrow \mathbf{G}_9^b \wedge \neg \text{Error} \wedge \text{Bad}_{\mathbf{G}, m_j^*}^b] - \Pr[1 \leftarrow \mathbf{G}_{10}^b \wedge \neg \text{Error} \wedge \text{Bad}_{\mathbf{G}, m_j^*}^b]| \\ & \stackrel{(c)}{\leq} \Pr[\neg \text{Error} \wedge \text{Bad}_{\mathbf{G}, m_j^*}^b : \mathbf{G}_{10}^b] \\ & \leq \Pr[\text{Bad}_{\mathbf{G}, m_j^*}^b : \mathbf{G}_{10}^b]. \end{aligned} \quad (75)$$

Here, (a) holds because the event that the game outputs 1 guarantees $\mathfrak{M}(\text{sID}^*) = \emptyset$ and $\text{sID}^* = s_{\text{init}}$, which implies the event **Error** does not occur. (b) follows from Eq. (72) and (c) follows from Eq. (74). Next, to compute $\Pr[\text{Bad}_{\mathbf{G}, m_j^*}^b : \mathbf{G}_{10}^b]$, we first define two new events as:

$\text{Bad}_{m_j^*}^b$ ($b \in \{0, 1\}$): This event is the same as $\text{Bad}_{\mathbf{G}, m_j^*}^b$, except that the set $S_{\mathbf{G}, m_j^*} := \{m_j^*\} \cup \{m : m \neq m_j^* \wedge \Pi_{\text{PKE}}.\text{Enc}(pk_{j^*}, m; \mathbf{G}(m)) = c_{j^*}\}$ is replaced by $S_{m_j^*} := \{m_j^*\}$.

$\text{Coll}_{m_j^*}$: There exists a $m \neq m_j^*$ such that $\Pi_{\text{PKE}}.\text{Enc}(pk_{j^*}, m; \mathbf{G}(m)) = c_{j^*}$.

Obviously, if $\text{Coll}_{m_j^*}$ does not occur, $\{m : m \neq m_j^* \wedge \Pi_{\text{PKE}}.\text{Enc}(pk_{j^*}, m; \mathbf{G}(m)) = c_{j^*}\}$ is an empty set and thus $S_{\mathbf{G}, m_j^*}$ and $S_{m_j^*}$ are identical. Therefore, we have

$$|\Pr[\text{Bad}_{\mathbf{G}, m_j^*}^b : \mathbf{G}_{10}^b] - \Pr[\text{Bad}_{m_j^*}^b : \mathbf{G}_{10}^b]| \leq \Pr[\text{Coll}_{m_j^*} : \mathbf{G}_{10}^b] \stackrel{(a)}{\leq} 2 \cdot \delta_{\text{P}}. \quad (76)$$

Here (a) uses the following corollary.

Corollary 1. For a δ -correct PKE scheme Π_{PKE} with message space $\mathcal{M}$ and randomness space $\mathcal{R}$, we have

$$\Pr[\exists m \neq m^*, \Pi_{\text{PKE}}.\text{Enc}(pk, m; \mathbf{G}(m)) = \Pi_{\text{PKE}}.\text{Enc}(pk, m^*; \mathbf{G}(m^*))] \leq 2 \cdot \delta,$$

where the probability is taken over $(pk, sk) \leftarrow \Pi_{\text{PKE}}.\text{KeyGen}$, $m^* \xleftarrow{\$} \mathcal{M}$ and $\mathbf{G} \xleftarrow{\$} \text{Fcs}[\mathcal{M}, \mathcal{R}]$.

Game $\mathbf{G}_{11}^b$: This game is the same as $\mathbf{G}_{10}^b$, except that in $\text{Challenge}(s_{\text{init}}, j^*)$ it replaces $\mathbf{G}(m_{j^*})$ by r^* , a fresh uniformly random value in $\mathcal{R}$.

Denote $\text{BadG}_{m_j^*}^b$ as the event that during the running of $\mathcal{A}$, $\mathbf{G}$ is ever queried with input m_j^* . Then, by the basic rules of random oracles, we have

$$\begin{aligned} |\Pr[1 \leftarrow \mathbf{G}_{10}^b] - \Pr[1 \leftarrow \mathbf{G}_{11}^b]| &\leq \Pr[\text{BadG}_{m_j^*}^b : \mathbf{G}_{11}^b], \\ |\Pr[\text{BadG}_{m_j^*}^b : \mathbf{G}_{10}^b] - \Pr[\text{BadG}_{m_j^*}^b : \mathbf{G}_{11}^b]| &\leq \Pr[\text{BadG}_{m_j^*}^b : \mathbf{G}_{11}^b]. \end{aligned} \quad (77)$$

As for the probability $\Pr[\text{BadG}_{m_j^*}^b : \mathbf{G}_{11}^b]$, we first construt a OW-CPA adversary $\mathcal{B}_b$ ($b \in \{0, 1\}$) against Π_{PKE} as follows, and then bound this probability by using $\mathcal{B}_b$'s advantage.

- The input of $\mathcal{B}_b$ is (pk_{j^*}, c_{j^*}) , where pk_{j^*} is the public key and c_{j^*} is the challenge ciphertext.
- $\mathcal{B}_b$ runs game $\mathbf{G}_{11}^b$, simulates the random oracles on-th-fly.
- When running $\text{Challenge}(s_{\text{init}}, j^*)$, $\mathcal{B}_b$ replaces the (pk_{j^*}, c_{j^*}) generated there by its own input.
- After $\mathcal{A}$ returns its output b' , $\mathcal{B}_b$ uniformly samples an element from the record lists of the random oracle $\mathbf{G}$, which we denote as m , and then returns m as its final output.

We stress that although $\mathcal{B}_b$ dose not have the secret key sk_{j^*} corresponding to pk_{j^*} and the plaintext m_{j^*} , it can still run game $\mathbf{G}_{11}^b$ normally, as the computations of this game no longer require sk_{j^*} and m_{j^*} . Obviously, for any $b \in \{0, 1\}$, the running time of $\mathcal{B}_b$ is roughly the same as those of $\mathcal{A}$, and $\text{BadG}_{m_j^*}^b$ occurring implies that $\mathcal{B}_b$ has an advantage at least $1/(q_H + q_G)$ to successfully guess the plaintext m_{j^*} of c_{j^*} . Thus, we have

$$\Pr[\text{BadG}_{m_j^*}^b : \mathbf{G}_{11}^b] \leq (q_H + q_G) \cdot \text{Adv}_{\Pi_{\text{PKE}}}^{\text{OW-CPA}}(\mathcal{B}_b). \quad (78)$$

The probability $\Pr[\text{BadG}_{m_j^*}^b : \mathbf{G}_{11}^b]$ can be similarly upper bounded, that is, there exists a OW-CPA adversary $\mathcal{B}'_b$ ($b \in \{0, 1\}$) (with roughly the same running time of $\mathcal{A}$) against Π_{PKE} such that

$$\Pr[\text{BadG}_{m_j^*}^b : \mathbf{G}_{11}^b] \leq (q_I + q_H) \cdot \text{Adv}_{\Pi_{\text{PKE}}}^{\text{OW-CPA}}(\mathcal{B}'_b). \quad (79)$$

Game $\mathbf{G}_{12}^b$: This game is the same as $\mathbf{G}_{11}^b$, except that the oracle TEST directly sets K_0^* as k^* , rather than $\text{sKey}[\text{sID}^*]$. Since outputting 1 in game $\mathbf{G}_{11}^b$ guarantees $\mathfrak{M}(\text{sID}^*) = \emptyset$, $\text{role}[\text{sID}^*] = \text{initiator}$, $\text{sID}^* = s_{\text{init}}$, $\text{peer}(\text{sID}^*) = j^*$, and sID^* is fresh, the K_0^* in game $\mathbf{G}_{11}^b$ is also k^* if it finally outputs 1. Thus, game $\mathbf{G}_{12}^b$ is only a conceptual change and

$$\Pr[1 \leftarrow \mathbf{G}_{11}^b] = \Pr[1 \leftarrow \mathbf{G}_{12}^b]. \quad (80)$$

Game $\mathbf{G}_{13}^b$: This game is the same as $\mathbf{G}_{12}^b$, except that in $\text{DER}_{\text{init}}(s_{\text{init}}, M')$ it directly sets $K_i := 0$. Similar to the above game hop, since outputting 1 in game $\mathbf{G}_{12}^b$ guarantees $\mathfrak{M}(\text{sID}^*) = \emptyset$, $\text{role}[\text{sID}^*] = \text{initiator}$, $\text{sID}^* = s_{\text{init}}$, $\text{peer}(\text{sID}^*) = j^*$, and sID^* is fresh, the adversary $\mathcal{A}$ never queries REVEAL with s_{init} to get the K_i

if the game finally outputs 1, and thus $\mathcal{A}$ cannot detect our change in game $\mathbf{G}_{13}^b$. Therefore, we have

$$\Pr [1 \leftarrow \mathbf{G}_{12}^b] = \Pr [1 \leftarrow \mathbf{G}_{13}^b]. \quad (81)$$

In game $\mathbf{G}_{13}^b$, K_0^* and K_1^* are both uniformly random values in $\mathcal{K}$, and thus

$$\Pr [1 \leftarrow \mathbf{G}_{13}^0] = \Pr [1 \leftarrow \mathbf{G}_{13}^1]. \quad (82)$$

At this point, by combining Eq. (59) to Eq. (64), Eq. (67) to Eq. (70), Eq. (75) to Eq. (82), and folding adversaries $\mathcal{B}_0, \mathcal{B}_1, \mathcal{B}'_0, \mathcal{B}'_1$ into one adversary $\mathcal{B}$, we obtain

$$\begin{aligned} \text{Case 3} \leq & 2S^2N \cdot \gamma(\Pi_{\text{KEM}}.\text{KeyGen}) + \frac{2SN(q_R + q_I)}{|\mathcal{M}|} \\ & + 4SN(q_G + 1) \cdot \delta_P + 8SN(q_G + 2q_H + q_I) \cdot \text{Adv}_{\Pi_{\text{PKE}}}^{\text{OW-CPA}}(\mathcal{B}). \end{aligned}$$

This completes the proof of Lemma 6. $\square$

A.4 Proof of Lemma 7

Proof. To prove this lemma, we introduce a sequence of games $\mathbf{G}_0^b$ to $\mathbf{G}_8^b$ ($b \in \{0, 1\}$) as shown in Fig. 14. In these games, H_{init} , H_{resp} , G , and H are random oracles, and the adversary $\mathcal{A}$ can also query a set of oracles $\mathcal{O}_{\text{AKE}} = \{\text{EST}, \text{INIT}, \text{DER}_{\text{resp}}, \text{DER}_{\text{init}}, \text{REVEAL}, \text{REV-STATE}, \text{CORRUPT}, \text{TEST}\}$.

Game $\mathbf{G}_0^b$: This game is the same as $\text{KeyIND}_{\Pi_{\text{AKE}}}^{\mathcal{A}, b}$ (Fig. 4), except that it additionally checks whether $\mathfrak{M}(\text{sID}^*) = \emptyset \wedge \text{role}[\text{sID}^*] = \text{responder}$ occurs, and directly returns 0 if this check fails. Thus,

$$\Pr [1 \leftarrow \mathbf{G}_0^b] = \Pr \left[1 \leftarrow \text{KeyIND}_{\Pi_{\text{AKE}}}^{\mathcal{A}, b} \wedge \mathfrak{M}(\text{sID}^*) = \emptyset \wedge \text{role}[\text{sID}^*] = \text{responder} \right]. \quad (83)$$

Game $\mathbf{G}_1^b$: This game is the same as $\mathbf{G}_0^b$, except that at the very beginning, it uniformly samples a session s_{resp} from $\{1, \dots, S\}$ and an index i^* from $\{1, \dots, N\}$, and checks whether $\text{sID}^* = s_{\text{resp}}$ and $\text{peer}(\text{sID}^*) = i^*$ at the end. If such check fails, game $\mathbf{G}_1^b$ directly returns 0.

In game $\mathbf{G}_1^b$, we make a random guess at the beginning, hoping to directly obtain sID^* and $\text{peer}(\text{sID}^*)$ (i.e., the s_{resp} and i^*). This guessing step is necessary; with it, we can embed the challenge ciphertext of Π_{PKE} into the computation of $\text{DER}_{\text{resp}}(s_{\text{resp}})$. As for the difference between games $\mathbf{G}_0^b$ and $\mathbf{G}_1^b$, we have

$$\Pr [1 \leftarrow \mathbf{G}_0^b] = SN \cdot \Pr [1 \leftarrow \mathbf{G}_1^b]. \quad (84)$$

Game $\mathbf{G}_2^b$: This game is the same as $\mathbf{G}_1^b$, except that it introduces a new procedure $\text{Challenge}(s_{\text{resp}}, i^*)$ and modifies the computations of DER_{resp} according to the internal random values generated by $\text{Challenge}(s_{\text{resp}}, i^*)$, as detailed below:

- $\text{Challenge}(s_{\text{resp}}, i^*)$ honestly simulates the generation of c_i in $\text{DER}_{\text{resp}}(s_{\text{resp}})$. Specifically, this procedure samples $m_{i^*} \xleftarrow{\$} \mathcal{M}$ and then computes $c_{i^*} := \Pi_{\text{PKE}}.\text{Enc}(pk_{i^*}, m_{i^*}; G(m_{i^*}))$.
- For DER_{resp} , if the input session is s_{resp} , it uses m_{i^*} to generate the key K_r (i.e. $m_i := m_{i^*}$) and sets $M' := (c_{i^*}, \tilde{c})$.

For all remaining input, DER_{resp} is in accordance with the specification given in Fig. 4.

In game $\mathbf{G}_2^b$, we simulate the generation of c_i in $\text{DER}_{\text{resp}}(s_{\text{resp}}, M)$ in advance by executing $\text{Challenge}(s_{\text{resp}}, i^*)$. We then perform a check on the input of DER_{resp} , and insert c_{i^*} generated during this simulation into M' . Since outputting 1 in both $\mathbf{G}_1^b$ and $\mathbf{G}_2^b$ guarantees $\text{sID}^* = s_{\text{resp}}$ and $\text{peer}(\text{sID}^*) = i^*$, the generation of c_i by $\text{DER}_{\text{resp}}(s_{\text{resp}}, M)$ in $\mathbf{G}_1^b$ is equivalent to that simulated by $\text{Challenge}(s_{\text{resp}}, i^*)$ in $\mathbf{G}_2^b$. As a result, the modifications introduced in $\mathbf{G}_2^b$ relative to $\mathbf{G}_1^b$ do not change the adversary's view under the condition that the game outputs 1, thus,

$$\Pr [1 \leftarrow \mathbf{G}_1^b] = \Pr [1 \leftarrow \mathbf{G}_2^b]. \quad (85)$$

GAMES $\mathbf{G}_0^b$ - $\mathbf{G}_{13}^b$ ($b \in \{0, 1\}$)	INIT(sID)
<pre> 1, cnt := 0, sID* := 0 2, H_{init}, H_{resp}, H $\xleftarrow{\\$}$ Fcs[{0, 1}* , K], G $\xleftarrow{\\$}$ Fcs[M, R] s_{resp} $\xleftarrow{\\$}$ {1, ..., S}, i* $\xleftarrow{\\$}$ {1, ..., N} //G₁^b-G₁₃^b G := G_{good} //G₅^b-G₇^b k* $\xleftarrow{\\$}$ K //G₁₀^b-G₁₃^b r* $\xleftarrow{\\$}$ R //G₁₁^b-G₁₃^b 3, for i ∈ {1, ..., N} (pk_i, sk_i) ← Π_{PKE}.KeyGen, s_i $\xleftarrow{\\$}$ M sk'_i := (sk_i, s_i), crp[i] := false end for run Challenge(s_{resp}, i*) //G₂^b-G₁₃^b 4, b' ← A^{OAKE, H_{init}, H_{resp}, G, H}(pk₁, ..., pk_N) 5, if sID* is not fresh, return 0 if M(sID*) ≠ ∅ ∨ role[sID*] = initiator, return 0 //G₀^b-G₁₃^b if sID* ≠ s_{resp} ∨ peer[sID*] ≠ i*, return 0 //G₁^b-G₁₃^b 6, return b' </pre>	<pre> 1, if holder[sID] = ⊥ ∨ sent[sID] ≠ ⊥, return ⊥ 2, role[sID] := initiator 3, (i, j) := (holder[sID], peer[sID]) (M, st) ← GC-AKE.Init(sk_i, pk_j) (sent[sID], st[sID]) := (M, st) return M DER_{init}(sID, M') 1, if holder[sID] = ⊥ ∨ st[sID] = ⊥ ∨ sKey[sID] ≠ ⊥ return ⊥ 2, (i, j) := (holder[sID], peer[sID]) Parse st[sID] as (M = (pk̃, c_j), sk̃, m_j) rece[sID] := M' if sID and s_{resp} are matching; K_i $\xleftarrow{\\$}$ K //G₉^b-G₁₃^b Parse M' as (c_i, c̃) if i = i*, j ≠ i*, K_i := H₁(c_i, m_j, K', i, j, M, M') //G₇^b-G₁₃^b if i = i*, j = i*, K_i := H₃(c_i, c_j, K', i, j, M, M') //G₇^b-G₁₃^b m'_i := Π_{PKE}.Dec(sk_i, c_i) K' := Π_{KEM}.Decap(sk̃, c̃) if m'_i = ⊥ ∨ Π_{PKE}.Enc(pk_i, m'_i; G(m'_i)) ≠ c_i K_i := H_{init}(s_i, c_i, m_j, c̃, i, j, M, M') if i = i*, K_i := F₁(c_i, m_j, c̃, i, j, M, M') //G₄^b-G₁₃^b else K_i := H(m'_i, m_j, K', i, j, M, M') sKey[sID] := K_i </pre>
DER _{resp} (sID, M)	TEST(sID) //only one query
<pre> 1, if holder[sID] = ⊥ ∨ sKey[sID] ≠ ⊥ ∨ role[sID] = initiator, return ⊥ 2, role[sID] := responder 3, (j, i) := (holder[sID], peer[sID]) Parse M as (pk̃, c_j) m_i $\xleftarrow{\\$}$ M, c_i := Π_{PKE}.Enc(pk_i, m_i; G(m_i)) (c̃, K*) := Π_{KEM}.Encap(pk̃), M' := (c_i, c̃) if sID ≠ s_{resp}, c_i = c_i*; K_r $\xleftarrow{\\$}$ K //G₉^b-G₁₃^b if sID = s_{resp}; m_i := m_i*, M' := (c_i*, c̃) //G₂^b-G₁₃^b if sID = s_{resp}, K_r := k* //G₁₀^b-G₁₃^b if sID = s_{resp}, K_r := 0 //G₁₃^b (sent[sID], rece[sID]) := (M', M) if sID = s_{resp}, s_{resp} has a match; K_r $\xleftarrow{\\$}$ K //G₉^b-G₁₃^b if j = i*, i ≠ i*, K_r := H₂(m_i, c_j, K*, i, j, M, M') //G₇^b-G₁₃^b if j = i*, i = i*, K_r := H₃(c_i, c_j, K*, i, j, M, M') //G₇^b-G₁₃^b m'_j := Π_{PKE}.Dec(sk_j) if m'_j = ⊥ ∨ Π_{PKE}.Enc(pk_j, m_j; G(m'_j)) ≠ c_j K_r := H_{resp}(s_j, m_i, c_j, K*, i, j, M, M') if j = i*, K_r := F_R(m_i, c_j, K*, i, j, M, M') //G₄^b-G₁₃^b else K_r := H(m_i, m'_j, K*, i, j, M, M') sKey[sID] := K_i return M' </pre>	<pre> 1, sID* := sID //test session's id 2, if sKey[sID*] = ⊥, return ⊥ 3, K₀* := sKey[sID*], K₁* $\xleftarrow{\\$}$ K K₀* := k*, K₁* $\xleftarrow{\\$}$ K //G₁₂^b-G₁₃^b 4, return K₀* CORRUPT(i ∈ {1, ..., N}) crp[i] := true if i = i*, return 0 //G₃^b-G₁₃^b return (sk_i, s_i) Challenge(s_{resp}, i*) 1, m_i* $\xleftarrow{\\$}$ M 2, c_i* := Π_{PKE}.Enc(pk_i*, m_i*; G(m_i*)) c_i* := Π_{PKE}.Enc(pk_i*, m_i*; r*) //G₁₁^b-G₁₃^b </pre>

Fig. 14: Games $\mathbf{G}_0^b$ to $\mathbf{G}_{13}^b$ used in the proof of Lemma 7. Here for brevity, we only list the oracles TEST, CORRUPT, INIT, DER_{resp} and DER_{init} that need to be modified in our game sequence. The definitions of all other unlisted oracles are identical to those given in Fig. 4.

Game $\mathbf{G}_3^b$: This game is the same as $\mathbf{G}_2^b$, except that the oracle CORRUPT directly returns 0 for the input i^* .

Notice that outputting 1 in $\mathbf{G}_2^b$ ensures $\mathfrak{M}(\text{sID}^*) = \emptyset$, $\text{role}[\text{sID}^*] = \text{responder}$, $\text{sID}^* = s_{\text{resp}}$, $\text{peer}(\text{sID}^*) = i^*$, and sID^* is fresh. Therefore, by Definition 11, the game outputting 1 implies that the adversary $\mathcal{A}$ never queries CORRUPT with i^* to get the long-term secret key sk_{i^*} . That is to say, under the condition that the game outputting 1, $\mathcal{A}$ cannot detect our modification to CORRUPT. Thus

$$\Pr[1 \leftarrow \mathbf{G}_2^b] = \Pr[1 \leftarrow \mathbf{G}_3^b]. \quad (86)$$

Game $\mathbf{G}_4^b$: This game introduces two new random oracles $\mathbf{F}_R$ and $\mathbf{F}_I$, and modifies the keys produced by DER_{resp} and DER_{init} . For $\text{DER}_{\text{resp}}(\text{sID}, M)$, if $\text{holder}[\text{sID}] = i^*$ and decryption of c_j fails, $K_r := \mathbf{F}_R(m_i, c_j, K^*, i, j, M, M')$; for $\text{DER}_{\text{init}}(\text{sID}, M')$, if $\text{holder}[\text{sID}] = i^*$ and decryption of c_i fails, $K_i := \mathbf{F}_I(c_i, m_j, \tilde{c}, i, j, M, M')$.

It is easy to see that these two games are equivalent, if oracles $\mathbf{H}_{\text{resp}}$ and $\mathbf{H}_{\text{init}}$ are never queried with input $(s_{i^*}, \dots)$ during the running of the adversary $\mathcal{A}$. Since in both $\mathbf{G}_3^b$ and $\mathbf{G}_4^b$, s_{i^*} is uniformly sampled from $\mathcal{M}$ and the running of $\mathcal{A}$ no longer depends on s_{i^*} , we have

$$|\Pr[1 \leftarrow \mathbf{G}_3^b] - \Pr[1 \leftarrow \mathbf{G}_4^b]| \leq \frac{q_R + q_I}{|\mathcal{M}|}. \quad (87)$$

Game $\mathbf{G}_5^b$: This game is the same as $\mathbf{G}_4^b$, except that $\mathbf{G}$ is set as $\mathbf{G}_{\text{good}}$, a function with domain $\mathcal{M}$ and codomain $\mathcal{R}$, and $\mathbf{G}_{\text{good}}(m)$ is uniformly sampled from $\mathcal{R}_{\text{good}}(pk_{i^*}, sk_{i^*}, m)$ (Eq. (42)) for any $m \in \mathcal{M}$. With the same analysis of the game hop $\mathbf{G}_7^b$ to $\mathbf{G}_8^b$ in the proof of Lemma 5, we have

$$|\Pr[1 \leftarrow \mathbf{G}_4^b] - \Pr[1 \leftarrow \mathbf{G}_5^b]| \leq q_G \cdot \delta_P. \quad (88)$$

Game $\mathbf{G}_6^b$: Unlike game $\mathbf{G}_5^b$, this game modifies the definition of random oracle $\mathbf{H}$ as follows.

$$\begin{aligned} & \mathbf{H}(m_1, m_2, K, i, j, M, M') \\ &= \begin{cases} \mathbf{H}_1(\Pi_{\text{PKE}}.\text{Enc}(pk_{i^*}, m_1; \mathbf{G}(m_1)), m_2, K, i, j, M, M') & i = i^*, j \neq i^*, \\ \mathbf{H}_2(m_1, \Pi_{\text{PKE}}.\text{Enc}(pk_{i^*}, m_2; \mathbf{G}(m_2)), K, i, j, M, M') & i \neq i^*, j = i^*, \\ \mathbf{H}_3 \left(\begin{array}{l} \Pi_{\text{PKE}}.\text{Enc}(pk_{i^*}, m_1; \mathbf{G}(m_1)), \\ \Pi_{\text{PKE}}.\text{Enc}(pk_{i^*}, m_2; \mathbf{G}(m_2)), K, i, j, M, M' \end{array} \right) & i = i^*, j = i^*, \\ \mathbf{H}_4(m_1, m_2, K, i, j, M, M') & \text{otherwise.} \end{cases} \end{aligned} \quad (89)$$

In addition, $\mathbf{G}_6^b$ also modifies the definition of random oracles $\mathbf{F}_I$ and $\mathbf{F}_R$ as follows.

$$\begin{aligned} \mathbf{F}_I(c, m, K, i, j, M, M') &= \begin{cases} \mathbf{F}_I^1(c, \Pi_{\text{PKE}}.\text{Enc}(pk_{i^*}, m; \mathbf{G}(m)), K, i, j, M, M') & i = j = i^*, \\ \mathbf{F}_I^2(c, m, K, i, j, M, M') & \text{otherwise,} \end{cases} \\ \mathbf{F}_R(m, c, K, i, j, M, M') &= \begin{cases} \mathbf{F}_R^1(\Pi_{\text{PKE}}.\text{Enc}(pk_{i^*}, m; \mathbf{G}(m)), c, K, i, j, M, M') & i = j = i^*, \\ \mathbf{F}_R^2(m, c, K, i, j, M, M') & \text{otherwise.} \end{cases} \end{aligned} \quad (90)$$

In the above definitions, $H_1, H_2, H_3, H_4, F_1^1, F_1^2, F_R^1$, and F_R^2 are fresh random oracles cannot be directly accessed by the adversary $\mathcal{A}$.

Notice that G has been set as G_{good} in games G_5^b and G_6^b , where $G_{\text{good}}(m)$ is uniformly sampled from $\mathcal{R}_{\text{good}}(pk_{i^*}, sk_{i^*}, m)$ for any $m \in \mathcal{M}$. Hence, by the definition of $\mathcal{R}_{\text{good}}(pk_{i^*}, sk_{i^*}, m)$ given in Eq. (42), we can conclude that the function $g(\cdot) := \Pi_{\text{PKE}}.\text{Enc}(pk_{i^*}, \cdot; G(\cdot))$ is an injective function. Thus, by the basic rules of random oracles, game G_6^b is only a conceptual change and

$$\Pr [1 \leftarrow G_5^b] = \Pr [1 \leftarrow G_6^b]. \quad (91)$$

Game G_7^b : This game is the same as G_6^b , except that it modifies the keys produced by DER_{resp} and DER_{init} as follows.

- For $\text{DER}_{\text{resp}}(\text{sID}, M)$, if $j = i^*$, and $i \neq i^*$ the key K_r is directly set to $H_2(m_i, c_j, K^*, i, j, M, M')$. Here $(j, i) = (\text{holder}[\text{sID}], \text{peer}[\text{sID}])$.
- For $\text{DER}_{\text{resp}}(\text{sID}, M)$, if $j = i^*$, and $i = i^*$ the key K_r is directly set to $H_3(c_i, c_j, K^*, i, j, M, M')$. Here $(j, i) = (\text{holder}[\text{sID}], \text{peer}[\text{sID}])$.
- For $\text{DER}_{\text{init}}(\text{sID}, M')$, if $i = i^*$, and $j \neq i^*$, the key K_i is directly set to $H_1(c_i, m_j, K', i, j, M, M')$. Here $(i, j) = (\text{holder}[\text{sID}], \text{peer}[\text{sID}])$.
- For $\text{DER}_{\text{init}}(\text{sID}, M')$, if $i = i^*$, and $j = i^*$, the key K_i is directly set to $H_3(c_i, c_j, K', i, j, M, M')$. Here $(i, j) = (\text{holder}[\text{sID}], \text{peer}[\text{sID}])$.

In this game, the secret key sk_{i^*} is no longer used to generated the final key K_r/K_i .

In game G_6^b , the final key produced by DER_{resp} for the input (sID, M) where $M = (\widetilde{pk}, c_j)$, $(j, i) = (\text{holder}[\text{sID}], \text{peer}[\text{sID}])$ and $j = i^*$ is

$$K_r = \begin{cases} F_R^2(m_i, c_j, K^*, i, j, M, M') & j = i^*, i \neq i^*, c_j \text{ decryption fails} & \textbf{(Branch 1)}, \\ H_2(m_i, c_j, K^*, i, j, M, M') & j = i^*, i \neq i^*, c_j \text{ decryption succeeds} & \textbf{(Branch 2)}, \\ F_R^1(c_i, c_j, K^*, i, j, M, M') & j = i^*, i = i^*, c_j \text{ decryption fails} & \textbf{(Branch 3)}, \\ H_3(c_i, c_j, K^*, i, j, M, M') & j = i^*, i = i^*, c_j \text{ decryption succeeds} & \textbf{(Branch 4)}. \end{cases}$$

Similarly, the final key produced by DER_{init} in game G_6^b for the input (sID, M') where $M' = (c_i, c^*)$, $(i, j) = (\text{holder}[\text{sID}], \text{peer}[\text{sID}])$ and $i = j^*$ is

$$K_i = \begin{cases} F_1^2(c_i, m_j, K', i, j, M, M') & i = i^*, j \neq i^*, c_i \text{ decryption fails} & \textbf{(Branch 5)}, \\ H_1(c_i, m_j, K', i, j, M, M') & i = i^*, j \neq i^*, c_i \text{ decryption succeeds} & \textbf{(Branch 6)}, \\ F_1^1(c_i, c_j, K', i, j, M, M') & i = i^*, j = i^*, c_i \text{ decryption fails} & \textbf{(Branch 7)}, \\ H_3(c_i, c_j, K', i, j, M, M') & i = i^*, j = i^*, c_i \text{ decryption succeeds} & \textbf{(Branch 8)}. \end{cases}$$

In the above equations, " c decryption fails" means $\Pi_{\text{PKE}}.\text{Dec}(sk_{i^*}, c) = \perp$, or $m := \Pi_{\text{PKE}}.\text{Dec}(sk_{i^*}, c) \neq \perp$ together with $\Pi_{\text{PKE}}.\text{Enc}(pk_{i^*}, m; G(m)) \neq c$. In contrast, " c decryption succeeds" means $m := \Pi_{\text{PKE}}.\text{Dec}(sk_{i^*}, c) \neq \perp$ together with $\Pi_{\text{PKE}}.\text{Enc}(pk_{i^*}, m; G(m)) = c$.

We first focus on **Branch 1** and **2**, for which the inputs to F_R^2 and H_2 are disjoint. Concretely, the input c_j to F_R^2 always incurs decryption failure, while the input c_j to H_2 always decrypts correctly. Note that the adversary $\mathcal{A}$ can only obtain

outputs of H_2 by querying H or CORRUPT , but by the definition of H given in Eq. (89), and the fact that G is replaced by G_{good} , $\mathcal{A}$ can never obtain $H_2(m, c, \dots)$ where c incurs decryption failure. Furthermore, by the definition of DER_{resp} , F_R^2 is only used to generate a key when c_j fails decryption. Therefore, even if $\mathcal{A}$ is allowed to query CORRUPT , it can never obtain $F_R^2(m, c, \dots)$ where c decrypts correctly. It follows that the input domain of F_R^2 and H_2 are disjoint in game $\mathbf{G}_6^b$. We can hence equivalently merge **Branch 1** and **2**: whenever $j = i^*$ and $i \neq i^*$, we directly set $K_r := H_2(m_i, c_j, K^*, i, j, M, M')$, removing the step that uses the secret key sk_{i^*} to verify whether c_j decrypts correctly. The analysis for **Branch 5** and **6** is analogous to that for **Branch 1** and **2**, and these two branches can also be merged with responses returned from H_1 , again without relying on sk_{i^*} .

We now turn to **Branch 3, 4, 7, and 8**. First, the ciphertext c_i in **Branch 4** always decrypts correctly, since inside DER_{resp} , $c_i = \Pi_{\text{PKE}}.\text{Enc}(pk_{i^*}, m_i; G(m_i))$ and the G is replaced by G_{good} . Similarly, c_j in **Branch 8** is guaranteed to decrypt correctly, as the c_j used by DER_{init} is generated by INIT , where $c_j = \Pi_{\text{PKE}}.\text{Enc}(pk_{i^*}, m_j; G(m_j))$ and the G is replaced by G_{good} . Observe that for the two ciphertexts c_i and c_j involved in **Branch 3** and **7**, at least one of them results in decryption failure. Consequently, the random oracle inputs for these two branches are disjoint from those of **Branch 4** and **8**. As for H_3 , although $\mathcal{A}$ can obtain outputs of H_3 by querying H or CORRUPT , by the definition of H given in Eq. (89), and the fact that G is replaced by G_{good} , $\mathcal{A}$ can never obtain $H_3(c, c', \dots)$ where either c or c' fails decryption. Regarding F_R^1 and F_I^1 , $\mathcal{A}$ can only learn their outputs indirectly by querying CORRUPT . From Eq. (90) and the definitions of DER_{resp} and DER_{init} , any $F_R^1(c, c', \dots)$ obtained by $\mathcal{A}$ must satisfy that c decrypts correctly while c' incurs decryption failure; in contrast, any $F_I^1(c, c', \dots)$ obtained by $\mathcal{A}$ must satisfy that c incurs decryption failure while c' decrypts correctly. This implies that the input domains of H_3 , F_R^1 and F_I^1 are pairwise disjoint in game $\mathbf{G}_6^b$. We can hence equivalently merge **Branch 3, 4, 7, and 8**: we directly use H_3 to generate K_r or K_i when $i = j = i^*$, removing the step that uses sk_{i^*} to verify whether c_i and c_j decrypts correctly. Combining the above analysis for **Branch 1** to **8**, K_r and K_i can be equivalently generated as follows:

$$\begin{aligned} K_r &= \begin{cases} H_2(m_i, c_j, K^*, i, j, M, M') & j = i^*, i \neq i^*, \\ H_3(c_i, c_j, K^*, i, j, M, M') & j = i^*, i = i^*, \end{cases} \\ K_i &= \begin{cases} H_1(c_i, m_j, K', i, j, M, M') & i = i^*, j \neq i^*, \\ H_3(c_i, c_j, K', i, j, M, M') & i = i^*, j = i^*. \end{cases} \end{aligned}$$

This exactly matches the construction of game $\mathbf{G}_7^b$, leading to

$$\Pr[1 \leftarrow \mathbf{G}_6^b] = \Pr[1 \leftarrow \mathbf{G}_7^b]. \quad (92)$$

Game $\mathbf{G}_8^b$: In this game, we switch back to using $G \stackrel{\$}{\leftarrow} \text{Fcs}[\mathcal{M}, \mathcal{R}]$ instead of G_{good} . With the same analysis of the game hop $\mathbf{G}_7^b$ to $\mathbf{G}_8^b$ in the proof of Lemma 5,

$$|\Pr[1 \leftarrow \mathbf{G}_7^b] - \Pr[1 \leftarrow \mathbf{G}_8^b]| \leq q_G \cdot \delta_P. \quad (93)$$

Game $\mathbf{G}_9^b$: This game is the same as $\mathbf{G}_8^b$, except that it modifies the keys produced by DER_{init} and DER_{resp} :

- For DER_{resp} with input $(\text{sID} \neq s_{\text{resp}}, M')$, if the ciphertext c_i equals c_{i^*} , it directly sets K_r as a uniformly random value of $\mathcal{K}$.
- For DER_{resp} with input (s_{resp}, M) , if there exists a $\text{sID} \neq s_{\text{resp}}$ such that sID and s_{resp} become matching after performing the assignment $(\text{sent}(s_{\text{resp}}), \text{rece}(s_{\text{resp}})) := (M', M)$, it directly sets K_i as a uniformly random value of $\mathcal{K}$.
- For DER_{init} with input (sID, M') , if sID and s_{resp} become matching after performing the assignment $\text{rece}(\text{sID}) := M'$, it sets K_i as a uniformly random value of $\mathcal{K}$.

These modifications are introduced to guarantee that sessions distinct from s_{resp} never query the random oracle with the same (M, M') used by s_{resp} for generating the final key; this property will be used in our subsequent game transitions.

Notice that outputting 1 in $\mathbf{G}_8^b$ ensures $\mathfrak{M}(\text{sID}^*) = \emptyset$, $\text{role}[\text{sID}^*] = \text{responder}$, and $\text{sID}^* = s_{\text{resp}}$. Hence, on the event that the game outputs 1, the adversary $\mathcal{A}$ cannot detect the latter two modifications made in game $\mathbf{G}_9^b$. Regarding the first modification, since c_i is honestly generated, the probability that it also produce c_{i^*} is at most $\gamma(\Pi_{\text{PKE}}.\text{Enc})$, and hence we have

$$|\Pr[1 \leftarrow \mathbf{G}_8^b] - \Pr[1 \leftarrow \mathbf{G}_9^b]| \leq S \cdot \gamma(\Pi_{\text{PKE}}.\text{Enc}). \quad (94)$$

Game $\mathbf{G}_{10}^b$: This game is the same as $\mathbf{G}_9^b$, except that in $\text{DER}_{\text{resp}}(s_{\text{resp}}, M')$ it no longer performs $m_i := m_{i^*}$ and directly sets $K_r := k^*$. Here k^* is a fresh uniformly random value in $\mathcal{K}$ sampled at the game beginning. Note that in this game, the plaintext m_{i^*} is no longer used by $\text{DER}_{\text{resp}}(s_{\text{resp}}, M)$ to generated the final key K_r .

Let $(j, i) := (\text{holder}[\text{sID}], \text{peer}[\text{sID}])$. Define **Error** as the event such that, just after the query $\text{DER}_{\text{resp}}(s_{\text{resp}}, M)$ finishes the assignment $(\text{sent}(s_{\text{resp}}), \text{rece}(s_{\text{resp}})) := (M', M)$, there exists a session $\text{sID} \neq s_{\text{resp}}$ for which sID and s_{resp} constitute a matching session pair. Now, according to the definition of game $\mathbf{G}_9^b$, the final key K_r generated by $\text{DER}_{\text{resp}}(s_{\text{resp}}, M = (pk, c_j))$ in this game is:

$$K_r = \begin{cases} k \xrightarrow{\$} \mathcal{K} & \text{Error occurs,} \\ k' & \text{Error does not occur,} \end{cases}$$

where

$$k' := \begin{cases} H_{\text{resp}}(s_j, m_{i^*}, c_j, K^*, i, j, M, M') & j \neq i^*, i = i^*, c_j \text{ decryption fails,} \\ H_1(c_{i^*}, m'_j, K^*, i, j, M, M') & j \neq i^*, i = i^*, c_j \text{ decryption succeeds,} \\ H_3(c_{i^*}, c_j, K^*, i, j, M, M') & j = i^*, i = i^*, \\ H_{\text{resp}}(s_j, m_{i^*}, c_j, K^*, i, j, M, M') & j \neq i^*, i \neq i^*, c_j \text{ decryption fails,} \\ H_4(m_{i^*}, m'_j, K^*, i, j, M, M') & j \neq i^*, i \neq i^*, c_j \text{ decryption succeeds,} \\ H_2(m_{i^*}, c_j, K^*, i, j, M, M') & j = i^*, i \neq i^*. \end{cases} \quad (95)$$

In game $\mathbf{G}_9^b$, $\neg \text{Error}$ implies that just after executing $\text{DER}_{\text{resp}}(s_{\text{resp}}, M)$, s_{resp} does not form a matching session with any session established prior to this execution. Consequently, for all random oracle queries submitted by DER_{init} to derivate key

before this execution, the value (M, M') contained in these queries must differ from the (M, M') used by s_{resp} . We recall that the first and third modifications introduced in $\mathbf{G}_9^b$ further guarantee that the (M, M') appearing in key-derivation random oracle queries issued by DER_{resp} and subsequent DER_{init} is distinct from the (M, M') used by s_{resp} . Therefore, conditioned on $\neg \text{Error}$, the adversary $\mathcal{A}$ cannot obtain k' via the strategy of first querying DER_{init} or DER_{resp} and then querying REVEAL . Instead, $\mathcal{A}$ can only obtain k' either by querying H_{resp} with input $(*, m_{i^*}, \dots)$, or by querying H with input $(m, \dots)$ where $m = m_{i^*}$ or $m \neq m_{i^*} \wedge \Pi_{\text{PKE}}.\text{Enc}(pk_{i^*}, m; \mathbf{G}(m)) = c_{i^*}$. Define events

$\text{Bad}_{\mathbf{G}, m_{i^*}}^b$ ($b \in \{0, 1\}$): During the running of $\mathcal{A}$, either H_{resp} is ever queried with input $(*, m_{i^*}, \dots)$ or H is ever queried with input $(m, \dots)$ where $m \in S_{\mathbf{G}, m_{i^*}}^*$ and $S_{\mathbf{G}, m_{i^*}}^* := \{m_{i^*}^*\} \cup \{m : m \neq m_{i^*} \wedge \Pi_{\text{PKE}}.\text{Enc}(pk_{i^*}, m; \mathbf{G}(m)) = c_{i^*}\}$.

Here b denotes whether the oracle TEST returns K_0^* or K_1^* . In fact, the above analysis implies that, conditioned on $\neg \text{Error} \wedge \neg \text{Bad}_{\mathbf{G}, m_{i^*}}^b$, $\mathcal{A}$'s view in game $\mathbf{G}_9^b$ is independent of the random oracle values in all branches corresponding to k' (Eq. (95)). Consequently, these values are uniformly random in $\mathcal{A}$'s view, and thus k' can be equivalently replaced by

$$k' = \begin{cases} k_1 \xleftarrow{\$} \mathcal{K} & j \neq i^*, i = i^*, c_j \text{ decryption fails,} \\ k_2 \xleftarrow{\$} \mathcal{K} & j \neq i^*, i = i^*, c_j \text{ decryption succeeds,} \\ k_3 \xleftarrow{\$} \mathcal{K} & j = i^*, i = i^*, \\ k_4 \xleftarrow{\$} \mathcal{K} & j \neq i^*, i \neq i^*, c_j \text{ decryption fails,} \\ k_5 \xleftarrow{\$} \mathcal{K} & j \neq i^*, i \neq i^*, c_j \text{ decryption succeeds,} \\ k_6 \xleftarrow{\$} \mathcal{K} & j = i^*, i \neq i^*. \end{cases}$$

At this point, the operation " $m_i := m_{i^*}$ " can be directly omitted since the generation of k' no longer uses it. Note that k' given in the above equation is obviously still a uniformly random value over $\mathcal{K}$. Thus, we have equivalently transitioned to the setting of game $\mathbf{G}_{10}^b$, which indicates that

$$\Pr \left[1 \leftarrow \mathbf{G}_9^b \wedge \neg \text{Error} \wedge \neg \text{Bad}_{\mathbf{G}, m_{i^*}}^b \right] = \Pr \left[1 \leftarrow \mathbf{G}_{10}^b \wedge \neg \text{Error} \wedge \neg \text{Bad}_{\mathbf{G}, m_{i^*}}^b \right], \quad (96)$$

$$\Pr \left[\neg \text{Error} \wedge \neg \text{Bad}_{\mathbf{G}, m_{i^*}}^b : \mathbf{G}_9^b \right] = \Pr \left[\neg \text{Error} \wedge \neg \text{Bad}_{\mathbf{G}, m_{i^*}}^b : \mathbf{G}_{10}^b \right]. \quad (97)$$

Since the event Error depends only on the game execution prior to the query $\text{DER}_{\text{resp}}(s_{\text{resp}}, M)$, and games $\mathbf{G}_9^b$ and $\mathbf{G}_{10}^b$ are identical up to that point, we have $\Pr[\neg \text{Error} : \mathbf{G}_9^b] = \Pr[\neg \text{Error} : \mathbf{G}_{10}^b]$. Combining this equation with Eq. (73),

$$\Pr \left[\neg \text{Error} \wedge \text{Bad}_{\mathbf{G}, m_{i^*}}^b : \mathbf{G}_9^b \right] = \Pr \left[\neg \text{Error} \wedge \text{Bad}_{\mathbf{G}, m_{i^*}}^b : \mathbf{G}_{10}^b \right]. \quad (98)$$

Now we can compute

$$\begin{aligned}
& |\Pr[1 \leftarrow \mathbf{G}_9^b] - \Pr[1 \leftarrow \mathbf{G}_{10}^b]| \\
& \stackrel{(a)}{=} |\Pr[1 \leftarrow \mathbf{G}_9^b \wedge \neg \text{Error}] - \Pr[1 \leftarrow \mathbf{G}_{10}^b \wedge \neg \text{Error}]| \\
& \stackrel{(b)}{=} \left| \Pr[1 \leftarrow \mathbf{G}_9^b \wedge \neg \text{Error} \wedge \text{Bad}_{\mathbf{G}, m_i^*}^b] - \Pr[1 \leftarrow \mathbf{G}_{10}^b \wedge \neg \text{Error} \wedge \text{Bad}_{\mathbf{G}, m_i^*}^b] \right| \quad (99) \\
& \stackrel{(c)}{\leq} \Pr[\neg \text{Error} \wedge \text{Bad}_{\mathbf{G}, m_i^*}^b : \mathbf{G}_{10}^b] \\
& \leq \Pr[\text{Bad}_{\mathbf{G}, m_i^*}^b : \mathbf{G}_{10}^b].
\end{aligned}$$

Here, (a) holds because the event that the game outputs 1 guarantees $\mathfrak{M}(\text{sID}^*) = \emptyset$ and $\text{sID}^* = s_{\text{resp}}$, which implies that the event **Error** does not occur. (b) follows from Eq. (96), and (c) follows from Eq. (98). Next, to compute $\Pr[\text{Bad}_{\mathbf{G}, m_i^*}^b : \mathbf{G}_{10}^b]$, we first define two new events as:

$\text{Bad}_{m_i^*}^b$ ($b \in \{0, 1\}$): This event is the same as $\text{Bad}_{\mathbf{G}, m_i^*}^b$, except that the set $S_{\mathbf{G}, m_i^*} := \{m_i^*\} \cup \{m : m \neq m_i^* \wedge \Pi_{\text{PKE}}.\text{Enc}(pk_{i^*}, m; \mathbf{G}(m)) = c_{i^*}\}$ is replaced by $S_{m_i^*} := \{m_i^*\}$.

$\text{Coll}_{m_i^*}$: There exists a $m \neq m_i^*$ such that $\Pi_{\text{PKE}}.\text{Enc}(pk_{i^*}, m; \mathbf{G}(m)) = c_{i^*}$.

Obviously, if $\text{Coll}_{m_i^*}$ does not occur, $\{m : m \neq m_i^* \wedge \Pi_{\text{PKE}}.\text{Enc}(pk_{i^*}, m; \mathbf{G}(m)) = c_{i^*}\}$ is an empty set and thus $S_{\mathbf{G}, m_i^*}$ and $S_{m_i^*}$ are identical. Therefore, we have

$$|\Pr[\text{Bad}_{\mathbf{G}, m_i^*}^b : \mathbf{G}_{10}^b] - \Pr[\text{Bad}_{m_i^*}^b : \mathbf{G}_{10}^b]| \leq \Pr[\text{Coll}_{m_i^*} : \mathbf{G}_{10}^b] \stackrel{(a)}{\leq} 2 \cdot \delta_P. \quad (100)$$

Here (a) uses Corollary 1.

Game $\mathbf{G}_{11}^b$: This game is the same as $\mathbf{G}_{10}^b$, except that in $\text{Challenge}(s_{\text{resp}}, i^*)$ it replaces $\mathbf{G}(m_{i^*})$ by r^* , a fresh uniformly random value in $\mathcal{R}$.

Denote $\text{BadG}_{m_i^*}^b$ as the event that during the procedure in which $\mathcal{A}$ produces b' , $\mathbf{G}$ is ever queried with input m_{i^*} . Then, by the basic rules of random oracles, we have

$$\begin{aligned}
& |\Pr[1 \leftarrow \mathbf{G}_{10}^b] - \Pr[1 \leftarrow \mathbf{G}_{11}^b]| \leq \Pr[\text{BadG}_{m_i^*}^b : \mathbf{G}_{11}^b], \\
& \left| \Pr[\text{Bad}_{m_i^*}^b : \mathbf{G}_{10}^b] - \Pr[\text{Bad}_{m_i^*}^b : \mathbf{G}_{11}^b] \right| \leq \Pr[\text{BadG}_{m_i^*}^b : \mathbf{G}_{11}^b]. \quad (101)
\end{aligned}$$

As for the probability $\Pr[\text{BadG}_{m_i^*}^b : \mathbf{G}_{11}^b]$, we first construt a OW-CPA adversary $\mathcal{B}_b$ ($b \in \{0, 1\}$) against Π_{PKE} as follows, and then bound this probability by using $\mathcal{B}_b$'s advantage.

- The input of $\mathcal{B}_b$ is (pk_{i^*}, c_{i^*}) , where pk_{i^*} is the public key and c_{i^*} is the challenge ciphertext.
- $\mathcal{B}_b$ runs game $\mathbf{G}_{11}^b$, simulates the random oracles on-th-fly.
- When running $\text{Challenge}(s_{\text{resp}}, i^*)$, $\mathcal{B}_b$ replaces the (pk_{i^*}, c_{i^*}) generated there by its own input.

- After $\mathcal{A}$ returns its output b' , $\mathcal{B}_b$ uniformly samples an element from the record lists of the random oracle $\mathbf{G}$, which we denote as m , and then returns m as its final output.

We stress that although $\mathcal{B}_b$ does not have the secret key sk_{i^*} corresponding to pk_{i^*} and the plaintext m_{i^*} , it can still run game $\mathbf{G}_{11}^b$ normally, as the computations of this game no longer require sk_{i^*} and m_{i^*} . Obviously, for any $b \in \{0, 1\}$, the running time of $\mathcal{B}_b$ is roughly the same as those of $\mathcal{A}$, and $\text{BadG}_{m_{i^*}}^b$ occurring implies that $\mathcal{B}_b$ has an advantage at least $1/(q_H + q_G)$ to successfully guess the plaintext m_{i^*} of c_{i^*} . Thus, we have

$$\Pr [\text{BadG}_{m_{i^*}}^b : \mathbf{G}_{11}^b] \leq (q_H + q_G) \cdot \text{Adv}_{\Pi_{\text{PKE}}}^{\text{OW-CPA}}(\mathcal{B}_b). \quad (102)$$

The probability $\Pr[\text{Bad}_{m_{i^*}}^b : \mathbf{G}_{11}^b]$ can be similarly upper bounded, that is, there exists a OW-CPA adversary $\mathcal{B}'_b$ ($b \in \{0, 1\}$) (with roughly the same running time of $\mathcal{A}$) against Π_{PKE} such that

$$\Pr [\text{Bad}_{m_{i^*}}^b : \mathbf{G}_{11}^b] \leq (q_R + q_H) \cdot \text{Adv}_{\Pi_{\text{PKE}}}^{\text{OW-CPA}}(\mathcal{B}'_b). \quad (103)$$

Game $\mathbf{G}_{12}^b$: This game is the same as $\mathbf{G}_{11}^b$, except that the oracle TEST directly sets K_0^* as k^* , rather than $\text{sKey}[\text{sID}^*]$. Since outputting 1 in game $\mathbf{G}_{11}^b$ guarantees $\mathfrak{M}(\text{sID}^*) = \emptyset$, $\text{role}[\text{sID}^*] = \text{responder}$, $\text{sID}^* = s_{\text{resp}}$, $\text{peer}(\text{sID}^*) = i^*$, and sID^* is fresh, the K_0^* in game $\mathbf{G}_{11}^b$ is also k^* if it finally outputs 1. Thus, game $\mathbf{G}_{12}^b$ is only a conceptual change and

$$\Pr [1 \leftarrow \mathbf{G}_{11}^b] = \Pr [1 \leftarrow \mathbf{G}_{12}^b]. \quad (104)$$

Game $\mathbf{G}_{13}^b$: This game is the same as $\mathbf{G}_{12}^b$, except that in $\text{DER}_{\text{resp}}(s_{\text{resp}}, M)$ it directly sets $K_r := 0$. Similar to the above game hop, since outputting 1 in game $\mathbf{G}_{12}^b$ guarantees $\mathfrak{M}(\text{sID}^*) = \emptyset$, $\text{role}[\text{sID}^*] = \text{responder}$, $\text{sID}^* = s_{\text{resp}}$, $\text{peer}(\text{sID}^*) = i^*$, and sID^* is fresh, the adversary $\mathcal{A}$ never queries REVEAL with s_{resp} to get the K_r if the game finally outputs 1, and thus $\mathcal{A}$ cannot detect our change in game $\mathbf{G}_{13}^b$. Therefore, we have

$$\Pr [1 \leftarrow \mathbf{G}_{12}^b] = \Pr [1 \leftarrow \mathbf{G}_{13}^b]. \quad (105)$$

In game $\mathbf{G}_{13}^b$, K_0^* and K_1^* are both uniformly random values in $\mathcal{K}$, and thus

$$\Pr [1 \leftarrow \mathbf{G}_{13}^0] = \Pr [1 \leftarrow \mathbf{G}_{13}^1]. \quad (106)$$

At this point, by combining Eq. (83) to Eq. (88), Eq. (91) to Eq. (94), Eq. (99) to Eq. (106), and folding adversaries $\mathcal{B}_0, \mathcal{B}_1, \mathcal{B}'_0, \mathcal{B}'_1$ into one adversary $\mathcal{B}$, we obtain

$$\begin{aligned} \text{Case 4} &\leq 2S^2N \cdot \gamma(\Pi_{\text{PKE}}.\text{Enc}) + \frac{2SN(q_R + q_I)}{|\mathcal{M}|} \\ &\quad + 4SN(q_G + 1) \cdot \delta_P + 8SN(q_G + q_H + q_R) \cdot \text{Adv}_{\Pi_{\text{PKE}}}^{\text{OW-CPA}}(\mathcal{B}). \end{aligned}$$

This completes the proof of Lemma 7. $\square$

A.5 Proof of Theorem 3

As mentioned before, the key point of this proof is to respectively establish the QROM versions of Lemma 4, Lemma 5, Lemma 6, and Lemma 7. Here, for brevity, we do not reproduce full proofs of these QROM versions. Instead, we first point out which game transitions used in the previous ROM proofs cannot directly lift to the QROM, and then give new analyses valid in the QROM. In the following, we first introduce necessary QROM tools, which will be used later.

Definition 13 (Semi-Classical Oracle [6]). For a subset $S \subseteq \mathcal{X}$, the indicator function f_S from $\mathcal{X}$ to $\{0, 1\}$ equals 1 iff its input $x \in S$. The semi-classical oracle, $\mathcal{O}_S^{\text{SC}}$, is a quantum accessible oracle that performs the following two operations on its query state $\sum_{x \in \mathcal{X}} \alpha_x |x\rangle$:

1. Initialize a single qubit register L with the state $|0\rangle_{\mathsf{L}}$.
2. Transform $\sum_{x \in \mathcal{X}} \alpha_x |x\rangle |0\rangle_{\mathsf{L}}$ into $\sum_{x \in \mathcal{X}} \alpha_x |x\rangle |f_S(x)\rangle_{\mathsf{L}}$ and then measure L .

During the execution of a quantum oracle algorithm $\mathcal{A}^{\mathcal{O}_S^{\text{SC}}}$, let **Find** be the classical event that the measurement of $\mathcal{O}_S^{\text{SC}}$ (on L) ever has outcome 1.

Lemma 8. Let $S \subseteq \mathcal{X}$ be random. Let z be a random bitstring. (S, z may have arbitrary joint distribution $\mathfrak{D}$.) Let $\mathcal{A}$ be a quantum oracle algorithm with query times q (not necessarily unitary). Let

$$P_{\text{left}} := \Pr[b = 1 : b \leftarrow \mathcal{A}^{\emptyset}(z), (S, z) \leftarrow \mathfrak{D}],$$

$$P_{\text{right}} := \Pr[b = 1 \wedge \neg \text{Find} : b \leftarrow \mathcal{A}^{\mathcal{O}_S^{\text{SC}}}(z), (S, z) \leftarrow \mathfrak{D}].$$

Here $\emptyset$ denotes an oracle that does nothing on the query register. Let $P_{\text{find}} := \Pr[\text{Find} : \mathcal{A}^{\mathcal{O}_S^{\text{SC}}}(z), (S, z) \leftarrow \mathfrak{D}]$. Then

$$|P_{\text{left}} - P_{\text{right}}| \leq \sqrt{(q+1) \cdot P_{\text{find}}} \text{ and } \left| \sqrt{P_{\text{left}}} - \sqrt{P_{\text{right}}} \right| \leq \sqrt{(q+1) \cdot P_{\text{find}}}.$$

Proof. This is a direct corollary of [6, Theorem 1] by setting the function H used by it as a constant zero function. $\square$

Lemma 9 (Search in Semi-Classical Oracle [6, Theorem 2]). Let $S \subseteq \mathcal{X}$ be random and z be a random bitstring. (S, z) may have an arbitrary joint distribution $\mathfrak{D}$. Let $\mathcal{A}$ be any quantum oracle algorithm making at most q queries to a semi-classical oracle with domain $\mathcal{X}$. Let $\mathcal{B}$ be an algorithm that on input z , chooses $i \leftarrow_{\$} [q]$; runs $\mathcal{A}^{\mathcal{O}_S^{\text{SC}}}(z)$ until (just before) the i -th query, measures the input register and outputs the measurement outcome x . Then

$$\Pr[\text{Find} : \mathcal{A}^{\mathcal{O}_S^{\text{SC}}}(z), (S, z) \leftarrow \mathfrak{D}] \leq 4q \cdot \Pr[x \in S : x \leftarrow \mathcal{B}(z), (S, z) \leftarrow \mathfrak{D}].$$

Lemma 10 (Generic quantum distinguishing with bounded probabilities [38, Lemma 2.9]). Let $\delta \in [0, 1]$ and $\mathcal{M}$ be a finite set. Let $N_1 : \mathcal{M} \rightarrow \{0, 1\}$ be a random function such that, for each $m \in \mathcal{M}$, $N_1(m) = 1$ with probability δ_m ($\delta_m \leq \delta$), and $N_1(m) = 0$ with probability $1 - \delta_m$. Let $N_2 : \mathcal{M} \rightarrow \{0, 1\}$ be a constant function such that $N_2(m) = 0$ for all $m \in \mathcal{M}$. For any algorithm $\mathcal{A}$ that makes at most q quantum queries, we have

$$\left| \Pr[b = 1 : b \leftarrow \mathcal{A}^{N_1}] - \Pr[b = 1 : b \leftarrow \mathcal{A}^{N_2}] \right| \leq 8(q+1)^2 \cdot \delta.$$

Lemma 11. Let $H \xleftarrow{\$} \text{Fcs}[\{0, 1\}^{m_1+m_2}, \{0, 1\}^n]$, $G \xleftarrow{\$} \text{Fcs}[\{0, 1\}^{m_2}, \{0, 1\}^n]$, and $x \xleftarrow{\$} \{0, 1\}^{m_1}$. Consider a quantum oracle algorithm $\mathcal{A}^{H, H(x, \cdot)}$ that makes at most q queries to its oracles. If x is independent from the $\mathcal{A}$'s view, we have

$$|\Pr[1 \leftarrow \mathcal{A}^{H, H(x, \cdot)}] - \Pr[1 \leftarrow \mathcal{A}^{H, G}]| \leq \frac{2q}{\sqrt{2^{m_1}}}.$$

Now, we respectively establish the QROM versions of Lemma 4, Lemma 5, Lemma 6, and Lemma 7.

Lemma 12 (QROM version of Lemma 4). For Case 1, where the QROM adversary $\mathcal{A}$ cannot obtain the internal state $\text{st}[\text{slD}^*]$, we have $\text{Case 1} \leq 2S \cdot \gamma(\Pi_{\text{PKE}}.\text{Enc}) + 4S^2N^2 \cdot \delta_P + 2S^2N^2 \cdot \delta_K + 8S^2N^2(q_H + 1) \cdot \sqrt{\text{Adv}_{\Pi_{\text{KEM}}}^{\text{OW-CPA}}(\mathcal{B}_K)}$. Here $\mathcal{B}_K$ has roughly the same running time as $\mathcal{A}$.

Proof. The only game transition in the proof of Lemma 4 that cannot be directly lifted to the QROM is game $\mathbf{G}_5^b$ to game $\mathbf{G}_6^b$ (Eq. (29)). The reason is that event $\text{Bad}_{K^*}^b$ is not well-defined in the QROM. Here, we use the semi-classical oracle technique to re-prove the upper bound of $|\Pr[1 \leftarrow \mathbf{G}_5^b] - \Pr[1 \leftarrow \mathbf{G}_6^b]|$. Denote $S := \{(*, *, K^*, \dots)\}$, and let $\mathcal{O}_S^{SC}$ be a semi-classical oracle with respect to the set S . Then, we introduce two new games:

Game $\mathbf{G}_{5a}^b$: This game is the same as $\mathbf{G}_5^b$, except that just before each query to H during the running of $\mathcal{A}$, the $\mathcal{O}_S^{SC}$ is queried on their input registers.

Game $\mathbf{G}_{6a}^b$: This game is the same as $\mathbf{G}_6^b$, except that just before each query to H during the running of $\mathcal{A}$, the $\mathcal{O}_S^{SC}$ is queried on their input registers.

Next, we design a new quantum oracle algorithm $\mathcal{C}_b$ ($b \in \{0, 1\}$) with oracle access to $O \in \{\emptyset, \mathcal{O}_S^{SC}\}$ and input

$$\text{input} := (i^*, j^*, s_{\text{init}}, s_{\text{resp}}, H, m_{i^*}, m_{j^*}, \widetilde{M}, \widetilde{M}', K_{\text{init}}, K_{\text{resp}}).$$

Here $(i^*, j^*, s_{\text{init}}, s_{\text{resp}}, H)$ is generated in the same way as in game $\mathbf{G}_5^b$, the remaining $(m_{i^*}, m_{j^*}, \widetilde{M}, \widetilde{M}', K_{\text{init}}, K_{\text{resp}})$ is generated by $\text{Session}(i^*, j^*)$, where $K_{\text{init}} = K_{\text{resp}} = H(m_{i^*}, m_{j^*}, K^*, i^*, j^*, \widetilde{M}, \widetilde{M}')$. The definition of $\mathcal{C}_b^O(\text{input})$ is given below.

- $\mathcal{C}_b^O(\text{input})$ runs game $\mathbf{G}_5^b$ by using its input and finally returns $\mathbf{G}_5^b$'s output b' . During the running, $\mathcal{C}$ simulates the random oracle H by using the function H of its input, simulates the other (quantum-accessible) random oracles using $2q$ -wise independent hash functions, and queries its oracle O on the input registers of H just before processing each query to H .

Note that $\mathcal{C}_b$ queries O at most q_H times. Thus, for games $\mathbf{G}_5^b$ and $\mathbf{G}_{5a}^b$, we have

$$\begin{aligned} \Pr[1 \leftarrow \mathbf{G}_5^b] &= \Pr[1 \leftarrow \mathcal{C}_b^\emptyset(\text{input})], \\ \Pr[\neg \text{Find} \wedge 1 \leftarrow \mathbf{G}_{5a}^b] &= \Pr[\neg \text{Find} \wedge 1 \leftarrow \mathcal{C}_b^{\mathcal{O}_S^{SC}}(\text{input})]. \end{aligned}$$

Let input' be a new input the same as input , except that the tuple $(K_{\text{init}}, K_{\text{resp}})$ is replaced by $(K_{\text{init}} := r^*, K_{\text{resp}} := r^*)$, where r^* is a fresh uniformly random value in $\mathcal{K}$. Since game $\mathbf{G}_6^b$ is the same as $\mathbf{G}_5^b$ except it directly sets $K_{\text{resp}} = K_{\text{init}} := r^*$, we have

$$\begin{aligned}\Pr[1 \leftarrow \mathbf{G}_6^b] &= \Pr[1 \leftarrow \mathcal{C}_b^\varnothing(\text{input}')], \\ \Pr[\neg\text{Find} \wedge 1 \leftarrow \mathbf{G}_{6a}^b] &= \Pr[\neg\text{Find} \wedge 1 \leftarrow \mathcal{C}_b^{\mathcal{O}_S^{SC}}(\text{input}')].\end{aligned}$$

By applying Lemma 8, we obtain

$$\begin{aligned}|\Pr[1 \leftarrow \mathbf{G}_5^b] - \Pr[\neg\text{Find} \wedge 1 \leftarrow \mathbf{G}_{5a}^b]| &\leq \sqrt{(q_H + 1) \cdot \Pr[\text{Find} : \mathcal{C}_b^{\mathcal{O}_S^{SC}}(\text{input})]}, \\ |\Pr[1 \leftarrow \mathbf{G}_6^b] - \Pr[\neg\text{Find} \wedge 1 \leftarrow \mathbf{G}_{6a}^b]| &\leq \sqrt{(q_H + 1) \cdot \Pr[\text{Find} : \mathcal{C}_b^{\mathcal{O}_S^{SC}}(\text{input}')]}.\end{aligned}$$

On the condition that Find does not occur, i.e., the projective measurement performed by $\mathcal{O}_S^{SC}$ always yields the result 0, $\mathbf{H}(x)$ for any x of the form $(*, *, K^*, \dots)$ is uniformly random in the adversary $\mathcal{A}$'s view. This means that

$$\begin{aligned}\Pr[\neg\text{Find} \wedge 1 \leftarrow \mathbf{G}_{5a}^b] &= \Pr[\neg\text{Find} \wedge 1 \leftarrow \mathbf{G}_{6a}^b], \\ \Pr[\text{Find} : \mathcal{C}_b^{\mathcal{O}_S^{SC}}(\text{input})] &= \Pr[\text{Find} : \mathcal{C}_b^{\mathcal{O}_S^{SC}}(\text{input}')].\end{aligned}$$

By using Lemma 9, we can upper bound the probability of Find during the running of $\mathcal{C}_b^{\mathcal{O}_S^{SC}}(\text{input}')$ using the success probability of the following quantum algorithm $\mathcal{B}_b$ ($b \in \{0, 1\}$).

- The input of $\mathcal{B}_b$ is $(\widetilde{pk}^*, \widetilde{c}^*)$, where $\widetilde{pk}^*$ is the public key and $\widetilde{c}^*$ is the challenge ciphertext.
- $\mathcal{B}_b$ runs $\mathcal{C}_b^\varnothing(\text{input}')$, simulates $\mathbf{H}$ by using the $2q$ -wise independent hash functions and replaces the $(\widetilde{pk}^*, \widetilde{c}^*)$ of input' by its own input.
- $\mathcal{B}_b$ samples $i \leftarrow \{1, \dots, q_H\}$, measures the i -th quantum query to $\mathbf{H}$. Denote the measurement outcome as $(*, *, K, \dots)$, then $\mathcal{B}_b$ returns K as its final output.

Note that $\mathcal{B}_b$ is actually a OW-CPA adversary against Π_{KEM} , and the running time of $\mathcal{B}_b$ is roughly the same as those of $\mathcal{A}$. Therefore, we have

$$\Pr[\text{Find} : \mathcal{C}_b^{\mathcal{O}_S^{SC}}(\text{input}')] \leq 4q_H \cdot \text{Adv}_{\Pi_{\text{KEM}}}^{\text{OW-CPA}}(\mathcal{B}_b).$$

Finally, by combining the above equations, we obtain

$$|\Pr[1 \leftarrow \mathbf{G}_5^b] - \Pr[1 \leftarrow \mathbf{G}_6^b]| \leq 4(q_H + 1) \cdot \sqrt{\text{Adv}_{\Pi_{\text{KEM}}}^{\text{OW-CPA}}(\mathcal{B}_b)}.$$

By using the above equation in place of Eq. (29) and Eq. (30), and following the subsequent proof of Lemma 4, we can complete the proof of Lemma 12. $\square$

Lemma 13 (QROM version of Lemma 5). *For Case 2, where the QROM adversary $\mathcal{A}$ cannot obtain the long-term secret key of the initiator for sID^* , we have*

$$\begin{aligned} \text{Case 2} \leq & 2S \cdot \gamma(\Pi_{\text{PKE}}.\text{Enc}) + 2S^3 N^2 \cdot \gamma(\Pi_{\text{KEM}}.\text{KeyGen}) \\ & + 2S^3 N^2 \cdot \gamma(\Pi_{\text{KEM}}.\text{Encap}) + 2S^2 N^2 \cdot \delta_K + 32S^2 N^2 (q_G + 2)^2 \cdot \delta_P \\ & + \frac{4S^2 N^2 (q_R + q_I)}{\sqrt{|\mathcal{M}|}} + 8S^2 N^2 (q_G + q_H + 1) \cdot \sqrt{\text{Adv}_{\Pi_{\text{PKE}}}^{\text{OW-CPA}}(\mathcal{B})}. \end{aligned}$$

Here $\mathcal{B}$ has roughly the same running time as $\mathcal{A}$.

Proof. Four equations in the proof of Lemma 5 cannot be directly lifted to QROM: Eq. (40), Eq. (46), Eq. (54), and Eq. (55). For Eq. (40), we cannot claim that the adversary $\mathcal{A}$ never queries H_{resp} and H_{init} on some particular input, as $\mathcal{A}$ has quantum access to these oracles. For Eq. (46), since $\mathcal{B}_b$ has quantum access to G or G_{good} in the QROM, $\mathcal{C}_b$ also has quantum-access to N_1 or N_2 , and thus the inequality (a) no longer holds. Since Eq. (54) has the same analysis as Eq. (46), it also fails to hold in the QROM. For Eq. (55), the event $\text{Bad}_{m_i^*}^b$ is not well-defined in the QROM. Here, we employ the following approach to lift Eq. (40), Eq. (46), Eq. (54), and Eq. (55) to the QROM.

- For Eq. (40), in the QROM, we can directly use Lemma 11 to derive a looser bound as $2(q_R + q_I)/\sqrt{|\mathcal{M}|}$.
- For Eq. (46) and Eq. (54), in the QROM, the inequality (a) can be derived using Lemma 10, yielding a looser bound as $8(q_G + 1)^2 \cdot \max_{m \in \mathcal{M}} \delta(pk_{i^*}, sk_{i^*}, m)$.
- For Eq. (55), in the QROM, by using the semi-classical oracle technique and a derivation very similar to that in the proof of Lemma 12, we can obtain

$$|\Pr[1 \leftarrow \mathbf{G}_{12}^b] - \Pr[1 \leftarrow \mathbf{G}_{13}^b]| \leq 4(q_G + q_H + 1) \cdot \sqrt{\text{Adv}_{\Pi_{\text{PKE}}}^{\text{OW-CPA}}(\mathcal{B}_b)}.$$

By following the proof strategy of Lemma 5 and using the above new bounds instead, we can complete the proof of Lemma 13. $\square$

Lemma 14 (QROM version of Lemma 6). *For Case 3, where $\mathfrak{M}(\text{sID}^*) = \emptyset$, $\text{role}[\text{sID}^*] = \text{initiator}$, and the adversary $\mathcal{A}$ has quantum access to its random oracles, we have*

$$\begin{aligned} \text{Case 3} \leq & 2S^2 N \cdot \gamma(\Pi_{\text{KEM}}.\text{KeyGen}) + \frac{2SN(q_R + q_I)}{\sqrt{|\mathcal{M}|}} \\ & + 6SN\sqrt{(q_I + q_H + 1) \cdot \delta_P} + 32SN(q_G + 1)^2 \cdot \delta_P \\ & + 22SN(q_I + q_H + q_G + 1)^{1.5} \cdot \sqrt{\text{Adv}_{\Pi_{\text{PKE}}}^{\text{OW-CPA}}(\mathcal{B}_b^*)}. \end{aligned}$$

Here $\mathcal{B}$ has roughly the same running time as $\mathcal{A}$.

Proof. Four parts in the proof of Lemma 6 cannot be directly lifted to the QROM: Eq. (63), Eq. (64), Eq. (69), and the game transition from $\mathbf{G}_9^b$ to $\mathbf{G}_{11}^b$. For Eq. (63), Eq. (64) and Eq. (69), we follow the approach used in the proof of Lemma

13 to derive a looser bound $2(q_R + q_I)/\sqrt{|\mathcal{M}|} + 8(q_G + 1)^2 \cdot \delta_P$. Regarding the game transition from $\mathbf{G}_9^b$ to $\mathbf{G}_{11}^b$, its analysis in the QROM is more involved; nevertheless, we are still able to establish an upper bound using the semi-classical oracle technique. Denote

$$S := \{(*, *, m_{j^*}, \dots)\} \cup \{m_j^*\} \cup \{m : m \neq m_{j^*} \wedge \Pi_{\text{PKE}}.\text{Enc}(pk_{j^*}, m; \mathbf{G}(m)) = c_{j^*}\}, \quad (107)$$

and let $\mathcal{O}_S^{SC}$ be a semi-classical oracle with respect to the set S . Then, we introduce two new games:

Game $\mathbf{G}_{9a}^b$: This game is the same as $\mathbf{G}_9^b$, except that just before each query to $\mathbf{H}_{\text{init}}$ and $\mathbf{H}$ during the running of $\mathcal{A}$, the $\mathcal{O}_S^{SC}$ is queried on their input registers.

Game $\mathbf{G}_{10a}^b$: This game is the same as $\mathbf{G}_{10}^b$, except that just before each query to $\mathbf{H}_{\text{init}}$ and $\mathbf{H}$ during the running of $\mathcal{A}$, the $\mathcal{O}_S^{SC}$ is queried on their input registers.

Next, we design a new quantum oracle algorithm $\mathcal{C}_b$ ($b \in \{0, 1\}$) with oracle access to $O \in \{\emptyset, \mathcal{O}_S^{SC}\}$ and input

$$\text{input} := (j^*, s_{\text{init}}, \mathbf{G}, m_{j^*}, \widetilde{pk}^*, \widetilde{sk}^*, c_{j^*}).$$

Here $(j^*, s_{\text{init}}, \mathbf{G})$ is generated in the same way as in game $\mathbf{G}_9^b$, $(m_{j^*}, \widetilde{pk}^*, \widetilde{sk}^*, c_{j^*})$ is generated by $\text{Challenge}(s_{\text{init}}, j^*)$, where $c_{j^*} := \Pi_{\text{PKE}}.\text{Enc}(pk_{j^*}, m_{j^*}; \mathbf{G}(m_{j^*}))$. The definition of $\mathcal{C}_b^O(\text{input})$ is given below.

- $\mathcal{C}_b^O(\text{input})$ runs game $\mathbf{G}_9^b$ by using its input and finally returns $\mathbf{G}_9^b$'s output b' . During the running, $\mathcal{C}$ simulates the random oracle $\mathbf{G}$ by using the function $\mathbf{G}$ of its input, simulates the other (quantum-accessible) random oracles using $2q$ -wise independent hash functions, and queries its oracle O on the input registers of $\mathbf{H}_{\text{init}}$ (resp. $\mathbf{H}$) just before processing each query to $\mathbf{H}_{\text{init}}$ (resp. $\mathbf{H}$).

Notice that $\mathcal{C}_b$ queries O at most $q_I + q_H$ times. Thus, for games $\mathbf{G}_9^b$ and $\mathbf{G}_{9a}^b$,

$$\begin{aligned} \Pr[1 \leftarrow \mathbf{G}_9^b] &= \Pr[1 \leftarrow \mathcal{C}_b^{\emptyset}(\text{input})], \\ \Pr[\neg \text{Find}_S \wedge 1 \leftarrow \mathbf{G}_{9a}^b] &= \Pr[\neg \text{Find}_S \wedge 1 \leftarrow \mathcal{C}_b^{\mathcal{O}_S^{SC}}(\text{input})]. \end{aligned}$$

Here Find_S denotes the event that the measurement of $\mathcal{O}_S^{SC}$ ever has outcome 1. Let $\mathcal{D}_b$ ($b \in \{0, 1\}$) be a new quantum oracle algorithm the same as $\mathcal{C}_b$, except that it runs game $\mathbf{G}_{10}^b$. Now we have

$$\begin{aligned} \Pr[1 \leftarrow \mathbf{G}_{10}^b] &= \Pr[1 \leftarrow \mathcal{D}_b^{\emptyset}(\text{input}')], \\ \Pr[\neg \text{Find}_S \wedge 1 \leftarrow \mathbf{G}_{10a}^b] &= \Pr[\neg \text{Find}_S \wedge 1 \leftarrow \mathcal{D}_b^{\mathcal{O}_S^{SC}}(\text{input}')]. \end{aligned}$$

By applying Lemma 8, we obtain

$$\begin{aligned} &|\Pr[1 \leftarrow \mathbf{G}_9^b] - \Pr[\neg \text{Find}_S \wedge 1 \leftarrow \mathbf{G}_{9a}^b]| \\ &\leq \sqrt{(q_I + q_H + 1) \cdot \Pr[\text{Find}_S : \mathcal{C}_b^{\mathcal{O}_S^{SC}}(\text{input})]} \\ &\stackrel{(a)}{=} \sqrt{(q_I + q_H + 1) \cdot \Pr[\text{Find}_S : \mathbf{G}_{9a}^b]}, \end{aligned} \quad (108)$$

and

$$\begin{aligned}
& \left| \Pr [1 \leftarrow \mathbf{G}_{10}^b] - \Pr [\neg \text{Find}_S \wedge 1 \leftarrow \mathbf{G}_{10a}^b] \right| \\
& \leq \sqrt{(q_I + q_H + 1) \cdot \Pr [\text{Find}_S : \mathcal{D}_b^{\mathcal{O}_{SC}}(\text{input})]} \\
& \stackrel{(b)}{=} \sqrt{(q_I + q_H + 1) \cdot \Pr [\text{Find}_S : \mathbf{G}_{10a}^b]}.
\end{aligned} \tag{109}$$

Here (a) and (b) follows from the definitions of $\mathcal{C}_b$ and $\mathcal{D}_b$.

Recall that **Error** denotes the event that just after the query $\text{DER}_{\text{init}}(s_{\text{init}}, M')$ finishes the assignment $(\text{sent}(s_{\text{init}}), \text{rece}(s_{\text{init}})) := (\widetilde{M}, M')$, there exists a session $\text{sID} \neq s_{\text{init}}$ for which sID and s_{init} constitute a matching session pair. Now, for the probability $\Pr[\neg \text{Find}_S \wedge 1 \leftarrow \mathbf{G}_{9a}^b]$, we can compute

$$\begin{aligned}
\Pr [\neg \text{Find}_S \wedge 1 \leftarrow \mathbf{G}_{9a}^b] & \stackrel{(a)}{=} \Pr [\neg \text{Find}_S \wedge \neg \text{Error} \wedge 1 \leftarrow \mathbf{G}_{9a}^b] \\
& \stackrel{(b)}{=} \Pr [\neg \text{Find}_S \wedge \neg \text{Error} \wedge 1 \leftarrow \mathbf{G}_{10a}^b].
\end{aligned} \tag{110}$$

Here, (a) holds because the event that the game outputs 1 guarantees $\mathfrak{M}(\text{sID}^*) = \emptyset$ and $\text{sID}^* = s_{\text{init}}$, which implies that the event **Error** does not occur. The analysis for establishing (b) is similar to that for Eq. (72). Specifically, conditioned on $\neg \text{Find}_S \wedge \neg \text{Error}$ occurring, the key k' finally output by $\text{DER}_{\text{init}}(s_{\text{init}}, M')$ in $\mathbf{G}_{9a}^b$ is uniformly random from the adversary $\mathcal{A}$'s view. As a result, $\mathcal{A}$ cannot detect our game transition from $\mathbf{G}_{9a}^b$ to $\mathbf{G}_{10a}^b$. For the probability $\Pr[\text{Find}_S : \mathbf{G}_{9a}^b]$, we can compute

$$\begin{aligned}
\Pr [\text{Find}_S : \mathbf{G}_{9a}^b] & = 1 - \Pr [\neg \text{Find}_S : \mathbf{G}_{9a}^b] \\
& = 1 - \Pr [\neg \text{Find}_S \wedge \text{Error} : \mathbf{G}_{9a}^b] - \Pr [\neg \text{Find}_S \wedge \neg \text{Error} : \mathbf{G}_{9a}^b] \\
& \stackrel{(a)}{=} 1 - \Pr [\neg \text{Find}_S \wedge \text{Error} : \mathbf{G}_{10a}^b] - \Pr [\neg \text{Find}_S \wedge \neg \text{Error} : \mathbf{G}_{9a}^b] \\
& \stackrel{(b)}{=} 1 - \Pr [\neg \text{Find}_S \wedge \text{Error} : \mathbf{G}_{10a}^b] - \Pr [\neg \text{Find}_S \wedge \neg \text{Error} : \mathbf{G}_{10a}^b] \\
& = 1 - \Pr [\neg \text{Find}_S : \mathbf{G}_{10a}^b] \\
& = \Pr [\text{Find}_S : \mathbf{G}_{10a}^b].
\end{aligned} \tag{111}$$

Here, (a) holds because conditioned on **Error** occurring, games $\mathbf{G}_{9a}^b$ and $\mathbf{G}_{10a}^b$ are equivalent. The analysis for establishing (b) is similar to that for establishing (b) in Eq. (110), that is, conditioned on $\neg \text{Find}_S \wedge \neg \text{Error}$ occurring, games $\mathbf{G}_{9a}^b$ and $\mathbf{G}_{10a}^b$ are also equivalent.

To further compute the probability $\Pr[\text{Find}_S : \mathbf{G}_{10a}^b]$, we first define a new set

$$S' := \{(*, *, m_{j*}, \dots)\} \cup \{m_j^*\}, \tag{112}$$

a new event

$$\text{Coll}_{m_j^*} : \text{There exists a } m \neq m_{j*} \text{ such that } \Pi_{\text{PKE}}.\text{Enc}(pk_{j*}, m; \mathbf{G}(m)) = c_{j*},$$

and a new game as:

Game $\mathbf{G}_{10b}^b$: This game is the same as $\mathbf{G}_{10a}^b$, except that the semi-classical Oracle $\mathcal{O}_S^{SC}$ is replaced by $\mathcal{O}_{S'}^{SC}$. That is, just before each query to $\mathbf{H}_{\text{init}}$ and $\mathbf{H}$ during the running of $\mathcal{A}$, the $\mathcal{O}_{S'}^{SC}$ is queried on their input registers.

Obviously, if $\text{Coll}_{m_j^*}$ does not occur, $\{m : m \neq m_{j^*} \wedge \Pi_{\text{PKE}}.\text{Enc}(pk_{j^*}, m; \mathbf{G}(m)) = c_{j^*}\}$ is an empty set and thus S (Eq. (107)) and S' (Eq. (112)) are identical. Therefore, we have

$$|\Pr[\text{Find}_S : \mathbf{G}_{10a}^b] - \Pr[\text{Find}_{S'} : \mathbf{G}_{10b}^b]| \stackrel{(a)}{\leq} \Pr[\text{Coll}_{m_j^*} : \mathbf{G}_{10b}^b] \stackrel{(b)}{\leq} 2 \cdot \delta_P. \quad (113)$$

Here, $\text{Find}_{S'}$ denotes the event that the measurement of $\mathcal{O}_{S'}^{SC}$ ever has outcome 1, (a) uses the difference lemma, and (b) relies on Corollary 1. Note that even though the random oracles in games $\mathbf{G}_{10a}^b$ and $\mathbf{G}_{11a}^b$ support quantum queries, the event $\text{Coll}_{m_j^*}$ is defined independently of the random oracle queries made by the adversary. We hence can directly apply the difference lemma to derive (a). At this point, by combining Eq. (108) to Eq. (111) and Eq. (113), we have

$$\begin{aligned} & |\Pr[1 \leftarrow \mathbf{G}_9^b] - \Pr[1 \leftarrow \mathbf{G}_{10}^b]| \\ & \leq 2\sqrt{(q_I + q_H + 1) \cdot (\Pr[\text{Find}_{S'} : \mathbf{G}_{10b}^b] + 2 \cdot \delta_P)}. \end{aligned} \quad (114)$$

Next, we consider the game transition from $\mathbf{G}_{10}^b$ to $\mathbf{G}_{11}^b$. Recall that game $\mathbf{G}_{11}^b$ is identical to $\mathbf{G}_{10}^b$, except that the c_{j^*} generated by $\text{Challenge}(s_{\text{init}}, j^*)$ is replaced by $\Pi_{\text{PKE}}.\text{Enc}(pk_{j^*}, m_{j^*}; r^*)$, where r^* is fresh uniformly random value of $\mathcal{R}$. This transition aims to convert c_{j^*} into a valid challenge ciphertext of scheme Π_{PKE} , which further ensures key indistinguishability based on the OW-CPA security of Π_{PKE} . Note that games $\mathbf{G}_{10}^b$ and $\mathbf{G}_{11}^b$ are equivalent if the adversary $\mathcal{A}$'s view is independent of $\mathbf{G}(m_{j^*})$. Hence, using an approach similar to that used in the proof of Lemma 12, we can construct a OW-CPA adversary $\mathcal{B}_b$ ($b \in \{0, 1\}$) such that $\mathcal{B}_b$ has roughly the same running time as $\mathcal{A}$ and

$$\begin{aligned} & |\Pr[1 \leftarrow \mathbf{G}_{10}^b] - \Pr[1 \leftarrow \mathbf{G}_{11}^b]| \\ & \leq 4(q_H + q_G + 1) \cdot \sqrt{\text{Adv}_{\Pi_{\text{PKE}}}^{\text{OW-CPA}}(\mathcal{B}_b)}. \end{aligned} \quad (115)$$

In addition to analyzing $|\Pr[1 \leftarrow \mathbf{G}_{10}^b] - \Pr[1 \leftarrow \mathbf{G}_{11}^b]|$, we also need to analyze $\Pr[\text{Find}_{S'} : \mathbf{G}_{10b}^b]$, as it appears in the upper bound of Eq. (114). We first define the following three games:

Game $\mathbf{G}_{10c}^b$: This game is the same as $\mathbf{G}_{10b}^b$, except that just before each query to $\mathbf{G}$ during the running of $\mathcal{A}$, the $\mathcal{O}_{S'}^{SC}$ is queried on their input registers.

Game $\mathbf{G}_{10d}^b$: This game is the same as $\mathbf{G}_{10c}^b$, except that in $\text{Challenge}(s_{\text{init}}, j^*)$ it replaces $\mathbf{G}(m_{j^*})$ by r^* , a fresh uniformly random value in $\mathcal{R}$.

Game $\mathbf{G}_{10e}^b$: This game is the same as $\mathbf{G}_{10d}^b$, except that it no longer queries $\mathcal{O}_{S'}^{SC}$ just before each query to $\mathbf{G}$.

It is easy to see that games $\mathbf{G}_{10b}^b$ and $\mathbf{G}_{10e}^b$ differ in exactly one point: the c_{j^*} produced by $\text{Challenge}(s_{\text{init}}, j^*)$ equals $\Pi_{\text{PKE}}.\text{Enc}(pk_{j^*}, m_{j^*}; \mathbf{G}(m_{j^*}))$ in $\mathbf{G}_{10b}^b$, while the corresponding c_{j^*} is $\Pi_{\text{PKE}}.\text{Enc}(pk_{j^*}, m_{j^*}; r^*)$ in $\mathbf{G}_{10e}^b$. Hence, although we introduce the semi-classical oracle in both games $\mathbf{G}_{10b}^b$ and $\mathbf{G}_{10e}^b$ ¹², these two games possess properties similar to those of games $\mathbf{G}_{10}^b$ and $\mathbf{G}_{11}^b$. Concretely, games $\mathbf{G}_{10b}^b$ and $\mathbf{G}_{10e}^b$ are also equivalent if the adversary $\mathcal{A}$'s view is independent of $\mathbf{G}(m_{j^*})$. Define $\text{Find}_{S'}^{\mathbf{G}}$ as the event that the measurement of $\mathcal{O}_{S'}^{SC}$ performed before $\mathbf{G}$ queries ever has outcome 1. Then, based on games $\mathbf{G}_{10c}^b$ and $\mathbf{G}_{10d}^b$ given above, and using an approach similar to that used in the proof of Lemma 12,

$$\begin{aligned} & \left| \sqrt{\Pr[\text{Find}_{S'} : \mathbf{G}_{10b}^b]} - \sqrt{\Pr[\neg \text{Find}_{S'}^{\mathbf{G}} \wedge \text{Find}_{S'} : \mathbf{G}_{10c}^b]} \right| \\ & \leq \sqrt{(q_H + q_G + 1) \cdot \Pr[\text{Find}_{S'}^{\mathbf{G}} : \mathbf{G}_{10c}^b]}, \\ & \left| \sqrt{\Pr[\text{Find}_{S'} : \mathbf{G}_{10e}^b]} - \sqrt{\Pr[\neg \text{Find}_{S'}^{\mathbf{G}} \wedge \text{Find}_{S'} : \mathbf{G}_{10d}^b]} \right| \\ & \leq \sqrt{(q_H + q_G + 1) \cdot \Pr[\text{Find}_{S'}^{\mathbf{G}} : \mathbf{G}_{10d}^b]}, \end{aligned}$$

and

$$\begin{aligned} \Pr[\neg \text{Find}_{S'}^{\mathbf{G}} \wedge \text{Find}_{S'} : \mathbf{G}_{10c}^b] &= \Pr[\neg \text{Find}_{S'}^{\mathbf{G}} \wedge \text{Find}_{S'} : \mathbf{G}_{10d}^b], \\ \Pr[\text{Find}_{S'}^{\mathbf{G}} : \mathbf{G}_{10c}^b] &= \Pr[\text{Find}_{S'}^{\mathbf{G}} : \mathbf{G}_{10d}^b]. \end{aligned}$$

In essence, we apply Lemma 8 in a nested fashion. However, in order to avoid nested square roots, we actually use an inequality of the form " $|\sqrt{P_{\text{left}}} - \sqrt{P_{\text{right}}}| \leq \sqrt{(q+1) \cdot P_{\text{find}}}$ " guaranteed by Lemma 8. Combining the above equations, we have

$$\begin{aligned} & \left| \sqrt{\Pr[\text{Find}_{S'} : \mathbf{G}_{10b}^b]} - \sqrt{\Pr[\text{Find}_{S'} : \mathbf{G}_{10e}^b]} \right| \\ & \leq 2\sqrt{(q_H + q_G + 1) \cdot \Pr[\text{Find}_{S'}^{\mathbf{G}} : \mathbf{G}_{10d}^b]}. \end{aligned} \tag{116}$$

For $\Pr[\text{Find}_{S'} : \mathbf{G}_{10e}^b]$, by using Lemma 9, we can construct a OW-CPA adversary $\mathcal{B}'_b$ ($b \in \{0, 1\}$) such that $\mathcal{B}'_b$ has roughly the same running time as $\mathcal{A}$ and

$$\Pr[\text{Find}_{S'} : \mathbf{G}_{10e}^b] \leq 4(q_I + q_H + 1) \cdot \text{Adv}_{\Pi_{\text{PKE}}}^{\text{OW-CPA}}(\mathcal{B}'_b). \tag{117}$$

Define $\text{Find}_{S'}^{\mathbf{G}, \mathbf{H}_I, \mathbf{H}}$ as the event that the measurement of $\mathcal{O}_{S'}^{SC}$ performed before either $\mathbf{G}$, $\mathbf{H}_{\text{init}}$, or $\mathbf{H}$ queries ever has outcome 1, then we have

$$\begin{aligned} \Pr[\text{Find}_{S'}^{\mathbf{G}} : \mathbf{G}_{10d}^b] &\leq \Pr[\text{Find}_{S'}^{\mathbf{G}, \mathbf{H}_I, \mathbf{H}} : \mathbf{G}_{10d}^b] \\ &\stackrel{(a)}{\leq} 4(q_I + q_H + q_G + 1) \cdot \text{Adv}_{\Pi_{\text{PKE}}}^{\text{OW-CPA}}(\mathcal{B}_b^{\diamond}). \end{aligned} \tag{118}$$

Here (a) uses Lemma 9 again, and $\mathcal{B}_b^{\diamond}$ ($b \in \{0, 1\}$) has roughly the same running time as $\mathcal{A}$. Finally, by combining Eq. (114), Eq. (116), Eq. (117), Eq. (118), and

¹² In game $\mathbf{G}_{10e}^b$, we do not query $\mathcal{O}_{S'}^{SC}$ before each query to $\mathbf{G}$. However, we are still required to query $\mathcal{O}_{S'}^{SC}$ just before every query to $\mathbf{H}_{\text{init}}$ and $\mathbf{H}$.

folding $\mathcal{B}'_b$ and $\mathcal{B}^\circ_b$ into one adversary $\mathcal{B}^*_b$, we have

$$\begin{aligned} & |\Pr[1 \leftarrow \mathbf{G}_9^b] - \Pr[1 \leftarrow \mathbf{G}_{10}^b]| \\ & \leq 3\sqrt{(q_I + q_H + 1) \cdot \delta_P} \\ & \quad + 18(q_I + q_H + q_G + 1)^{1.5} \cdot \sqrt{\text{Adv}_{\Pi_{\text{PKE}}}^{\text{OW-CPA}}(\mathcal{B}_b^*)}. \end{aligned} \quad (119)$$

At this point, we have successfully lifted the game transition from $\mathbf{G}_9^b$ to $\mathbf{G}_{11}^b$ to the QROM, i.e., the above Eq. (115) and Eq. (119). Then, following the remaining proof strategy of Lemma 6, we can complete the proof of Lemma 14. $\square$

Lemma 15 (QROM version of Lemma 7). *For Case 4, where $\mathfrak{M}(\text{sID}^*) = \emptyset$, $\text{role}[\text{sID}^*] = \text{responder}$, and the adversary $\mathcal{A}$ has quantum access to its random oracles, we have*

$$\begin{aligned} \text{Case 4} & \leq 2S^2N \cdot \gamma(\Pi_{\text{PKE}}.\text{Enc}) + \frac{2SN(q_R + q_I)}{\sqrt{|\mathcal{M}|}} \\ & \quad + 6SN\sqrt{(q_R + q_H + 1) \cdot \delta_P} + 32SN(q_G + 1)^2 \cdot \delta_P \\ & \quad + 22SN(q_R + q_H + q_G + 1)^{1.5} \cdot \sqrt{\text{Adv}_{\Pi_{\text{PKE}}}^{\text{OW-CPA}}(\mathcal{B}_b^*)}. \end{aligned}$$

Here $\mathcal{B}$ has roughly the same running time as $\mathcal{A}$.

Proof. The proof is similar to that of Lemma 14. We omit the details here. $\square$

Finally, by combining Lemma 12, Lemma 13, Lemma 14, Lemma 15, and folding the OW-CPA adversaries against Π_{PKE} into one adversary $\mathcal{B}_P$, we obtain

$$\begin{aligned} \text{Adv}_{\text{GC-AKE}}^{\text{KI-CK}^+}(\mathcal{A}) & \leq 2S(SN + 2) \cdot \gamma(\Pi_{\text{PKE}}.\text{Enc}) + 4S^3N^2 \cdot \gamma(\Pi_{\text{KEM}}.\text{KeyGen}) \\ & \quad + 2S^3N^2 \cdot \gamma(\Pi_{\text{KEM}}.\text{Encap}) + 4S^2N^2 \cdot \delta_K + 96S^2N^2(q_G + 3) \cdot \delta_P \\ & \quad + \frac{8S^2N^2(q_R + q_I)}{\sqrt{|\mathcal{M}|}} + 8S^2N^2(q_H + 1) \cdot \sqrt{\text{Adv}_{\Pi_{\text{KEM}}}^{\text{OW-CPA}}(\mathcal{B}_K)} \\ & \quad + 52S^2N^2(q_G + 2q_H + q_I + q_R + 1)^{1.5} \cdot \sqrt{\text{Adv}_{\Pi_{\text{PKE}}}^{\text{OW-CPA}}(\mathcal{B}_P)}. \end{aligned}$$

This completes the proof of Theorem 3.